Istio & Kubernetes

Agenda

1. Kubernetes - Grundlagen
    - Anatomie einer Webanwendung mit der Gateway API (ohne Istio)
2. Grundlagen ServiceMesh & Istio
    - Einführung in Istio & Service Mesh-Architekturen
    - Warum ein Service Mesh?
    - Herausforderung & Vorteile
    - Architektur & Komponenten von Istio (Ambient: istiod, ztunnel, Waypoint, istio-cni)
    - Waypoint: was bedeutet --for service, workload und all?
    - Istio Ingress Gateway vs. Kubernetes Gateway API
    - Wildcards bei principals in der AuthorizationPolicy
3. istioctl
    - istioctl cheatsheet zum Debuggen
4. Installation von istio (ambient mode)
    - istio-installation mit istioctl - ambient (Installation per istioctl, um ein Gefühl dafür zu bekommen)
    - Bookinfo demo in ambient ausrollen
    - istio (ambient) deinstallieren - nur als Referenz
5. Installation auf OCP/Azure (ambient mode)
    - Istio Ambient auf OpenShift (OCP): OSSM 3 per Operator, Unterschiede und Einschränkungen
    - Rancher auf Azure im Hub-and-Spoke-Netz: Verbindungen, Firewall, Istio Ambient auf den Downstream-Clustern
6. Steuerung des Netzwerkverhaltens in Istio
    - Gateway API als Nachfolger von IngressGateway (Istio), VirtualService und DestinationRule
    - Virtual Services, Destination Rules
7. Übungen Steuerung des Netzwerkverhaltens (Gateway API)
    - Übung: Header-basiertes Routing
    - Übung: Traffic-Shifting / Load-Balancing
    - Pfadbasiertes Routing
    - Übung: LoadBalancing (LEAST_CONN) - Regeln
    - Übung: Bookinfo mit HTTPS und Let's Encrypt (cert-manager, Gateway API)
    - Übung (Ambient): Kong als API-Gateway vor Istio Ambient (mTLS, STRICT, Waypoint-Umgehung)
8. Service Resilience & Fehlertoleranz (Gateway API)
    - Circuit Breaker
    - Circuit Breaker - Gateway API (Ambient + Waypoint)
    - Chaos Engineering mit Istio
9. Sichere Service-Kommunikation mit istio (Sicherheit)
    - Sicherheit & Zero Trust mit Istio
    - Was ist in istio deep-defense (defense in depth) ?

- Überblick 3 Säulen - Sichere Service-Kommunikation
- Übung Säule 1 (Sidecar): mTLS prüfen und aktivieren/deaktivieren
- Übung Säule 1 (Ambient): mTLS prüfen und aktivieren/deaktivieren (ztunnel, HBONE, PeerAuthentication)
- Übung Säule 2 + 3 (Sidecar): Authentication und Authorization mit JWT und RBAC
- Übung Säule 2 + 3 (Ambient): Authentication und Authorization mit JWT und RBAC (Waypoint, targetRefs)
- Übung (Ambient): Zero Trust für bookinfo Schritt für Schritt (ztunnel Layer 4, Waypoint Layer 7)
- Übung (Sidecar): ZeroTrust für bookinfo Namespace bookinfo einrichten
- Überblick: CloudNativePG (Postgres) im Ambient-Mesh - lohnt sich das? (Grafiken)
- Übung (Ambient): CloudNativePG im Ambient-Mesh absichern (Operator Port 8000, Replikation, NetworkPolicy Port 15008)

10. Sicherheit, Fehlertoleranz & Observability

- Istio Authorization Policies (ingress/egress Security)
- Übung (Ambient): Egress-Gateway mit Waypoint - externe Ziele steuern, TLS-Origination, Weg per NetworkPolicy erzwingen

11. Debugging

- Debugging mit debug/run pod
- Übung: Fehlersuche im Ambient-Mesh - ztunnel-Log und Waypoint-Log
- Übung (Ambient): Gateway umgeht den Waypoint - in den Logs erkennen und beheben (ingress-use-waypoint)

12. Monitoring, Logging & Observability

- Installation Jaeger
- Distributed Tracing mit Jaeger
- Metriken & Dashboards mit Prometheus & Grafana
- Installation Prometheus
- Metriken mit Prometheus auswerten
- Installation Grafana
- Grafana Dashboards für istio
- Sinnvolle istio Metriken und Dashboards - Teil 1
- Sinnvolle istio Metriken und Dashboards - Teil 2
- Installation Kiali
- Visualisierung mit Kiali
- Analyse & Debugging von Service-Mesh-Daten

13. Vergleich Sidecar und Ambient

- Istio Proxy-Konzepte
- Istio Sidecar-Injection
- Sidecar-Overhead & Ressourcenoptimierung
- Vergleich ohne istio, istio sidecar, istio ambient
- Was Ambient (noch) nicht kann, was Sidecar kann (Stand 10.11.2026)
- Ressourcenverbrauch messen: Sidecar-Modus gegen Ambient-Modus mit denselben Pods
- Bookinfo von Sidecar auf Ambient umstellen
- Canary Update von istiod mit tags und revisions

14. Multi-Cluster (nur Theorie)

- Multi-Cluster- & Hybrid-Umgebungen mit Istio (Überblick mit Grafiken)

- Istio Federation & Cross-Cluster Traffic

15. Erweiterbarkeit von Istio (nur Theorie)
    - Was ist Wasm, wofür brauche ich es in Istio, geht es mit Ambient? (mit Anwendungsfall)
    - Wo läuft WASM (WebAssembly) im Rahmen von istio ?
16. Automatisierung mit GitOps & ArgoCD
    - Was ist ArgoCD
    - istio ambient und bookinfo mit argocd ausrollen
    - Übung: Externe CA für Istio mit ArgoCD ausliefern (cert-manager, Secret cacerts)
    - Übung (Ambient): cert-manager als CA für das Mesh mit istio-csr (istiod ist keine CA mehr; von Hand und mit ArgoCD)

Backlog

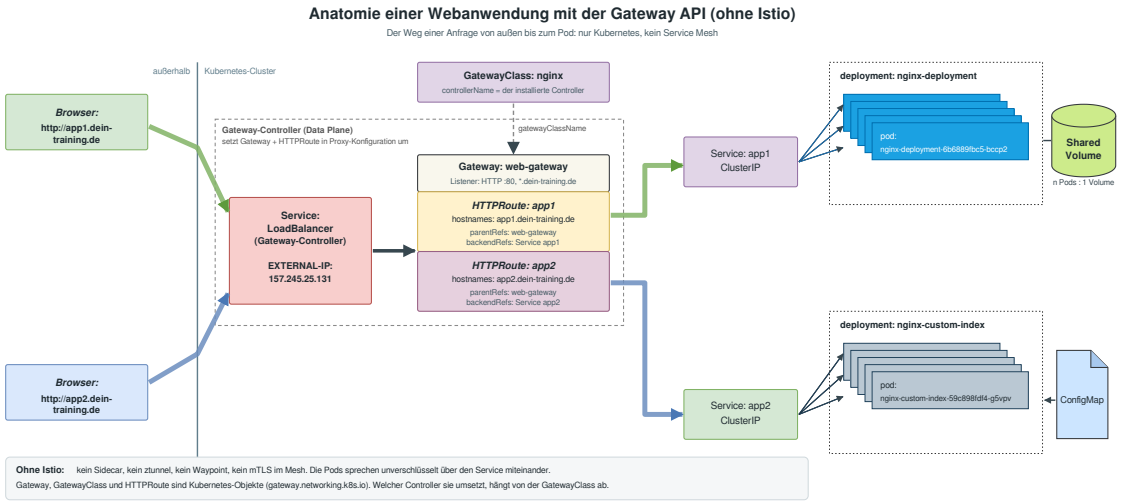
1. Installation von istio (ambient mode) mit helm
    - istio-installation with helm - ambient
2. Setup Cluster (Self-Service)
    - Self-Service Cluster ausrollen
    - Self-Service Cluster destroy
3. istio sidecar (Grundlagen)
    - Systemanforderungen & Kubernetes-Cluster-Vorbereitung
4. Installation/Konfiguration/Deinstallation von istio (sidecar - Installation mit istioctl)
    - Istio-Installation mit istioctl und der IstioOperator - Resource
    - Installations-Config-Profile
    - Wie ändere ich die Config/Installation von istio - Beispiel egressGateway
    - Deinstallation von Istio mit istioctl
5. Installation von istio (sidecar) mit helm-charts
    - Systemanforderungen & Kubernetes-Cluster-Vorbereitung
    - Istio-Installation mit helm charts
    - istioctl als debugging - tool installieren
6. Demo-App (Gateway API, sidecar)
    - Istio demo-app *bookinfo* installieren (Gateway API - Variante)
7. CNI-Plugin istio
    - Vorteile und Installation mit helm
8. Service Resilience & Fehlertoleranz (nur VirtualService)
    - Bemerkungen zu nur VirtualService und nicht Gateway API - httproute
    - Retries VirtualService only !
    - Fehlerinjektion (z.B. 500er) VirtualService only !
9. Service Resilience & Fehlertoleranz (nur Sidecar, EnvoyFilter)
    - Local Rate Limit mit EnvoyFilter - nur Sidecar !

10. Demo-App (istio api)
    - Istio demo-app *bookinfo* installieren (istio API - Variante)
11. Übungen Steuerung des Netzwerkverhaltens (istio api)
    - Übung: Header-basiertes Routing - istio api
12. Skalierung, Erweiterbarkeit & Performance-Optimierung
    - Skalierung von Istio - Möglichkeiten/Tipps&Tricks
    - Performance-Optimierung
13. Erweiterte Routing-Techniken & Traffic-Optimierung (Istio API)
    - Canary Releases & Progressive Deployments / Traffic - Shifting
    - A/B-Tests & Traffic Mirroring
    - Blue-Green- und Canary-Deployments mit Istio
14. Erweiterbarkeit von Istio (WebAssembly, Übung)
    - Eigene Istio-Erweiterungen mit WebAssembly schreiben (RateLimit-Plugin)
15. Misc
    - Grenze bei der Header-Größe Istio vs. Nginx
    - Istio als API Gateway
    - Mehrere Namepaces bei allowedRoutes bei Gateway - Objekt / Gateway API
    - Global Ratelimiting, je nach Kunde
    - zu istio beitragen

Kubernetes - Grundlagen

Anatomie einer Webanwendung mit der Gateway API (ohne Istio)

Bevor Istio ins Spiel kommt, hier der Weg einer Anfrage von außen bis zum Pod, nur mit Kubernetes-Bordmitteln und der **Gateway API**. Das Schaubild entspricht der klassischen "Anatomie einer Webanwendung" mit Ingress, nur dass die Ingress-Objekte durch Gateway-API-Objekte ersetzt sind.



Der Weg einer Anfrage

Schritt	Was passiert	Objekt
1	Der Browser löst `app1.dein-training.de` auf und landet auf der **EXTERNAL-IP**	DNS, `Service` vom Typ `LoadBalancer` (gehört zum Gateway-Controller)
2	Der Gateway-Controller nimmt die Anfrage am **Listener** (Port 80) an	`Gateway`
3	Anhand des Hostnamens wählt er die passende Route	`HTTPRoute` (`hostnames`, `parentRefs`)
4	Die Route verweist auf einen Service	`backendRefs` auf `Service` (`ClusterIP`)
5	Der Service verteilt auf die Pods des Deployments	`Deployment`, `Pod`
6	Die Pods lesen Konfiguration oder Daten	`ConfigMap`, `PersistentVolume` (gemeinsam genutzt, n Pods : 1 Volume)

Die Objekte im Einzelnen

Objekt	Aufgabe	Wer legt es an
`GatewayClass`	Legt fest, **welcher Controller** (z. B. NGINX Gateway Fabric, Envoy Gateway, Istio) die Gateways umsetzt	Plattform (einmal pro Cluster)
`Gateway`	Der Eingang: Listener mit Port, Protokoll, Hostname, optional TLS	Plattform oder Team

`HTTPRoute`	Regeln: Hostname, Pfad, Header, Ziel-Service	Team (im eigenen Namespace)
`Service` (`ClusterIP`)	Stabile interne Adresse und Lastverteilung auf die Pods	Team
`Deployment`	Hält die gewünschte Zahl Pods am Laufen	Team

Der **Gateway-Controller** ist die eigentliche Software, die Datenverkehr verarbeitet. Er liest `Gateway` und `HTTPRoute` und setzt sie in die Konfiguration seines Proxys um. `Gateway` , `GatewayClass` und `HTTPRoute` sind nur Beschreibungen (API-Gruppe `gateway.networking.k8s.io`).

Beispiel: Gateway und HTTPRoute

Der Wert von `gatewayClassName` hängt vom installierten Controller ab (`kubectl get gatewayclass`).

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      hostname: "*.dein-training.de"
      allowedRoutes:
        namespaces:
          from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app1
spec:
  parentRefs:
    - name: web-gateway
  hostnames:
    - app1.dein-training.de
  rules:
    - backendRefs:
        - name: app1
          port: 80
```

Was Istio später ergänzt

Ohne Istio gibt es kein Sidecar, keinen ztunnel und keinen Waypoint. Die Pods sprechen unverschlüsselt über den Service miteinander, und es gibt keine Identität pro Workload. Genau diese Lücken schließt Istio im weiteren Verlauf des Trainings (mTLS, Autorisierung, Resilience, Observability). Die Gateway API bleibt dabei dieselbe: Istio kann als Controller hinter einer `GatewayClass` stehen.

Grundlagen ServiceMesh & Istio

Einführung in Istio & Service Mesh-Architekturen

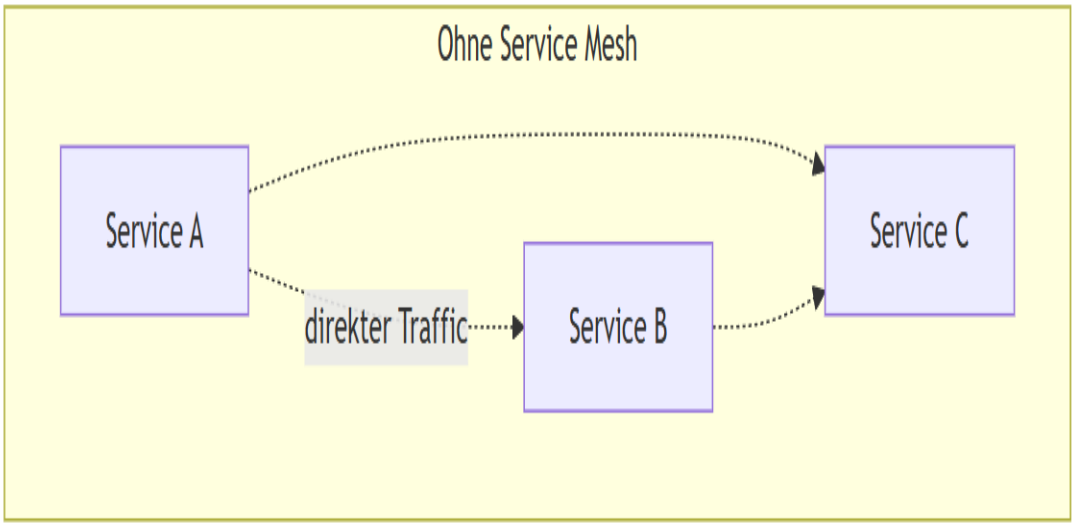
Was ist ein Service Mesh?

- Dedizierte Infrastrukturschicht für Service-zu-Service-Kommunikation
- Transparente Zwischenschicht ohne Code-Änderungen
- Zentrale Steuerung von Retry, Timeout, Verschlüsselung, Monitoring

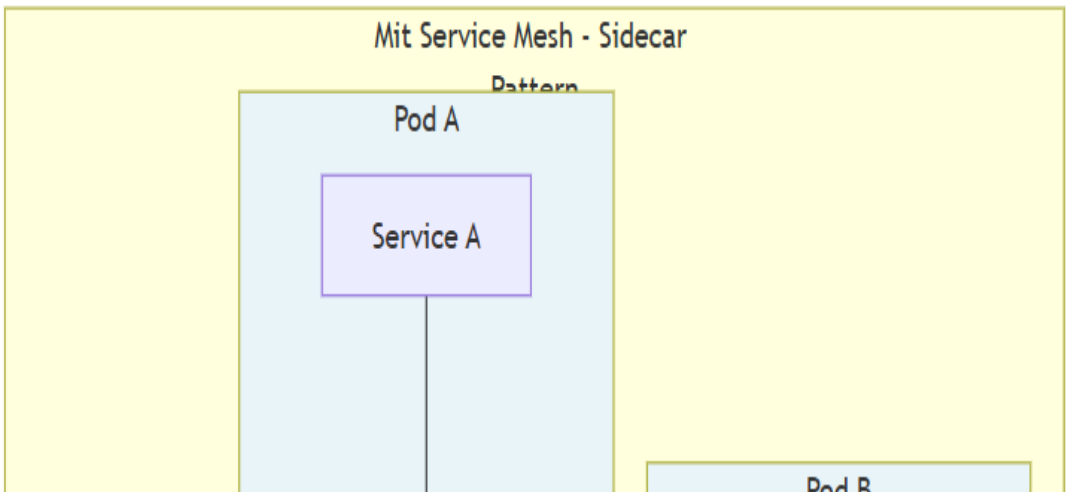
Istio im Überblick:

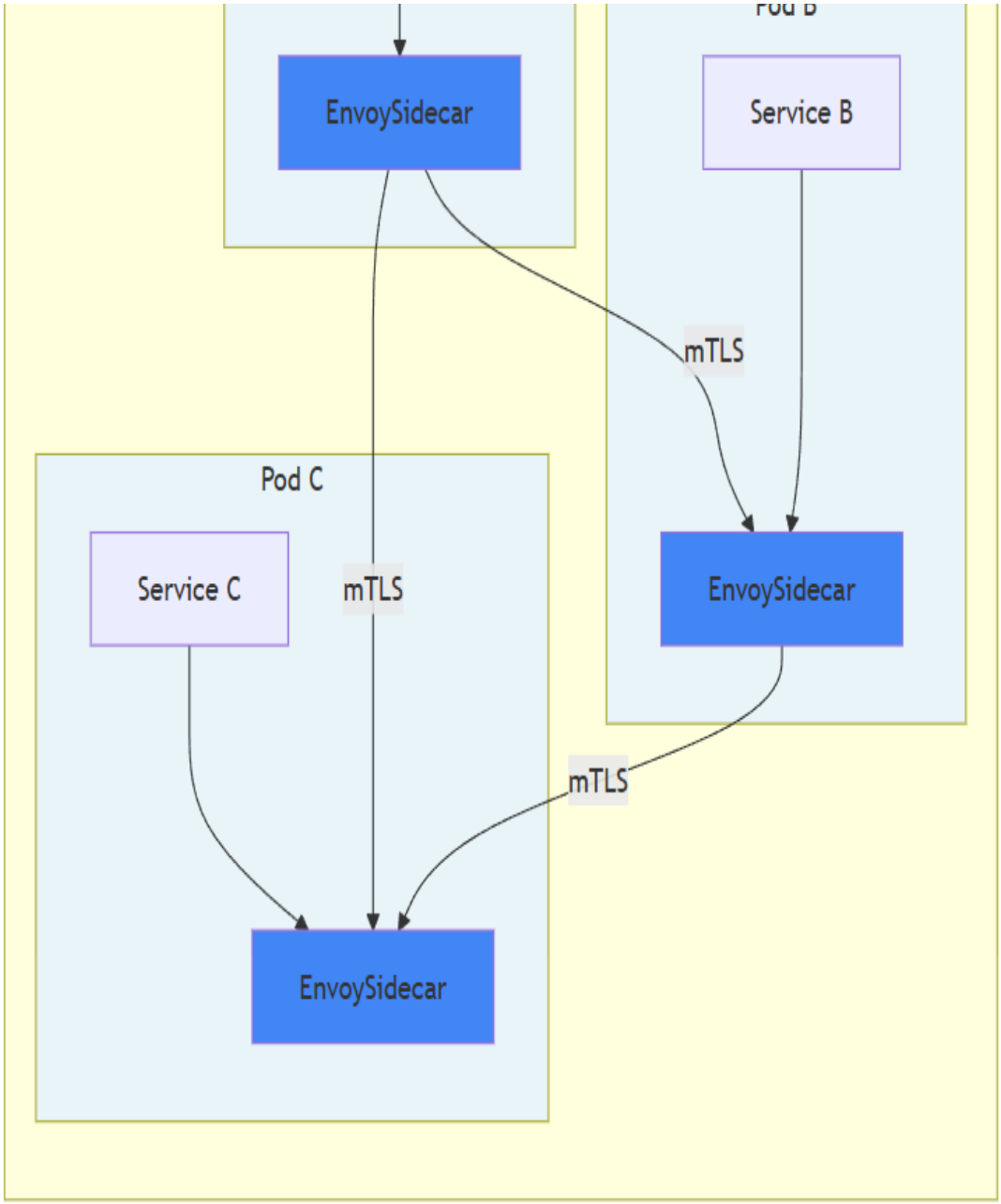
- Open-Source Service Mesh (Google, IBM, Lyft)
- Nutzt Envoy-Proxies als Sidecars
- Fängt gesamten Netzwerkverkehr ab

Vorher: Ohne Service-Mesh



Nachher: Mit Service-Mesh





Mermaid-Quelltexte

```
graph TB
    subgraph "Ohne Service Mesh"
    direction LR
    A1[Service A] -.direkter Traffic.-> B1[Service B]
    A1 -.-> C1[Service C]
    B1 -.-> C1
    end
```

```
graph TB
    subgraph "Mit Service Mesh - Sidecar Pattern"
    direction TB
```

```
    subgraph Pod1["Pod A"]
    direction LR
    SA[Service A] --> EA[EnvoySidecar]
    end

    subgraph Pod2["Pod B"]
    direction LR
    SB[Service B] --> EB[EnvoySidecar]
    end

    subgraph Pod3["Pod C"]
    direction LR
    SC[Service C] --> EC[EnvoySidecar]
    end

    EA -->|mTLS| EB
    EA -->|mTLS| EC
    EB -->|mTLS| EC
    end

    style EA fill:#4285f4
    style EB fill:#4285f4
    style EC fill:#4285f4
    style Pod1 fill:#e8f4f8
    style Pod2 fill:#e8f4f8
    style Pod3 fill:#e8f4f8
```

Warum ein Service Mesh?

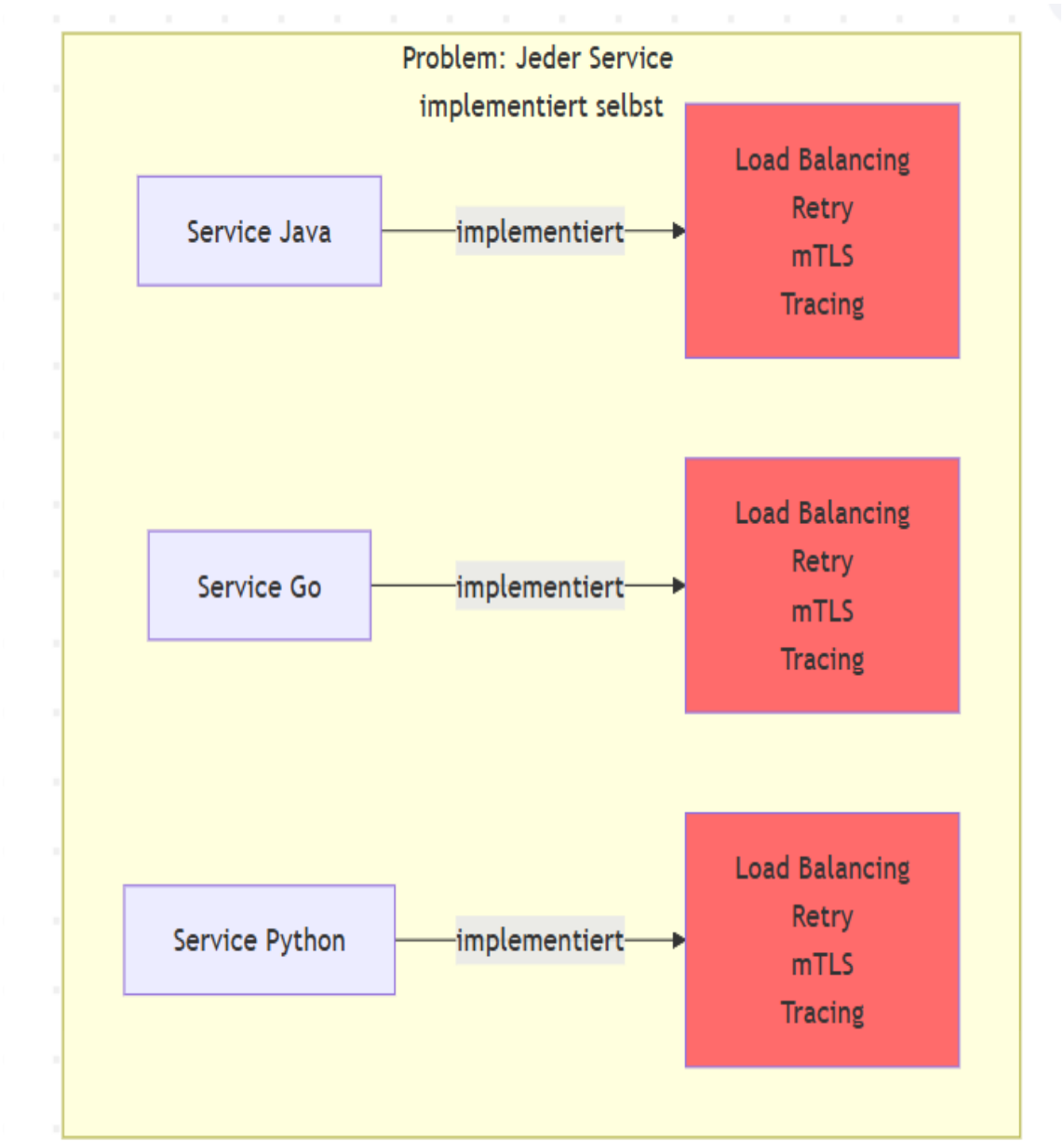
Probleme in Microservices:

- Dutzende/Hunderte Services → komplexe Kommunikation
- Jeder Service muss selbst implementieren:
  - Circuit Breaking
  - Load Balancing
  - mTLS-Verschlüsselung
  - Distributed Tracing
  - Retry-Logik
- Inkonsistente Implementierung über Sprachen hinweg
- Hoher Wartungsaufwand

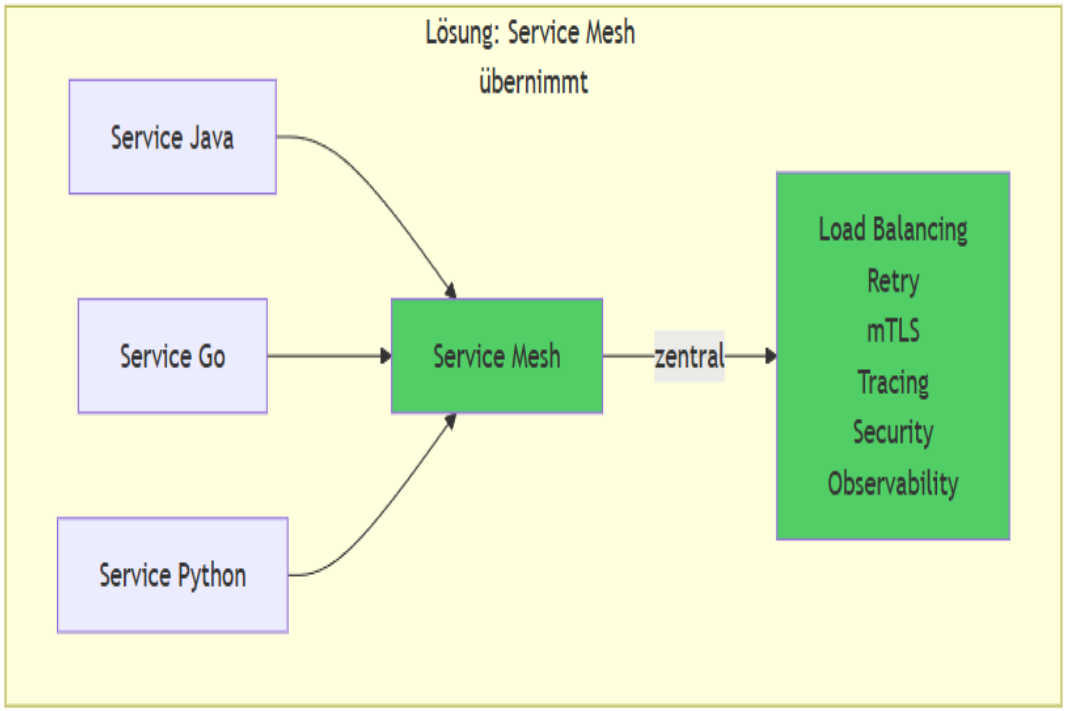
Lösung:

- Komplexität aus Anwendungscode → Infrastruktur
- Platform-Teams: zentrale Policies
- Entwickler: Fokus auf Business-Logik

Vorher: Ohne ServiceMesh



Nachher: Mit ServiceMesh




```
graph TD
    subgraph "Problem: Jeder Service implementiert selbst"
    SJ[Service Java] --> |implementiert| LJ[Load
Balancing<br/>Retry<br/>mTLS<br/>Tracing]
    SG[Service Go] --> |implementiert| LG[Load
Balancing<br/>Retry<br/>mTLS<br/>Tracing]
    SP[Service Python] --> |implementiert| LP[Load
Balancing<br/>Retry<br/>mTLS<br/>Tracing]
    end

    subgraph "Lösung: Service Mesh übernimmt"
    S1[Service Java] --> SM[Service Mesh]
    S2[Service Go] --> SM
    S3[Service Python] --> SM
    SM --> |zentral| F[Load
Balancing<br/>Retry<br/>mTLS<br/>Tracing<br/>Security<br/>Observability]
    end

    style LJ fill:#ff6b6b
    style LG fill:#ff6b6b
    style LP fill:#ff6b6b
    style SM fill:#51cf66
    style F fill:#51cf66
```

Herausforderung & Vorteile

✅ Vorteile:

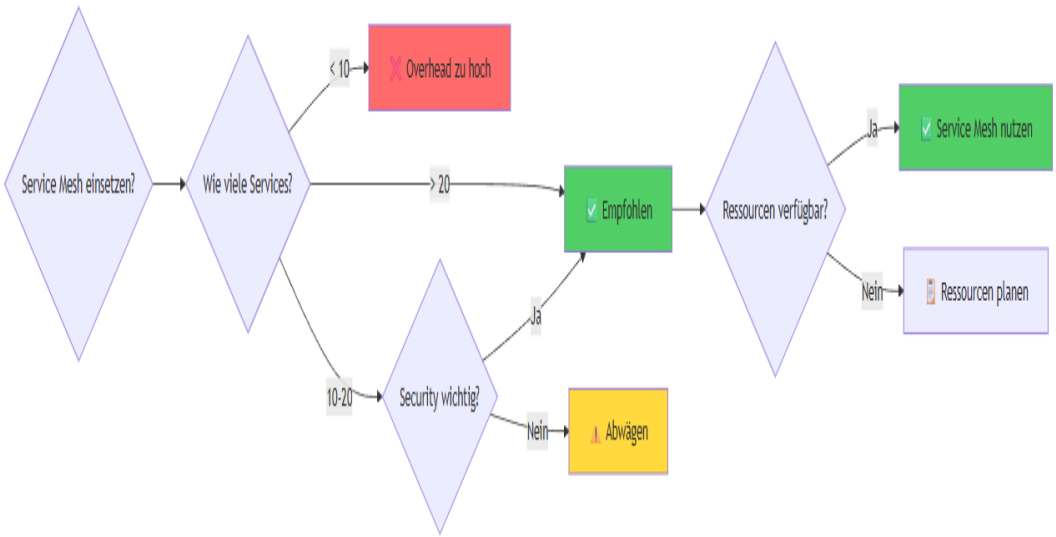
- Automatische mTLS zwischen allen Services
- Traffic-Steuerung: Canary, Blue-Green, A/B-Testing
- Einheitliches Observability (Metrics, Traces, Logs)
- Zentrale Security-Policies
- Keine Code-Änderungen nötig

⚠ Herausforderungen:

- Ressourcen-Overhead: CPU/RAM pro Sidecar
- Zusätzliche Latenz (Proxy-Hops)
- Steile Lernkurve
- Komplexeres Debugging

Wann lohnt es sich?

- Ab ~20-30 Services
- Hohe Security/Compliance-Anforderungen
- Multi-Team-Umgebungen




```
graph LR
    START{Service Mesh einsetzen?}

    START --> Q1{Wie viele Services?}
    Q1 -->|< 10| NEIN[❌ Overhead zu hoch]
    Q1 -->|10-20| Q2{Security wichtig?}
    Q1 -->|> 20| JA[✅ Empfohlen]

    Q2 -->|Ja| JA
    Q2 -->|Nein| MAYBE[⚠ Abwägen]
```

```
JA --> CHECK{Ressourcen verfügbar?}
CHECK -->|Ja| GO[✅ Service Mesh nutzen]
CHECK -->|Nein| PLAN[  Ressourcen planen]

style NEIN fill:#ff6b6b
style JA fill:#51cf66
style GO fill:#51cf66
style MAYBE fill:#ffd93d
```

Architektur & Komponenten von Istio (Ambient: istiod, ztunnel, Waypoint, istio-cni)

Im Ambient Mode steckt kein Envoy-Sidecar mehr in jedem Pod. Die Aufgaben sind stattdessen auf **zwei Ebenen** verteilt:

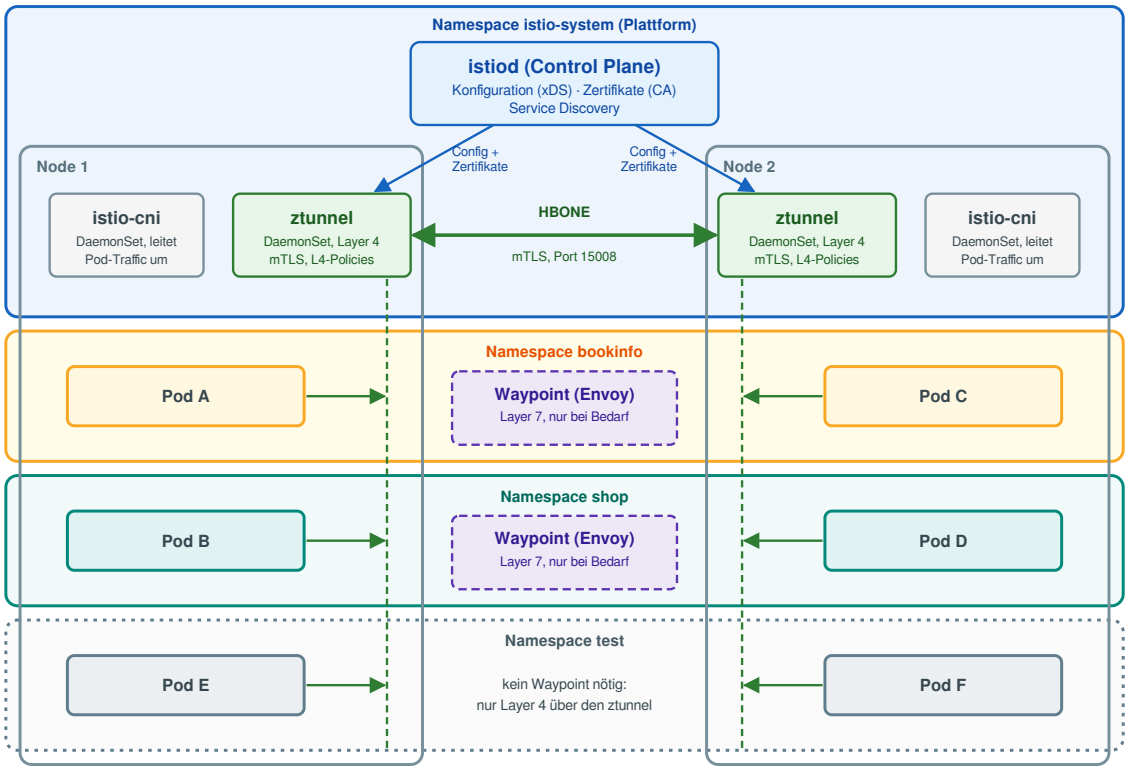
Ebene	Was	Wo
Layer 4 (immer)	mTLS, Identität, L4-Policies, TCP-Metriken	`ztunnel`, ein Agent pro Node
Layer 7 (nur bei Bedarf)	Routing, Retries, Timeouts, L7-Policies, HTTP-Metriken	`Waypoint`, ein Envoy pro Namespace

Dazu kommen der `istiod` als Control Plane und `istio-cni`, der den Traffic der Pods zum ztunnel umleitet.

Überblick

Die Grafik hat zwei Achsen: Die **Spalten** sind die Nodes, die **Zeilen** die Namespaces. `istio-system` gehört der Plattform (istiod, ztunnel, istio-cni). Die Anwendungs-Namespaces erstrecken sich über mehrere Nodes, und ein Waypoint liegt in dem Namespace, für den er Layer 7 bereitstellt.

Istio Ambient Mode: Komponenten, Nodes und Namespaces



Spalten = Nodes. Pro Node laufen ztunnel und istio-cni (immer). Die grüne Linie: Der Traffic aller Pods eines Nodes wird zum ztunnel umgeleitet.
Zeilen = Namespaces. istio-system gehört der Plattform (istiod, ztunnel, istio-cni). Die Namespaces der Anwendungen erstrecken sich über mehrere Nodes.
Waypoint: liegt im Namespace der Anwendung, nicht in istio-system. Er ist optional (gestrichelt) und bringt Layer 7 nur dorthin, wo man es braucht.
istiod konfiguriert ztunnel und Waypoints. Der Traffic der Anwendungen läuft nie durch istiod.

Die Komponenten im Einzelnen

Komponente	Läuft als	Aufgabe	Zuständig (bei Plattform-Betrieb)
istiod	Deployment, `istio-system`	Verteilt Konfiguration per xDS, stellt Zertifikate aus (CA), Service Discovery	Plattform
istio-cni	DaemonSet, ein Pod pro Node, `istio-system`	Richtet die Umleitung des Pod-Traffics zum ztunnel ein. Dafür sind keine `NET_ADMIN`-Rechte in den Anwendungs-Pods nötig	Plattform
ztunnel	DaemonSet, ein Pod pro Node, `istio-system`	Layer 4: baut mTLS-Tunnel (HBONE) auf, prüft L4-Policies, liefert TCP-Metriken. Hat pro Pod auf dem Node eine eigene Identität (SPIFFE)	Plattform
Waypoint	Deployment im Namespace der Anwendung, per Gateway API	Layer 7: HTTPRoute, Retries, Timeouts, Fault Injection, L7-`AuthorizationPolicy`, HTTP-Metriken	**Namespace-Team**

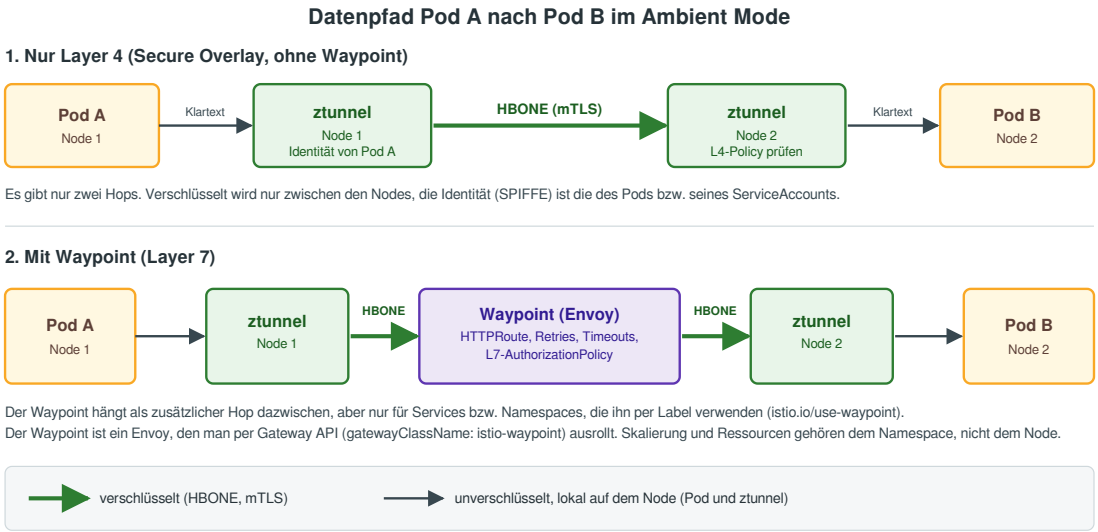
	(`gatewayClassName: istio-waypoint`)		
Ingress Gateway	Deployment, per Gateway API (`gatewayClassName: istio`)	Eingang von außen in das Mesh	Namespace-Team oder Plattform

Die Spalte ganz rechts ist für das Training wichtig ("Istio als Mieter"): ztunnel, istio-cni, istiod und die CRDs gehören zur Plattform. Im eigenen Namespace sind nur das **Label** (`istio.io/dataplane-mode=ambient`), der **Waypoint**, die **Routen** und die **Policies** Sache des Teams.

HBONE in zwei Sätzen

HBONE (HTTP-Based Overlay Network Environment) ist der Tunnel zwischen den ztunnels: HTTP/2 `CONNECT` über mTLS auf **Port 15008**. Mehrere Verbindungen zwischen zwei Nodes laufen gemultiplext durch denselben Tunnel, und die Identität des Absenders (SPIFFE) reist im Zertifikat mit.

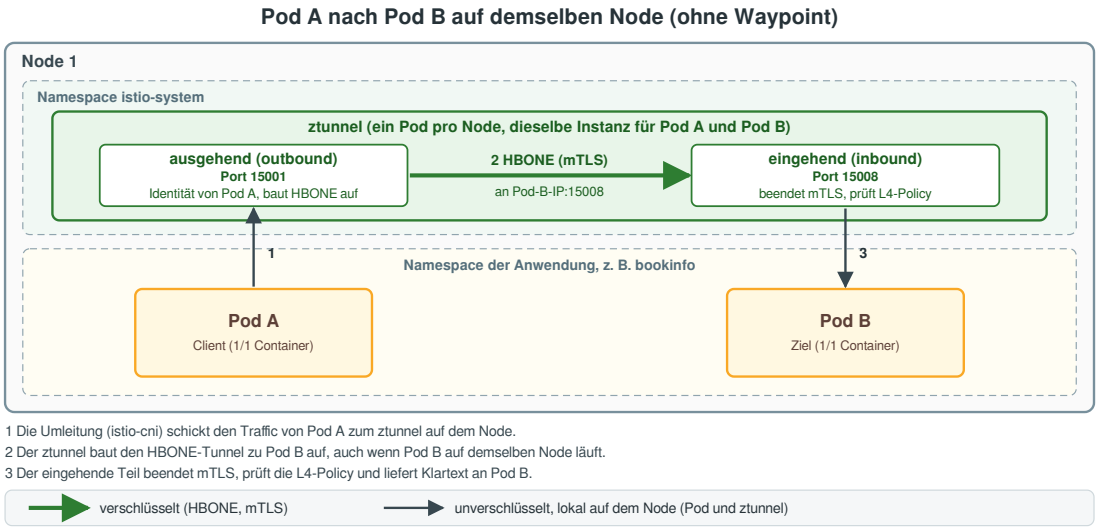
Datenpfad: ohne und mit Waypoint



- **Ohne Waypoint** (Secure Overlay): zwei Hops, nur Layer 4. Reicht für mTLS und einfache Policies (wer darf mit wem sprechen).
- **Mit Waypoint:** ein zusätzlicher Hop für alles, was HTTP-Wissen braucht. Man aktiviert ihn gezielt pro Namespace oder Service (`istio.io/use-waypoint`).
- Beides kann im selben Cluster nebeneinander laufen, Namespace für Namespace.

Sonderfall: Pod und Ziel auf demselben Node

Auch wenn beide Pods auf demselben Node laufen, geht der Traffic **über ztunnel und HBONE**. Es gibt keine Abkürzung am ztunnel vorbei. Der Grund: mTLS, Identität und L4-Policies sollen unabhängig davon gelten, wo der Scheduler die Pods hingelegt hat.



- Quell- und Ziel-Pod teilen sich **denselben ztunnel** (ein Pod pro Node). Er ist in dem Fall Absender und Empfänger zugleich.
- Im ztunnel gibt es zwei Seiten: **ausgehend** nimmt den umgeleiteten Traffic des Pods auf Port **15001** an, **eingehend** nimmt HBONE auf Port **15008** an. (Klartext von Pods außerhalb des Meshes landet auf Port 15006.)
- Das Paket verlässt den Node nicht, wird aber trotzdem verschlüsselt (HBONE auf Port 15008 der Ziel-Pod-IP) und gegen die L4-Policy geprüft.
- Ist ein Waypoint konfiguriert, ändert sich nur der Weg: Pod A → ztunnel → Waypoint → ztunnel → Pod B. Jeder Hop ist ein HBONE-Tunnel, auch wenn alle Teilnehmer auf einem Node liegen.
- Der Vorteil für Betrieb und Fehlersuche: Das Verhalten ist auf einem Node und über Nodes hinweg identisch.

Im ztunnel-Log sieht man das Beispiel direkt. Hier ruft `reviews-v2` den Service `ratings` auf, und reviews, ratings und der Waypoint liegen alle auf demselben Node (gekürzt):

```
## ausgehend: reviews-v2 spricht per HBONE mit dem Waypoint (Port 15008)
src.workload="reviews-v2-..."
src.identity="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-reviews"
dst.addr=192.168.11.96:15008 dst.hbone_addr=10.108.46.136:9080
dst.workload="waypoint-..." direction="outbound"

## eingehend: der Waypoint spricht per HBONE mit ratings (Port 15008)
src.workload="waypoint-..."
src.identity="spiffe://cluster.local/ns/bookinfo/sa/waypoint"
dst.addr=192.168.11.98:15008 dst.hbone_addr=192.168.11.98:9080 dst.workload="ratings-
v1-..." direction="inbound"
```

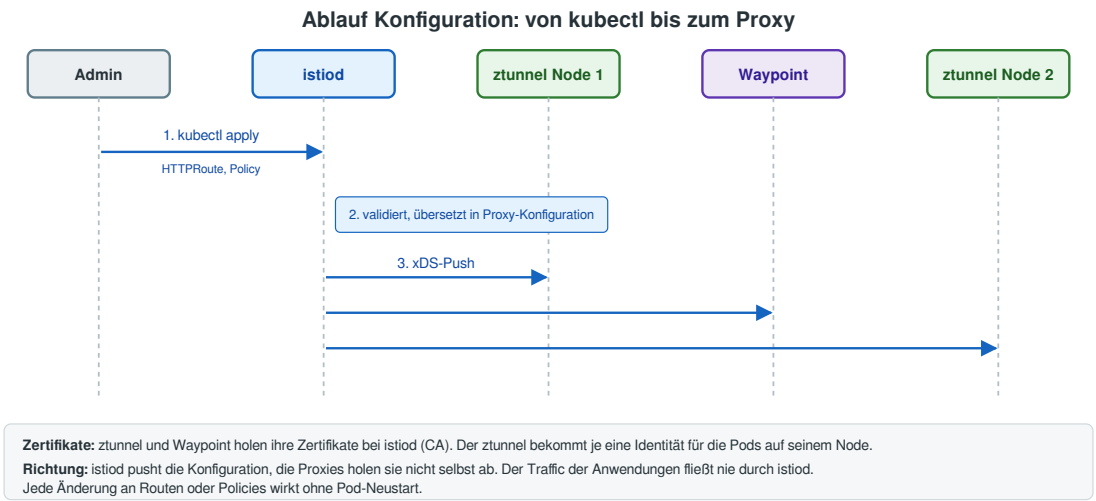
Selbst nachsehen:

```
## Auf welchem Node liegt welcher Pod? (gleiche Spalte NODE = gleicher Node)
kubectl get pods -n bookinfo -o wide

## ztunnel-Pod des Nodes finden und dessen Access-Log lesen
kubectl get pods -n istio-system -o wide | grep ztunnel
kubectl logs -n istio-system <ztunnel-pod-auf-dem-node> | grep "connection complete"
```

```
## Spricht der Ziel-Pod HBONE? (Spalte PROTOCOL)
istioctl ztunnel-config workload
```

Ablauf: Konfiguration

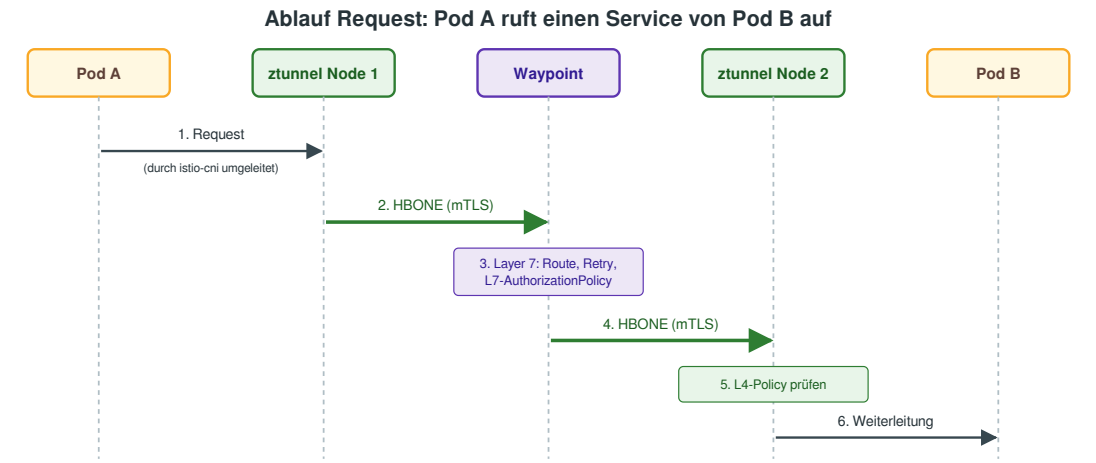


Zertifikate: ztunnel und Waypoint holen ihre Zertifikate bei istiod (CA). Der ztunnel bekommt je eine Identität für die Pods auf seinem Node.
Richtung: istiod pusht die Konfiguration, die Proxies holen sie nicht selbst ab. Der Traffic der Anwendungen fließt nie durch istiod.
Jede Änderung an Routen oder Policies wirkt ohne Pod-Neustart.

1. Der Admin legt eine Route oder Policy an (`kubectl apply`).
2. istiod validiert sie und übersetzt sie in Proxy-Konfiguration.
3. istiod schickt die Konfiguration per xDS an ztunnel und Waypoint.

Die ztunnels holen dabei auch die Zertifikate für die Pods auf ihrem Node, der Waypoint holt seines bei istiod (CA). Änderungen wirken ohne Pod-Neustart.

Ablauf: Request



Ohne Waypoint entfallen die Schritte 3 und 4: ztunnel Node 1 spricht direkt per HBONE mit ztunnel Node 2.
Metriken kommen nicht über istiod: ztunnel (Layer 4) und Waypoint (Layer 7) werden von Prometheus abgefragt.
Antworten laufen denselben Weg zurück (nicht eingezeichnet).

1. Ein Pod ruft einen Service auf. `istio-cni` hat den Traffic zum lokalen ztunnel umgeleitet.

2. Der ztunnel schickt ihn per HBONE (mTLS) weiter, mit Waypoint zuerst dorthin.
3. Der Waypoint wendet die Layer-7-Regeln an (Routing, Retries, L7-Policies).
4. Weiter geht es per HBONE zum ztunnel auf dem Ziel-Node.
5. Der ztunnel prüft die L4-Policy.
6. Er übergibt den Request an den Ziel-Pod.

Ohne Waypoint entfallen die Schritte 3 und 4: Der ztunnel auf Node 1 spricht direkt mit dem ztunnel auf Node 2.

Ambient im Vergleich zu Sidecar

	Sidecar	Ambient
Wo läuft der Proxy?	Envoy in **jedem Pod**	ztunnel **pro Node**, Waypoint **pro Namespace** (optional)
Pod-Neustart zum Aktivieren?	Ja, der Sidecar wird injiziert	Nein, Label setzen genügt
Ressourcen	CPU/RAM pro Pod	deutlich weniger Overhead
Layer 7	immer (Envoy im Pod)	nur mit Waypoint
mTLS	Sidecar zu Sidecar	ztunnel zu ztunnel (HBONE)
Upgrade von Istio	Pods müssen neu starten	ztunnel/CNI als DaemonSet, Anwendungs-Pods bleiben unberührt
Koexistenz	ja, beide Modi können im selben Cluster laufen	

Die klassische Sidecar-Architektur steht in overview-classic-sidecar.md und 07-istio-proxy-concepts-sidecar.md.

Selbst nachschauen (auf dem eigenen Cluster)

```
## Control Plane und Node-Agents: je 1 ztunnel und 1 istio-cni pro Node, 1 istiod
kubectl get deploy,ds -n istio-system
kubectl get pods -n istio-system -o wide

## Die Anwendungs-Pods zeigen nur ihren eigenen Container (1/1)
kubectl get pods -n bookinfo

## Ist der Namespace im Mesh, und nutzt er einen Waypoint?
kubectl get ns bookinfo --show-labels
kubectl get gateway -n bookinfo

## Was weiß ztunnel? Welcher Pod spricht HBONE, welcher hat einen Waypoint?
istioctl ztunnel-config workload
```

Merksätze

- **ztunnel = Layer 4, pro Node, immer da.** Er verschlüsselt und prüft, wer mit wem sprechen darf.
- **Waypoint = Layer 7, pro Namespace, nur wenn man ihn braucht.** Er routet und wiederholt Requests.
- **istiod verteilt nur Konfiguration und Zertifikate**, der Traffic fließt nie durch ihn.
- Ambient verändert die Pods nicht, deshalb ist der Einstieg ein Label und der Ausstieg ebenfalls.

Waypoint: was bedeutet --for service, workload und all?

Beim Erzeugen eines Waypoints gibt es eine Option, über die oft gestolpert wird:

```
istioctl waypoint generate --namespace bookinfo --for service
```

Die Option `--for` legt fest, **für welchen Traffic der Waypoint zuständig ist**. Der Unterschied hängt davon ab, **wie ein Client sein Ziel anspricht**.

Die einfache Erklärung

Ein Pod kann ein anderes Ziel auf zwei Arten erreichen:

Der Client ruft auf	Beispiel	Das ist ein Aufruf an
den **Namen eines Services**	`http://reviews:9080`	einen **Service**
die **IP-Adresse eines Pods**	`http://10.244.1.17:9080`	einen **Workload** (Pod)

Der Waypoint kann sich auf eine Art, auf beide oder auf keine festlegen:

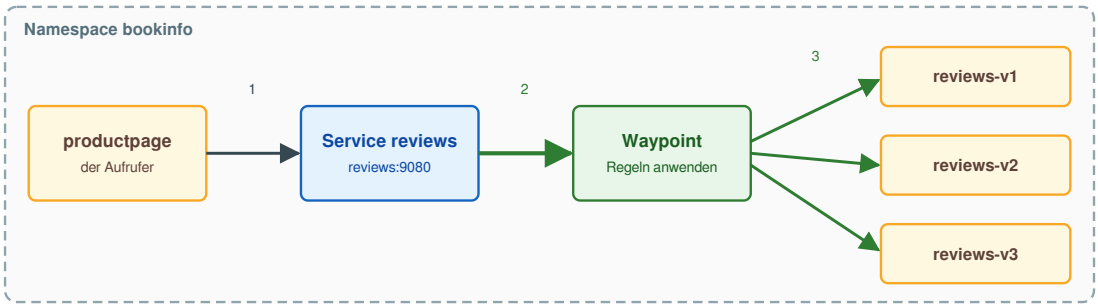
Wert	Der Waypoint bearbeitet
`service` (Standard)	Aufrufe an einen **Service-Namen**
`workload`	Aufrufe an eine **Pod-IP**
`all`	**beides**

Es gibt noch den Wert `none`. Er ist für Sonderfälle gedacht und spielt im Training keine Rolle.

Fall 1: Aufruf über den Service-Namen (der Normalfall)

Fast jede Anwendung ruft andere Anwendungen über den Service-Namen auf, bei Bookinfo zum Beispiel `reviews:9080`. Dann ist `--for service` genau richtig: Der Aufruf geht zum Waypoint, der Waypoint wendet seine Regeln an und verteilt auf die Pods.

Aufruf über den Service-Namen: der Waypoint ist dabei



1 Der Aufrufer nennt den Namen des Services. 2 Der Service-Name führt zum Waypoint. 3 Der Waypoint verteilt auf die Pods.
Retries, Routing und AuthorizationPolicy wirken.

Der Waypoint ist hier der Ort, an dem Layer-7-Regeln wirken. Das sind zum Beispiel:

- Routing (`HTTPRoute`), also Header-basiert, pfadbasiert oder Traffic-Shifting

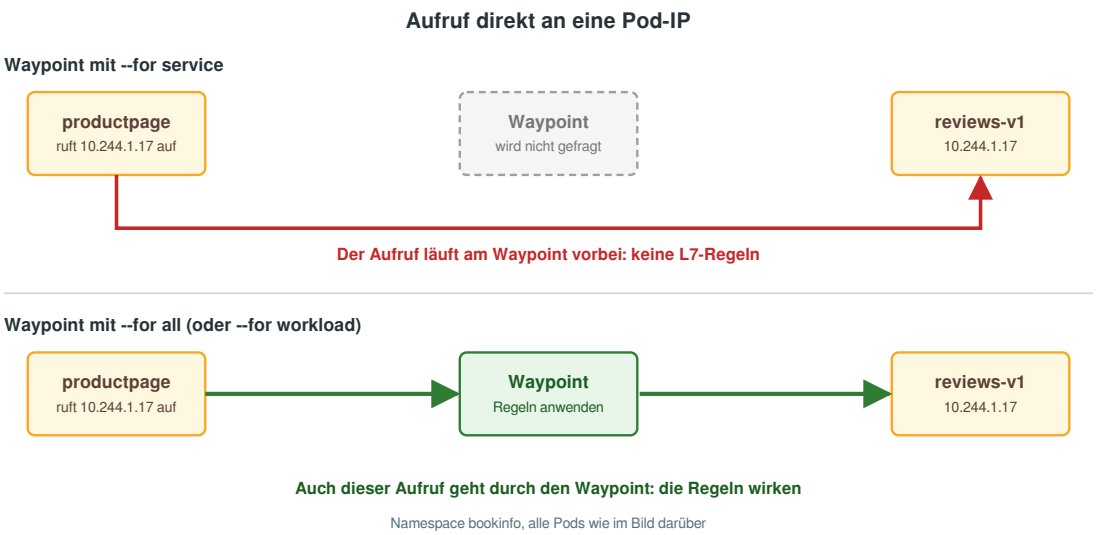
- Retries und Timeouts
- `AuthorizationPolicy` auf HTTP-Ebene

Die Pods in der Grafik liegen im selben Namespace wie der Waypoint. Der ztunnel auf dem Weg ist weggelassen, weil er für diese Frage keine Rolle spielt.

Fall 2: Aufruf direkt an die Pod-IP

Ruft ein Client direkt eine Pod-IP auf, gilt Folgendes:

- Mit `--for service` wird der Waypoint **nicht gefragt**. Der Aufruf geht am Waypoint vorbei und bekommt nur Layer 4, also mTLS durch den ztunnel.
- Mit `--for workload` oder `--for all` geht auch dieser Aufruf durch den Waypoint, und die Regeln wirken.



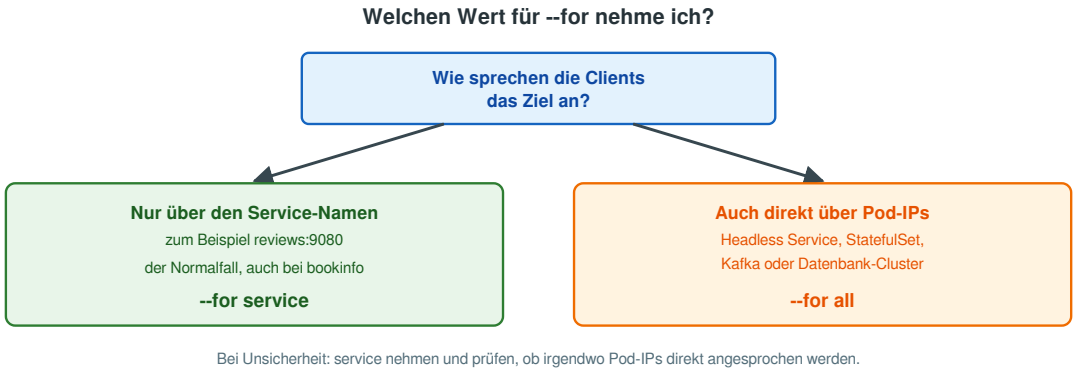
Wichtig dabei: Eine L7-Regel, die du am Waypoint definierst, gilt dann nur für den Traffic, der auch wirklich durch den Waypoint läuft.

Wann ruft ein Client eine Pod-IP direkt auf?

Das ist selten, kommt aber vor:

- **Headless Services** (`clusterIP: None`). DNS liefert dort direkt die Pod-IPs. Das ist typisch bei StatefulSets.
- **Anwendungen, die ihre Partner selbst suchen**, zum Beispiel Kafka, Datenbank-Cluster oder Protokolle, die sich gegenseitig melden.
- **Clients, die Pod-IPs aus der Service-Discovery** (Endpoints) lesen und direkt ansprechen.

Was nehme ich?



Bei Unsicherheit: service nehmen und prüfen, ob irgendwo Pod-IPs direkt angesprochen werden.

In der Praxis gilt:

- **Meistens reicht `--for service`.** Es ist auch der Standard.
- `--for all` nimmst du, wenn du sicher auch den direkten Weg über Pod-IPs abdecken willst, oder wenn du nicht sicher weißt, wie deine Clients die Ziele ansprechen.
- `--for workload` allein ist selten sinnvoll, weil dann der normale Aufruf über den Service-Namen am Waypoint vorbeigeht.

In unserer Übung Bookinfo demo in ambient ausrollen steht `--for all`. Das funktioniert. Bookinfo ruft aber nur über Service-Namen auf, deshalb würde dort auch `--for service` genügen.

Wie sehe ich, was eingestellt ist?

Die Option setzt ein Label am Waypoint-Gateway:

```
## Label am Waypoint anschauen
kubectl -n bookinfo get gateway waypoint -o jsonpath='{.metadata.labels}'
```

Dort steht `istio.io/waypoint-for` mit dem gewählten Wert. Das fertige YAML siehst du schon vor dem Ausrollen:

```
istioctl waypoint generate --namespace bookinfo --for service
```

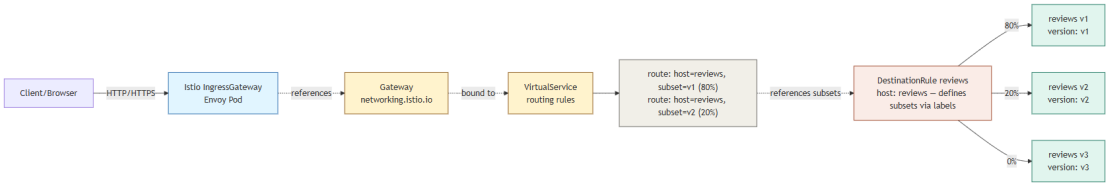
Selbst-Check

1. Ein Client ruft `http://reviews:9080` auf. Der Waypoint wurde mit `--for service` erzeugt. Geht der Aufruf durch den Waypoint?
2. Derselbe Client ruft `http://10.244.1.17:9080` auf, die IP eines reviews-Pods. Geht der Aufruf durch den Waypoint?
3. Welchen Wert nimmst du, wenn ein StatefulSet mit Headless Service im Namespace liegt und dort auch L7-Regeln gelten sollen?

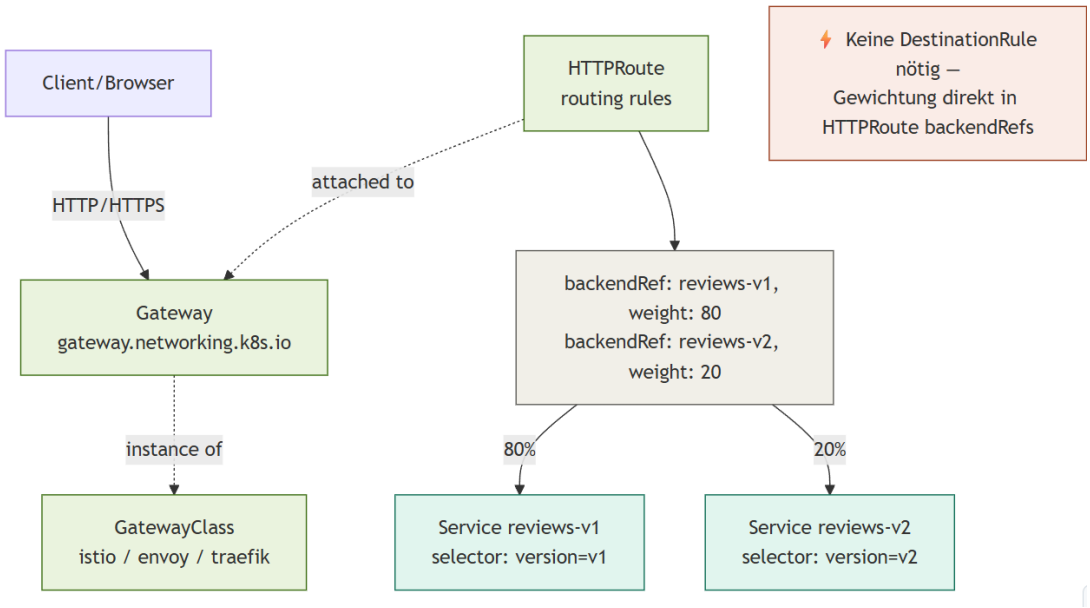
▶ Lösung

Istio Ingress Gateway vs. Kubernetes Gateway API

Istio Ingress-Gateway-Flow (bookinfo/productpage)

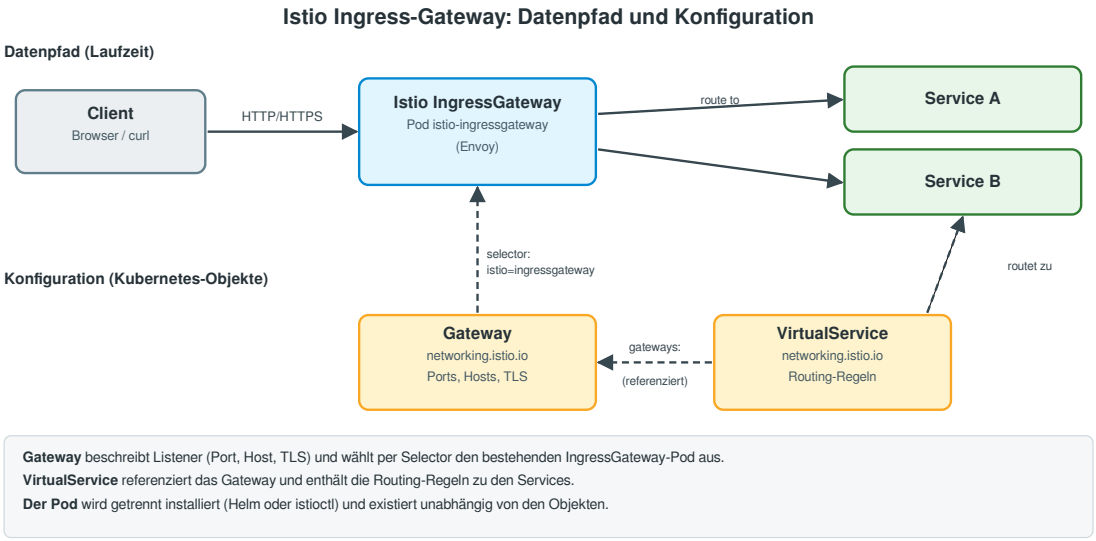


Gateway API -> Gateway Flow (bookinfo/productpage)

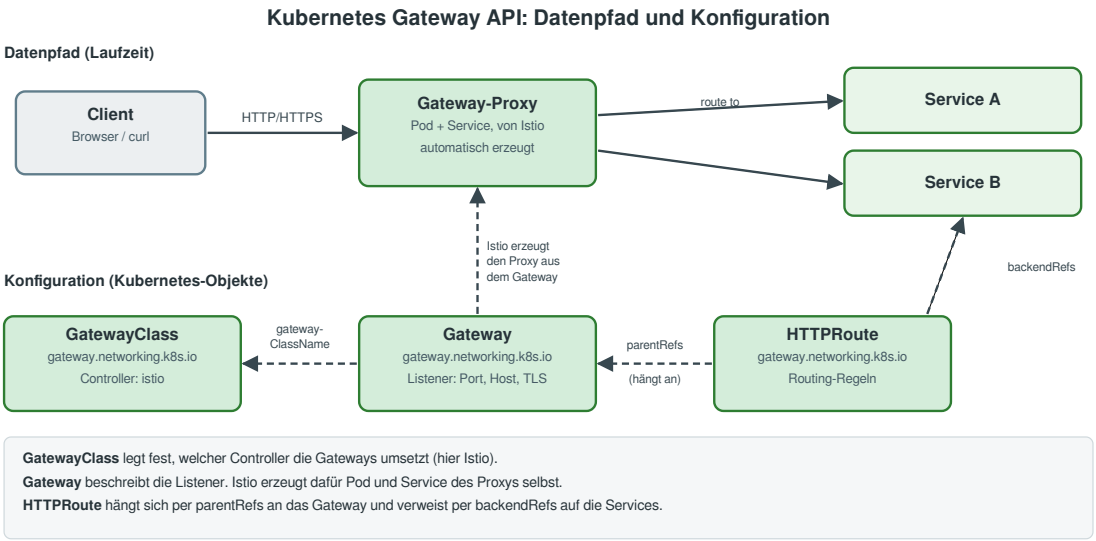


- Achtung: VirtualService und DestinationRule werden teilweise immer noch benötigt, auch wenn man mit der Gateway API arbeitet, um bestimmte Feature abzubilden

Schema: Istio Ingress-Gateway



Schema: Kubernetes Gateway API



Wildcards bei principals in der AuthorizationPolicy

Stand: 10.11.2026. Aus allgemeinem Istio-Wissen zusammengefasst, **nicht auf dem Trainings-Cluster getestet** (ungeprüft).

Kurzfassung

Ja, aber nur eingeschränkt. Istio kennt bei `principals` **kein freies Globbing**, nur drei Formen:

Form	Beispiel	Bedeutung
Präfix	`cluster.local/ns/team-a/sa/*`	alle ServiceAccounts im Namespace `team-a`
Suffix	`*/ns/team-a/sa/frontend`	`frontend` in `team-a`, egal welche Trust Domain

Presence	`*`	jede authentifizierte Identität (Request mit mTLS)

Dazu kommt der exakte Treffer ohne Wildcard.

Aufbau einer Principal

```
<trust-domain>/ns/<namespace>/sa/<serviceaccount>
```

- Standard-Trust-Domain ist `cluster.local`.
- **Kein** `spiffe://` davor schreiben (in der Zertifikats-URI steht es, in der Policy nicht).

Beispiel: `cluster.local/ns/bookinfo/sa/bookinfo-reviews`

Beispiel: alle ServiceAccounts eines Namespace

```yaml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-team-a
  namespace: backend
spec:
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/team-a/sa/*"]
```

Was nicht geht

- **`*` in der Mitte**, zum Beispiel `cluster.local/ns/*/sa/frontend`. Das wird als exakter String behandelt und trifft nie.
- **Präfix und Suffix zugleich**, zum Beispiel `*/ns/team-a/sa/*`. Pro Eintrag ist nur ein `*` am Anfang **oder** am Ende möglich.
- **Reguläre Ausdrücke.**

Wer "frontend in beliebigem Namespace" braucht, muss die Namespaces einzeln auflisten.

Stolperfallen

- **`principals: ["*"]` heißt nicht "alle".** Plaintext-Traffic ohne Identität trifft nicht. Will man wirklich alle zulassen, lässt man `from` weg.
- Für "alle aus einem Namespace" ist `namespaces: ["team-a"]` meist sauberer als das Präfix in `principals`.
- `notPrincipals` kennt dieselben Wildcard-Formen.
- Dieselben Formen (exakt, Präfix, Suffix, Presence) gelten auch bei anderen Feldern der Policy, etwa `hosts`, `paths`, `methods`. Nicht dagegen bei `ipBlocks`, dort steht CIDR.

Ambient

Principal-basierte Regeln wertet schon **ztunnel auf Layer 4** aus, ein Waypoint ist dafür nicht nötig. Die Wildcards verhalten sich wie beim Sidecar.

Regeln mit Layer-7-Feldern (`methods` , `paths` , `hosts`) brauchen dagegen einen Waypoint. Mehr dazu in der Übung Zero Trust Ambient.

Weiterlesen

- Istio-Doku: Authorization Policy, Referenz (Abschnitt `Source`)

istioctl

istioctl cheatsheet zum Debuggen

> *Ohne Install-/Uninstall-/Manifest-/Profile-Kommandos. Alle Kommandos verifiziert gegen die offizielle Istio v1.29 Referenz.*

Version & Preflight

```
## Client- und Control-Plane-Version anzeigen
istioctl version
```

Proxy Status (ps)

Zeigt den Sync-Status aller Envoy-Proxies mit Istiod.

```
## Alle Proxies im Mesh
istioctl proxy-status
istioctl ps                    # Kurzform

## Nach Namespace filtern
istioctl ps --namespace bookinfo

## Diff zwischen Envoy-Config und Istiod für einen bestimmten Proxy
istioctl ps <pod-name>.<namespace>
```

Proxy Config (pc)

Envoy-Konfiguration eines Pods inspizieren.

```
## Listeners
istioctl proxy-config listeners <pod>.<ns>
istioctl pc l <pod>.<ns>

## Routes
istioctl pc routes <pod>.<ns>
istioctl pc r <pod>.<ns>

## Clusters (Upstream-Services)
istioctl pc clusters <pod>.<ns>
istioctl pc c <pod>.<ns>

## Endpoints
```

```
istioctl pc endpoints <pod>.<ns>
istioctl pc ep <pod>.<ns>

## Bootstrap-Konfiguration
istioctl pc bootstrap <pod>.<ns>
istioctl pc b <pod>.<ns>

## ECDS (Extension Config Discovery)
istioctl pc ecds <pod>.<ns>

## Alles auf einmal (JSON-Dump)
istioctl pc all <pod>.<ns> -o json

## Envoy Log-Level abfragen (einzelner Pod)
istioctl pc log <pod>.<ns>

## Envoy Log-Level setzen (einzelner Pod)
istioctl pc log <pod>.<ns> --level debug
istioctl pc log <pod>.<ns> --level info    # zurücksetzen

## Einzelne Envoy-Logger gezielt setzen
istioctl pc log <pod>.<ns> --level connection:debug,router:debug
istioctl pc log <pod>.<ns> --level rbac:debug,conn_handler:warning

## Per Deployment — iteriert über ALLE Pods im Deployment
istioctl pc log deploy/productpage-v1 -n bookinfo              # abfragen
istioctl pc log deploy/productpage-v1 -n bookinfo --level debug # setzen
istioctl pc log deploy/productpage-v1 -n bookinfo --level
rbac:debug,conn_handler:warning

## Funktioniert analog auch mit svc/ und rs/ (ReplicaSet)
istioctl pc log svc/productpage -n bookinfo --level debug
istioctl pc log rs/productpage-v1-abc123 -n bookinfo --level debug

## Andere pc-Subcommands akzeptieren ebenfalls deploy/svc/rs
istioctl pc l deploy/productpage-v1 -n bookinfo
```

Analyze

Konfiguration auf Fehler und Warnungen prüfen.

```
## Aktuellen Namespace analysieren
istioctl analyze

## Bestimmten Namespace
istioctl analyze -n bookinfo

## Alle Namespaces
istioctl analyze --all-namespaces

## Lokale YAML-Dateien prüfen (ohne Cluster)
```

```
istioctl analyze my-virtualservice.yaml --use-kube=false

## Bestimmte Meldungen unterdrücken
istioctl analyze -n default --suppress "IST0102=Namespace default"

## Mehrere Meldungen unterdrücken + Wildcards
istioctl analyze --all-namespaces \
  --suppress "IST0102=Namespace frod" \
  --suppress "IST0107=Pod *.baz"
```

Validate

YAML-Dateien gegen das Istio-Schema validieren.

```
istioctl validate -f my-resource.yaml

## Kurzform
istioctl v -f my-resource.yaml

## Ganzes Verzeichnis
istioctl validate -f samples/bookinfo/networking/

## Aus stdin
kubectl get vs -o yaml | istioctl validate -f -
```

Dashboard (dash / d)

Dashboards per Port-Forward öffnen.

```
istioctl dashboard kiali
istioctl dashboard grafana
istioctl dashboard jaeger
istioctl dashboard zipkin

## Envoy Admin UI eines Pods
istioctl dashboard envoy <pod>.<ns>
istioctl dash envoy deploy/productpage-v1

## Proxy Dashboard (auch für ztunnel/waypoint)
istioctl dashboard proxy <pod>.<ns>

## ControlZ UI (Istiod)
istioctl dashboard controlz deploy/istiod.istio-system
```

Kube-Inject

Sidecar manuell in ein Deployment injizieren.

```
## On-the-fly beim Apply
kubectl apply -f <(istioctl kube-inject -f deployment.yaml)

## In eine Datei schreiben
istioctl kube-inject -f deployment.yaml -o deployment-injected.yaml

## Bestimmte Revision verwenden
istioctl kube-inject -f deployment.yaml --revision canary
```

Admin Log

Istiod-Logging-Level abrufen und ändern.

```
## Aktuelle Log-Level von Istiod
istioctl admin log

## Bestimmten Istiod-Pod abfragen
istioctl admin log <istiod-pod>

## Log-Level ändern
istioctl admin log --level ads:debug,authorization:debug

## Alle zurücksetzen
istioctl admin log --log-reset
```

Bug Report

Diagnose-Bundle für Support erstellen.

```
## Vollständiger Bug-Report
istioctl bug-report

## Auf bestimmte Namespaces beschränken
istioctl bug-report --include default,bookinfo

## Zeitraum begrenzen
istioctl bug-report --duration 30m
```

Tag (Revision Tags)

Revision-Tags für Canary-Upgrades verwalten.

```
## Alle Tags auflisten
istioctl tag list

## Tag erstellen/setzen
istioctl tag set prod --revision 1-22-0
```

```
## Tag entfernen
istioctl tag remove prod
```

Waypoint (Ambient Mode)

Waypoint-Proxies für Ambient Mode verwalten.

```
## Waypoint deployen
istioctl waypoint apply -n default
istioctl waypoint apply -n default --name my-waypoint

## Für Workloads statt Services
istioctl waypoint apply -n default --name wp --for workload

## YAML generieren (ohne Apply)
istioctl waypoint generate -n default --for service

## Alle Waypoints auflisten
istioctl waypoint list -n default
istioctl waypoint list -A                # clusterweilt

## Status prüfen
istioctl waypoint status -n default

## Waypoint löschen
istioctl waypoint delete my-waypoint -n default
istioctl waypoint delete --all -n default
```

Ztunnel Config (Ambient Mode)

Ztunnel-Konfiguration inspizieren.

```
## Workloads
istioctl ztunnel-config workload
istioctl ztunnel-config workload <ztunnel-pod>.<ns> --node <node-name>

## Services
istioctl ztunnel-config service

## Zertifikate
istioctl ztunnel-config certificates --node <node-name>

## Policies
istioctl ztunnel-config policies

## Logging
istioctl ztunnel-config log <ztunnel-pod>.<ns>
istioctl ztunnel-config log <ztunnel-pod>.<ns> --level debug
```

```
## Alles (JSON-Dump)
istioctl ztunnel-config all <ztunnel-pod>.<ns> -o json
```

Experimental (x) Kommandos

```
## AuthorizationPolicy eines Pods prüfen
istioctl x authz check <pod>.<ns>

## Pod beschreiben (mTLS-Status, Policies, Traffic)
istioctl x describe pod <pod> -n <ns>

## Sidecar-Injection-Status prüfen
istioctl x check-inject <pod>.<ns>
istioctl x check-inject deploy/<name> -n <ns>

## Envoy-Stats abrufen
istioctl x envoy-stats <pod>.<ns>
istioctl x envoy-stats deploy/<name> --type clusters

## Service-Metriken (benötigt Prometheus)
istioctl x metrics productpage-v1.default

## Root-CA vergleichen (Multi-Cluster)
istioctl x rootca-compare <pod1>.<ns1> <pod2>.<ns2>

## Internal Debug (Istiod xDS)
istioctl x internal-debug syncz

## VM-Workload konfigurieren
istioctl x workload entry configure -f workloadgroup.yaml -o config
istioctl x workload group create --name foo --namespace bar
```

Globale Flags

Flag	Beschreibung
`-n, --namespace`	Kubernetes Namespace
`-c, --kubeconfig`	Kubeconfig-Datei
`--context`	Kubernetes Context
`--istioNamespace`	Istio Control Plane Namespace (default: `istio-system`)
`--revision`	Istio Revision auswählen
`-o, --output`	Ausgabeformat: `json`, `yaml`, `short`

Aliase & Kurzformen

Langform	Kurzform

proxy-status	ps
proxy-config	pc
proxy-config listeners	pc l
proxy-config routes	pc r
proxy-config clusters	pc c
proxy-config endpoints	pc ep
proxy-config bootstrap	pc b
dashboard	dash / d
experimental	x
validate	v

Debugging-Workflow (Kurzreferenz)

```
1. istioctl ps                              # Proxies synced?
2. istioctl analyze -n <ns>                 # Config-Fehler?
3. istioctl x describe pod <pod> -n <ns>    # Pod-Details
4. istioctl pc l <pod>.<ns> --port <port>   # Listener OK?
5. istioctl pc r <pod>.<ns> --name <port>   # Routes OK?
6. istioctl pc c <pod>.<ns> --fqdn <svc>    # Cluster/Upstream OK?
7. istioctl pc ep <pod>.<ns> --cluster <c>  # Endpoints OK?
```

Quelle: istio.io/latest/docs/reference/commands/istioctl — Stand: Istio v1.29

Installation von istio (ambient mode)

istio-installation mit istioctl - ambient (Installation per istioctl, um ein Gefühl dafür zu bekommen)

Getestet mit Istio 1.31.4, Kubernetes v1.37 und Calico (Iptables-Dataplane).

Step 1: Gateway API CRDs

Ab Istio 1.30 werden die Gateway API CRDs in Version v1.5.x benötigt. Ohne sie werden z.B. TLSRoute und ReferenceGrant von istiod nicht gesehen.

```
kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.5.1/standard-install.yaml
```

Step 2: Installation von istio (Profil ambient)

```
istioctl version --remote=false
istioctl install --set profile=ambient --skip-confirmation
```

Step 3: Überprüfen

```
kubectl -n istio-system get pods
istioctl version
```

Erwartet: istiod (1x), istio-cni-node (1x pro Node) und ztunnel (1x pro Node) laufen, Client, Control Plane und Data Plane melden dieselbe Version.

Hinweis zu Calico

Bei der Installation kann diese Meldung erscheinen:

```
detected Calico CNI with 'bpfConnectTimeLoadBalancing=TCP'; this must be set to
'bpfConnectTimeLoadBalancing=Disabled'
```

- Sie betrifft nur Calico im **eBPF-Modus** (`bpfEnabled: true`).
- Mit dem Iptables-Dataplane (so sind die Trainings-Cluster aufgesetzt) ist die Einstellung wirkungslos, die Meldung kann ignoriert werden. Getestet: Bookinfo im Ambient-Namespace, mTLS und STRICT funktionieren.

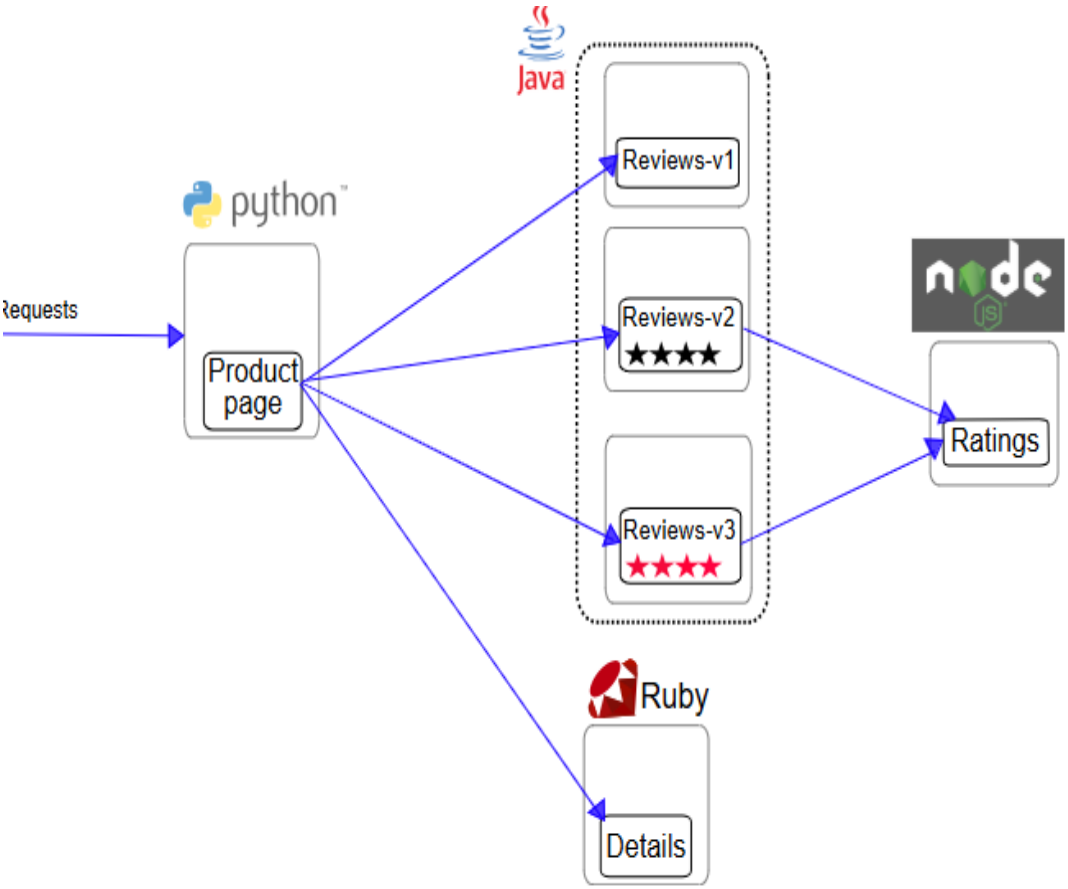
```
## Prüfen, welcher Dataplane läuft
kubectl get felixconfiguration default -o jsonpath='{.spec.bpfEnabled}{"\n"}'
```

Nur wenn das `true` ausgibt:

```
kubectl patch felixconfiguration default --type=merge -p '{"spec":
{"bpfConnectTimeLoadBalancing":"Disabled","bpfHostNetworkedNATWithoutCTLB":"Disabled"}}'
```

Bookinfo demo in ambient ausrollen

Überblick



(Optional) Alte Installation entfernen

```
kubectl delete ns bookinfo
```

Vorbereitung

```
kubectl create ns bookinfo
kubectl label namespace bookinfo istio.io/dataplane-mode=ambient
```

```
kubectl describe namespace bookinfo
```

Waypoint Proxy ausrollen

```
cd
mkdir -p manifests/waypoint
cd manifests/waypoint
```

```
## YAML generieren (dry-run)
istioctl waypoint generate --namespace bookinfo --for all  > waypoint.yaml
```

```
## Anschauen was passiert
cat waypoint.yaml
```

```
## Ausrollen
kubectl apply -f waypoint.yaml
kubectl label namespace bookinfo istio.io/use-waypoint=waypoint
```

```
## Überprüfen
kubectl -n bookinfo get gateways
istioctl waypoint list --namespace bookinfo
```

Bookinfo App ausrollen

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/platform/kube/bookinfo-
versions.yaml
kubectl -n bookinfo get all
```

Testen ob die App funktioniert

```
kubectl -n bookinfo exec "$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

```
## Einfachere Variante
kubectl -n bookinfo exec deployments/ratings-v1 -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

App mit Gateway API nach aussen öffnen

```
## That's what we do ....
cat ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
```

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/gateway-api/bookinfo-
gateway.yaml
kubectl -n bookinfo get gateways
kubectl -n bookinfo get httproutes -o yaml
```

```
## Das Gateway erstellt automatisch einen Service
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

Zugriff auf die App

Auf den Trainings-Clustern läuft MetalLB. Der Service des Gateways bekommt deshalb automatisch eine `EXTERNAL-IP` (die öffentliche IP eines Workers), und das Gateway zeigt `PROGRAMMED True`:

```
kubectl -n bookinfo get gateways
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

Direkt nach dem Anlegen steht das Gateway noch kurz auf `PROGRAMMED False` und die `EXTERNAL-IP` auf `<pending>`. Nach wenigen Sekunden (Befehle wiederholen) steht dort `True` und die IP.

```
## IP des Gateways merken
GATEWAY_IP=$(kubectl -n bookinfo get svc bookinfo-gateway-istio -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo $GATEWAY_IP
```

```
curl -s http://$GATEWAY_IP/productpage | grep -o "<title>.*</title>"
```

Im Browser: `http://<GATEWAY_IP>/productpage`

Hinweise:

- Die IP ist öffentlich erreichbar, auch von eurem eigenen Rechner aus.
- Pro Cluster gibt es zwei IPs (die beiden Worker), also höchstens **zwei** Services vom Typ `LoadBalancer` gleichzeitig. Ein weiterer bleibt auf `<pending>`. Nicht mehr benötigte Gateways oder Services vorher löschen.

Ref

- https://istio.io/latest/docs/ambient/getting-started/deploy-sample-app/

istio (ambient) deinstallieren - nur als Referenz

Nur als Referenz. Reihenfolge einhalten: erst Namespaces aus dem Mesh nehmen, dann Istio entfernen.

Step 1: Namespaces aus dem Mesh nehmen

```
## Beispiel: Namespace bookinfo
kubectl label namespace bookinfo istio.io/dataplane-mode-
kubectl label namespace bookinfo istio.io/use-waypoint-
istioctl waypoint delete --all -n bookinfo
```

Die Anwendungs-Pods müssen nicht neu gestartet werden (ambient hat keine Sidecars).

Step 2: Istio entfernen

```
istioctl uninstall -y --purge
kubectl delete namespace istio-system

## Falls Istio-CRDs übrig bleiben (siehe Step 4), von Hand löschen:
kubectl get crd -o name | grep istio.io | xargs -r kubectl delete
```

Step 3: Gateway API CRDs entfernen (optional)

Nur wenn nichts anderes im Cluster davon abhängt.

```
kubectl delete -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.5.1/standard-install.yaml
```

Step 4: Überprüfen

```
kubectl get crd | grep -E 'istio.io|gateway.networking.k8s.io'
kubectl get pods -A | grep -E 'istio|ztunnel'
```

Beide Befehle sollten nichts ausgeben.

Installation auf OCP/Azure (ambient mode)

Istio Ambient auf OpenShift (OCP): OSSM 3 per Operator, Unterschiede und Einschränkungen

Stand: 2026-11-09. Quelle: Red-Hat-Doku und Blog zu OSSM 3.0 bis 3.3 (Links unten). Punkte mit **(ungeprueft)** stammen aus Erfahrungswissen und sind noch nicht gegen die englische Doku verifiziert.

Kurzfassung

- Ambient laeuft auf OpenShift ueber **Red Hat OpenShift Service Mesh 3 (OSSM 3)**.
- OSSM 3 basiert auf dem **Sail Operator** und wird ueber **OperatorHub**/**OLM** installiert, **nicht** ueber Helm-Charts.
- Produktiv sinnvoll ab **OSSM 3.2** (Ambient GA, braucht OCP 4.18 oder neuer).
- Pro Cluster gibt es **ein** Ambient-Mesh. Mischbetrieb mit Sidecars ist nur Preview.

Support-Stand pro Version

OSSM	Istio	Ambient-Status
3.0	1.24	Developer Preview
3.1	1.26	Technology Preview, nur auf Clustern ohne bestehende OSSM-Installation
3.2	1.27	**GA** fuer ztunnel und waypoint. Bekannt: nicht auf FIPS-Clustern
3.3	-	FIPS-Support fuer ztunnel. Multi-Cluster-Ambient als Tech Preview

Upgrade von OSSM 2.x: erst auf 3.0, dann 3.1, dann 3.2.

Installation

Zwei Stufen.

1. Operator per OLM installieren

Web-Konsole: Operators, OperatorHub, "Red Hat OpenShift Service Mesh 3", Install. Oder als YAML (Paketname und Channel vorher mit `oc get packagemanifest` pruefen):

```yaml
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: servicemeshoperator3
  namespace: openshift-operators
spec:
  channel: stable
  name: servicemeshoperator3
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

2. Mesh ueber Custom Resources anlegen

Die Namespaces `istio-system`, `istio-cni` und `ztunnel` vorher anlegen. `apiVersion` und Felder gegen die CRDs der installierten Version pruefen (`oc explain istio.spec`).

```
apiVersion: sailoperator.io/v1
kind: Istio
metadata: { name: default }
spec:
  namespace: istio-system
  profile: ambient
  values:
    pilot:
      trustedZtunnelNamespace: ztunnel
---
apiVersion: sailoperator.io/v1
kind: IstioCNI
metadata: { name: default }
spec:
  namespace: istio-cni
  profile: ambient
---
apiVersion: sailoperator.io/v1
kind: ZTunnel
metadata: { name: default }
spec:
  namespace: ztunnel
```

Namespace ins Mesh aufnehmen: `oc label namespace <ns> istio.io/dataplane-mode=ambient`. L7-Funktionen ueber einen Waypoint (Gateway API, `gatewayClassName: istio-waypoint`).

Was OperatorHub ist

- OLM (Operator Lifecycle Manager) ist Teil von OpenShift und laeuft im Cluster.
- Kataloge sind `CatalogSource` -Objekte in `openshift-marketplace` (`redhat-operators`, `certified-operators`, `community-operators`, `redhat-marketplace`).
- Die Web-Konsole (Operators, OperatorHub) ist nur eine Oberflaeche. Dahinter entsteht eine `Subscription`. Dasselbe geht per `oc apply` und damit per GitOps.
- Zum Installieren braucht man cluster-admin oder entsprechende Rechte.

Unterschiede: OSSM (Operator) gegen Upstream-Istio (Helm)

Thema	OSSM 3 (Operator)	Upstream-Istio per Helm
Support	Red Hat	keiner von Red Hat fuer den Mesh-Teil
Ausrollen	Subscription plus CRs (`Istio`, `IstioCNI`, `ZTunnel`)	`helm install` (base, istiod, cni, ztunnel)
Sichtbar als	Operator, keine Helm-Releases	Helm-Releases
Konfiguration	`spec.values` (entspricht den Istio-Helm-Values)	`values.yaml`
SCCs, privilegierte	vom Operator behandelt	selbst loesen

Pods		
Upgrades	Revisionen und Canary ueber `IstioRevision`, `IstioRevisionTag`	selbst steuern
Versionen	an OSSM-Release gebunden (z. B. 3.2 gleich Istio 1.27)	frei waehlbar, immer aktuell
Gateway-API-CRDs	auf OCP vom Ingress Operator verwaltet, OSSM darauf abgestimmt	selbst abstimmen, nicht ueberschreiben
Kiali, Monitoring	integriert und getestet	selbst integrieren
Kataloge	braucht OLM und Katalog (bei Air-Gap gespiegelt)	braucht nur eine Chart-Quelle

Intern nutzt der Sail Operator die Istio-Helm-Charts zum Rendern. Du siehst sie aber nicht.

Upstream bringt eigene OpenShift-Profile mit (`openshift` , `openshift-ambient`) **(ungeprueft)**.

Einschraenkungen von Ambient in OSSM (Stand 3.3)

Thema	Status
ztunnel (Kern)	GA
Waypoint (Kern)	GA
Waypoint mit Gateway API Stable Channel (HTTPRoute, GRPCRoute)	GA
Sidecar- und Ambient-Namespaces im selben Mesh mischen	Developer Preview
Dual-Stack und IPv6-only	Technology Preview
Multi-Cluster, Multi-Primary	Technology Preview
Andere Multi-Cluster-Topologien	nicht verfuegbar
Ambient auf Cluster mit bestehendem Sidecar-Mesh	nicht verfuegbar
Mehrere Ambient-Meshes in einem Cluster	nicht verfuegbar
FIPS-Cluster	nicht in 3.2, ab 3.3 unterstuetzt
Post-Quantum-Kryptografie (X25519MLKEM768) auf FIPS-Clustern	nicht unterstuetzt

Wichtig fuer IPv6-Umgebungen: IPv6-only ist bei Ambient in OSSM nur Tech Preview.

Weitere Punkte beim Betrieb

- **L4 ohne Waypoint:** mTLS und `AuthorizationPolicy` auf Identitaet. L7-Policies brauchen einen Waypoint pro Namespace oder Service.
- **NetworkPolicies und AdminNetworkPolicies:** HBONE-Port 15008 und Health-Probes muessen durchgelassen werden **(ungeprueft)**.
- **CNI:** OVN-Kubernetes ist der Normalfall. ztunnel und CNI-Chaining dort vor dem Training testen.
- **Ingress:** Istio-Gateway kann parallel zum OpenShift-Router (HAProxy) laufen.
- **Observability:** ueber Kiali und den OpenShift-Monitoring-Stack (User-Workload-Monitoring). Eigener Konfigurationsblock.

- **Air-Gap:** ohne Internet braucht der Cluster einen gespiegelten Operator-Katalog (`oc-mirror` , eigene `CatalogSource`).
- **Rechte:** Operator-Installation nur mit cluster-admin.
- **OKD:** OperatorHub gibt es, aber mit `community-operators` . Der OSSM-Operator liegt in `redhat-operators` und setzt eine OCP-Subscription voraus.
- **GitOps:** Reihenfolge einhalten. Subscription zuerst, warten bis die CRDs da sind, dann die CRs (z. B. Sync-Waves in Argo CD).

Empfehlung

- OSSM 3.2 oder 3.3 per Operator als Standard. 3.3, falls FIPS gebraucht wird.
- Ein Ambient-Mesh pro Cluster, kein Mischbetrieb.
- Vor der Planung klaeren: connected oder disconnected, wer darf Operatoren installieren, braucht ihr Multi-Cluster oder IPv6-only (dann nur Tech Preview).
- Helm/Upstream nur, wenn es keine Red-Hat-Subscription gibt (OKD) oder ein Feature noetig ist, das OSSM nicht liefert.

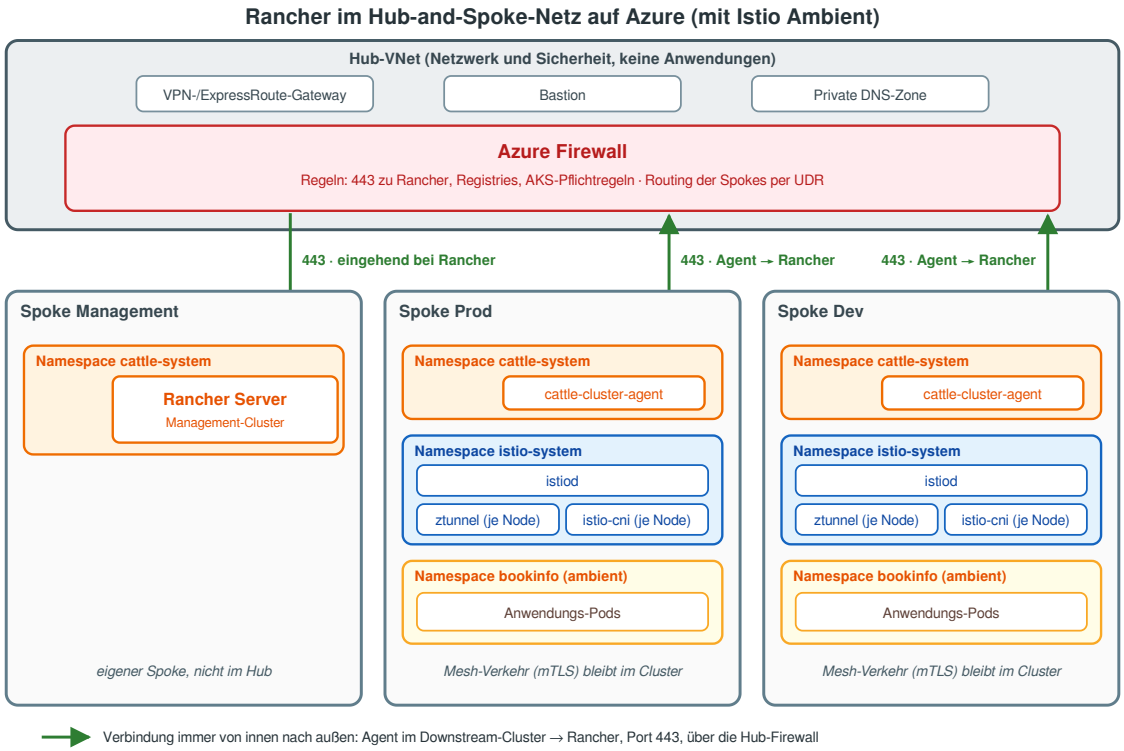
Quellen

- OSSM 3.3 Feature-Support-Tabellen
- OSSM 3.2 Release Notes
- OSSM 3.3 Release Notes
- Blog: OSSM 3.2 und Istio Ambient Mode
- Blog: OSSM 3.1
- Blog: OSSM 3.3 und Post-Quantum-Kryptografie

Rancher auf Azure im Hub-and-Spoke-Netz: Verbindungen, Firewall, Istio Ambient auf den Downstream-Clustern

Stand: 2026-11-09. Punkte mit **(ungeprueft)** stammen aus Erfahrungswissen und sind noch nicht gegen die offizielle Doku der eingesetzten Versionen verifiziert.

Schwerpunkt ist **Rancher**: wo er in einem Hub-and-Spoke-Netz läuft, welche Verbindungen er braucht und was das für Istio Ambient auf den Downstream-Clustern bedeutet.



Kurzfassung

- Rancher gehört in einen **eigenen Spoke** ("Management" oder "Platform"), nicht in den Hub.
- Die Downstream-Cluster (Prod, Dev) liegen in eigenen Spokes und melden sich **von innen nach außen** über die Hub-Firewall bei Rancher (Port 443).
- Istio Ambient wird auf den Downstream-Clustern **genauso installiert wie sonst**. Der Mesh-Verkehr zwischen den Pods bleibt im Cluster und läuft nicht über den Hub.
- Fast alle Probleme liegen bei Firewall, Route oder DNS, nicht bei Rancher oder Istio.

Hub and Spoke in einem Satz

Der **Hub** ist ein zentrales VNet mit Netzwerk- und Sicherheitsdiensten (Firewall, VPN-/ExpressRoute- Gateway, Bastion, DNS). Die **Spokes** sind eigene VNets für Workloads, per Peering mit dem Hub verbunden. Faustregel: Der Hub transportiert und kontrolliert, die Spokes führen aus. Rancher ist ein Dienst, also kommt er in einen Spoke.

```
Hub (Firewall, VPN, DNS)
 ├── Spoke Management  → Rancher (Management-Cluster)
 ├── Spoke Prod        → Downstream-Cluster (AKS, Istio Ambient)
 └── Spoke Dev         → Downstream-Cluster (AKS, Istio Ambient)
```

Warum nicht im Hub: Er soll schlank und leicht prüfbar bleiben. Anwendungen und Cluster gehören dort nicht hin.

Ausnahme: Bei einem einzelnen Cluster darf Rancher im selben Spoke wie die Workloads laufen.

Welche Verbindungen Rancher braucht

Von	Nach	Port	Wozu
Downstream-Cluster (`cattle-`	Rancher-	443	Agent baut die Verbindung auf, Rancher wählt sich

`cluster-agent`, `cattle-node-agent`)	URL		**nicht** in den Cluster ein
Rancher-Management-Cluster	Azure-APIs	443	nur wenn Rancher die AKS-Cluster selbst anlegt oder verwaltet (Azure Resource Manager, Entra ID) **(ungeprueft)**
Nutzer, Admins	Rancher-URL	443	Web-Oberfläche, API, `kubectl` über Rancher
Alle Cluster	Container-Registries	443	Rancher-, Istio- und Anwendungs-Images

Wichtig für die Firewall-Regeln im Hub:

- Die Regel gilt **vom Spoke Prod/Dev zum Spoke Management**, mit Routing (UDR) über die Hub-Firewall in beide Richtungen, sonst kommt die Antwort nicht zurück (asymmetrisches Routing).
- Rancher muss unter einem **DNS-Namen** erreichbar sein, den die Downstream-Cluster auflösen können (private DNS-Zone, am Hub verlinkt). Die Agenten verwenden diesen Namen.
- **TLS-Inspection** der Firewall (z. B. Azure Firewall Premium) bricht die Agent-Verbindung, weil die Agenten das Zertifikat von Rancher prüfen. Für die Rancher-URL die Inspection ausnehmen **(ungeprueft)**.
- Zusätzlich braucht **AKS selbst** Pflicht-Egress-Regeln, wenn der Ausgang über die Firewall läuft (`outboundType: userDefinedRouting`). Ohne sie scheitert schon die Cluster-Erstellung. Siehe die Microsoft-Doku "Outbound network and FQDN rules for AKS" **(ungeprueft)**.

Rancher und Istio Ambient auf den Downstream-Clustern

Die Reihenfolge ist: Cluster anlegen oder importieren, Agent verbindet sich mit Rancher, danach Istio installieren. Istio ändert an der Rancher-Verbindung nichts.

- **Istio selbst installieren** (`istioctl` oder Helm, Profil `ambient`), wie im Training. Rancher bringt eine eigene Istio-App im Katalog mit. Ob sie Ambient unterstützt, ist offen **(ungeprueft)**. Nicht mit einer Eigeninstallation mischen: genau eine Istio-Installation pro Cluster.
- **Rancher-Namespaces nicht in das Mesh aufnehmen.** `cattle-system` , `cattle-fleet-system` und die System-Namespaces bekommen kein `istio.io/dataplane-mode=ambient` . Ambient nur für die eigenen Anwendungs-Namespaces setzen. Der Agent-Verkehr zu Rancher läuft sonst durch ztunnel, was ohne Not eine Fehlerquelle ist.
- **Project Network Isolation** (Netzwerkregeln pro Rancher-Projekt): Falls aktiv, muss **Port 15008 (HBONE)** zwischen den Nodes und Pods erlaubt sein, sonst funktioniert ztunnel nicht **(ungeprueft)**.
- **Zuständigkeit:** Rancher verwaltet Cluster und Zugriffsrechte (Projekte, Nutzer, RBAC). Istio verwaltet den Verkehr zwischen Services (mTLS, Routen, Policies). Das ergänzt sich, überschneidet sich aber nicht.
- **Mehrere Cluster in einem Mesh** (Multi-Cluster): nicht nötig für den Anfang. Der Verkehr zwischen den Spokes liefe über den Hub, und der Reifegrad von Multi-Cluster-Ambient ist vor einer Planung in der Istio-Doku zu prüfen. Rancher ersetzt kein Multi-Cluster-Mesh.

Typische Fehlerbilder

Symptom	Wahrscheinliche Ursache
Cluster in Rancher bleibt "Unavailable" oder "Waiting"	Firewall-Regel oder Route zu Rancher (443) fehlt, DNS-Name nicht auflösbar, TLS-Inspection
`cattle-cluster-agent` im Zustand `CrashLoopBackOff`	wie oben: `kubectl -n cattle-system logs deploy/cattle-cluster-agent` zeigt, ob die Rancher-URL erreichbar ist

`ImagePullBackOff` im Spoke	Registry in der Firewall nicht freigegeben
AKS-Erstellung hängt oder schlägt fehl	Pflicht-Egress-Regeln von AKS fehlen
Antworten kommen nicht zurück	Route nur in eine Richtung, Peering nicht transitiv

Merksatz: Die Fehlersuche hat eine Ebene mehr als bei einem normalen Cluster. Zuerst Firewall, Route und DNS prüfen, dann erst Rancher und Istio.

Prüfen vom Spoke aus

```
## Rancher-URL aus dem Downstream-Cluster erreichbar?
kubectl run -it --rm nettest --image=curlimages/curl --restart=Never -- \
  curl -sS -o /dev/null -w '%{http_code}\n' https://<rancher-url>/ping
```

Erwartet: `200`. Kommt ein Zertifikatsfehler oder ein Timeout, liegt es am Netz, nicht an Rancher.

Quellen

- Azure Architecture Center: Hub-and-Spoke-Netzwerktopologie
- Rancher-Doku: Port-Anforderungen
- Microsoft Learn: Outbound network and FQDN rules for AKS

Steuerung des Netzwerkverhaltens in Istio

Gateway API als Nachfolger von IngressGateway (Istio), VirtualService und DestinationRule

Wichtig Hinweis:

- In Istio werden immer mehr Sachen von der Gateway API übernommen
- Aber noch nicht alles ist migriert, deswegen brauche ich telweise immer noch die Objekte (VirtualService und DestinationRule für bestimmte Features

Was Gateway API ersetzt

Kurze Antwort: Für Routing nein, für einige Istio-spezifische Features ja.

Was Gateway API vollständig ersetzt (Stand: 03/2026)

Feature	VirtualService	Gateway API Equivalent
Request Routing (Header, Path, Method)	✅	HTTPRoute `matches`
Traffic Splitting / Canary	✅	HTTPRoute `backendRefs` mit `weight`
URL Rewrite	✅	HTTPRoute `URLRewrite` Filter
Request Header Modification	✅	HTTPRoute `RequestHeaderModifier`
Redirects	✅	HTTPRoute `RequestRedirect`
Request Mirroring	✅	HTTPRoute `RequestMirror`
gRPC Routing	✅	GRPCRoute

Was du weiterhin als Istio-API brauchst

DestinationRule — kein Gateway API Equivalent für:

- Circuit Breaking / Outlier Detection
- Connection Pool Settings
- Load Balancing Algorithmus (LEAST_REQUEST, CONSISTENT_HASH)
- TLS-Modus zum Backend

VirtualService — noch nötig für:

- **Fault Injection** (Delay, Abort) → gibt es in Gateway API nicht
- **Retries** (Anzahl, Timeout, retryOn Conditions)
- **Timeouts** (request-level)
- **CORS Policy**
- **Regex-basiertes URI-Matching** (Gateway API kann nur Exact und PathPrefix)

Praxis-Empfehlung

```
Neues Projekt:
  Routing            → Gateway API (HTTPRoute)
  Fault Injection    → VirtualService (solange nötig)
  Backend-Policies   → DestinationRule

Bestehendes Projekt:
  Migration schrittweise, beides kann parallel existieren
```

Achtung: Nicht beide gleichzeitig (httpRoute und VirtualService) auf denselben Service anwenden — das führt zu undefiniertem Verhalten. Entweder HTTPRoute **oder** VirtualService pro Service, nicht mischen.

Virtual Services, Destination Rules

Überblick

Ressource	Aufgabe
VirtualService	*Wohin* wird Traffic geroutet? (Routing-Regeln, Matching, Rewrites)
DestinationRule	*Wie* genau geht der Traffic? (Subsets, Load Balancing, TLS, Connection Pools)

Beide arbeiten zusammen: Der VirtualService wählt das Ziel, die DestinationRule definiert die Eigenschaften des Ziels.

VirtualService

Ein VirtualService fängt Traffic ab und leitet ihn anhand von Regeln weiter.

Routing-Möglichkeiten

Feature	Beschreibung
Traffic Splitting (Canary)	Über `weight` auf mehrere Subsets verteilen
Header-basiertes Routing	`match.headers` mit `exact`, `prefix`, `regex`
URI-basiertes Routing	`match.uri` mit `exact`, `prefix`, `regex`
Fault Injection	`fault.delay` und `fault.abort` mit Prozentwerten

Timeouts & Retries	`timeout`, `retries.attempts`, `retries.perTryTimeout`, `retries.retryOn`

DestinationRule

Eine DestinationRule definiert Policies **nach** dem Routing (Subsets, Load Balancing, Circuit Breaking, TLS).

Traffic Policies

Feature	Feld	Optionen
Load Balancing	`trafficPolicy.loadBalancer.simple`	`ROUND_ROBIN`, `RANDOM`, `LEAST_CONN`, `PASSTHROUGH`
Consistent Hashing	`trafficPolicy.loadBalancer.consistentHash`	`httpHeaderName`, `httpCookie`, `useSourceIp`
Circuit Breaking	`trafficPolicy.connectionPool`	`tcp.maxConnections`, `http.http1MaxPendingRequests`, `http.http2MaxRequests`
Outlier Detection	`trafficPolicy.outlierDetection`	`consecutive5xxErrors`, `interval`, `baseEjectionTime`, `maxEjectionPercent`
mTLS	`trafficPolicy.tls.mode`	`DISABLE`, `SIMPLE`, `MUTUAL`, `ISTIO_MUTUAL`

Subset-spezifische `trafficPolicy` überschreibt die globale Policy nur für das jeweilige Subset.

Zusammenspiel

1. **DestinationRule** erstellen → Subsets definieren (z.B. `v1`, `v2`, `v3`) + globale Policies (TLS, LB)
2. **VirtualService** erstellen → Routing-Regeln mit Verweis auf die Subsets

---

Cheat Sheet

Was willst du?	Ressource	Feld
Canary Deployment	VirtualService	`weight`
A/B Testing (Header)	VirtualService	`match.headers`
Fehler simulieren	VirtualService	`fault`
Timeouts setzen	VirtualService	`timeout` / `retries`
Subsets (v1/v2)	DestinationRule	`subsets`
Load Balancing	DestinationRule	`trafficPolicy.loadBalancer`
Circuit Breaker	DestinationRule	`outlierDetection`
mTLS	DestinationRule	`trafficPolicy.tls`
Connection Limits	DestinationRule	`connectionPool`

Übungen Steuerung des Netzwerkverhaltens (Gateway API)

Übung: Header-basiertes Routing

Prerequisites

- Bookinfo - Projekt aufgesetzt.

Schritt 1: Vorbereitung: Status review-pods

- Status: alle Pods sind unter einem Service erreichbar

```
## Es gibt 3 verschieden review-pods (v1, v2, v3)
kubectl -n bookinfo get pods --show-labels | grep review
```

```
## Ein Service zeigt auf alle pods (Alle versionen der Review - Pods)
kubectl -n bookinfo get svc | grep reviews
```

Schritt 2: Vorher (ohne request routing)

- Es werden alle Pods angezeigt, die das Label: app:reviews haben
- D.h. jedesmal wenn ich die Seite öffne, wird eine andere Version angegezeigt (v1, v2 oder v3) - * d.h. es werden ganz normal die Services von Kubernetes verwendet **
- Service (selector: app:reviews)

```
kubectl -n bookinfo get svc reviews -o yaml
kubectl -n bookinfo get pods -l app=reviews --show-labels
```

```
## Gateway wurde in der Übung vorher angelegt
## du findest so die IP des gateways raus
kubectl -n bookinfo get gateway
```

```
GATEWAY_URL=<ip-aus-der-vorigen-Ausgabe-eintragen>
```

```
## Im Browser mehrmals ausführen
## Im Block mit den Reviews wechselt die Version
$GATEWAY_URL/productpage
```

Schritt 3: Übung (jetzt request - routing)

Voraussetzung:

- Bookinfo-App läuft bereits im Namespace `bookinfo`
- Service Reviews existiert
- Es gibt 3 verschieden Pods an Reviews (v1, v2 und v3)
- Ingress/Gateway + `GATEWAY_URL` (IP: http://164.90.237.35/productpage aus der vorherigen Übung vorhanden

0. Vorbereitung

```
mkdir -p ~/manifests/requests
cd ~/manifests/requests

## Die Service-Versionen anlegen
cp -a ~/istio/samples/bookinfo/platform/kube/bookinfo-versions.yaml bookinfo-
```

```
versions.yaml
kubectl -n bookinfo apply -f .
```

---

1. HTTPRoute: Alle Requests → `reviews-v1`

```
cat <<EOF > ~/manifests/requests/httproute-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-v1.yaml
kubectl -n bookinfo get httproute reviews -n bookinfo
```

```
## Vergleich mit allen httproutes
kubectl -n bookinfo get httproute
## booking for north - south traffik
```

```
## Anzeige im Browser - es ist immer die v1
http://164.90.237.35/productpage
```

---

2. HTTPRoute anpassen: User `jason` → `reviews-v2`, Rest → `reviews-v1`

```
cat <<EOF > ~/manifests/requests/httproute-reviews-jason-v2.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - matches:
```

```
    - headers:
      - name: end-user
        value: jason
    backendRefs:
    - name: reviews-v2
      port: 9080
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-jason-v2.yaml
kubectl -n bookinfo get httproute reviews -n bookinfo -o yaml
```

---

3. Testen im Browser

```
echo "$GATEWAY_URL"
## Beispiel: http://<IP>:<PORT>

## 1. Im Browser: $GATEWAY_URL/productpage aufrufen (nicht eingeloggt oder anderer
User)
##    → Reviews ohne Sterne (v1)

## 2. Im Browser: als User "jason" einloggen
##    → Reviews mit Sternen (v2)
```

---

4. Aufräumen

```
kubectl delete -f httproute-reviews-v1.yaml --ignore-not-found
kubectl delete -f httproute-reviews-jason-v2.yaml --ignore-not-found
```

Reference:

- https://istio.io/latest/docs/examples/bookinfo/#define-the-service-versions

Übung: Traffic-Shifting / Load-Balancing

- Schrittweise Umleitung von Netzwerk-Traffic zwischen zwei Service-Versionen

0. Vorbereitung

```
mkdir -p ~/manifests/traffic-shifting
cd ~/manifests/traffic-shifting

## Die Service-Versionen anlegen
cp -a ~/istio/samples/bookinfo/platform/kube/bookinfo-versions.yaml bookinfo-
versions.yaml
kubectl -n bookinfo apply -f .
```

1. 100% Traffic -> reviews.v1

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v1.yaml
kubectl get httproute -n bookinfo reviews
kubectl get httproute -n bookinfo reviews -o yaml | less
```

```
## Status ist interessant !
```

```
status:
  parents:
  - conditions:
    - lastTransitionTime: "2025-11-17T10:32:34Z"
      message: Route was valid
      observedGeneration: 3
```

2. Testen

```
## Seite öffnen
http://<deine-ip>/productpage

## Egal wie oft du die Seite lädst, es bleibt immer v1
```

3. 50% (v1) /50% (v3) Traffic

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-50-50.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
```

```
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
      weight: 50
    - name: reviews-v3
      port: 9080
      weight: 50
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-50-50.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 40
```

4. Testen

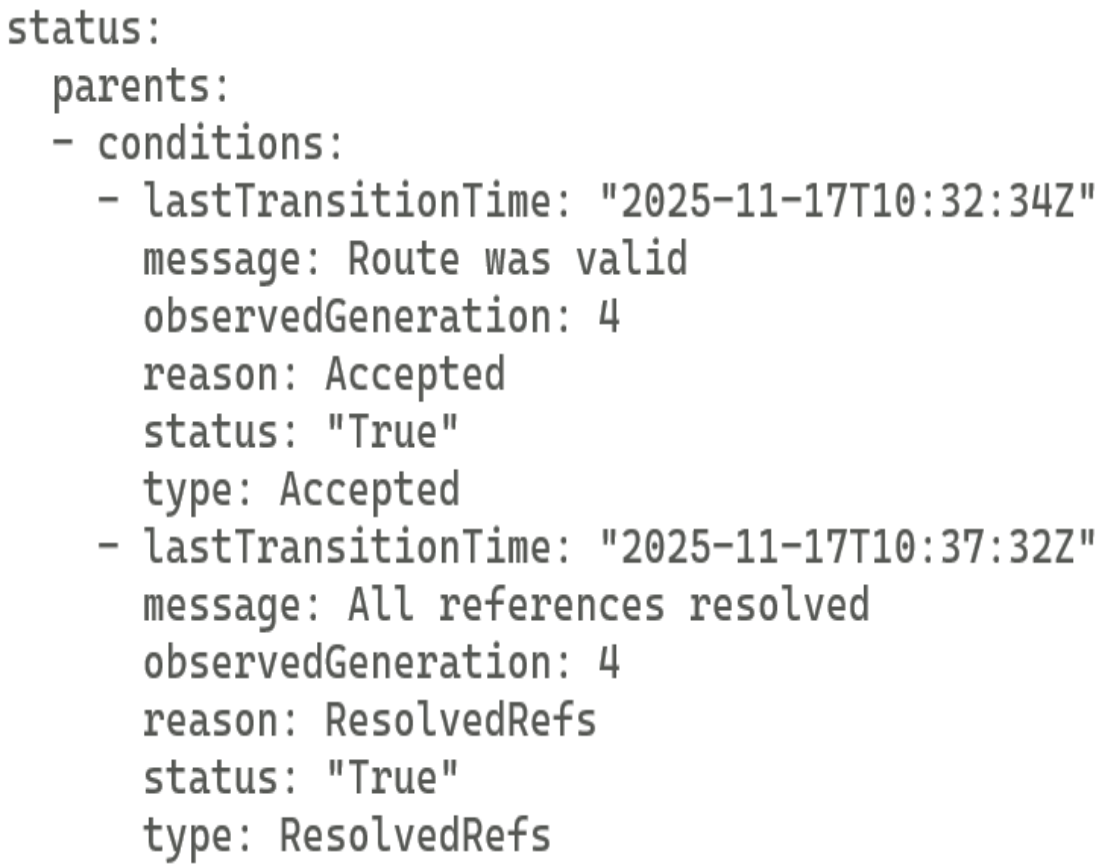
```
## Seite öffnen
http://<deine-ip>/productpage

## Abwechselnd bei mehrmals laden v1 (keine Sterne) und v3 (sterne)
```

5. 100% auf v3

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-v3.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v3
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v3.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 50
```




```
## Seite öffnen
http://<deine-ip>/productpage

## bei mehrmals laden immer v3
```

6. Aufräumen

```
kubectl delete -n bookinfo httproute reviews --ignore-not-found
```

Reference:

- https://istio.io/latest/docs/tasks/traffic-management/traffic-shifting/

Pfadbasiertes Routing

```
## Pfadbasiertes Routing ist bereits eingerichtet
## Achtung: Nord/Süd - Traffik / httproute zeigt auf das Gateway
kubectl -n booking get httproute bookinfo -o yaml
```

Übung: LoadBalancing (LEAST_CONN) - Regeln

Ziel

Den Unterschied zwischen LEAST_CONN und ROUND_ROBIN sichtbar machen. Dafür deployen wir einen Service mit schnellen und einem langsamen Pod. Bei parallelen Requests zeigt LEAST_CONN messbar besseres Verhalten.

Voraussetzungen

- Kubernetes-Cluster mit Istio (inkl. Gateway API CRDs)
- `kubectl` und `istioctl` konfiguriert

Schritt 0: traefik deinstallieren

- Wir haben aktuelle keine freien IP's. Traefik ist installiert und belegt eine IP, brauchen wir aber aktuell nicht

```
helm list -A | grep -i traefik
helm -n ingress uninstall traefik
```

Schritt 1: Namespace vorbereiten

```
kubectl create namespace lb-demo
kubectl label namespace lb-demo istio-injection=enabled
```

Schritt 2: Backend deployen (schnelle + langsamer Pod)

Alle Pods laufen hinter dem gleichen Service. Der einzige Unterschied: der langsame Pod wartet 2 Sekunden vor jeder Antwort.

```
cd
mkdir -p manifests/lb-least-conn
cd manifests/lb-least-conn
```

```
nano backend.yaml
```

```
## backend.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-server
  namespace: lb-demo
data:
  server.py: |
    from http.server import HTTPServer, BaseHTTPRequestHandler
    import time, os

    DELAY = float(os.environ.get('RESPONSE_DELAY', '0'))

    class Handler(BaseHTTPRequestHandler):
        def do_GET(self):
            if DELAY > 0:
                time.sleep(DELAY)
            self.send_response(200)
            self.send_header('Content-Type', 'text/plain')
            self.end_headers()
            host = os.environ.get('HOSTNAME', 'unknown')
            self.wfile.write(f"{host} delay={DELAY}s\n".encode())
```

```
        def log_message(self, format, *args):
            pass

    HTTPServer(('', 8080), Handler).serve_forever()
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-fast
  namespace: lb-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
      variant: fast
  template:
    metadata:
      labels:
        app: backend
        variant: fast
    spec:
      containers:
        - name: server
          image: python:3-alpine
          command: ["python3", "/app/server.py"]
          env:
            - name: RESPONSE_DELAY
              value: "0"
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: server-script
              mountPath: /app
      volumes:
        - name: server-script
          configMap:
            name: backend-server
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-slow
  namespace: lb-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
      variant: slow
  template:
    metadata:
```

```
      labels:
        app: backend
        variant: slow
    spec:
      containers:
        - name: server
          image: python:3-alpine
          command: ["python3", "/app/server.py"]
          env:
            - name: RESPONSE_DELAY
              value: "2"
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: server-script
              mountPath: /app
      volumes:
        - name: server-script
          configMap:
            name: backend-server
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: lb-demo
spec:
  selector:
    app: backend       # selektiert ALLE Pods – schnelle und langsame
  ports:
    - port: 80
      targetPort: 8080
```

```
kubectl apply -f backend.yaml
kubectl get pods -n lb-demo -w
```

Ergebnis: 3 Pods hinter einem Service – 2 schnelle (delay=0s), 1 langsamer (delay=2s).

Schritt 3: Gateway + HTTPRoute

```
nano gateway.yaml
```

```
## gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: lb-gateway
  namespace: lb-demo
spec:
  gatewayClassName: istio
```

```
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend-route
  namespace: lb-demo
spec:
  parentRefs:
    - name: lb-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /lb-test
      backendRefs:
        - name: backend
          port: 80
```

```
kubectl apply -f gateway.yaml

export GATEWAY_IP=$(kubectl get gateway lb-gateway -n lb-demo \
  -o jsonpath='{.status.addresses[0].value}')
echo "Gateway: http://$GATEWAY_IP/lb-test"
```

Schnelltest:

```
curl -s http://$GATEWAY_IP/lb-test
```

Erwartete Antwort: `backend-fast-xxxxx delay=0s` oder `backend-slow-xxxxx delay=2s`

Schritt 4: DestinationRule mit LEAST_CONN

```
nano destination-rule.yaml
```

```
## destination-rule.yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: backend
  namespace: lb-demo
spec:
  host: backend
```

```
  trafficPolicy:
    loadBalancer:
      simple: LEAST_CONN
```

```
kubectl apply -f destination-rule.yaml
```

Schritt 5: Envoy-Konfiguration prüfen

```
GWPOD=$(kubectl get pods -n lb-demo -l gateway.networking.k8s.io/gateway-name=lb-
gateway -o jsonpath='{.items[0].metadata.name}')
```

```
istioctl proxy-config cluster $GWPOD -n lb-demo   --fqdn "backend.lb-
demo.svc.cluster.local" -o yaml | grep "LEAST"
## expected output:
##  lbPolicy: LEAST_REQUEST
```

Schritt 6: Test-Funktion definieren

Die Funktion sendet 30 parallele Requests (10 gleichzeitig) und zeigt:

- **Gesamtdauer** – wie lange alle Requests zusammen dauern
- **Verteilung** – wie viele Requests jeder Pod bekommen hat

```
test_lb() {
  local TMPDIR=$(mktemp -d)

  echo "--- Sende 30 Requests (10 parallel) ---"
  SECONDS=0
  seq 1 30 | xargs -P 10 -I{} \
    sh -c 'curl -s http://'"$GATEWAY_IP"'/lb-test > '"$TMPDIR"'/{}'
  echo "--- Gesamtdauer: ${SECONDS}s ---"

  echo "--- Verteilung pro Pod ---"
  cat "$TMPDIR"/* | sort | uniq -c | sort -rn

  rm -rf "$TMPDIR"
}
```

Schritt 7: LEAST_CONN vs ROUND_ROBIN vergleichen

Test mit LEAST_CONN

```
echo "=== LEAST_CONN ==="
test_lb
```

Erwartete Ausgabe:

```
--- Sende 30 Requests (10 parallel) ---
--- Gesamtdauer: 4s ---
--- Verteilung pro Pod ---
     14 backend-fast-abc123 delay=0s
     13 backend-fast-def456 delay=0s
      3 backend-slow-xyz789 delay=2s
```

> *Der langsame Pod bekommt deutlich weniger Requests, weil seine Verbindungen 2s offen bleiben. LEAST_CONN weicht auf die schnellen Pods aus.*

Umschalten auf ROUND_ROBIN

```
kubectl patch destinationrule backend -n lb-demo --type merge \
  -p '{"spec":{"trafficPolicy":{"loadBalancer":{"simple":"ROUND_ROBIN"}}}}'
```

```
## Unbedingt warten, bis keine Ausgabe LEAST_ mehr kommt
istioctl proxy-config cluster $GWPOD -n lb-demo   --fqdn "backend.lb-
demo.svc.cluster.local" -o yaml | grep "lbPolicy"
```

```
sleep 2

echo "=== ROUND_ROBIN ==="
test_lb
```

Erwartete Ausgabe:

```
--- Sende 30 Requests (10 parallel) ---
--- Gesamtdauer: x?s ---
--- Verteilung pro Pod ---
     10 backend-fast-abc123 delay=0s
     10 backend-fast-def456 delay=0s
     10 backend-slow-xyz789 delay=2s
```

> *ROUND_ROBIN verteilt stur gleichmäßig – der langsame Pod bekommt genauso viele Requests. Die Gesamtdauer ist deutlich höher.*

Zurück auf LEAST_CONN

```
kubectl patch destinationrule backend -n lb-demo --type merge \
  -p '{"spec":{"trafficPolicy":{"loadBalancer":{"simple":"LEAST_CONN"}}}}'
```

Schritt 8: Ergebnis

Metrik	LEAST_CONN	ROUND_ROBIN
Requests an langsamen Pod	wenige (~3-5)	gleichmäßig (~10)
Gesamtdauer (30 Req)	~4s	~8s
Prinzip	Bevorzugt freie Pods	Verteilt stur reihum

LEAST_CONN erkennt, dass der langsame Pod noch beschäftigt ist (offene Verbindung), und schickt neue Requests an die freien, schnellen Pods.

Schritt 9: Wann welchen Algorithmus?

Algorithmus	Envoy-Name	Wann sinnvoll?
ROUND_ROBIN	`ROUND_ROBIN`	Default. Gleichförmige, kurze Requests
LEAST_CONN	`LEAST_REQUEST`	Variable Latenzen, Long-Running Connections (gRPC, WebSocket)
RANDOM	`RANDOM`	Große Cluster, vermeidet Hot-Spots
PASSTHROUGH	–	Kein LB, direkt an Upstream-IP

Bonusaufgabe: HTTPRoute East-West mit Service parentRef

Seit der GAMMA-Initiative unterstützt Istio `HTTPRoute` auch für **East-West Traffic** (Service-zu-Service). Dafür setzt man `parentRefs` auf einen **Service** statt auf einen Gateway.

Client-Pod deployen

```
nano client.yaml
```

```yaml
## client.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client
  namespace: lb-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client
  template:
    metadata:
      labels:
        app: client
    spec:
      containers:
        - name: curl
          image: curlimages/curl:latest
          command: ["sleep", "infinity"]
```

```
kubectl apply -f client.yaml
```

Zweiten Backend-Service anlegen

```
## backend-fast-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-fast
  namespace: lb-demo
spec:
  selector:
    app: backend
    variant: fast
  ports:
    - port: 80
      targetPort: 8080
```

```
kubectl apply -f backend-fast-svc.yaml
```

HTTPRoute: East-West Traffic Split

Alle Calls an `backend:80` werden zu 100% an die schnellen Pods umgeleitet:

```
## httproute-east-west.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend-east-west
  namespace: lb-demo
spec:
  parentRefs:
    - group: ""
      kind: Service
      name: backend
      port: 80
  rules:
    - backendRefs:
        - name: backend-fast
          port: 80
          weight: 100
```

```
kubectl apply -f httproute-east-west.yaml
```

Testen aus dem Client-Pod

```
CLIENT=$(kubectl get pods -n lb-demo -l app=client -o
jsonpath='{.items[0].metadata.name}')

## Vorher: alle Pods (inkl. slow) erreichbar
## Nachher: nur noch fast-Pods antworten
for i in $(seq 1 10); do
```

```
  kubectl exec -n lb-demo $CLIENT -- curl -s http://backend/
done
```

> *Alle Antworten kommen nur noch von `backend-fast` Pods – der langsame Pod ist per East-West HTTPRoute komplett aus dem Routing genommen.*

Test von aussen

- Das geht auch !

```
curl -s http://$GATEWAY_IP/lb-test
```

Aufräumen

```
kubectl delete namespace lb-demo
```

Zusammenfassung

Ebene	Ressource	parentRef	Aufgabe
North-South	`Gateway` + `HTTPRoute`	`kind: Gateway`	Externer Eintrittspunkt + Routing
East-West	`HTTPRoute`	`kind: Service`	Service-zu-Service Traffic Split
LB-Algorithmus	`DestinationRule`	–	LEAST_CONN, RANDOM, Consistent Hash

Merke:

- `HTTPRoute` kann beides: North-South (`parentRef` → Gateway) **und** East-West (`parentRef` → Service)
- Für East-West setzt man `kind: Service` + `group: ""` in `parentRefs`
- `DestinationRule` definiert den LB-Algorithmus – unabhängig von der Routing-Ressource

Übung: Bookinfo mit HTTPS und Let's Encrypt (cert-manager, Gateway API)

Ziel

Bookinfo ist danach unter `https://bookinfo.<dein-user>.do.t3isp.de/productpage` mit einem echten Zertifikat erreichbar, `http://` leitet auf `https://` um. cert-manager holt das Zertifikat bei Let's Encrypt (HTTP-01) und erneuert es automatisch.

Voraussetzung

- Uebung Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo`, Gateway `bookinfo-gateway`, HTTPRoute `bookinfo`).
- Der DNS-Eintrag `bookinfo.<dein-user>.do.t3isp.de` existiert schon (siehe Merkkasten).

> ***Merkkasten: DNS ist schon eingerichtet***
>
> *Der Trainer hat fuer jeden Cluster vorab einen DNS-Eintrag angelegt, z. B. `bookinfo.tln5.do.t3isp.de`. Er zeigt auf die oeffentliche IP deines ersten Workers. Du musst **nichts** im DNS eintragen. Die IP holst du dir in Schritt 1 per `dig` und gibst sie dem Gateway mit.*

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir letsencrypt
cd letsencrypt
```

```
export HOST=bookinfo.$(whoami).do.t3isp.de
export IP=$(dig +short $HOST)
echo $HOST $IP
```

Beide Werte muessen gefuellt sein. Ist `IP` leer: Trainer fragen.

Schritt 2: cert-manager installieren

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
```

```
helm install cert-manager jetstack/cert-manager \
  --version v1.21.3 \
  --namespace cert-manager --create-namespace \
  --set crds.enabled=true \
  --set config.apiVersion=controller.config.cert-manager.io/v1alpha1 \
  --set config.kind=ControllerConfiguration \
  --set config.enableGatewayAPI=true \
  --wait
```

```
kubectl -n cert-manager get pods
```

Erwartet: 3 Pods `Running`.

Schritt 3: Issuer anlegen

```
## vi 01-issuer.yml
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: letsencrypt
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-account-key
    solvers:
    - http01:
        gatewayHTTPRoute:
          parentRefs:
          - kind: Gateway
            name: bookinfo-gateway
```

```
kubectl -n bookinfo apply -f 01-issuer.yml
kubectl -n bookinfo get issuer
```

Erwartet: `READY True`.

Schritt 4: Gateway mit HTTPS

```
## vi 02-gateway.yml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: bookinfo-gateway
  annotations:
    cert-manager.io/issuer: letsencrypt
spec:
  gatewayClassName: istio
  addresses:
  - type: IPAddress
    value: ${IP}
  listeners:
  - name: http
    hostname: ${HOST}
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same
  - name: https
    hostname: ${HOST}
    port: 443
    protocol: HTTPS
    tls:
      mode: Terminate
      certificateRefs:
      - name: bookinfo-tls
    allowedRoutes:
      namespaces:
        from: Same
```

```
envsubst < 02-gateway.yml | kubectl -n bookinfo apply -f -
```

```
kubectl -n bookinfo get gateway bookinfo-gateway
```

Erwartet: `PROGRAMMED True`, `ADDRESS` = deine IP.

Schritt 5: Zertifikat abwarten

```
kubectl -n bookinfo get certificate
```

Wiederholen, bis `READY True` (ca. 1 Minute).

Schritt 6: HTTP auf HTTPS umleiten

```
## vi 03-httproute-redirect.yml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: http-to-https
spec:
  parentRefs:
  - name: bookinfo-gateway
    sectionName: http
  rules:
  - filters:
    - type: RequestRedirect
      requestRedirect:
        scheme: https
        statusCode: 301
```

```
## vi 04-httproute-bookinfo.yml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bookinfo
spec:
  parentRefs:
  - name: bookinfo-gateway
    sectionName: https
  rules:
  - matches:
    - path:
        type: Exact
        value: /productpage
    - path:
        type: PathPrefix
        value: /static
    - path:
        type: Exact
        value: /login
    - path:
        type: Exact
        value: /logout
    - path:
        type: PathPrefix
        value: /api/v1/products
    backendRefs:
    - name: productpage
      port: 9080
```

```
kubectl -n bookinfo apply -f 03-httproute-redirect.yml -f 04-httproute-bookinfo.yml
```

Schritt 7: Testen

```
curl -s https://$HOST/productpage | grep -o "<title>.*</title>"
```

Erwartet: `<title>Simple Bookstore App</title>`

```
curl -sI http://$HOST/productpage
```

Erwartet: `HTTP/1.1 301 Moved Permanently` und `location: https://...`

```
echo https://$HOST/productpage
```

Die URL im Browser oeffnen: Schloss in der Adressleiste, Aussteller `Let's Encrypt` .

Fehlersuche

Beobachtung	Pruefen
`Certificate` bleibt `READY False`	`kubectl -n bookinfo describe challenge`
Gateway nicht `PROGRAMMED True`	`IP` gesetzt? `echo $IP`
Nichts passiert nach dem Gateway	cert-manager mit `config.enableGatewayAPI=true` installiert?
Let's Encrypt meldet `too many ...`	Limit der CA erreicht (5 Fehlversuche pro Stunde je Hostname), spaeter erneut

Aufraeumen

```
kubectl -n bookinfo delete httproute http-to-https
kubectl -n bookinfo delete gateway bookinfo-gateway
kubectl -n bookinfo delete httproute bookinfo
kubectl -n bookinfo delete issuer letsencrypt
kubectl -n bookinfo delete secret bookinfo-tls letsencrypt-account-key
```

```
helm uninstall cert-manager -n cert-manager
kubectl delete namespace cert-manager
kubectl get crd -o name | grep cert-manager.io | xargs kubectl delete
```

Bookinfo wieder ohne HTTPS:

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/gateway-api/bookinfo-
gateway.yaml
```

Ref

- https://cert-manager.io/docs/usage/gateway/

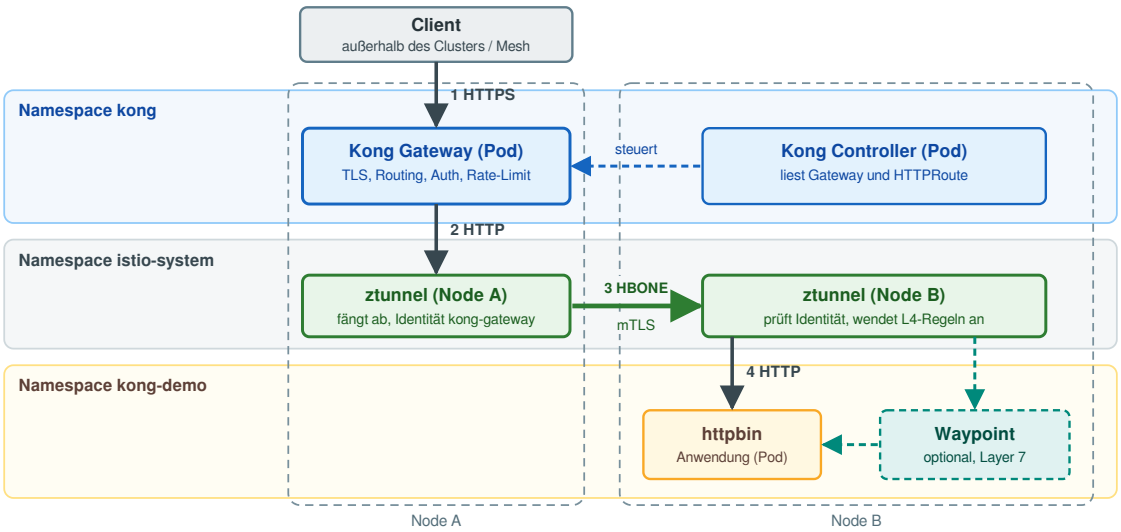
Übung (Ambient): Kong als API-Gateway vor Istio Ambient (mTLS, STRICT, Waypoint-Umgehung)

Ziel

Kong nimmt Anfragen von aussen an (Routing, Authentifizierung, Rate-Limit, TLS). Istio Ambient sichert den Weg von Kong zur Anwendung (mTLS, Identitaet, Regeln). In dieser Uebung bauen wir beides zusammen und pruefen:

- Kong laeuft als normaler Workload im Ambient-Mesh, ohne Sidecar.
- Der Weg Kong zur Anwendung ist mTLS (HBONE), mit der Identitaet von Kong.
- `STRICT` funktioniert nur, wenn auch Kong im Mesh ist.
- Eine Regel kann festlegen: **nur Kong darf an die Anwendung.**
- **Stolperfalle:** Layer-7-Regeln am Waypoint werden von Kong standardmaessig umgangen. Wir zeigen das und zwei Loesungen.

Kong vor Istio Ambient: der Weg einer Anfrage



Kong (Nord-Süd): nimmt Anfragen von außen an, terminiert TLS, prüft API-Keys oder Token und wählt per HTTPRoute das Ziel.
Ambient (Ost-West): der ztunnel verschlüsselt den Weg von Kong zur Anwendung (mTLS) und liefert die Identität von Kong.
Waypoint (gestrichelt): nur nötig, wenn die Anwendung Layer-7-Regeln des Mesh braucht. Dann siehe Grafik zur Umgehung.

Voraussetzung

- Istio Ambient ist installiert, siehe [istio-installation mit istioctl - ambient].
- Die Gateway-API-CRDs sind da (Istio braucht sie fuer jeden Waypoint, Kong fuer seine `HTTPRoute`):

```
kubectl get crd gateways.gateway.networking.k8s.io
```

- `helm` ist auf dem Client installiert (`helm version`).
- Bookinfo wird fuer diese Uebung **nicht** gebraucht. Alles liegt in eigenen Namespaces.

Hintergrund

Frage	Antwort
Ersetzt Kong den Istio-Ingress?	Ja. Kong ist hier das Gateway fuer Verkehr von aussen. Ein zweites Gateway von Istio laeuft nicht dahinter.
Braucht Kong einen Sidecar?	Nein. Der Namespace von Kong bekommt das Label `istio.io/dataplane-mode=ambient`, der ztunnel uebernimmt.
Welche GatewayClass	`kong`. Istio nutzt `istio`. Beide koennen im selben Cluster laufen, ohne sich zu

nutzt Kong?	stoeren.
Wer ist fuer was zustaendig?	Kong: TLS von aussen, API-Keys, OAuth, Rate-Limit, Routing. Ambient: mTLS zwischen den Pods, Identitaet, Regeln.
Woran erkennt die Anwendung Kong?	An der Identitaet `spiffe://cluster.local/ns/kong/sa/kong-gateway`. Sie steht im Zertifikat des ztunnel.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir kong-ambient
cd kong-ambient
```

Schritt 2: Anwendung im Mesh und Test-Client ausserhalb

Die Anwendung `httpbin` in einem Namespace im Ambient-Mesh. Ein curl-Pod in einem Namespace **ohne** Mesh simuliert den Client von aussen:

```
kubectl create namespace kong-demo
kubectl label namespace kong-demo istio.io/dataplane-mode=ambient
kubectl -n kong-demo apply -f ~/istio/samples/httpbin/httpbin.yaml
kubectl create namespace kong-client
kubectl -n kong-client apply -f ~/istio/samples/curl/curl.yaml
kubectl -n kong-demo rollout status deploy/httpbin
kubectl -n kong-client rollout status deploy/curl
```

Schritt 3: Kong installieren und ins Mesh holen

Der Namespace bekommt das Label **vor** der Installation. Dann laeuft Kong von Anfang an im Mesh. `ClusterIP` genuegt, weil wir aus dem Cluster testen:

```
kubectl create namespace kong
kubectl label namespace kong istio.io/dataplane-mode=ambient
helm repo add kong https://charts.konghq.com
helm repo update
helm install kong kong/ingress --version 0.24.0 -n kong --set
gateway.proxy.type=ClusterIP --wait --timeout 5m
```

```
kubectl -n kong get pods
```

Erwartet: `kong-controller-...` und `kong-gateway-...`, beide `1/1 Running`. Es gibt keinen zweiten Container (kein Sidecar).

Der ztunnel kennt beide Pods als Mesh-Teilnehmer:

```
istioctl ztunnel-config workload | grep -E "NAMESPACE|kong"
```

In der Spalte `PROTOCOL` steht fuer die Kong-Pods `HBONE`. Der curl-Pod aus `kong-client` hat `TCP`, er ist nicht im Mesh.

Schritt 4: Gateway und Route fuer Kong

Kong braucht eine `GatewayClass`, ein `Gateway` und eine `HTTPRoute`. Das ist dieselbe Gateway API wie bei Istio, nur mit `gatewayClassName: kong`:

```
## vi 01-gateway.yml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: kong
  annotations:
    konghq.com/gatewayclass-unmanaged: "true"
spec:
  controllerName: konghq.com/kic-gateway-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: kong
  namespace: kong
spec:
  gatewayClassName: kong
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: All
```

Die Route liegt im Namespace der Anwendung und haengt sich an das Gateway in `kong`. Kong schneidet den Praefix `/httpbin` ab:

```
## vi 02-httproute.yml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: httpbin
spec:
  parentRefs:
  - name: kong
    namespace: kong
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /httpbin
    filters:
    - type: URLRewrite
      urlRewrite:
        path:
          type: ReplacePrefixMatch
```

```
          replacePrefixMatch: /
    backendRefs:
    - name: httpbin
      port: 8000
```

```
kubectl apply -f 01-gateway.yml
kubectl apply -f 02-httproute.yml -n kong-demo
```

```
kubectl get gatewayclass
kubectl get gateway -n kong
```

Nach etwa 15 Sekunden: Die GatewayClass `kong` ist `ACCEPTED True`, das Gateway `PROGRAMMED True`.

Schritt 5: Erster Aufruf und der Weg im ztunnel

Vom Client ausserhalb des Mesh durch Kong zur Anwendung:

```
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "%{http_code}\n"
http://kong-gateway-proxy.kong/httpbin/get
```

Erwartet: `200`. Kommt direkt nach dem Anlegen noch `404` (`no Route matched with those values`), hat Kong die Route noch nicht uebernommen: 10 Sekunden warten, dann wiederholen.

Die Antwort von `/headers` zeigt, dass Kong dabei war (`Via: 1.1 kong/...` , `X-Forwarded-Path`):

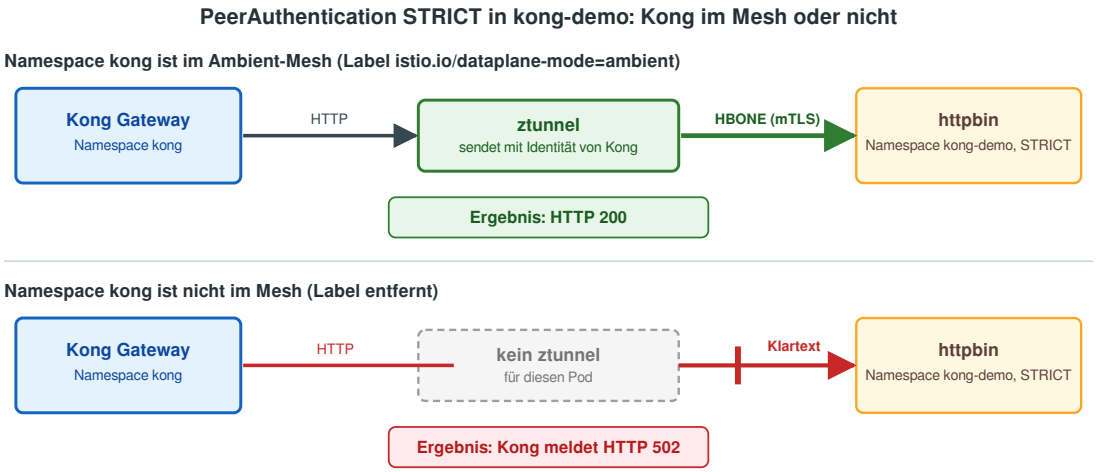
```
kubectl -n kong-client exec deploy/curl -- curl -s http://kong-gateway-
proxy.kong/httpbin/headers
```

Den Weg von Kong zur Anwendung sehen wir im ztunnel-Log auf dem Node des Kong-Pods:

```
NODE=$(kubectl -n kong get pods -o wide --no-headers | grep kong-gateway | awk '{print
$7}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
kubectl -n istio-system logs $ZT --tail=200 | grep 'src.workload="kong-gateway' | grep
'dst.workload="httpbin' | tail -2
```

Erwartet: Zeilen mit `src.identity="spiffe://cluster.local/ns/kong/sa/kong-gateway"` , `dst.identity="spiffe://cluster.local/ns/kong-demo/sa/httpbin"` , `dst.addr=...:15008` (HBONE) und `direction="outbound"` . Kong hat selbst nur HTTP gesprochen, die Verschluesselung kommt vom ztunnel.

Schritt 6: STRICT - Kong muss im Mesh sein



Oben: der ztunnel baut den HBONE-Tunnel mit der Identität von Kong auf. STRICT ist erfüllt.
Unten: Kong spricht Klartext. Der ztunnel von httpbin lehnt ihn wegen STRICT ab, bei Kong kommt ein 502 an.
Folge: Wer STRICT einsetzt, muss auch den Kong-Namespace ins Mesh nehmen. Ein Neustart des Kong-Pods ist dafür nicht nötig.

`STRICT` fuer `kong-demo`:

```
## vi 03-peerauthentication-strict.yml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT
```

```
kubectl apply -f 03-peerauthentication-strict.yml -n kong-demo
```

Durch Kong (Kong ist im Mesh):

```
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "%{http_code}\n"
http://kong-gateway-proxy.kong/httpbin/get
```

Erwartet: `200`. Direkt an `httpbin`, vom Client ausserhalb des Mesh:

```
kubectl -n kong-client exec deploy/curl -- curl -s -m 8 -o /dev/null -w "%
{http_code}\n" http://httpbin.kong-demo:8000/get
```

Erwarteter Fehler: `000` und `command terminated with exit code 56` (bei anderer curl-Version auch `52`). Der ztunnel lehnt Klartext ab.

Jetzt Kong aus dem Mesh nehmen. Ein Neustart ist nicht noetig:

```
kubectl label namespace kong istio.io/dataplane-mode-
sleep 5
kubectl -n kong-client exec deploy/curl -- curl -s -m 8 -o /dev/null -w "%
{http_code}\n" http://kong-gateway-proxy.kong/httpbin/get
```

Erwarteter Fehler: `502`. Kong spricht Klartext, `STRICT` lehnt ihn ab.

Wieder zurueck:

```
kubectl label namespace kong istio.io/dataplane-mode=ambient
sleep 5
kubectl -n kong-client exec deploy/curl -- curl -s -m 8 -o /dev/null -w "%
{http_code}\n" http://kong-gateway-proxy.kong/httpbin/get
```

Erwartet: wieder `200`.

Schritt 7: Nur Kong darf an die Anwendung

Die Identitaet von Kong steht im Zertifikat. Damit laesst sich der Zugang einschraenken (Layer 4, der ztunnel prueft):

```
## vi 04-authorizationpolicy-nur-kong.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: nur-kong
spec:
  selector:
    matchLabels:
      app: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/kong/sa/kong-gateway"]
```

```
kubectl apply -f 04-authorizationpolicy-nur-kong.yml -n kong-demo
sleep 5
```

Ein zweiter curl-Pod, diesmal **im** Mesh, im Namespace der Anwendung:

```
kubectl -n kong-demo apply -f ~/istio/samples/curl/curl.yaml
kubectl -n kong-demo rollout status deploy/curl
```

Durch Kong, und direkt aus dem Mesh:

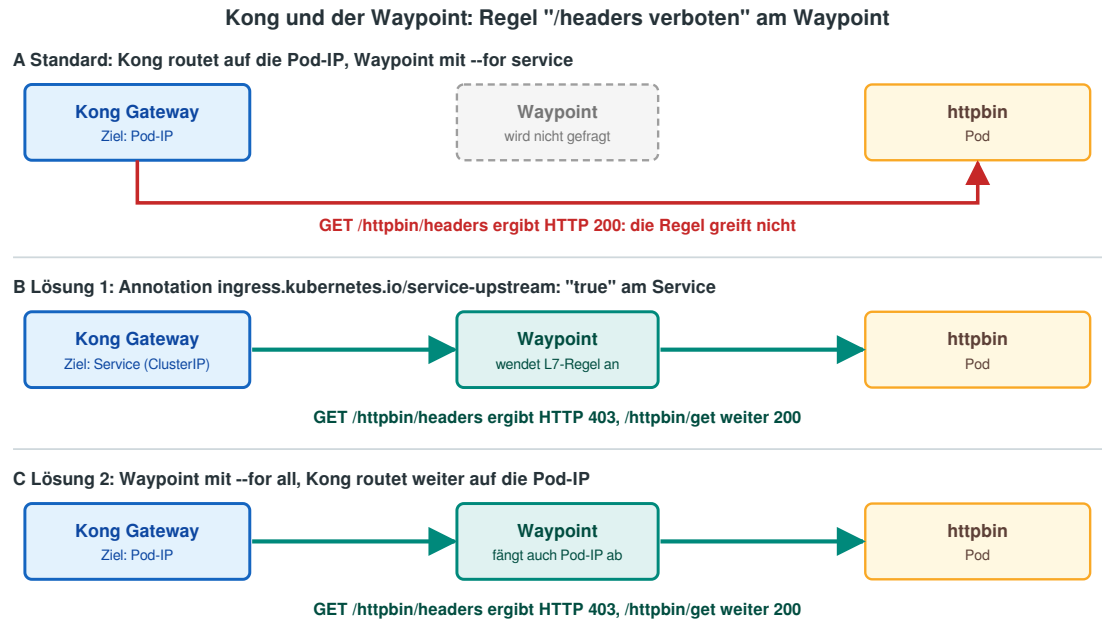
```
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "durch Kong: %
{http_code}\n" http://kong-gateway-proxy.kong/httpbin/get
kubectl -n kong-demo exec deploy/curl -- curl -s -m 8 -o /dev/null -w "direkt aus dem
Mesh: %{http_code}\n" http://httpbin:8000/get
```

Erwartet: `durch Kong: 200`. **Erwarteter Fehler:** `direkt aus dem Mesh: 000` (Exit-Code 56). Die Anwendung ist nur noch ueber Kong erreichbar.

Regel wieder weg:

```
kubectl delete -f 04-authorizationpolicy-nur-kong.yml -n kong-demo
```

Schritt 8: Die Stolperfalle - Kong umgeht den Waypoint



Warum A? Der Kong Ingress Controller trägt die Pod-IPs der Endpoints als Ziele ein, nicht die Service-IP.
Ohne --for all ist der Waypoint nur für Aufrufe an den Service zuständig.

Lösung 1 ändert Kong: Er ruft den Service auf, der ztunnel schickt den Aufruf zum Waypoint. Gilt pro Service.

Lösung 2 ändert den Waypoint: Auch Aufrufe an Pod-IPs laufen hindurch, im Namespace also für jeden Aufrufer.

Prüfen: immer mit einem Pfad testen, den die Regel verbietet. Ein 200 auf dem verbotenen Pfad zeigt die Umgehung.

Gemessen auf Istio 1.31.4, Kong Gateway 3.9 (Chart kong/ingress 0.24.0), Kubernetes 1.37.2.

Fuer Layer-7-Regeln (Pfade, Methoden, JWT) braucht die Anwendung einen Waypoint. Wir legen einen an und verbieten den Pfad `/headers`:

```
istioctl waypoint apply -n kong-demo --enroll-namespace --wait
```

```
## vi 05-deny-headers.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-headers
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/headers"]
```

```
kubectl apply -f 05-deny-headers.yml -n kong-demo
sleep 8
```

Die Warnung `will deny all traffic to TCP ports ... recommended to explicitly specify the port` kannst du hier ignorieren.

Kontrolle, **ohne Kong**, aus dem Mesh (der Aufruf geht ueber den Service, also durch den Waypoint):

```
kubectl -n kong-demo exec deploy/curl -- curl -s -o /dev/null -w "/headers direkt: %
{http_code}\n" http://httpbin:8000/headers
kubectl -n kong-demo exec deploy/curl -- curl -s -o /dev/null -w "/get direkt: %
{http_code}\n" http://httpbin:8000/get
```

Erwartet: `/headers direkt: 403` und `/get direkt: 200`. Die Regel wirkt.

Jetzt **durch Kong**:

```
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/headers ueber
Kong: %{http_code}\n" http://kong-gateway-proxy.kong/httpbin/headers
```

Ueberraschung: `200`. Die verbotene Regel greift nicht. Der Kong Ingress Controller traegt die Pod-IPs der Endpoints als Ziele ein, nicht die Service-IP. Ein Waypoint mit `--for service` (Standard) ist aber nur fuer Aufrufe an den Service zustaendig. Das siehst du, wenn du die Endpoints ansiehst:

```
kubectl -n kong-demo get endpoints httpbin
```

Loesung 1: Kong soll den Service aufrufen

Eine Annotation am Service der Anwendung:

```
kubectl -n kong-demo annotate service httpbin ingress.kubernetes.io/service-
upstream=true
sleep 10
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/headers ueber
Kong: %{http_code}\n" http://kong-gateway-proxy.kong/httpbin/headers
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/get ueber Kong: %
{http_code}\n" http://kong-gateway-proxy.kong/httpbin/get
```

Erwartet: `/headers ueber Kong: 403`, `/get ueber Kong: 200`. Die Annotation gilt pro Service.

Loesung 2: Der Waypoint faengt auch Pod-IPs ab

Erst die Annotation wieder entfernen, dann den Waypoint mit `--for all` neu anlegen:

```
kubectl -n kong-demo annotate service httpbin ingress.kubernetes.io/service-upstream-
istioctl waypoint apply -n kong-demo --for all --overwrite --wait
sleep 12
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/headers ueber
Kong: %{http_code}\n" http://kong-gateway-proxy.kong/httpbin/headers
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/get ueber Kong: %
{http_code}\n" http://kong-gateway-proxy.kong/httpbin/get
```

Erwartet: `/headers ueber Kong: 403`, `/get ueber Kong: 200`. Hier bleibt Kong unveraendert. Dafuer laufen im Namespace alle Aufrufe an Pod-IPs durch den Waypoint, auch die anderer Aufrufer. Mehr dazu: Waypoint: was bedeutet --for service, workload und all?.

Zur Gegenprobe wieder auf den Standard zurueck. Der Verbotene Pfad kommt wieder durch:

```
istioctl waypoint apply -n kong-demo --for service --overwrite --wait
sleep 12
kubectl -n kong-client exec deploy/curl -- curl -s -o /dev/null -w "/headers ueber
Kong: %{http_code}\n" http://kong-gateway-proxy.kong/httpbin/headers
```

Erwartet: `200`.

Aufraeumen

```
helm uninstall kong -n kong
kubectl delete namespace kong kong-demo kong-client
kubectl delete gatewayclass kong
kubectl get crd -o name | grep konghq.com | xargs -r kubectl delete
cd
rm -rf manifests/kong-ambient
```

`helm uninstall` laesst die Kong-CRDs (`konghq.com`) stehen, deshalb die letzte `kubectl delete`-Zeile. Die Gateway-API-CRDs und alles von Istio bleiben unberuehrt.

Zusammenfassung

Szenario	Ergebnis
Kong im Ambient-Mesh (Namespace-Label)	Kein Sidecar, ztunnel verschluesselt Kong zur Anwendung (HBONE)
Aufruf ueber Kong, `STRICT` in `kong-demo`, Kong im Mesh	200
Direktaufruf von aussen bei `STRICT`	000, Klartext abgelehnt
Kong nicht im Mesh, `STRICT`	502
`AuthorizationPolicy` mit Principal von Kong	Nur Kong darf an die Anwendung
L7-Regel am Waypoint, Kong mit Standard (Pod-IP)	**Wird umgangen** (200 auf verbotenem Pfad)
Annotation `service-upstream` am Service	Regel wirkt (403)
Waypoint mit `--for all`	Regel wirkt (403), Kong unveraendert

Referenz

- Getestet mit Istio 1.31.4 (Ambient), Kubernetes 1.37.2, Kong Gateway 3.9 (Helm-Chart `kong/ingress` 0.24.0, Kong Ingress Controller mit Gateway API).
- Nicht getestet: Kong als `LoadBalancer` (`--set gateway.proxy.type=LoadBalancer`), Kong-Plugins (Authentifizierung, Rate-Limit), Kong mit eigener Datenbank.
- Kong-Doku zur Annotation: `ingress.kubernetes.io/service-upstream`.

Service Resilience & Fehlertoleranz (Gateway API)

Circuit Breaker

Aufgabe des Circuit Breakers:

```
Der Circuit Breaker in Istio schützt Deine Services davor, einen überlasteten oder fehlerhaften Backend-Service weiter mit Requests zu bombardieren.
```

Wie ?

Du konfigurierst im `DestinationRule` unter `trafficPolicy.connectionPool` und `outlierDetection` zwei Dinge:

1. **Connection Pool Limits** (`connectionPool`) — begrenzt die Anzahl gleichzeitiger TCP-Connections und wartender HTTP-Requests. Wird das Limit überschritten, werden neue Requests sofort mit 503 abgelehnt statt endlos zu warten.
2. **Outlier Detection** (`outlierDetection`) — erkennt ungesunde Endpoints. Wenn ein Pod z.B. 5× hintereinander 5xx zurückgibt, wird er temporär aus dem Load-Balancing-Pool entfernt ("evicted"). Nach einer Pause darf er wieder rein.

Voraussetzungen

- Istio ist installiert (Sidecar Mode).
- Namespace `bookinfo` ist mit automatischer Sidecar-Injection gelabelt:

```
kubectl label namespace bookinfo istio-injection=enabled --overwrite
```

- Das Istio-Repo liegt unter `~/istio` (Samples unter `~/istio/samples/...`).

---

1 Arbeitsverzeichnis anlegen

```
mkdir -p ~/manifests/circuit-breaker
cd ~/manifests/circuit-breaker
```

---

2 httpbin im Namespace `bookinfo` deployen

```
kubectl apply -n bookinfo -f ~/istio/samples/httpbin/httpbin.yaml
kubectl get pods -n bookinfo -l app=httpbin
```

Warte, bis der Pod `Running` ist.

---

3 DestinationRule mit Circuit Breaker anlegen

Erzeuge eine Manifestdatei im Übungsverzeichnis:

```
nano dr-httpbin-circuit-breaker.yaml
```

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
```

```
  name: httpbin
  namespace: bookinfo
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
```

- Erklärung TCP/http
- Request 1 → bekommt die eine Connection → wird verarbeitet
- Request 2 → Connection belegt → geht in die Pending Queue (1 Platz)
- Request 3 → Connection belegt, Queue voll → sofort 503

```
consecutive5xxErrors: 1 — Ein einziger 5xx-Fehler reicht, und der Endpoint fliegt
raus. In Produktion eher 5–10, damit nicht ein einzelner Timeout sofort zur Eviction
führt.

interval: 1s — Envoy prüft jede Sekunde, ob ein Endpoint auffällig ist. Das ist der
Auswertungszyklus. Default wäre 10s. Heißt: nach einem 5xx dauert es maximal 1
Sekunde, bis der Pod evicted wird.

baseEjectionTime: 3m — Ein evicted Endpoint bleibt 3 Minuten draußen, bevor er wieder
in den Pool darf. Wichtig: bei wiederholter Eviction multipliziert sich die Zeit —
beim zweiten Mal 6 Minuten, beim dritten 9 Minuten usw. Das bestraft
"Wiederholungstäter".

maxEjectionPercent: 100 — Es dürfen alle Endpoints gleichzeitig evicted werden.
Default ist 10%. Mit 100% kann es passieren, dass der Service komplett unerreichbar
wird, weil kein einziger Endpoint mehr im Pool ist. In Produktion gefährlich — da
willst du immer mindestens einen Endpoint behalten. Für die Demo ist es aber
praktisch, weil du klar zeigen kannst: ein Fehler → Pod fliegt raus → alles 503.
```

> ***Hinweis (mTLS):*** *Wenn dein Mesh* ***strict mTLS*** *nutzt und du 503er bekommst, ergänze in* `trafficPolicy` *noch:*
>
> ```
>   tls:
>     mode: ISTIO_MUTUAL
> ```

Apply & prüfen:

```
kubectl apply -f dr-httpbin-circuit-breaker.yaml
kubectl get destinationrule httpbin -n bookinfo -o yaml
```

4 Fortio-Client im Mesh deployen

- Fortio ist ein Open-Source Load-Testing-Tool,

```
kubectl apply -n bookinfo -f ~/istio/samples/httpbin/sample-client/fortio-deploy.yaml
kubectl get pods -n bookinfo -l app=fortio
```

Exportiere den Pod-Namen:

```
export FORTIO_POD=$(kubectl get pods -n bookinfo -l app=fortio -o 'jsonpath=
{.items[0].metadata.name}')
echo "$FORTIO_POD"
```

---

5 Sanity-Check: Ein einzelner Request

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio curl -quiet http://httpbin:8000/get
```

Erwartung: **HTTP 200 OK** mit JSON-Antwort.

---

6 Circuit Breaker „anrucken“ (2 Verbindungen)

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio --   /usr/bin/fortio load -c 2 -qps
0 -n 20 -loglevel Warning   http://httpbin:8000/get 2>&1 | grep "Code "
```

- `-c 2` → 2 gleichzeitige Verbindungen
- `-n 20` → 20 Requests

Erwartung: Die meisten Requests sind 200, einige ggf. 503.

---

7 Circuit Breaker deutlich auslösen (3 Verbindungen)

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning \
  http://httpbin:8000/get 2>&1 | grep "Code "
```

Erwartung: **Deutlich mehr 503** (Circuit Breaking greift).

---

8 Envoy-Stats im Sidecar prüfen

```
kubectl exec -n bookinfo "$FORTIO_POD" -c istio-proxy -- \
  pilot-agent request GET stats | grep httpbin.bookinfo.svc.cluster.local | grep
pending
```

Achte auf Werte wie `upstream_rq_pending_overflow` → zeigt an, wie viele Requests wegen Circuit Breaking abgewiesen wurden.

---

9 Aufräumen

```
## Circuit-Breaker-Regel entfernen
kubectl delete -f dr-httpbin-circuit-breaker.yaml

## Fortio-Client entfernen
kubectl delete -n bookinfo -f ~/istio/samples/httpbin/sample-client/fortio-
deploy.yaml

## httpbin entfernen
kubectl delete -n bookinfo -f ~/istio/samples/httpbin/httpbin.yaml
```

Circuit Breaker - Gateway API (Ambient + Waypoint)

Aufgabe des Circuit Breakers

```
Der Circuit Breaker in Istio schützt Deine Services davor, einen überlasteten oder
fehlerhaften Backend-Service weiter mit Requests zu bombardieren.
```

Warum `DestinationRule` und nicht `HTTPRoute` ?

```
Die HTTPRoute (Gateway API) legt fest, WOHIN ein Request geht (Pfad, Header, Gewichte,
Backend). Wie sich der Aufrufer gegenüber einem Backend verhält (Verbindungslimits,
Ausreißer-Erkennung, Load-Balancing-Algorithmus, TLS), kennt die Gateway API nicht.
Das bleibt in Istio Aufgabe der DestinationRule, die Du mit HTTPRoute und Waypoint
einfach kombinierst.
```

Wie ?

Du konfigurierst in der `DestinationRule` unter `trafficPolicy.connectionPool` und `outlierDetection` zwei Dinge:

1. **Connection Pool Limits** (`connectionPool`) — begrenzt die Anzahl gleichzeitiger TCP-Connections und wartender HTTP-Requests. Wird das Limit überschritten, werden neue Requests sofort mit 503 abgelehnt statt endlos zu warten.
2. **Outlier Detection** (`outlierDetection`) — erkennt ungesunde Endpoints. Wenn ein Pod z.B. 5× hintereinander 5xx zurückgibt, wird er temporär aus dem Load-Balancing-Pool entfernt ("evicted"). Nach einer Pause darf er wieder rein.

Wo wird das durchgesetzt? (Ambient)

Im Ambient Mode gibt es keinen Sidecar. Die Regeln der `DestinationRule` setzt immer der **Envoy** durch, der den Request an das Backend schickt (ztunnel kann nur L4):

Weg	Wer setzt den Circuit Breaker durch

Teil A: `fortio` → `httpbin` (im Mesh)	der **Waypoint** (`bookinfo` hat `istio.io/use-waypoint=waypoint`)
Teil B: Internet → Gateway → `httpbin` (Gateway API `HTTPRoute`)	der **Gateway-Pod** `bookinfo-gateway-istio`

Voraussetzungen

- Istio Ambient ist installiert, `bookinfo` ist nach `ambient/04-install-demo-app.md` eingerichtet:
  - Label `istio.io/dataplane-mode=ambient` und `istio.io/use-waypoint=waypoint`
  - Waypoint `waypoint` ist `PROGRAMMED`
  - Gateway `bookinfo-gateway` (Gateway API) existiert (nur für Teil B)

```
kubectl get ns bookinfo --show-labels
kubectl get gateway -n bookinfo
```

- Das Istio-Repo liegt unter `~/istio` (Samples unter `~/istio/samples/...`).

---

1 Arbeitsverzeichnis anlegen

```
mkdir -p ~/manifests/circuit-breaker-gateway-api
cd ~/manifests/circuit-breaker-gateway-api
```

---

2 httpbin und Fortio im Namespace `bookinfo` deployen

```
kubectl apply -n bookinfo -f ~/istio/samples/httpbin/httpbin.yaml
kubectl apply -n bookinfo -f ~/istio/samples/httpbin/sample-client/fortio-deploy.yaml
kubectl get pods -n bookinfo -l 'app in (httpbin,fortio)'
```

Warte, bis beide Pods `Running` sind (`1/1` , kein Sidecar-Container, das ist Ambient).

```
export FORTIO_POD=$(kubectl get pods -n bookinfo -l app=fortio -o 'jsonpath=
{.items[0].metadata.name}')
echo "$FORTIO_POD"
```

---

3 Sanity-Check und Baseline (ohne Circuit Breaker)

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio curl -quiet http://httpbin:8000/get | head -12
```

Erwartung: **HTTP 200 OK**. Im Header steht `server: istio-envoy` — die Antwort kam über den **Waypoint**.

Last mit 3 Verbindungen, noch ohne Regel:

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning \
```

```
  http://httpbin:8000/get 2>&1 | grep "Code "
```

Erwartung: `Code 200 : 30 (100.0 %)`

---

4 DestinationRule mit Circuit Breaker anlegen

```
nano dr-httpbin-circuit-breaker.yaml
```

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
  namespace: bookinfo
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
```

- Erklärung TCP/http
- Request 1 → bekommt die eine Connection → wird verarbeitet
- Request 2 → Connection belegt → geht in die Pending Queue (1 Platz)
- Request 3 → Connection belegt, Queue voll → sofort 503

```
consecutive5xxErrors: 1 — Ein einziger 5xx-Fehler reicht, und der Endpoint fliegt
raus. In Produktion eher 5–10, damit nicht ein einzelner Timeout sofort zur Eviction
führt.

interval: 1s — Envoy prüft jede Sekunde, ob ein Endpoint auffällig ist. Das ist der
Auswertungszyklus. Default wäre 10s.

baseEjectionTime: 3m — Ein evicted Endpoint bleibt 3 Minuten draußen, bevor er wieder
in den Pool darf. Bei wiederholter Eviction multipliziert sich die Zeit — beim zweiten
Mal 6 Minuten, beim dritten 9 Minuten usw.

maxEjectionPercent: 100 — Es dürfen alle Endpoints gleichzeitig evicted werden.
Default ist 10%. In Produktion gefährlich — da willst du immer mindestens einen
Endpoint behalten.
```

> ***Hinweis (mTLS):*** *Im Ambient Mode läuft der Verkehr zum Waypoint ohnehin über mTLS (HBONE). Du musst hier kein* `tls: ISTIO_MUTUAL` *ergänzen.*

Apply & prüfen:

```
kubectl apply -f dr-httpbin-circuit-breaker.yaml
kubectl get destinationrule httpbin -n bookinfo -o yaml
```

---

5 Teil A: Circuit Breaker im Mesh auslösen (Waypoint)

Ein einzelner Request geht weiterhin durch:

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio curl -quiet http://httpbin:8000/get | head -3
```

2 Verbindungen („anrucken“):

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel Warning \
  http://httpbin:8000/get 2>&1 | grep "Code "
```

- `-c 2` → 2 gleichzeitige Verbindungen
- `-n 20` → 20 Requests

Erwartung: alle oder fast alle 200.

3 Verbindungen (deutlich auslösen):

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning \
  http://httpbin:8000/get 2>&1 | grep "Code "
```

Erwartung: **Deutlich mehr 503** (Circuit Breaking greift). Beispiel: `Code 200 : 18` , `Code 503 : 12` . Die genaue Zahl schwankt von Lauf zu Lauf.

---

6 Metriken im Waypoint prüfen

Der Waypoint ist ein normaler Pod (`deploy/waypoint`), die Zähler holst Du dort:

```
kubectl exec -n bookinfo deploy/waypoint -- \
  pilot-agent request GET stats 2>/dev/null \
  | grep istio_requests_total | grep httpbin \
  | grep -o 'response_code\.[0-9]*\.grpc_response_status\.response_flags\.[A-Z-]*\|:
[0-9]*$' | paste - -
```

Erwartung (Zahlen variieren):

```
response_code.200.grpc_response_status.response_flags.- : 69
response_code.503.grpc_response_status.response_flags.UO        : 12
```

`response_flags.UO` = **U**pstream **O**verflow: Envoy hat den Request wegen des Circuit Breakers sofort abgewiesen. Die Zahl hinter `UO` entspricht den 503ern aus Fortio.

> ***i*** *Im Sidecar Mode zeigt* `upstream_rq_pending_overflow` *im* `istio-proxy` *des Clients den Zähler. Der Waypoint exportiert diesen Cluster-Zähler nicht, deshalb nehmen wir hier die Metrik mit dem Flag* `UO` *.*

---

7 Teil B: Circuit Breaker auch am Gateway (HTTPRoute)

Jetzt dieselbe `DestinationRule` auf dem Weg **Gateway → httpbin**. Dafür veröffentlichen wir `httpbin` über eine `HTTPRoute` am vorhandenen `bookinfo-gateway` .

```
nano httproute-httpbin.yaml
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: httpbin
  namespace: bookinfo
spec:
  parentRefs:
  - name: bookinfo-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /httpbin
    filters:
    - type: URLRewrite
      urlRewrite:
        path:
          type: ReplacePrefixMatch
          replacePrefixMatch: /
    backendRefs:
    - name: httpbin
      port: 8000
```

- `/httpbin/get` wird zu `/get` umgeschrieben (`URLRewrite`), httpbin kennt den Prefix nicht.
- Die Circuit-Breaker-Regeln stehen **nicht** in der HTTPRoute, sie kommen aus der DestinationRule `httpbin` (Schritt 4).

```
kubectl apply -f httproute-httpbin.yaml
kubectl get httproute httpbin -n bookinfo
```

Ein Request von außen:

```
GATEWAY_IP=$(kubectl get gateway bookinfo-gateway -n bookinfo -o
jsonpath='{.status.addresses[0].value}')
echo $GATEWAY_IP
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/httpbin/get
```

Erwartung: `200` .

> ⚠ *Hat der Listener des Gateways einen `hostname` (z. B. nach Übung 17: `bookinfo.tln<N>.do.t3isp.de`), brauchst Du den Host-Header: `curl -H "Host: bookinfo.tln<N>.do.t3isp.de" ...` bzw. direkt die URL mit dem Hostnamen.*

Last erzeugen: mit Fortio **im Cluster** gegen den Gateway-Service, damit die Verbindungen parallel ankommen:

```
kubectl exec -n bookinfo "$FORTIO_POD" -c fortio -- \
  /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning \
  http://bookinfo-gateway-istio.bookinfo.svc.cluster.local/httpbin/get 2>&1 | grep
"Code "
```

(Mit Hostname am Listener zusätzlich `-H "Host: bookinfo.tln<N>.do.t3isp.de"` vor der URL.)

Erwartung: wieder 200 **und** 503. Die 503 kommen diesmal vom **Gateway-Pod**:

```
kubectl exec -n bookinfo deploy/bookinfo-gateway-istio -- \
  pilot-agent request GET stats 2>/dev/null \
  | grep istio_requests_total | grep httpbin \
  | grep -o 'response_code\.[0-9]*\.grpc_response_status\.response_flags\.[A-Z-]*\|:
[0-9]*$' | paste - -
```

Wieder `response_flags.UO` bei `503`.

---

8 Aufräumen

```
## HTTPRoute entfernen (Teil B)
kubectl delete -f httproute-httpbin.yaml

## Circuit-Breaker-Regel entfernen
kubectl delete -f dr-httpbin-circuit-breaker.yaml

## Fortio-Client entfernen
kubectl delete -n bookinfo -f ~/istio/samples/httpbin/sample-client/fortio-
deploy.yaml

## httpbin entfernen
kubectl delete -n bookinfo -f ~/istio/samples/httpbin/httpbin.yaml
```

Chaos Engineering mit Istio

- https://istio.io/latest/docs/examples/microservices-istio/production-testing/

Sichere Service-Kommunikation mit istio (Sicherheit)

Sicherheit & Zero Trust mit Istio

1 Strong Authentication (mTLS überall)

- Jeder Pod bekommt automatisch ein **X.509-Zertifikat** (~SpiffeID).
- **Alle** Services sprechen untereinander **mutual TLS**.

- Istio überprüft:
    - Ist der Client wirklich der, der er vorgibt zu sein?
    - Passt die SPIFFE Identity?

Nichts darf unverschlüsselt, nichts darf anonym miteinander reden.

---

2 No implicit trust

Mit einer deny - policy (oder einer einzigen Allow-Policy) gilt heute in Istio: **Alles ist DENY, bis du ALLOW definierst.**

D. h. nur weil zwei Services im gleichen Namespace laufen, dürfen sie sich **nicht automatisch** gegenseitig aufrufen.

→ Du definierst explizit **RequestAuthentication** und **AuthorizationPolicy**.

---

3 Fine-grained Authorization

Istio entscheidet:

- **Wer** (Service identity / JWT claims)
- **darf** (ALLOW)
- **was** (HTTP-Verb, Pfad, Port)
- **wohin** (Service, Namespace)
- **von wo** (IP, Namespace, Principals)

Beispiel: „Nur `reviews` darf `/ratings/*` aufrufen – aber nur GET, nicht POST."

Das ist Zero-Trust.

---

4 Policy enforcement auf Infrastruktur-Ebene

Alle Regeln gelten **zentral**, unabhängig vom Code des Services. Das heißt:

- Keine ACLs mehr im Code
- Keine Bibliotheken anpassen
- Keine Firewall-Regeln auf Node-Ebene

→ Der Sidecar (oder Waypoint) erzwingt die Security.

---

5 Identity-based Security (nicht IP-basiert)

In Kubernetes ändern sich IPs permanent → unbrauchbar.

Istio arbeitet stattdessen mit **Identitäten**, z. B.:

```
spiffe://cluster.local/ns/bookinfo/sa/productpage
```

→ Diese Identität wird über mTLS geprüft

Merksatz

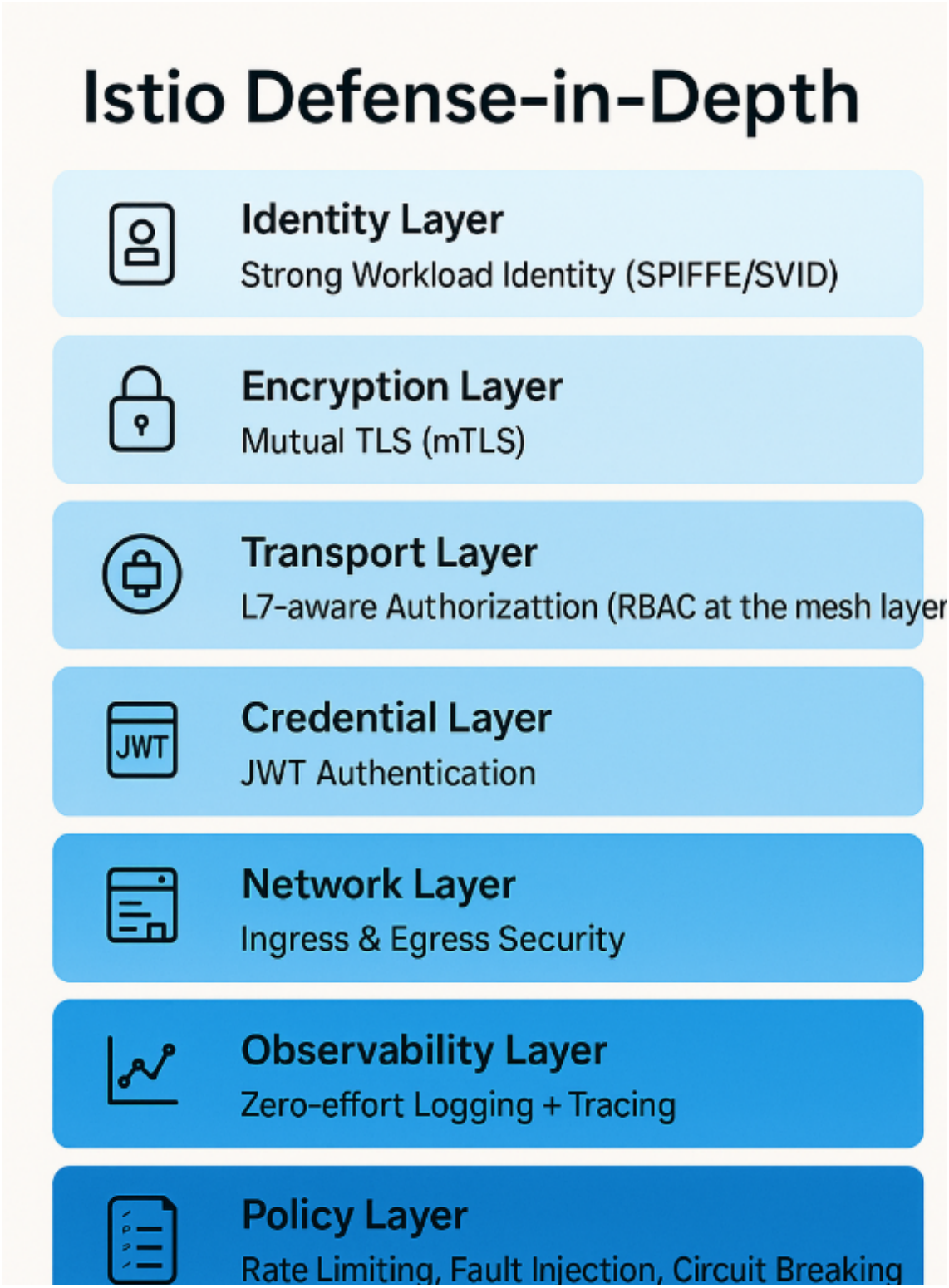
Zero-Trust in Istio: „Authenticate everything, authorize explicitly, trust nobody automatically."

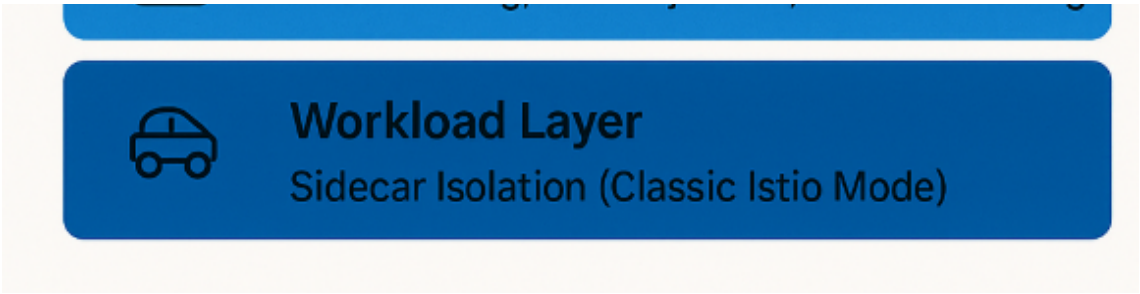
Was ist in istio deep-defense (defense in depth) ?

Istio secures your microservices at **multiple layers**, not just at one point. Each layer is independent, so a failure in Layer A does *not* break Layer B or C.

Let's walk through the layers:

Diagramm





---

1 Identity Layer — Strong Workload Identity (SPIFFE/SVID)

Istio gives each workload a strong cryptographic identity:

- SPIFFE ID: `spiffe://cluster.local/ns/bookinfo/sa/reviews`
- Stored in an mTLS certificate
- Rotated automatically every few minutes/hours

Why this is depth:

✔ Even if the network is compromised, identity is still secure. ✔ Even if someone spoofs an IP, they *cannot spoof* the SPIFFE ID.

---

2 Encryption Layer — Mutual TLS (mTLS)

Across **all service-to-service traffic**, Istio enforces:

- Encryption
- Authentication (client cert)
- Integrity (no tampering)
- Replay protection

mTLS is handled by the sidecar (or waypoint in Ambient).

Why this is depth:

✔ Even if a pod is compromised, an attacker cannot sniff traffic of other services. ✔ Even if a malicious service sends traffic, it must prove its identity cryptographically.

---

3 Transport Layer — L7-aware Authorization (RBAC at the mesh layer)

Istio AuthorizationPolicies allow very fine-grained access control:

Examples:

- Only service A may call service B
- Only `GET /api/v1/orders` is allowed
- Only traffic with a specific JWT can reach a workload
- Deny all by default + whitelist exceptions

Why this is depth:

✔ Even if mTLS is on, you *still* get application-layer RBAC. ✔ Even if a token leaks, policies can enforce additional identity checks.

---

4 Credential Layer — JWT Authentication

Istio allows binding of **end-user identity** (JWT/OIDC) into the mesh:

- Validate JWT
- Verify iss / aud
- Use JWKS URI to fetch signing keys
- Bind user identity to traffic and enforce RBAC

Why this is depth:

✔ Even if an attacker gets inside the cluster, they *still need valid end-user credentials*. ✔ Services cannot impersonate users.

---

5 Network Layer — Ingress & Egress security

Istio can enforce:

Ingress:

- TLS termination
- JWT validation
- Rate limiting
- Web Application Firewall integration (Envoy filters)

Egress:

- Strict allowlists
- TLS origination
- Domain-based restrictions

Why this is depth:

✔ Even if an internal service is compromised, it cannot exfiltrate data (egress controls). ✔ Even if the user-facing API is attacked, gateway protections apply before workload.

---

6 Observability Layer — Zero-effort logging + tracing

Istio gives:

- Distributed tracing
- Access logs (source, destination, identity)
- Metrics per service / route / error code
- mTLS metric visibility

Why this is depth:

✔ Even if someone tries to bypass security, you see clear traces. ✔ Unauthorized or weird traffic stands out immediately.

---

7 Policy Layer — Rate limiting, Fault injection, Circuit breaking

Istio uses Envoy to enforce:

- Retry budgets
- Local or global rate limits

- Timeouts
- Circuit breakers
- Head-of-line attack protection

Why this is depth:

✔ Even if traffic is valid but malicious (DoS), rate limits protect workloads. ✔ Segments failure domains so one service cannot overload another.

---

8 Workload Layer — Sidecar isolation (Classic Istio Mode)

In sidecar mode:

- Workloads can't talk directly to the network
- Envoy controls all outbound and inbound traffic
- Pod cannot bypass mTLS or policies (unless sysadmin-level access)

Why this is depth:

✔ Even if app code is vulnerable, the network guardrail remains intact. ✔ Compromised code ≠ compromised network.

(With Ambient mode, this moves to per-node waypoints; same idea but different architecture.)

Summary — Istio's Defense in Depth

Layer	Purpose	Benefit
mTLS	Encryption + authentication	Prevent sniffing + spoofing
SPIFFE identities	Strong workload identity	Prevent impersonation
AuthZ policies	L7 firewall	Prevent unauthorized access
JWT Rules	End-user identity	Stop token reuse / user spoofing
Ingress/Egress rules	Cluster edge security	Stop data exfil / limit attack surface
Rate limits / CBs	Traffic control	Prevent overload attacks
Sidecar isolation	Network control	Prevent bypass
Observability	Full visibility	Detect attacks early

Together, these layers form a **mesh-level, enterprise-grade security architecture** — which is exactly what *Defense in Depth* means.

Überblick 3 Säulen - Sichere Service-Kommunikation

Überblick

Sichere Kommunikation im Istio Service Mesh basiert auf drei Säulen:

Säule	Ressource	Funktion
Verschlüsselung & Identität	`PeerAuthentication`	mTLS zwischen Services

Zugriffskontrolle	`AuthorizationPolicy`	Wer darf mit wem kommunizieren
End-User-Authentifizierung	`RequestAuthentication`	JWT-Token-Validierung

Erst das Zusammenspiel aller drei ergibt ein vollständiges Zero-Trust-Modell.

---

1. mTLS mit PeerAuthentication

Was passiert unter der Haube?

- Istio stellt jedem Workload automatisch ein **X.509-Zertifikat** aus (SPIFFE-ID)
- Format: `spiffe://<trust-domain>/ns/<namespace>/sa/<service-account>`
- Zertifikate werden über die **Istio CA (istiod)** ausgestellt und regelmäßig rotiert
- Der Traffic zwischen Sidecars (Classic Mode) bzw. ztunnel (Ambient Mode) wird transparent verschlüsselt

Modi

Modus	Verhalten
`PERMISSIVE`	Akzeptiert mTLS **und** Plaintext (Default, gut für Migration)
`STRICT`	**Nur** mTLS erlaubt — Plaintext wird abgelehnt
`DISABLE`	mTLS deaktiviert
`UNSET`	Erbt Einstellung vom Parent (Mesh → Namespace → Workload)

Beispiel: Mesh-weit STRICT

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system
spec:
  mtls:
    mode: STRICT
```

Beispiel: Namespace-Level

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
    mode: STRICT
```

Beispiel: Port-Level Exception

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: my-service
  namespace: production
spec:
  selector:
    matchLabels:
      app: my-service
  mtls:
    mode: STRICT
  portLevelMtls:
    8080:
      mode: PERMISSIVE  # Health-Check-Port ohne mTLS
```

Vererbungshierarchie

```
Mesh-Policy (istio-system)
  └── Namespace-Policy
        └── Workload-Policy (mit selector)
              └── Port-Level Override
```

Die spezifischste Policy gewinnt.

---

2. Zugriffskontrolle mit AuthorizationPolicy

Warum reicht mTLS nicht?

mTLS stellt sicher, **wer** ein Service ist (Identität) und verschlüsselt den Traffic. Aber ohne AuthorizationPolicy kann **jeder** authentifizierte Service **jeden** anderen erreichen. Das ist kein Zero Trust.

Aktionstypen

Action	Verhalten
`ALLOW`	Erlaubt Zugriff wenn Regeln matchen
`DENY`	Verweigert Zugriff wenn Regeln matchen (hat Vorrang vor ALLOW)
`CUSTOM`	Delegiert Entscheidung an externen Authorizer

Evaluierungsreihenfolge

```
1. CUSTOM → 2. DENY → 3. ALLOW → 4. Implizites Verhalten
```

Wenn **keine** AuthorizationPolicy existiert → alles erlaubt.
Wenn **nur** ALLOW-Policies existieren → alles andere wird implizit verweigert.

Beispiel: Nur frontend darf auf backend zugreifen

```yaml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
```

```
metadata:
  name: backend-access
  namespace: production
spec:
  selector:
    matchLabels:
      app: backend
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/production/sa/frontend"]
    to:
    - operation:
        methods: ["GET", "POST"]
        paths: ["/api/*"]
```

Beispiel: Deny-Rule für sensible Endpoints

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-admin
  namespace: production
spec:
  selector:
    matchLabels:
      app: backend
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/admin*"]
    from:
    - source:
        notPrincipals: ["cluster.local/ns/production/sa/admin-service"]
```

Wichtige Source-Felder (L4)

Feld	Beschreibung
`principals`	SPIFFE-ID des aufrufenden Service
`namespaces`	Source-Namespace
`ipBlocks`	Source-IP-Bereiche

Wichtige Operation-Felder (L7)

Feld	Beschreibung
`methods`	HTTP-Methoden (GET, POST, ...)

`paths`	URL-Pfade
`hosts`	Ziel-Hostname
`ports`	Ziel-Ports

> ***Hinweis:*** *L7-Felder (methods, paths, hosts) erfordern im Ambient Mode einen* ***Waypoint Proxy****. Der ztunnel arbeitet nur auf L4.*

---

3. End-User-Authentifizierung mit RequestAuthentication

Wofür?

PeerAuthentication sichert **Service-zu-Service**. RequestAuthentication validiert **End-User-Tokens** (JWTs), die von einem externen Identity Provider stammen (z.B. Keycloak, Auth0, Dex).

Beispiel: JWT-Validierung

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: jwt-auth
  namespace: production
spec:
  selector:
    matchLabels:
      app: backend
  jwtRules:
  - issuer: "https://keycloak.example.com/realms/myrealm"
    jwksUri: "https://keycloak.example.com/realms/myrealm/protocol/openid-
connect/certs"
    forwardOriginalToken: true
```

Verhalten

- Request **ohne** Token → wird **durchgelassen** (RequestAuthentication prüft nur vorhandene Tokens!)
- Request **mit gültigem** Token → wird durchgelassen, Claims werden extrahiert
- Request **mit ungültigem** Token → wird abgelehnt (401)

Kombination mit AuthorizationPolicy

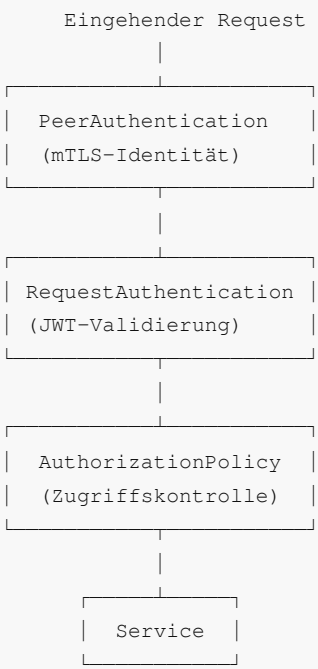
Um Requests **ohne** Token abzulehnen, braucht man eine ergänzende AuthorizationPolicy:

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: production
spec:
  selector:
    matchLabels:
      app: backend
  action: ALLOW
  rules:
```

```
  - from:
    - source:
        requestPrincipals: ["https://keycloak.example.com/realms/myrealm/*"]
    when:
    - key: request.auth.claims[groups]
      values: ["admin", "editor"]
```

> ***Wichtig:*** `RequestAuthentication` *allein erzwingt* ***keine*** *Authentifizierung. Erst zusammen mit einer* `AuthorizationPolicy` *auf* `requestPrincipals` *wird ein Token tatsächlich* ***erforderlich****.*

Zusammenspiel der drei Säulen



Typische Konfigurationsreihenfolge

1. **PeerAuthentication** auf `STRICT` setzen (Mesh-weit oder Namespace)
2. **AuthorizationPolicy** erstellen (wer darf wohin)
3. **RequestAuthentication** hinzufügen (wenn End-User-Tokens validiert werden sollen)
4. **AuthorizationPolicy** mit `requestPrincipals` ergänzen (Token erzwingen + Claims prüfen)

Debugging & Verifizierung

mTLS-Status prüfen (sidecar)

```
## Prüfen ob mTLS aktiv ist
istioctl x describe pod <pod-name> -n <namespace>

## PeerAuthentication-Policies anzeigen (auch für ambient)
kubectl get peerauthentication --all-namespaces
```

```
## TLS-Verbindung verifizieren
istioctl proxy-config secret <pod-name> -n <namespace>
```

AuthorizationPolicy testen

```
## RBAC-Debug-Logs aktivieren
istioctl proxy-config log <pod-name> --level rbac:debug

## Logs prüfen
kubectl logs <pod-name> -c istio-proxy | grep rbac

## Typische Log-Einträge:
## "enforced denied" → Policy hat geblockt
## "enforced allowed" → Policy hat erlaubt
```

JWT-Probleme debuggen

```
## Token manuell prüfen
kubectl exec <pod-name> -c istio-proxy -- \
  curl -s http://localhost:15000/config_dump | \
  jq '.configs[] | select(.["@type"] | contains("listener"))'

## JWT-Rejection im Access-Log
## Response-Flag "UAEX" = Unauthorized External (JWT ungültig)
```

Classic Mode vs. Ambient Mode

Aspekt	Classic (Sidecar)	Ambient (ztunnel + Waypoint)
mTLS	Sidecar-zu-Sidecar	ztunnel-zu-ztunnel
L4-Policy	Im Sidecar	Im ztunnel
L7-Policy	Im Sidecar	**Nur mit Waypoint Proxy**
RequestAuthentication	Im Sidecar	**Nur mit Waypoint Proxy**
Zertifikate	Pro Pod	Pro Node (ztunnel) / Pro SA (Waypoint)

> ***Praxis-Tipp:*** *Im Ambient Mode zunächst nur L4-Policies (principals, namespaces) nutzen. Für L7 (paths, methods, JWT) einen Waypoint deployen.*

Übung Säule 1 (Sidecar): mTLS prüfen und aktivieren/deaktivieren

> ***Sidecar-Fassung.*** *Diese Übung gilt für den Sidecar-Mode (Envoy-Proxy in jedem Pod). Für den Ambient-Mode (ztunnel, Waypoint) gibt es eine eigene Fassung: 02-check-mtls-enable-disable-ambient.md.*

Ziel

mTLS-Status im Mesh inspizieren, PeerAuthentication-Policies anwenden und die Auswirkungen auf den Traffic anhand der Bookinfo-Anwendung verifizieren.

Voraussetzungen

- Kubernetes-Cluster mit Istio im **Sidecar-Mode** (für Ambient siehe die Ambient-Fassung)
- `istioctl` installiert
- Namespace `bookinfo` mit Istio-Injection:

```
kubectl create ns bookinfo
kubectl label ns bookinfo istio-injection=enabled
```

Schritt 1: Bookinfo deployen

- Bereit deployed

Architektur-Überblick:

```
productpage → details
            → reviews-v1 (keine Sterne)
            → reviews-v2 → ratings (schwarze Sterne)
            → reviews-v3 → ratings (rote Sterne)
```

Funktionstest

```
kubectl -n bookinfo exec deploy/productpage-v1 -- \
  curl -s productpage:9080/productpage | grep -o "<title>.*</title>"
```

Erwartete Ausgabe: `<title>Simple Bookstore App</title>`

Schritt 2: Aktuellen mTLS-Status prüfen

2a) Mit istioctl

```
## mTLS-Status der productpage prüfen
istioctl x describe pod \
  $(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].metadata.name}') \
  -n bookinfo

## mTLS-Status von ratings prüfen
istioctl x describe pod \
  $(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}') \
  -n bookinfo
```

Frage: Welchen mTLS-Mode zeigt Istio standardmäßig an?

2b) Bestehende PeerAuthentication-Policies anzeigen

```
## Mesh-weit (istio-system)
kubectl get peerauthentication -A
```

```
## Im Namespace
kubectl get peerauthentication -n bookinfo
```

> ***Hinweis:*** *PeerAuthentication ist eine* ***Namespaced Resource*** *(* `NAMESPACED=true` *). Die Geltung hängt davon ab, wo sie erstellt wird:*
>
> - *Im* ***App-Namespace*** *(z.B.* `bookinfo` *) → gilt nur für diesen Namespace*
> - *In* ***`istio-system`*** *→ wirkt als Mesh-weite Policy (quasi cluster-weit)*
>
> *Es gibt keine echte Cluster-Resource – die „globale" Wirkung entsteht nur durch die Konvention, dass Istio eine PeerAuthentication in* `istio-system` *als Mesh-Default behandelt.*
>
> ```
> kubectl api-resources | grep peerauthentication
> # peerauthentications   pa   security.istio.io/v1   true   PeerAuthentication
> ```
>
> ***Wichtig:*** *Wenn weder im App-Namespace noch in* `istio-system` *eine PeerAuthentication gesetzt ist, zeigt* `kubectl get peerauthentication -A` *ein leeres Ergebnis (* `No resources found` *). In diesem Fall greift automatisch* ***PERMISSIVE*** *– dieser Default ist im Istio-Quellcode hardcoded und nicht über die meshConfig konfigurierbar. Prüfen lässt sich der effektive Mode mit:*
>
> ```
> istioctl x describe pod <pod-name> -n bookinfo
> # Ausgabe enthält: "mTLS mode: PERMISSIVE"
> ```

2c) Verifizierung über Request-Header

```
## sleep als Debug-Client deployen
kubectl -n bookinfo apply -f https://raw.githubusercontent.com/istio/istio/release-
1.25/samples/sleep/sleep.yaml
kubectl -n bookinfo wait --for=condition=ready pod -l app=sleep --timeout=60s

## Header bei ratings prüfen - XFCC-Header zeigt mTLS-Identität
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -v http://ratings:9080/ratings/0 2>&1 | grep -i x-forwarded-client-cert
```

Frage: Welche `X-Forwarded-Client-Cert`-Header sehen Sie? Was bedeutet das?

Schritt 3: mTLS auf STRICT setzen

3a) Namespace-weite STRICT Policy

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: bookinfo
spec:
  mtls:
    mode: STRICT
EOF
```

3b) Verifizieren – Kommunikation im Mesh funktioniert weiterhin

```
## Falls nicht vorher schon erstellt
kubectl -n bookinfo apply -f https://raw.githubusercontent.com/istio/istio/release-
1.25/samples/sleep/sleep.yaml

## productpage kann weiterhin details und reviews erreichen
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage:9080/productpage

## sleep kann ratings erreichen
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings:9080/ratings/0
```

3c) Zugriff ohne Sidecar testen

Starten Sie einen Pod **ohne** Sidecar und versuchen Sie, die Bookinfo-Services zu erreichen:

```
## Pod ohne Sidecar in einem Namespace ohne Injection
kubectl create ns no-mesh
kubectl -n no-mesh apply -f https://raw.githubusercontent.com/istio/istio/release-
1.25/samples/sleep/sleep.yaml
kubectl -n no-mesh wait --for=condition=ready pod --all --timeout=60s

## Zugriff auf productpage versuchen
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage

## Zugriff auf ratings versuchen
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0
```

Erwartetes Ergebnis: Die Verbindungen schlagen fehl, da der Client kein mTLS-Zertifikat präsentiert. curl gibt den HTTP-Statuscode 000 aus (keine HTTP-Antwort) und bricht mit Exit Code 56 (CURLE_RECV_ERROR) ab – Envoy resettet die Verbindung bereits auf TLS-Ebene, bevor es zur HTTP-Kommunikation kommt.

Schritt 4: mTLS auf PERMISSIVE setzen

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: bookinfo
spec:
  mtls:
    mode: PERMISSIVE
EOF
```

Verifizieren

```
## Aus dem Mesh – funktioniert (mit mTLS)
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage:9080/productpage

## Von außerhalb des Mesh – funktioniert jetzt auch (ohne mTLS)
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage
```

Frage: Warum ist PERMISSIVE der Istio-Default und wann ist STRICT besser?

Schritt 5: Workload-spezifisches mTLS

mTLS lässt sich gezielt für einzelne Services konfigurieren. Hier setzen wir nur den ratings-Service auf STRICT, während der Rest PERMISSIVE bleibt:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratings-strict
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  mtls:
    mode: STRICT
EOF
```

Verifizieren

```
## ratings ist jetzt STRICT – von außerhalb nicht erreichbar
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0

## productpage ist weiterhin PERMISSIVE – von außerhalb erreichbar
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage

## reviews (im Mesh) kann ratings weiterhin erreichen
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings:9080/ratings/0
```

Frage: In welcher Reihenfolge wertet Istio PeerAuthentication-Policies aus?

```
## Aufräumen
kubectl -n bookinfo delete peerauthentication ratings-strict
```

Schritt 6: Port-Level mTLS (Bonus)

mTLS lässt sich auch pro Port konfigurieren:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratings-per-port
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  mtls:
    mode: STRICT
  portLevelMtls:
    9080:
      mode: PERMISSIVE
EOF
```

Frage: Für welches Szenario ist Port-Level-Konfiguration sinnvoll?

```
kubectl -n bookinfo delete peerauthentication ratings-per-port
```

Schritt 7: mTLS mit DestinationRule steuern

Die **Client-Seite** wird über DestinationRules konfiguriert:

```
kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: ratings-mtls
  namespace: bookinfo
spec:
  host: ratings.bookinfo.svc.cluster.local
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
EOF
```

Hinweis: Seit Istio 1.x sendet der Sidecar automatisch mTLS, wenn das Ziel einen Sidecar hat (Auto-mTLS). Eine explizite DestinationRule ist nur nötig, wenn man Auto-mTLS überschreiben will.

```
kubectl -n bookinfo delete destinationrule ratings-mtls
```

Schritt 8: mTLS-Traffic mit tcpdump verifizieren (Bonus)

Terminal 1 – Listener auf dem ratings-Pod:

```
kubectl -n bookinfo debug -it \
  $(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}') \
  --image=nicolaka/netshoot -- tcpdump -i eth0 -A -c 20 port 9080
```

Terminal 2 – Request auslösen:

```
kubectl -n bookinfo exec deploy/sleep -- curl -s ratings:9080/ratings/0
```

Frage: Sehen Sie Klartext oder verschlüsselte Daten im tcpdump-Output?

Aufräumen

```
kubectl -n bookinfo delete pod sleep
kubectl -n bookinfo delete sa sleep
kubectl delete ns no-mesh
```

Zusammenfassung

Konzept	Beschreibung
PeerAuthentication	Server-Seite: akzeptiert mTLS (STRICT) oder beides (PERMISSIVE)
DestinationRule	Client-Seite: sendet mTLS (ISTIO_MUTUAL) oder Klartext (DISABLE)
Auto-mTLS	Istio aktiviert mTLS automatisch, wenn beide Seiten im Mesh sind
PERMISSIVE	Default – erlaubt Migration ohne Downtime
STRICT	Zero-Trust – nur mTLS-Traffic erlaubt
Policy-Hierarchie	Mesh-weit → Namespace → Workload (spezifischste gewinnt)

Lösungen

▶ Antworten zu den Fragen

Übung Säule 1 (Ambient): mTLS prüfen und aktivieren/deaktivieren (ztunnel, HBONE, PeerAuthentication)

> *Das ist die **Ambient**-Fassung der Uebung. Die Fassung fuer Sidecars steht in 02-check-mtls-enable-disable.md.*

Ziel

Du pruefst, ob der Verkehr in Bookinfo mit mTLS laeuft, und steuerst mit `PeerAuthentication`, ob Clients **ausserhalb** des Mesh noch durchkommen. Dabei lernst du, dass im Ambient-Mode der **ztunnel** das mTLS macht und nicht mehr ein Sidecar im Pod.

Hintergrund: Was ist im Ambient-Mode anders?

	Sidecar	Ambient
Wer macht mTLS?	Envoy-Sidecar in jedem Pod	**ztunnel** (ein Pod pro Node)
Tunnel	mTLS direkt zwischen den Sidecars	**HBONE** (mTLS) ueber Port **15008**
Identitaet	SPIFFE-ID pro ServiceAccount	dieselbe SPIFFE-ID pro ServiceAccount
Status pruefen	`istioctl x describe pod`	`istioctl ztunnel-config` und ztunnel-Log
Pod hat	2 Container (App + `istio-proxy`)	1 Container (nur die App)
Client ohne Mesh	wird mit STRICT abgelehnt	wird mit STRICT abgelehnt

Merksatz: **Zwischen Pods im Mesh laeuft immer mTLS.** `PeerAuthentication` entscheidet nur, ob zusaetzlich noch Klartext von Clients **ausserhalb** des Mesh angenommen wird (PERMISSIVE) oder nicht (STRICT).

Voraussetzungen

- Uebung Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo` mit Waypoint, alle Pods `1/1 Running`).
- `istioctl` ist installiert.
- Im Namespace gibt es keine PeerAuthentication und keine AuthorizationPolicy:

```
kubectl get peerauthentication -A
kubectl -n bookinfo get authorizationpolicy
```

Erwartet: `No resources found`. Sonst vorher loeschen, z. B. `kubectl -n bookinfo delete peerauthentication --all`

Der Namespace muss im Ambient-Mode sein (kein `istio-injection`!):

```
kubectl get ns bookinfo --show-labels
```

Erwartet: `istio.io/dataplane-mode=ambient` (und `istio.io/use-waypoint=waypoint`)

Schritt 1: Vorbereitung und Funktionstest

Ein kleiner Client im Mesh:

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo rollout status deploy/sleep
```

```
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s productpage:9080/productpage | grep -o "<title>.*</title>"
```

Erwartete Ausgabe: `<title>Simple Bookstore App</title>`

Die Pods haben nur **einen** Container, es gibt keinen Sidecar:

```
kubectl -n bookinfo get pods
```

Schritt 2: Aktuellen mTLS-Status pruefen

2a) Warum `istioctl x describe` hier nicht hilft

In der Sidecar-Uebung haben wir den Status mit `istioctl x describe pod` gelesen. Probiere es hier aus:

```
istioctl x describe pod \
  $(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].metadata.name}') \
  -n bookinfo
```

Du siehst eine Meldung wie:

```
WARNING: productpage-v1-... is not part of mesh; no Istio sidecar
Error: failed to execute command on sidecar: ...
```

Frage: Ist der Pod wirklich nicht im Mesh? Woher kommt die Meldung?

`describe` fragt den Envoy-Sidecar im Pod ab (Port 15000). Im Ambient-Mode gibt es den nicht. Die Meldung "not part of mesh" ist deshalb **irrefuehrend**: Der Pod ist im Mesh, nur ohne Sidecar.

2b) Wird der Pod vom ztunnel bedient? (`ztunnel-config workload`)

```
istioctl ztunnel-config workload | grep -E "NAMESPACE|bookinfo"
```

Du siehst in der Spalte `PROTOCOL` fuer die Bookinfo-Pods `HBONE` und in der Spalte `WAYPOINT` den Namen `waypoint`. Der Waypoint selbst und das Gateway stehen mit `TCP` da.

Frage: Was bedeutet `HBONE`, was bedeutet `TCP`?

2c) Welche Zertifikate (SPIFFE-IDs) hat der ztunnel?

`ztunnel-config certificate` braucht den Namen des ztunnel-Pods. Wir nehmen den ztunnel auf dem Node, auf dem `ratings` laeuft:

```
NODE=$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
echo $ZT
```

```
istioctl ztunnel-config certificate $ZT.istio-system | grep -E "CERTIFICATE|bookinfo"
```

Du siehst je ServiceAccount ein `Leaf`-Zertifikat (kurz gueltig) und das gemeinsame `Root`-Zertifikat, z. B.:

```
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-ratings   Leaf   Available   true   ...
```

Frage: Warum stehen hier nur die ServiceAccounts, deren Pods auf diesem Node laufen?

2d) Bestehende PeerAuthentication-Policies anzeigen

```
## Ueberall im Cluster
kubectl get peerauthentication -A
```

```
## Im Namespace
kubectl -n bookinfo get peerauthentication
```

> ***Hinweis:*** *`PeerAuthentication` ist eine **Namespaced Resource**. Die Geltung haengt davon ab, wo sie liegt:*
>
> - *Im **App-Namespace** (z. B. `bookinfo`) gilt sie fuer diesen Namespace (ohne `selector`) oder fuer einzelne Workloads (mit `selector`).*
> - *In **`istio-system`** wirkt sie als Mesh-weiter Default.*
>
> *Ist nirgends eine PeerAuthentication gesetzt (`No resources found`), nimmt der ztunnel auch Klartext von Clients ausserhalb des Mesh an. Das pruefen wir in Schritt 3.*

2e) Verifizierung ueber das ztunnel-Log

Im Sidecar-Mode haben wir den Header `X-Forwarded-Client-Cert` angeschaut. Im Ambient-Mode steht die Identitaet im **ztunnel-Log**.

Zuerst einen Aufruf ausloesen:

```
kubectl -n bookinfo exec deploy/sleep -- curl -s -o /dev/null ratings:9080/ratings/0
```

Dann den ztunnel auf dem Node des **Aufrufers** (`sleep`) anschauen:

```
NODE_SLEEP=$(kubectl -n bookinfo get pod -l app=sleep -o
jsonpath='{.items[0].spec.nodeName}')
ZT_SLEEP=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE_SLEEP -o jsonpath='{.items[0].metadata.name}')
kubectl -n istio-system logs $ZT_SLEEP --tail=100 | grep 'src.workload="sleep' | tail
-1
```

Die Zeile sieht gekuerzt so aus:

```
info access connection complete src.workload="sleep-..." src.namespace="bookinfo"
  src.identity="spiffe://cluster.local/ns/bookinfo/sa/sleep"
  dst.addr=192.168.x.x:15008 dst.hbone_addr=10.x.x.x:9080
dst.service="ratings.bookinfo.svc.cluster.local"
  dst.workload="waypoint-..."
dst.identity="spiffe://cluster.local/ns/bookinfo/sa/waypoint" ...
```

Fragen:

- Welche SPIFFE-ID hat der Aufrufer (`src.identity`)?
- Auf welchem Port geht die Verbindung raus (`dst.addr`)? Was ist das fuer ein Port?
- Warum ist das Ziel (`dst.workload`) der **Waypoint** und nicht `ratings`?

Schritt 3: mTLS auf STRICT setzen

3a) Vorher: Client ohne Mesh

Ein Pod in einem Namespace **ohne** Ambient-Label ist nicht im Mesh:

```
kubectl create ns no-mesh
kubectl -n no-mesh apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n no-mesh rollout status deploy/sleep
```

```
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage

kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0
```

Erwartet: zweimal `200`. Ohne Policy kommt auch Klartext von ausserhalb durch.

3b) Namespace-weite STRICT Policy

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: bookinfo
spec:
  mtls:
    mode: STRICT
EOF
```

3c) Verifizieren - die Kommunikation im Mesh funktioniert weiterhin

```
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage:9080/productpage

kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings:9080/ratings/0
```

Erwartet: zweimal `200`.

3d) Zugriff ohne Mesh testen

```
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage

kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0
```

Erwartetes Ergebnis: Beide Aufrufe schlagen fehl. curl gibt `000` aus (keine HTTP-Antwort) und bricht mit einem Fehler ab (Exit-Code 52 oder 56, je nachdem, wie die Verbindung geschlossen wird). Der ztunnel des Ziel-Pods schliesst die Verbindung, bevor es zu HTTP kommt.

3e) Warum? Der Grund steht im ztunnel-Log des Ziels

```
NODE=$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
kubectl -n istio-system logs $ZT --tail=50 | grep 'src.namespace="no-mesh"' | tail -1
```

Am Ende der Zeile steht:

```
direction="inbound" ... error="connection closed due to policy rejection: explicitly
denied by: istio-system/istio_converted_static_strict"
```

Frage: Welche Seite lehnt ab, die Quelle oder das Ziel? Woran siehst du das (`direction`)?

Schritt 4: mTLS auf PERMISSIVE setzen

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: bookinfo
spec:
  mtls:
    mode: PERMISSIVE
EOF
```

Verifizieren

```
## Aus dem Mesh - funktioniert (mit mTLS)
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage:9080/productpage

## Von ausserhalb des Mesh - funktioniert jetzt wieder (ohne mTLS)
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage
```

Frage: Ist der Aufruf aus dem Mesh bei PERMISSIVE verschluesselt? Pruefe es mit dem ztunnel-Log aus Schritt 2e (`dst.addr` Port 15008).

Schritt 5: Workload-spezifisches mTLS

Wir setzen nur `ratings` auf STRICT, der Rest bleibt PERMISSIVE:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratings-strict
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  mtls:
    mode: STRICT
EOF
```

Verifizieren

```
## ratings ist jetzt STRICT - von ausserhalb nicht erreichbar
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0

## productpage ist weiterhin PERMISSIVE - von ausserhalb erreichbar
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage

## Aus dem Mesh kommt ratings weiterhin an
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings:9080/ratings/0
```

Frage: In welcher Reihenfolge wertet Istio PeerAuthentication-Policies aus?

> ***Wichtig:*** *Lass* `ratings-strict` ***stehen****, wir brauchen die Policy gleich in Schritt 6.*

Schritt 6: Port-Level-mTLS (Bonus)

mTLS laesst sich auch pro Port konfigurieren:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratings-per-port
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  mtls:
    mode: STRICT
  portLevelMtls:
    9080:
      mode: PERMISSIVE
EOF
```

Die Policy `ratings-strict` aus Schritt 5 ist noch da. Warte ca. 10 Sekunden und teste von ausserhalb des Mesh:

```
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0
```

Erwartet: der Aufruf schlaegt fehl. curl gibt `000` aus (Exit-Code 52 oder 56), obwohl Port 9080 in `ratings-per-port` auf PERMISSIVE steht.

Frage: Warum greift die Ausnahme fuer Port 9080 nicht? (Tipp: Schau dir die Policies sortiert nach Alter an.)

```
kubectl -n bookinfo get peerauthentication --sort-by=.metadata.creationTimestamp
```

Jetzt loeschen wir die aeltere Policy `ratings-strict`, warten wieder ca. 10 Sekunden und testen noch einmal:

```
kubectl -n bookinfo delete peerauthentication ratings-strict
```

```
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings.bookinfo:9080/ratings/0
```

Erwartet: `200`, weil jetzt nur noch `ratings-per-port` gilt und Port 9080 dort auf PERMISSIVE steht, obwohl der Workload sonst STRICT ist.

> ***Info: Mehrere PeerAuthentications auf denselben Workload***
>
> *Passen mehrere workload-spezifische PeerAuthentications (mit `selector`) auf denselben Workload, gilt nur **eine**: die **aelteste**. Die anderen werden ignoriert, sie werden nicht zusammengefuehrt. Eine aeltere, reine STRICT-Policy ueberstimmt deshalb eine juengere mit Port-Ausnahme, unabhaengig davon, welche "strenger" ist.*
>
> *Aus der Istio-Doku (Abschnitt "Selector field"):*
>
> - *"When more than one workload-specific peer authentication policy matches, Istio picks the oldest one."*
> - *"When you configure multiple mesh- or namespace-wide peer authentication policies for the same mesh or namespace, Istio ignores the newer policies."*
>
> *Quelle: https://istio.io/latest/docs/concepts/security/#selector-field*
>
> *Zu `portLevelMtls` sagt die PeerAuthentication-Referenz: "The port refers to the port of the workload, not the port of the Kubernetes service." Eingetragen wird also der Port des Containers (`targetPort`), nicht der des Service. Bei `ratings` sind beide `9080`.*
>
> *Praxis: Pro Workload nur **eine** PeerAuthentication mit `selector` anlegen und die Port-Ausnahme dort mit `portLevelMtls` setzen.*

Frage: Fuer welches Szenario ist Port-Level-Konfiguration sinnvoll?

```
kubectl -n bookinfo delete peerauthentication ratings-per-port
```

Schritt 7: Client-Seite - was aendert sich im Ambient-Mode?

mTLS auf der Client-Seite passiert in **beiden** Modi automatisch (Auto-mTLS): Im Sidecar-Mode sendet der Sidecar des Clients mTLS, sobald das Ziel einen Sidecar hat. Im Ambient-Mode baut der ztunnel der Quelle zu jedem Ziel im Mesh den HBONE-Tunnel auf.

Der Unterschied ist der **Hebel zum Ueberschreiben**:

	Sidecar	Ambient
mTLS zum Ziel	automatisch (Auto-mTLS)	automatisch (HBONE durch den ztunnel)
Ueberschreiben per `DestinationRule` (`tls.mode: DISABLE` oder `ISTIO_MUTUAL`)	moeglich, der Sidecar des Clients richtet sich danach	**wirkungslos**, mTLS ist fest

Du kannst es nachstellen: `ratings` auf STRICT setzen und dazu eine `DestinationRule` mit `tls.mode: DISABLE` anlegen:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratings-strict
```

```
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  mtls:
    mode: STRICT
EOF

kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: ratings-disable
  namespace: bookinfo
spec:
  host: ratings.bookinfo.svc.cluster.local
  trafficPolicy:
    tls:
      mode: DISABLE
EOF
```

```
kubectl -n bookinfo exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" ratings:9080/ratings/0
```

Erwartet: `200`. Der Aufruf kommt trotz `DISABLE` und STRICT durch und laeuft weiter ueber Port 15008 (ztunnel-Log wie in Schritt 2e). Im Sidecar-Mode wuerde `DISABLE` Klartext senden, und STRICT wuerde ihn ablehnen.

```
kubectl -n bookinfo delete destinationrule ratings-disable
kubectl -n bookinfo delete peerauthentication ratings-strict
```

Frage: Wo wuerdest du im Ambient-Mode eine `DestinationRule` trotzdem noch brauchen? (Tipp: Waypoint, Layer 7)

Aufraeumen

```
kubectl -n bookinfo delete peerauthentication --all
kubectl -n bookinfo delete deploy sleep
kubectl -n bookinfo delete svc sleep
kubectl -n bookinfo delete sa sleep
kubectl delete ns no-mesh
```

Zusammenfassung

Konzept	Beschreibung
ztunnel	Macht mTLS fuer alle Pods auf seinem Node (HBONE, Port 15008)
Status pruefen	`istioctl ztunnel-config workload` (`HBONE`/`TCP`), `ztunnel-config certificate`, ztunnel-Log
`x describe`	Nur fuer Sidecars, im Ambient-Mode nutzlos und irrefuehrend

PeerAuthentication	Entscheidet, ob Klartext von Clients ausserhalb des Mesh angenommen wird (PERMISSIVE) oder nicht (STRICT)
Ohne Policy	Clients ausserhalb des Mesh kommen durch (wie PERMISSIVE)
STRICT	Zero-Trust: nur mTLS, der ztunnel des Ziels lehnt Klartext ab
Client-Seite	mTLS ist automatisch (wie bei Sidecars, dort Auto-mTLS). Neu im Ambient-Mode: Der ztunnel der Quelle baut den Tunnel, `tls.mode` in einer `DestinationRule` ist wirkungslos
Policy-Hierarchie	Mesh-weit (`istio-system`) -> Namespace -> Workload (die spezifischste gewinnt)

Loesungen

▶ Antworten zu den Fragen

Übung Säule 2 + 3 (Sidecar): Authentication und Authorization mit JWT und RBAC

> *Das ist die **Sidecar**-Fassung der Übung. Die Fassung für Ambient (Waypoint, `targetRefs`) steht in gateway-api/12-jwt-rbac-ambient.md.*

Grundlagen

Was ist JWKS ?

JWKS = JSON Web Key Set

Ein JSON-Dokument, das die **öffentlichen Schlüssel** enthält, mit denen JWTs (JSON Web Tokens) verifiziert werden können.

Typisches Format:

```
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "abc123",
      "use": "sig",
      "n": "0vx7agoebGc...",
      "e": "AQAB"
    }
  ]
}
```

Wie es funktioniert:

1. Ein Identity Provider (Keycloak, Auth0, Google etc.) signiert JWTs mit seinem **Private Key**
2. Der JWKS-Endpunkt (z.B. `https://idp.example.com/.well-known/jwks.json`) stellt die zugehörigen **Public Keys** bereit
3. Istio (oder ein anderer Verifier) holt sich die Keys von dort und prüft damit die JWT-Signatur

In Istio konkret — `RequestAuthentication` :

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
```

```
spec:
  jwtRules:
  - issuer: "https://idp.example.com"
    jwksUri: "https://idp.example.com/.well-known/jwks.json"
```

Istio cached die Keys und rotiert automatisch mit, wenn der IDP neue Keys veröffentlicht (via `kid` — Key ID).

Kurz: JWKS ist der standardisierte Weg, wie ein JWT-Verifier an die Public Keys kommt, ohne sie manuell konfigurieren zu müssen.

Begriffe

`kid` (Key ID) — Eindeutige ID des Schlüssels. Damit weiß der Verifier, welcher Key aus dem Set zum JWT passt (das JWT hat `kid` im Header).

`use` (Public Key Use) — Wofür der Key gedacht ist:

- `"sig"` = Signaturprüfung (Standard bei JWT)
- `"enc"` = Verschlüsselung

`n` (Modulus) — Der RSA-Modulus, Base64url-kodiert. Das ist der mathematische Kern des öffentlichen RSA-Schlüssels.

`e` (Exponent) — Der RSA-Exponent, Base64url-kodiert. Fast immer `"AQAB"` (= 65537 dezimal), der Standard-Public-Exponent.

Zusammenspiel: `n` und `e` zusammen **sind** der öffentliche RSA-Schlüssel. Damit kann die JWT-Signatur mathematisch verifiziert werden, ohne den Private Key zu kennen.

Step 0: Preparation

```
cd
mkdir -p manifests/jwt
cd manifests/jwt
```

Step 1: Create http-bin and curl workloads

```
kubectl create ns foo
kubectl apply -f <(istioctl kube-inject -f ~/istio/samples/httpbin/httpbin.yaml) -n
foo
kubectl apply -f <(istioctl kube-inject -f ~/istio/samples/curl/curl.yaml) -n foo
```

Step 2: Can we connect ?

```
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl http://httpbin.foo:8000/ip -sS -o
/dev/null -w "%{http_code}\n"
```

Step 3: Create a RequestAuthentication

```
nano 01-ra.yml
```

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: "jwt-example"
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
  jwtRules:
  - issuer: "testing@secure.istio.io"
    jwksUri: "https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/jwks.json"
```

```
kubectl apply -f .
```

Step 4: Check with an invalid jwt

- Invalid is restricted, so we do not get access (no 200)

```
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -H "Authorization: Bearer invalidToken" -w "%{http_code}\n"
```

Step 5: But: without a jwt -> its work

- ... Because ! -> There is no AuthorizationPolicy

```
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -w "%{http_code}\n"
```

Step 6: We create an AuthorizationPolicy

> *[!NOTE] requestPrincipal set to testing@secure.istio.io/testing@secure.istio.io. Istio constructs the requestPrincipal by combining the iss and sub of the JWT token with a / separator.*
>
> *iss = issuer sub = subject*

```
nano 02-ap.yml
```

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
  action: ALLOW
  rules:
```

```
  - from:
    - source:
       requestPrincipals: ["testing@secure.istio.io/testing@secure.istio.io"]
```

```
kubectl apply -f 02-ap.yml
```

Step 7: Test access

- jwt consists of 3 parts
  - HEADER / PAYLOAD / SIGNATURE
  - Each part is base64 encoded
- cut -d. -f2 -> gets the 2nd part -> the payload

```
## This is the way we get the token
TOKEN=$(curl https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/demo.jwt -s) && echo "$TOKEN" | cut -d '.' -f2 - |
base64 --decode
```

```
echo $TOKEN
```

```
## Testing with allowed jwt
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -H "Authorization: Bearer $TOKEN" -w "%{http_code}\n"
```

```
## Testing without a jwt
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -w "%{http_code}\n"
```

Step 8: Update AuthorizationPolicy also needing a specific group

```
nano 02-ap-group.yml
```

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
       requestPrincipals: ["testing@secure.istio.io/testing@secure.istio.io"]
    when:
```

```
    - key: request.auth.claims[groups]
      values: ["group1"]
```

```
kubectl apply -f 02-ap-group.yml
```

Step 9: get token included a claim for a group

- Get the JWT that sets the groups claim to a list of strings: group1 and group2:

```
TOKEN_GROUP=$(curl https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/groups-scope.jwt -s) && echo "$TOKEN_GROUP" | cut -d
'.' -f2 - | base64 --decode
```

Step 10: Test it with that token (so group1 must be included)

```
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -H "Authorization: Bearer $TOKEN_GROUP" -w "%{http_code}\n"
```

Step 11: Test with a token without group included

- We use that TOKEN before, which had not group

```
kubectl exec "$(kubectl get pod -l app=curl -n foo -o jsonpath=
{.items..metadata.name})" -c curl -n foo -- curl "http://httpbin.foo:8000/headers" -sS
-o /dev/null -H "Authorization: Bearer $TOKEN" -w "%{http_code}\n"
```

Step 12: Cleanup

```
kubectl delete namespace foo
```

Reference:

- https://istio.io/latest/docs/tasks/security/authorization/authz-jwt/
- JWT Debugger: https://www.jwt.io/

Übung Säule 2 + 3 (Ambient): Authentication und Authorization mit JWT und RBAC (Waypoint, targetRefs)

> *Das ist die **Ambient**-Fassung der Übung. Die Fassung für Sidecars steht in 12-jwt-rbac.md.*

Was ist im Ambient-Mode anders?

Die JWT-Prüfung (`RequestAuthentication`) und Regeln auf JWT-Claims (`requestPrincipals` , `request.auth.claims`) sind **Layer-7-Funktionen** (HTTP). Der **ztunnel** kann im Ambient-Mode nur Layer 4. Layer 7 erledigt der **Waypoint**.

	Sidecar	Ambient
Wer prüft das JWT?	Envoy-Sidecar im Pod von httpbin	**Waypoint**
Policy zuordnen	`selector` (Pod-Label)	**`targetRefs`** (Service oder Waypoint-Gateway)

mit		
Workloads	`istioctl kube-inject`	kein Inject, der Namespace hat `istio.io/dataplane-mode=ambient`

> *[!WARNING] Nimmst du wie in der Sidecar-Übung `selector`, wirkt die Policy **nicht wie gewünscht**: Eine `RequestAuthentication` mit `selector` prüft kein JWT (alles liefert `200`), und eine `AuthorizationPolicy` mit `selector` und `requestPrincipals` wird vom ztunnel ausgewertet. Der ztunnel kennt keine HTTP-Attribute, lässt die Regel weg, und der Service ist für alle gesperrt (`503`). Das probierst du am Ende in "Step 13" selbst aus.*

Voraussetzungen

- Übung Bookinfo demo in ambient ausrollen ist durchgeführt (Namespace `bookinfo` mit Waypoint, alle Pods `Running`).
- Der Namespace ist im Ambient-Mode **und** nutzt den Waypoint:

```
kubectl get ns bookinfo --show-labels
```

Erwartet: `istio.io/dataplane-mode=ambient` und `istio.io/use-waypoint=waypoint`

- Es gibt noch keine JWT-Policies im Namespace:

```
kubectl -n bookinfo get requestauthentication,authorizationpolicy
```

Erwartet: `No resources found`. Sonst vorher löschen.

Grundlagen

Was ist JWKS ?

JWKS = JSON Web Key Set

Ein JSON-Dokument, das die **öffentlichen Schlüssel** enthält, mit denen JWTs (JSON Web Tokens) verifiziert werden können.

Typisches Format:

```
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "abc123",
      "use": "sig",
      "n": "0vx7agoebGc...",
      "e": "AQAB"
    }
  ]
}
```

Wie es funktioniert:

1. Ein Identity Provider (Keycloak, Auth0, Google etc.) signiert JWTs mit seinem **Private Key**

2. Der JWKS-Endpunkt (z.B. `https://idp.example.com/.well-known/jwks.json`) stellt die zugehörigen **Public Keys** bereit
3. Istio (oder ein anderer Verifier) holt sich die Keys von dort und prüft damit die JWT-Signatur

In Istio konkret — `RequestAuthentication`:

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
spec:
  jwtRules:
  - issuer: "https://idp.example.com"
    jwksUri: "https://idp.example.com/.well-known/jwks.json"
```

Istio cached die Keys und rotiert automatisch mit, wenn der IDP neue Keys veröffentlicht (via `kid` — Key ID).

Kurz: JWKS ist der standardisierte Weg, wie ein JWT-Verifier an die Public Keys kommt, ohne sie manuell konfigurieren zu müssen.

Begriffe

`kid` (Key ID) — Eindeutige ID des Schlüssels. Damit weiß der Verifier, welcher Key aus dem Set zum JWT passt (das JWT hat `kid` im Header).

`use` (Public Key Use) — Wofür der Key gedacht ist:

- `"sig"` = Signaturprüfung (Standard bei JWT)
- `"enc"` = Verschlüsselung

`n` (Modulus) — Der RSA-Modulus, Base64url-kodiert. Das ist der mathematische Kern des öffentlichen RSA-Schlüssels.

`e` (Exponent) — Der RSA-Exponent, Base64url-kodiert. Fast immer `"AQAB"` (= 65537 dezimal), der Standard-Public-Exponent.

Zusammenspiel: `n` und `e` zusammen **sind** der öffentliche RSA-Schlüssel. Damit kann die JWT-Signatur mathematisch verifiziert werden, ohne den Private Key zu kennen.

Step 0: Preparation

```
cd
mkdir -p manifests/jwt
cd manifests/jwt
```

Step 1: Create http-bin and curl workloads

```
kubectl -n bookinfo apply -f ~/istio/samples/httpbin/httpbin.yaml
kubectl -n bookinfo apply -f ~/istio/samples/curl/curl.yaml
kubectl -n bookinfo rollout status deploy/httpbin deploy/curl
```

Step 2: Can we connect ?

- Erwartet: `200`

```
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl http://httpbin.bookinfo:8000/ip
-sS -o /dev/null -w "%{http_code}\n"
```

Step 3: Create a RequestAuthentication

```
nano 01-ra.yml
```

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: "jwt-example"
  namespace: bookinfo
spec:
  targetRefs:
  - group: ""
    kind: Service
    name: httpbin
  jwtRules:
  - issuer: "testing@secure.istio.io"
    jwksUri: "https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/jwks.json"
```

```
kubectl apply -f .
```

> *[!NOTE]* `targetRefs` *mit* `kind: Service` *bindet die Policy an den* ***Waypoint****, der für den Service* `httpbin` *zuständig ist. Mit* `selector` *(wie bei Sidecars) würde die Policy nicht greifen.*

Step 4: Check with an invalid jwt

- Invalid is restricted, so we do not get access (no 200)
- Erwartet: `401`

```
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -H "Authorization: Bearer
invalidToken" -w "%{http_code}\n"
```

Step 5: But: without a jwt -> its work

- ... Because ! -> There is no AuthorizationPolicy
- Erwartet: `200`

```
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -w "%{http_code}\n"
```

Step 6: We create an AuthorizationPolicy

> *[!NOTE] requestPrincipal set to testing@secure.istio.io/testing@secure.istio.io. Istio constructs the requestPrincipal by combining the iss and sub of the JWT token with a / separator.*

> *iss = issuer sub = subject*

```
nano 02-ap.yml
```

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: bookinfo
spec:
  targetRefs:
  - group: ""
    kind: Service
    name: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
       requestPrincipals: ["testing@secure.istio.io/testing@secure.istio.io"]
```

```
kubectl apply -f 02-ap.yml
```

Prüfen, ob der Waypoint die Policy übernommen hat:

```
kubectl -n bookinfo get authorizationpolicy require-jwt -o
jsonpath='{.status.conditions[*].message}{"\n"}'
```

Erwartet: `bound to bookinfo/waypoint`

Step 7: Test access

- jwt consists of 3 parts
  - HEADER / PAYLOAD / SIGNATURE
  - Each part is base64 encoded
- cut -d. -f2 -> gets the 2nd part -> the payload

```
## This is the way we get the token
TOKEN=$(curl https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/demo.jwt -s) && echo "$TOKEN" | cut -d '.' -f2 - |
base64 --decode
```

```
echo $TOKEN
```

```
## Testing with allowed jwt  (erwartet: 200)
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -H "Authorization: Bearer
$TOKEN" -w "%{http_code}\n"
```

```
## Testing without a jwt  (erwartet: 403)
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
```

```
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -w "%{http_code}\n"
```

Step 8: Update AuthorizationPolicy also needing a specific group

```
nano 02-ap-group.yml
```

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: bookinfo
spec:
  targetRefs:
  - group: ""
    kind: Service
    name: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
       requestPrincipals: ["testing@secure.istio.io/testing@secure.istio.io"]
    when:
    - key: request.auth.claims[groups]
      values: ["group1"]
```

```
kubectl apply -f 02-ap-group.yml
```

Step 9: get token included a claim for a group

- Get the JWT that sets the groups claim to a list of strings: group1 and group2:

```
TOKEN_GROUP=$(curl https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/groups-scope.jwt -s) && echo "$TOKEN_GROUP" | cut -d
'.' -f2 - | base64 --decode
```

Step 10: Test it with that token (so group1 must be included)

- Erwartet: `200`

```
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -H "Authorization: Bearer
$TOKEN_GROUP" -w "%{http_code}\n"
```

Step 11: Test with a token without group included

- We use that TOKEN before, which had not group
- Erwartet: `403`

```
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -H "Authorization: Bearer
$TOKEN" -w "%{http_code}\n"
```

Step 12: Overview of the results

Aufruf	Nur RequestAuthentication	+ AuthorizationPolicy	+ Gruppe `group1`
ohne Token	`200`	`403`	`403`
ungültiges Token	`401`	`401`	`401`
gültiges Token (ohne Gruppe)	`200`	`200`	`403`
Token mit `group1`	`200`	`200`	`200`

- `401` : Das JWT ist ungültig, die Prüfung der `RequestAuthentication` im Waypoint lehnt ab.
- `403` : Das JWT ist gültig oder fehlt, aber die `AuthorizationPolicy` erlaubt es nicht.

Step 13: Stolperstein: selector statt targetRefs (optional)

Jetzt siehst du selbst, was bei Sidecars funktioniert und im Ambient-Mode nicht. Erst die Policies aus den Steps davor löschen:

```
kubectl -n bookinfo delete authorizationpolicy/require-jwt requestauthentication/jwt-
example
```

Die `AuthorizationPolicy` mit `selector` (wie in der Sidecar-Übung):

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt-selector
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
       requestPrincipals: ["testing@secure.istio.io/testing@secure.istio.io"]
EOF
```

Der Status verrät das Problem:

```
kubectl -n bookinfo get authorizationpolicy require-jwt-selector -o
jsonpath='{.status.conditions[*].message}{"\n"}'
```

Erwartet: `ztunnel does not support HTTP attributes (found: requestPrincipals). In ambient mode you must use a waypoint proxy to enforce HTTP rules. ...`

Auch das gültige Token kommt jetzt nicht mehr durch (erwartet: `503`):

```
TOKEN=$(curl https://raw.githubusercontent.com/istio/istio/release-
1.29.1/security/tools/jwt/samples/demo.jwt -s)
kubectl exec "$(kubectl get pod -l app=curl -n bookinfo -o jsonpath=
{.items..metadata.name})" -c curl -n bookinfo -- curl
"http://httpbin.bookinfo:8000/headers" -sS -o /dev/null -H "Authorization: Bearer
$TOKEN" -w "%{http_code}\n"
```

Merke: Im Ambient-Mode gehören Layer-7-Policies (JWT, HTTP-Methoden, Claims) per `targetRefs` an den Waypoint.

Step 14: Cleanup

```
kubectl -n bookinfo delete authorizationpolicy/require-jwt-selector
kubectl -n bookinfo delete -f ~/istio/samples/httpbin/httpbin.yaml -f
~/istio/samples/curl/curl.yaml
cd
```

Reference:

- https://istio.io/latest/docs/tasks/security/authorization/authz-jwt/
- JWT Debugger: https://www.jwt.io/

Übung (Ambient): Zero Trust für bookinfo Schritt für Schritt (ztunnel Layer 4, Waypoint Layer 7)

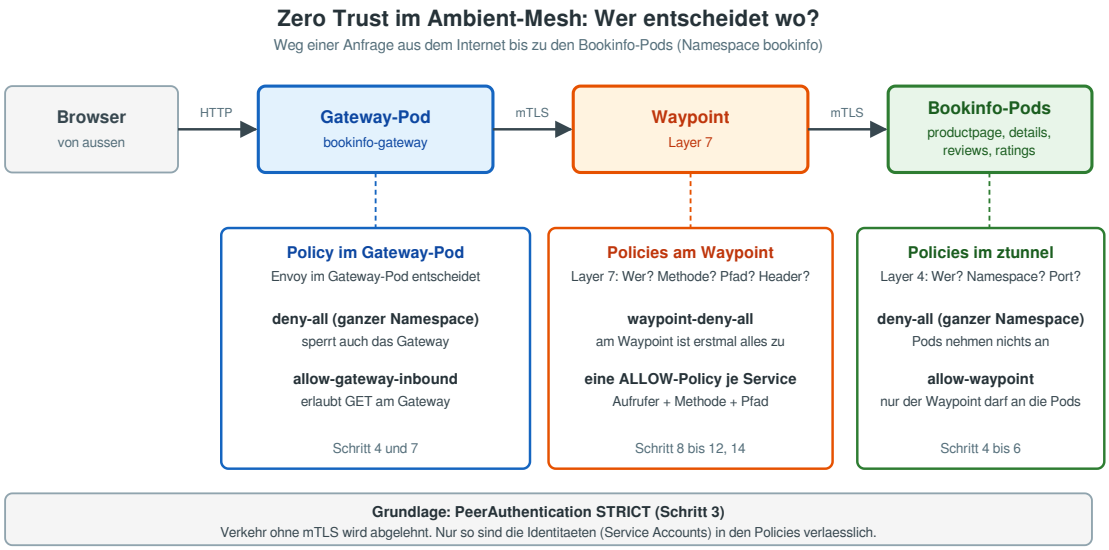
Ziel

Du setzt **Zero Trust** im Ambient-Mesh Schritt fuer Schritt um: **Erst alles verbieten, dann genau das erlauben, was Bookinfo braucht.**

Jeder Service darf nur mit den Services reden, die er wirklich braucht, und nur mit den erlaubten HTTP-Methoden und Pfaden.

Im Ambient-Modus gibt es dafuer **zwei Stellen**, die du nacheinander einrichtest:

Ebene	Wer entscheidet	Was kann die Regel pruefen
Layer 4	**ztunnel** auf dem Node des Ziel-Pods	Identitaet (Service Account), Namespace, Port
Layer 7	**Waypoint**	zusaetzlich HTTP-Methode, Pfad, Header



Hinweis: Die gleiche Uebung fuer den **Sidecar-Modus** steht in Zero Trust mit Bookinfo (Sidecar).

Voraussetzung

- Uebung Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo` mit Waypoint `waypoint`, Gateway `bookinfo-gateway`, alle Pods `1/1 Running`).
- Falls du Uebung Bookinfo mit HTTPS und Let's Encrypt gemacht hast: Dort zuerst den Abschnitt **Aufraeumen** ausfuehren, damit das Gateway wieder nur HTTP spricht.
- Im Namespace gibt es keine Policies:

```
kubectl -n bookinfo get authorizationpolicy,peerauthentication
```

Erwartet: `No resources found in bookinfo namespace.` Sonst vorher loeschen: `kubectl -n bookinfo delete authorizationpolicy --all` und `kubectl -n bookinfo delete peerauthentication --all`

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir zero-trust-ambient
cd zero-trust-ambient
```

Ein kleiner Client im Mesh, von dem aus wir Services direkt aufrufen:

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo rollout status deploy/sleep
```

Die IP des Gateways:

```
GATEWAY_IP=$(kubectl -n bookinfo get svc bookinfo-gateway-istio -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo $GATEWAY_IP
```

Drei kleine Hilfen, die wir immer wieder benutzen:

```
## HTTP-Statuscode der Produktseite ueber das Gateway (von aussen)
gw() { curl -s -m 10 -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/productpage;
}

## HTTP-Statuscode eines Aufrufs vom Pod "sleep" aus (aus dem Mesh)
call() { kubectl -n bookinfo exec deploy/sleep -- curl -s -m 10 -o /dev/null -w "%
{http_code}\n" "$@"; }

## Produktseite 8-mal laden und zaehlen, was darauf steht (Bookinfo waehlt zufaellig
reviews v1, v2 oder v3)
seite() {
  for i in $(seq 1 8); do
    curl -s -m 10 http://$GATEWAY_IP/productpage \
      | grep -o -E "Error fetching product details|Error fetching product
reviews|Ratings service is currently unavailable|glyphicon-star" \
      | sort -u | tr '\n' ',' ; echo
  done | sort | uniq -c
}
```

Schritt 2: Ausgangszustand - alles ist erlaubt

```
gw
call http://productpage:9080/productpage
call http://ratings:9080/ratings/0
```

Erwartet: dreimal `200`.

Der Pod `sleep` hat mit Bookinfo nichts zu tun und kommt trotzdem an `ratings` heran. Ohne Policy darf im Mesh **jeder alles**. Das aendern wir jetzt.

```
seite
```

Erwartet: Die Seite ist vollstaendig. Je nach `reviews`-Version steht eine leere Zeile (v1, keine Sterne) oder `glyphicon-star,` (v2, v3).

Schritt 3: mTLS erzwingen

Policies auf Identitaeten (`principals`) sind nur verlaesslich, wenn jede Verbindung mTLS spricht. Im Mesh macht der ztunnel das von selbst. Was ist mit einem Pod **ausserhalb** des Mesh?

Ein Pod in einem Namespace ohne Mesh:

```
kubectl create ns zt-ohne-mesh
kubectl -n zt-ohne-mesh apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n zt-ohne-mesh rollout status deploy/sleep
```

```
kubectl -n zt-ohne-mesh exec deploy/sleep -- curl -s -m 10 -o /dev/null -w "%
{http_code}\n" http://productpage.bookinfo:9080/productpage
```

Erwartet: `200`. Der ztunnel nimmt auch unverschluesselte Verbindungen an. Das schliessen wir:

```
## vi 01-peerauthentication.yml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
kubectl -n zt-ohne-mesh exec deploy/sleep -- curl -s -m 10 -o /dev/null -w "%
{http_code}\n" http://productpage.bookinfo:9080/productpage
```

Erwartet: `000` und `command terminated with exit code 56` (manchmal `52`). Die Verbindung wird abgelehnt, weil kein mTLS-Zertifikat im Spiel ist.

```
gw
call http://productpage:9080/productpage
```

Erwartet: zweimal `200`. Im Mesh und ueber das Gateway funktioniert weiter alles.

Schritt 4: Layer 4 - Deny All im ganzen Namespace

Die Basis von Zero Trust: **Erstmal alles verbieten.** Eine `AuthorizationPolicy` ohne `rules` laesst nichts durch. Ohne `targetRefs` und ohne `selector` gilt sie fuer den **ganzen Namespace**.

```
## vi 02-deny-all.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-all
spec: {}
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
gw
call http://productpage:9080/productpage
call http://ratings:9080/ratings/0
```

Erwartet:

Aufruf	Status	Wer lehnt ab
`gw` (von aussen ueber das	`403`	das Gateway selbst (die Policy gilt auch fuer den Gateway-

Gateway)		Pod)
`call productpage, call ratings`	`503`	der ztunnel am Ziel-Pod (siehe Schritt 5)

Schritt 5: Ursache im ztunnel-Log ansehen

Wir wollen sehen, **wer** am Ziel abgelehnt wird. Dazu brauchen wir den ztunnel auf dem Node des Ziel-Pods `productpage`:

```
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
echo $NODE $ZT
```

```
call http://productpage:9080/productpage
kubectl -n istio-system logs $ZT --since=1m | grep "policy rejection" | tail -2
```

Erwartet (gekuerzt):

```
error  access  connection complete  src.workload="waypoint-..."
src.identity="spiffe://cluster.local/ns/bookinfo/sa/waypoint" ...
dst.workload="productpage-v1-..." direction="inbound" ... error="connection closed due
to policy rejection: allow policies exist, but none allowed"
```

Wichtig ist `src.identity`: Am Pod kommt **nicht** `sleep` an, sondern der **Waypoint** (`sa/waypoint`). Der Aufruf von `sleep` laeuft ueber den Waypoint, und der Waypoint ruft dann den Pod auf. Mehr zum Lesen dieser Zeilen steht in der Uebung Fehlersuche im Ambient-Mesh.

Schritt 6: Layer 4 - nur der Waypoint darf an die Pods

Die Pods sollen nur noch Verbindungen vom Waypoint annehmen. Wer am Waypoint vorbei direkt an einen Pod will, wird vom ztunnel abgelehnt.

```
## vi 03-allow-waypoint.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-waypoint
spec:
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/waypoint"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
call http://productpage:9080/productpage
call http://ratings:9080/ratings/0
gw
```

Erwartet: `200`, `200` und `403`.

Im Mesh geht es wieder, weil der Waypoint durchgelassen wird. Vom Waypoint kommt aber **jeder** Aufrufer durch, denn am Waypoint ist noch keine Regel (das machen wir ab Schritt 8). Von aussen ueber das Gateway bleibt es bei `403`.

Schritt 7: Das Gateway erlauben und durch den Waypoint schicken

7a: Den Gateway-Pod freigeben

Der `403` kommt aus dem Gateway-Pod. Wir erlauben dort `GET`. Die Policy zeigt mit `targetRefs` auf das Gateway:

```
## vi 04-allow-gateway-inbound.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-gateway-inbound
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: bookinfo-gateway
  action: ALLOW
  rules:
  - to:
    - operation:
        methods: ["GET"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
gw
```

Erwartet: `503`. Das Gateway ist jetzt durch, aber der naechste Schritt scheitert. Warum? Wieder ins ztunnel-Log am Ziel (`productpage`) schauen:

```
kubectl -n istio-system logs $ZT --since=1m | grep "policy rejection" | tail -1
```

Erwartet: `src.identity="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio"` mit `none allowed`. Das Gateway ruft den Pod **direkt** auf, am Waypoint vorbei. Der ztunnel erlaubt aber nur den Waypoint.

7b: Das Gateway ueber den Waypoint leiten

Ein Ingress-Gateway nutzt den Waypoint nicht von selbst. Wir sagen es ihm mit einem Label am Namespace:

```
kubectl label namespace bookinfo istio.io/ingress-use-waypoint=true
sleep 15
gw
```

Erwartet: `200`.

Hinweis: Laut Istio-Doku wirkt das Label nur, wenn istiod die Variable `ENABLE_INGRESS_WAYPOINT_ROUTING` aktiv hat (Standard `false`). Auf unseren Clustern (Istio 1.31.4) ist sie nicht gesetzt, und es klappt trotzdem. Bleibt bei dir der `503`, ist das die erste Stelle zum Pruefen.

Jetzt laeuft **aller** Verkehr durch den Waypoint, auch der von aussen. Nur so wirken die Layer-7-Regeln der naechsten Schritte auch fuer Anfragen aus dem Internet. (Haetten wir stattdessen das Gateway im ztunnel erlaubt, wuerde es den Waypoint umgehen und keine Layer-7-Regel haette Einfluss.)

Schritt 8: Layer 7 - Deny All am Waypoint

Die Layer-4-Regeln aus Schritt 4 bis 6 sagen nur: "Der Waypoint darf an die Pods". Wer **am Waypoint** was darf, regeln wir jetzt. Auch hier zuerst alles verbieten. Die Policy zeigt mit `targetRefs` auf den Waypoint:

```
## vi 05-waypoint-deny-all.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: waypoint-deny-all
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
gw
call http://productpage:9080/productpage
```

Erwartet: zweimal `403`. Diesmal sagt der **Waypoint** nein (Layer 7). Der ztunnel ist unbeteiligt.

Merke den Unterschied: `503` kam aus dem ztunnel (Verbindung abgelehnt), `403` kommt aus einem Envoy (Gateway oder Waypoint).

Schritt 9: Gateway darf productpage aufrufen

Jetzt erlauben wir Weg fuer Weg. Die Regeln gehoeren an den Service, den der Waypoint schuetzt (`targetRefs` auf den Service). Die Identitaet des Aufrufers steht im Waypoint als `principals`.

```
## vi 06-allow-productpage-from-gateway.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-productpage-from-gateway
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: productpage
  action: ALLOW
```

```
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/productpage", "/static/*", "/login", "/logout", "/api/v1/products*"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
seite
```

Erwartet: Die Seite laedt, aber `Error fetching product details,Error fetching product reviews,` (8-mal). `productpage` darf noch niemanden aufrufen.

Schritt 10: productpage darf details aufrufen

```
## vi 07-allow-details.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-details
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: details
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-productpage"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/details/*"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
seite
```

Erwartet: `Error fetching product details` ist weg, es bleibt `Error fetching product reviews,` .

Schritt 11: productpage darf reviews aufrufen

```
## vi 08-allow-reviews.yml
apiVersion: security.istio.io/v1
```

```
kind: AuthorizationPolicy
metadata:
  name: allow-reviews
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: reviews
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-productpage"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/reviews/*"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
seite
```

Erwartet: Die Fehlermeldung zu den Reviews ist weg. Bei `reviews` v2 und v3 steht jetzt `Ratings service is currently unavailable,` (v1 zeigt nichts an, das ist die leere Zeile).

Schritt 12: reviews darf ratings aufrufen

```
## vi 09-allow-ratings.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-ratings
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: ratings
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-reviews"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/ratings/*"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
seite
```

Erwartet: nur noch leere Zeilen (v1) und `glyphicon-star,` (v2, v3). Keine Fehlermeldung mehr.

Schritt 13: Gesamtstatus und Negativtests

```
kubectl -n bookinfo get authorizationpolicy
```

Erwartet: `deny-all`, `allow-waypoint`, `allow-gateway-inbound`, `waypoint-deny-all`, `allow-productpage-from-gateway`, `allow-details`, `allow-reviews`, `allow-ratings`.

Was der ztunnel kennt (Layer 4) und was am Waypoint haengt:

```
istioctl ztunnel-config policies | grep -E "NAMESPACE|bookinfo"
```

Erwartet: `deny-all` und `allow-waypoint` mit `SCOPE Namespace`. Die anderen Policies siehst du hier nicht, die setzt der Waypoint durch.

Negativtests - Nicht erlaubte Kommunikation

```
## Test 1: sleep darf ratings NICHT aufrufen
call http://ratings:9080/ratings/0
```

Erwartet: `403`. (In Schritt 2 war es noch `200`.)

```
## Test 2: sleep darf productpage nicht aufrufen und schon gar nicht mit POST
call http://productpage:9080/productpage
call -X POST http://productpage:9080/productpage
```

Erwartet: zweimal `403`.

```
## Test 3: details darf NICHT reviews aufrufen
POD=$(kubectl -n bookinfo get pods -l app=details -o
jsonpath='{.items[0].metadata.name}')
kubectl -n bookinfo debug $POD -it --image=curlimages/curl --profile=general --
target=details -- sh
```

Im Container (er hat die Identitaet des `details`-Pods):

```
curl -s -o /dev/null -w "%{http_code}\n" http://reviews:9080/reviews/0
exit
```

Erwartet: `403`.

```
## Gegenprobe: reviews darf ratings aufrufen
POD=$(kubectl -n bookinfo get pods -l app=reviews,version=v2 -o
jsonpath='{.items[0].metadata.name}')
kubectl -n bookinfo debug $POD -it --image=curlimages/curl --profile=general --
target=reviews -- sh
```

Im Container:

```
curl -s -o /dev/null -w "%{http_code}\n" http://ratings:9080/ratings/0
exit
```

Erwartet: `200`.

Schritt 14: DENY-Policy - gezielt etwas blockieren

DENY-Policies werden **vor** ALLOW ausgewertet. Sie eignen sich fuer zusaetzliche Einschraenkungen. Beispiel: Aufrufe von `ratings` mit dem Header `x-debug: true` sperren.

```
## vi 10-deny-ratings-debug.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-ratings-debug
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: ratings
  action: DENY
  rules:
  - when:
    - key: request.headers[x-debug]
      values: ["true"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

```
POD=$(kubectl -n bookinfo get pods -l app=reviews,version=v2 -o
jsonpath='{.items[0].metadata.name}')
kubectl -n bookinfo debug $POD -it --image=curlimages/curl --profile=general --
target=reviews -- sh
```

Im Container:

```
curl -s -o /dev/null -w "%{http_code}\n" http://ratings:9080/ratings/0
curl -s -o /dev/null -w "%{http_code}\n" -H "x-debug: true"
http://ratings:9080/ratings/0
exit
```

Erwartet: `200` und `403`.

Schritt 15 (Bonus): Andere Namespaces aussperren

Defense in Depth: Selbst wenn spaeter eine zu weit gefasste ALLOW-Policy dazukommt, bleibt Verkehr aus anderen Namespaces draussen. DENY gewinnt immer.

```
## vi 11-deny-other-namespaces.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-other-namespaces
```

```
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint
  action: DENY
  rules:
  - from:
    - source:
        notNamespaces: ["bookinfo"]
```

```
kubectl -n bookinfo apply -f .
sleep 15
```

Verifizieren

Ein Pod aus einem anderen Namespace, der **im Mesh** ist:

```
kubectl create ns zt-fremd
kubectl label namespace zt-fremd istio.io/dataplane-mode=ambient
kubectl -n zt-fremd apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n zt-fremd rollout status deploy/sleep
```

```
kubectl -n zt-fremd exec deploy/sleep -- curl -s -m 10 -o /dev/null -w "%
{http_code}\n" http://productpage.bookinfo:9080/productpage
gw
```

Erwartet: `403` (fremder Namespace) und `200` (Gateway im Namespace `bookinfo`).

Zusammenfassung - Zero Trust Checkliste

Prinzip	Umsetzung im Ambient-Mesh	Wer setzt es durch
mTLS erzwingen	`PeerAuthentication` mit `mode: STRICT`	ztunnel
Default Deny, Layer 4	`AuthorizationPolicy` ohne Regeln im Namespace	ztunnel
Nur ueber den Waypoint	`allow-waypoint`: Pods nehmen nur den Waypoint an	ztunnel
Default Deny, Layer 7	`AuthorizationPolicy` mit `targetRefs` auf den Waypoint, ohne Regeln	Waypoint
Least Privilege	je Service eine ALLOW-Policy mit Aufrufer, Methode und Pfad	Waypoint
Defense in Depth	DENY-Policies (Header, fremde Namespaces)	Waypoint

Unterschied zum Sidecar-Modus

	Sidecar	Ambient
Wo steht die Regel	`selector` auf den Pod	`targetRefs` auf Service oder Waypoint

Wer sieht den Aufrufer	der Sidecar des Ziels, `principals` = Aufrufer	am Pod nur der Waypoint, `principals` = Aufrufer gilt **am Waypoint**
Debuggen	`rbac:debug` am Sidecar, `istioctl x authz check`	ztunnel-Log (Layer 4), Waypoint-Log (Layer 7), `istioctl ztunnel-config policies`

Evaluierungsreihenfolge

```
1. CUSTOM Policies  (externe Authorizer)
2. DENY Policies    (blockt wenn eine Regel passt)
3. ALLOW Policies   (erlaubt wenn eine Regel passt)
4. Keine Policy     -> Default: ALLOW
5. Leere Policy     -> Default: DENY (keine Regel kann passen)
```

Diese Reihenfolge gilt getrennt je Ebene: einmal im ztunnel (Layer 4), einmal am Waypoint (Layer 7).

Troubleshooting

Beobachtung	Pruefen
Eine neue Policy wirkt nicht	15 Sekunden warten und den Test wiederholen, sie braucht einen Moment im Mesh
`503`	Verbindung am Ziel abgelehnt: ztunnel-Log auf dem Node des Ziels (`grep "policy rejection"`), Feld `src.identity` ansehen
`403`	Ein Envoy hat abgelehnt: Gateway oder Waypoint. Welche Policy? `kubectl -n bookinfo get authorizationpolicy`, dann `targetRefs` der Policy pruefen
`403` obwohl eine passende ALLOW-Policy existiert	`principals` stimmen? Service Account des Aufrufers: `kubectl -n bookinfo get pod <pod> -o jsonpath='{.spec.serviceAccountName}'`
`gw` liefert `000`	`echo $GATEWAY_IP` gefuellt? Gateway `PROGRAMMED True`?
Policy am Waypoint wirkt nicht fuer Anfragen von aussen	Label `istio.io/ingress-use-waypoint=true` am Namespace gesetzt?
`istioctl x authz check` zeigt einen Fehler	Normal bei Ambient. Stattdessen `istioctl ztunnel-config policies`
Waypoint-Log zeigt nichts	Der Waypoint schreibt Anfragen nur mit einer `Telemetry`-Resource, siehe Fehlersuche im Ambient-Mesh

Aufraeumen

```
kubectl -n bookinfo delete authorizationpolicy --all
kubectl -n bookinfo delete peerauthentication default
kubectl label namespace bookinfo istio.io/ingress-use-waypoint-
```

```
kubectl -n bookinfo delete -f ~/istio/samples/sleep/sleep.yaml
kubectl delete namespace zt-ohne-mesh zt-fremd
```

```
gw
```

Erwartet: `200`. Bookinfo ist wieder wie nach der Demo-Installation.

Ref

- https://istio.io/latest/docs/ambient/usage/l7-features/
- https://istio.io/latest/docs/ambient/usage/waypoint/
- https://istio.io/latest/docs/ambient/usage/troubleshoot-ztunnel/

Übung (Sidecar): ZeroTrust für bookinfo Namespace bookinfo einrichten

> *Diese Übung gilt für den **Sidecar-Modus**. Für den **Ambient-Modus** (ztunnel und Waypoint) siehe Zero Trust mit Bookinfo im Ambient-Mesh.*

Ziel

In dieser Übung setzen wir **Zero Trust** im Mesh um:
Deny All → explizit erlauben, was nötig ist.

Jeder Service darf nur mit den Services kommunizieren, die er tatsächlich braucht – und nur über die erlaubten HTTP-Methoden und Pfade.

Voraussetzungen

- Kubernetes-Cluster mit Istio (Sidecar)
- Bookinfo-App deployed im Namespace `bookinfo`
- mTLS im Mesh aktiv (Default: `STRICT` oder `PERMISSIVE` mit PeerAuthentication)
- Gateway konfiguriert (Gateway API)

Architektur – Bookinfo Traffic Flow

```
Client → Gateway → productpage → reviews → ratings
                                 → details
```

- `productpage` ruft `reviews` und `details` auf
- `reviews` (v2/v3) ruft `ratings` auf
- `details` hat keine Downstream-Dependencies

Schritt 1: GATEWAY_IP setzten

```
## IP notieren
kubectl -n bookinfo get gateway
GATEWAY_IP=<ip-des-gateways>
```

Schritt 2: Projekt-Ordner setzen

```
cd
mkdir -p manifests/zero-trust-rbac
cd manifests/zero-trust-rbac
```

Schritt 3: Ausgangszustand herstellen.

- Wir wollen das mit Gateway API und httproute testen

```
## just in case
kubectl -n bookinfo delete virtualservice details productpage reviews
kubectl -n bookinfo delete destinationrule details productpage ratings reviews
```

```
## alles auf gateway ausrichten
kubectl -n bookinfo apply -f  ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo get all
```

Schritt 4: mTLS verifizieren

Bevor Authorization Policies greifen, muss mTLS aktiv sein. Ohne mTLS funktionieren `source.principals` und `source.namespaces` nicht.

```
## PeerAuthentication prüfen
kubectl get peerauthentication -A
```

```
## Falls nicht vorhanden, mesh-weit STRICT setzen
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system
spec:
  mtls:
    mode: STRICT
EOF
```

Verifizieren

```
## Aus einem Pod ohne Sidecar sollte der Zugriff fehlschlagen
## weil dieser die Verbindung plaintext und nicht verschlüsselt aufbaut
kubectl run test-no-mesh --image=busybox --rm -it -n default
```

```
## in der shell
wget -O - http://productpage.bookinfo:9080/productpage
## ** Erwartung: ** Connection reset by peer (kein mTLS-Zertifikat)
```

Schritt 5: Deny All – Alles sperren

Die Basis von Zero Trust: **Erstmal alles verbieten.**

Eine leere `AuthorizationPolicy` ohne Rules mit impliziter `ALLOW` -Action blockiert sämtlichen Traffic im Namespace.

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
```

```
kind: AuthorizationPolicy
metadata:
  name: deny-all
  namespace: bookinfo
spec: {}
EOF
```

Verifizieren

```
## Über den Browser oder curl auf die Productpage zugreifen
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/productpage
```

```
##Erwartung: 403
```

> ***Erklärung:*** *Eine Policy mit* `spec: {}` *(keine* `action`*, keine* `rules`*) bedeutet:*
> *Action ist implizit* `ALLOW`*, aber es gibt keine Rules die matchen → kein Request wird erlaubt.*

Schritt 6: Gateway → productpage erlauben

Der erste erlaubte Pfad: Das Gateway darf die Productpage erreichen.

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-productpage-from-gateway
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: productpage
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/productpage", "/static/*", "/login", "/logout",
"/api/v1/products*"]
EOF
```

> ***Hinweis:*** *Bei Gateway API mit Waypoint Proxy oder anderem Setup muss der* `principals` *-Wert angepasst werden.*
> *Den korrekten Service Account findest du mit:*
>
> ```
> kubectl get pods -n istio-system -l app=istio-ingressgateway -o
> jsonpath='{.items[0].spec.serviceAccountName}'
> ```

Verifizieren

```
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/productpage
```

```
## Erwartung: 200 OK - aber die Seite zeigt Fehler für Reviews und Details (noch
blockiert)
## --> Aber: Es ist 403.
```

Schritt 7: Problem rbac debuggen (am productpage - sidecar)

```
## Kommt die Anfrage am productpage (sidecar- container an) ?
## Dazu müssen wir zunächst wissen, ob rbac:debug gesetzt ist, nur dann sehen wir
etwas
istioctl proxy-config log deploy/productpage-v1 -n bookinfo | grep rbac
## Falls nicht rbac: debug,  jetzt setzen:
istioctl proxy-config log deploy/productpage-v1 -n bookinfo --level rbac:debug
```

```
## Anfrage nochmal absetzen und Log überprüfen
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/productpage
## Es kommt nichts an
kubectl logs -n bookinfo deploy/productpage-v1 -c istio-proxy --tail=30 | grep rbac
```

- **Resultat**: Da kommt nichts an, das Problem muss also davor sein: AM Gateway

Schritt 8: Problem rbac (am gateway - pod) identifizieren

Situation:

- Anfrage kommt garnicht bis productpage -> Sidecar
- Problem muss also vorher bereits existieren
- Vermutung: Am Gateway

Debuggen

```
## Gateway verifizieren
kubectl -n bookinfo get gateway
## Dies erzeugt einen Pod -> Das ist unser Freund
kubectl -n bookinfo get pods | grep gateway
```

```
## rbac einschalten / Abfrage absenden und nochmal überprüfen
istioctl proxy-config log deploy/bookinfo-gateway-istio -n bookinfo --level rbac:debug
curl -s http://$GATEWAY_IP/productpage
kubectl logs -n bookinfo deploy/bookinfo-gateway-istio -c istio-proxy --tail=30 | grep
rbac
```

 Ausgabe vom Pod

Schritt 9: Problem rbac (am gateway - pod) lösen

- Anfragen werden nicht durchgelassen, die vom Web-Browser kommen

- Diese müssen erlaubt werden

```
## Hier sehen wir die labels
kubectl -n bookinfo get pods --show-labels | grep gateway
```

```
nano allow-gateway-inbound.yaml
```

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-gateway-inbound
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      gateway.networking.k8s.io/gateway-name: bookinfo-gateway
  rules:
  - to:
    - operation:
        methods: ["GET"]
        paths: ["/productpage", "/static/*", "/login", "/logout", "/api/v1/products*"]
```

```
kubectl apply -f .
```

```
## Nochmal testen
## Gibt es für das gateway jetzt eine Policy
istioctl x authz check deploy/bookinfo-gateway-istio -n bookinfo
```

```
## Nochmal aufrufen
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP/productpage
```

```
## For fun: wie sieht der RBAC - Eintrag aus
kubectl logs -n bookinfo deploy/bookinfo-gateway-istio -c istio-proxy --tail=30 | grep
rbac
```

- Ergebnis: Es klappt. Eintrag: allowed by default or by policy

Schritt 10: productpage → details erlauben

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-details
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: details
  action: ALLOW
```

```
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-productpage"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/details/*"]
EOF
```

Verifizieren

```
## Achtung es kann einen Moment dauern, bis die Rechte im Mesh bekannt sind
curl -s http://$GATEWAY_IP/productpage | grep -o "Book Details"
## Erwartung: "Book Details" erscheint - Details-Section wird angezeigt
```

Schritt 11: productpage → reviews erlauben

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-reviews
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: reviews
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-productpage"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/reviews/*"]
EOF
```

Schritt 12: reviews → ratings erlauben

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-ratings
  namespace: bookinfo
spec:
```

```
  selector:
    matchLabels:
      app: ratings
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-reviews"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/ratings/*"]
EOF
```

Verifizieren

```
curl -s http://$GATEWAY_IP/productpage | grep -o "Reviews"
## Erwartung: Ratings werden angezeigt (Sterne bei reviews v2/v3)
```

Schritt 13: Gesamtstatus prüfen

```
## Alle Authorization Policies auflisten
kubectl get authorizationpolicies -n bookinfo

## Erwartete Ausgabe:
## NAME                              AGE
## deny-all                          ...
## allow-gateway-inbound             ...
## allow-productpage-from-gateway    ...
## allow-details                     ...
## allow-reviews                     ...
## allow-ratings                     ...
```

Negativtests – Nicht erlaubte Kommunikation

```
## Test 1: ratings darf NICHT direkt von productpage aufgerufen werden
```

```
## Debug Container verwenden

POD=$(kubectl get pods -n bookinfo -l app=productpage -o
jsonpath='{.items[0].metadata.name}')
kubectl debug $POD -n bookinfo --image=curlimages/curl -it -- sh
```

Dann im Container:

```
curl -s -o /dev/null -w "%{http_code}\n" http://ratings:9080/ratings/0
## Erwartung: 403
```

```
## Test 2: details darf NICHT reviews aufrufen
POD=$(kubectl get pods -n bookinfo -l app=details -o
jsonpath='{.items[0].metadata.name}')
kubectl debug $POD -n bookinfo --image=curlimages/curl -it -- sh
```

Dann im Container:

```
curl -s -o /dev/null -w "%{http_code}\n" http://reviews:9080/reviews/0
## Erwartung: 403
```

```
## Test 3: POST auf productpage ist nicht erlaubt (nur GET)
kubectl apply -f ~/istio/samples/sleep/sleep.yaml -n bookinfo
kubectl exec deploy/sleep -n bookinfo -c sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://productpage:9080/productpage
## Erwartung: 403
```

Schritt 14: DENY-Policy – Explizit blockieren

DENY-Policies werden **vor** ALLOW evaluiert und eignen sich für zusätzliche Einschränkungen.

Beispiel: Zugriff auf `/ratings` mit bestimmtem Header blockieren:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-ratings-debug
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: ratings
  action: DENY
  rules:
  - when:
    - key: request.headers[x-debug]
      values: ["true"]
EOF
```

Verifizieren

```
POD=$(kubectl get pods -n bookinfo -l app=reviews -o
jsonpath='{.items[0].metadata.name}')
kubectl debug $POD -n bookinfo --image=curlimages/curl -it -- sh
```

Dann im Container:

```
## Normaler Zugriff – Erwartung: 200
curl -s -o /dev/null -w "%{http_code}\n" http://ratings:9080/ratings/0
```

```
## Mit x-debug Header – Erwartung: 403
curl -s -o /dev/null -w "%{http_code}\n" -H "x-debug: true"
http://ratings:9080/ratings/0
```

Schritt 15: Alternative Namespace-Isolation (Bonus) - statt

Services aus anderen Namespaces komplett blockieren:

```
kubectl apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-other-namespaces
  namespace: bookinfo
spec:
  action: DENY
  rules:
  - from:
    - source:
        notNamespaces: ["bookinfo", "istio-system"]
EOF
```

Zusammenfassung – Zero Trust Checkliste

Prinzip	Umsetzung
mTLS erzwingen	`PeerAuthentication` mit `mode: STRICT`
Default Deny	Leere `AuthorizationPolicy` im Namespace
Least Privilege	Pro Service nur erlaubte Quellen, Methoden und Pfade
Defense in Depth	DENY-Policies für zusätzliche Einschränkungen
Namespace-Isolation	`notNamespaces` um Cross-Namespace-Zugriff zu blockieren

Evaluierungsreihenfolge

```
1. CUSTOM Policies  (externe Authorizer)
2. DENY Policies    (blockt wenn match)
3. ALLOW Policies   (erlaubt wenn match)
4. Keine Policy     → Default: ALLOW
5. Leere Policy     → Default: DENY (kein match möglich)
```

Aufräumen

```
kubectl delete authorizationpolicy -n bookinfo --all
## When no peerauthentication is set
```

```
## it will be PERMISSIVE by default
kubectl delete peerauthentication default -n istio-system
```

Troubleshooting

```
## Envoy-Logs auf RBAC-Entscheidungen prüfen
kubectl logs deploy/productpage-v1 -n bookinfo -c istio-proxy | grep "rbac"

## Debug-Level für RBAC setzen
istioctl proxy-config log deploy/productpage-v1 -n bookinfo --level rbac:debug

## Effektive Policies für einen Pod anzeigen
istioctl x authz check deploy/productpage-v1 -n bookinfo
```

Überblick: CloudNativePG (Postgres) im Ambient-Mesh - lohnt sich das? (Grafiken)

Worum geht es?

- **CloudNativePG (CNPG)** betreibt Postgres-Datenbanken in Kubernetes: Replikas, Failover, Backups.
- Das **Mesh (Istio Ambient)** sichert den Netzwerkverkehr zwischen den Pods ab und macht ihn sichtbar.

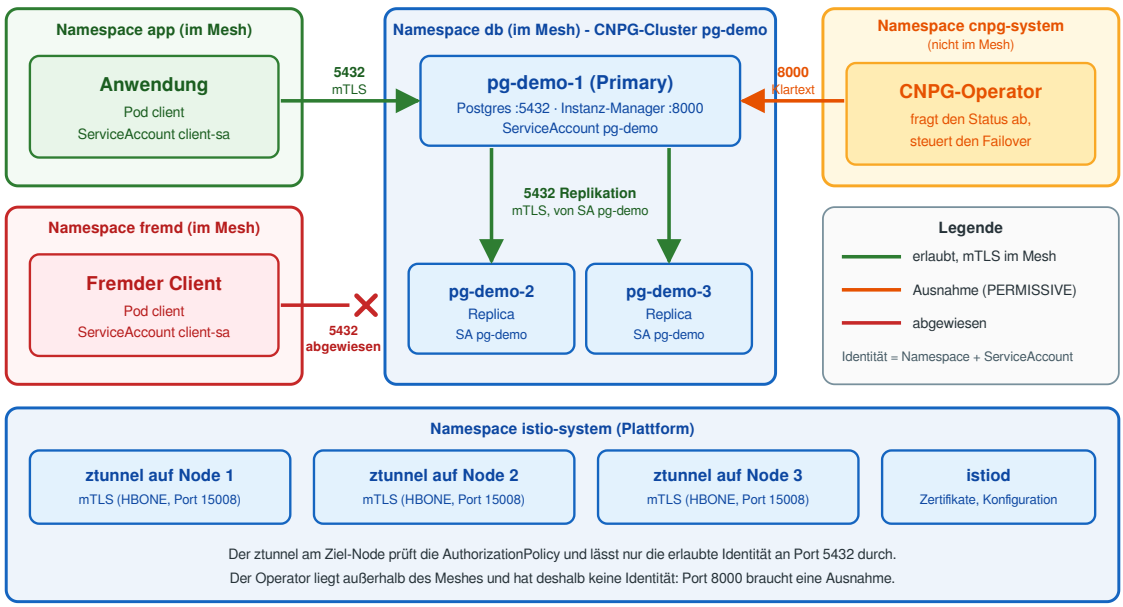
Die Frage: **Soll der Verkehr zur Datenbank zusätzlich durchs Mesh laufen?**

Antwort in einem Satz: Das geht mit Ambient gut, bringt aber nur dann etwas, wenn du die **Identität** des Absenders prüfen willst. Für die reine Frage "wer darf an die Datenbank" reicht eine NetworkPolicy.

Das Gesamtbild

Die Grafik zeigt, **in welchem Namespace was läuft** und welche Wege der Verkehr nimmt. Die Datenbank (`db`) und die Anwendung (`app`) liegen im Mesh, der Operator (`cnpg-system`) bleibt draußen.

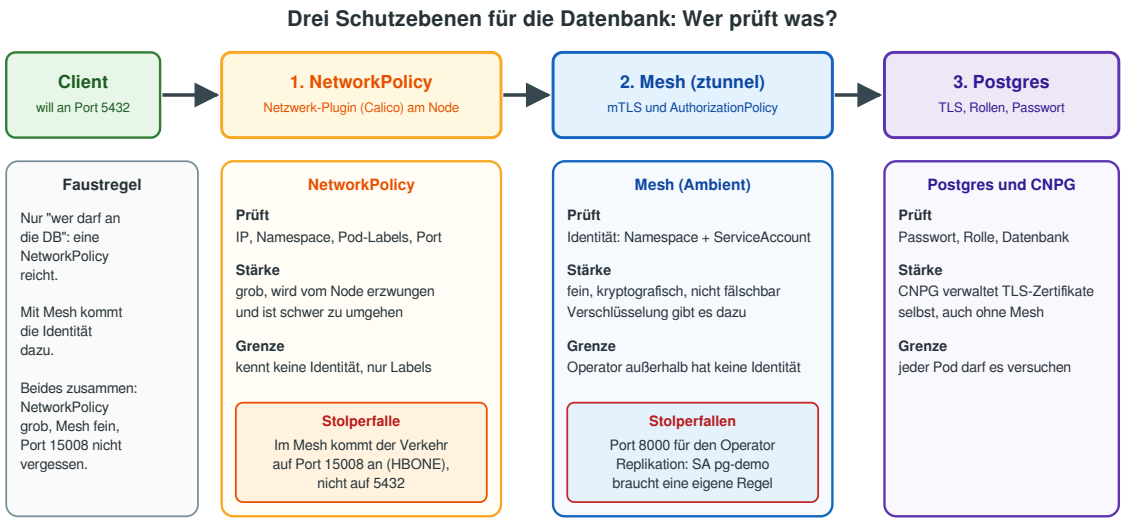
CloudNativePG im Ambient-Mesh: Namespaces, Ports und Wege



Weg	Port	Was passiert
Anwendung → Primary	5432	mTLS im Mesh, erlaubt durch die `AuthorizationPolicy`
Fremder Client → Primary	5432	wird vom ztunnel abgewiesen (falsche Identität)
Primary → Replicas	5432	Replikation, mTLS, Identität `pg-demo`
Operator → DB-Pods	8000	Status und Failover, Ausnahme (PERMISSIVE) nötig

Drei Schutzebenen

Eine Verbindung zur Datenbank läuft durch drei Prüfungen. Jede sieht etwas anderes.



- **NetworkPolicy:** grob und robust, kennt aber nur IP, Namespace, Labels und Port.
- **Mesh:** fein, weil es den **ServiceAccount** prüft (kryptografisch, nicht fälschbar).
- **Postgres und CNPG:** Passwort, Rollen und TLS. CNPG verwaltet die Zertifikate selbst.

Vorteile und Nachteile

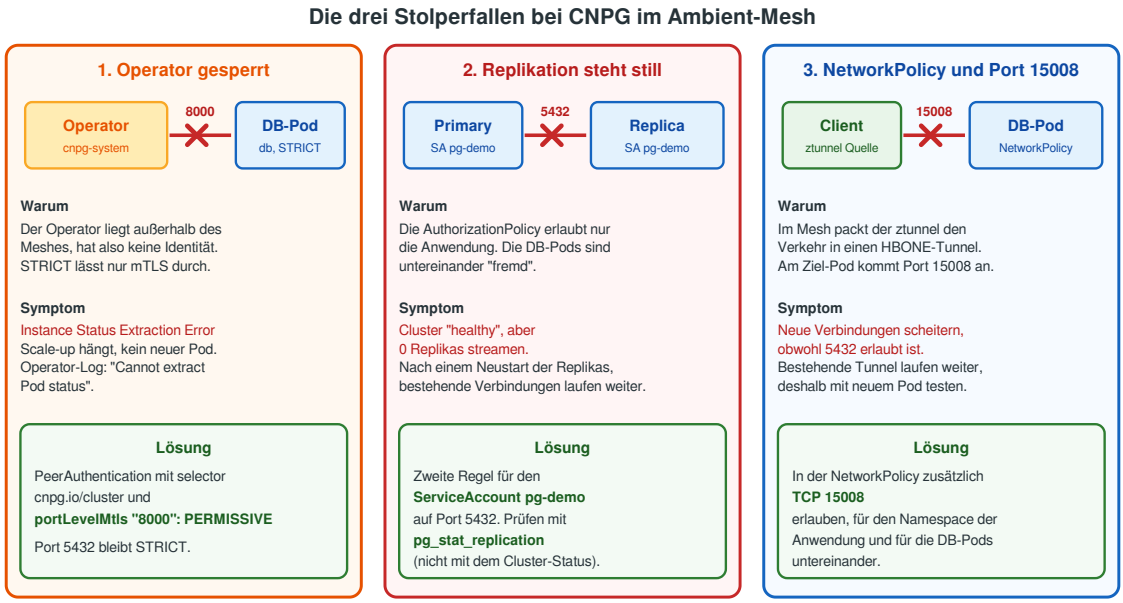
	Vorteil	Nachteil
Zugriff	Nur ein bestimmter ServiceAccount kommt an die DB, andere werden vor dem Passwort abgewiesen	Mehr Regeln, die zusammenpassen müssen
Verschlüsselung	kommt ohne Konfiguration	kein echter Gewinn, CNPG verschlüsselt schon selbst
Sichtbarkeit	TCP-Metriken wie bei allen anderen Diensten	Zusätzliche Fehlerquelle: bei Problemen ist unklar, ob Postgres, CNPG oder das Mesh schuld ist
Betrieb mit Ambient	Kein Sidecar: Join-Pods und Failover laufen ohne Sonderbehandlung	Es gibt **keine offizielle Unterstützung** durch CNPG, du bist auf Community-Wissen angewiesen

Netzwerk		Port 8000 braucht eine Ausnahme, Port 15008 muss in NetworkPolicies erlaubt sein

Sidecar-Modus für Datenbank-Pods: nicht empfohlen. Die Bootstrap-Pods erben keine Annotationen von CNPG, und mit Sidecars gibt es Probleme mit Start und Beenden.

Die drei Stolperfallen

Alle drei wurden auf einem Ambient-Cluster nachgestellt (Istio 1.31.4, CNPG 1.30.1, Postgres 17, Calico).



Die zweite Falle ist die gefährlichste: CNPG meldet `Cluster in healthy state`, obwohl keine Replika mehr Daten bekommt. **Deshalb nach jeder Policy die Replikation mit `pg_stat_replication` prüfen, nicht den Cluster-Status.**

Empfehlung

Situation	Empfehlung
Kein Mesh im Einsatz und keins geplant	CNPG allein, dazu eine **NetworkPolicy**
Ambient wird ohnehin betrieben, nur "wer darf an die DB"	**NetworkPolicy** reicht
Ambient wird betrieben und die **Identität** soll geprüft werden (zum Beispiel für Zero Trust oder Audit)	CNPG im Ambient-Mesh, mit `AuthorizationPolicy`, und die drei Stolperfallen beachten
Datenbank-Pods mit Sidecar	vermeiden

Wenn du beides kombinierst: **NetworkPolicy grob, Mesh fein**, und Port 15008 nicht vergessen.

Noch nicht getestet

- Der Operator selbst im Ambient-Mesh (Port 9443 für die Webhooks unter STRICT).

- Waypoint (Layer 7) für Postgres, Backups und der Pooler (PgBouncer).

Übung

- Übung (Ambient): CloudNativePG im Ambient-Mesh absichern

Quellen

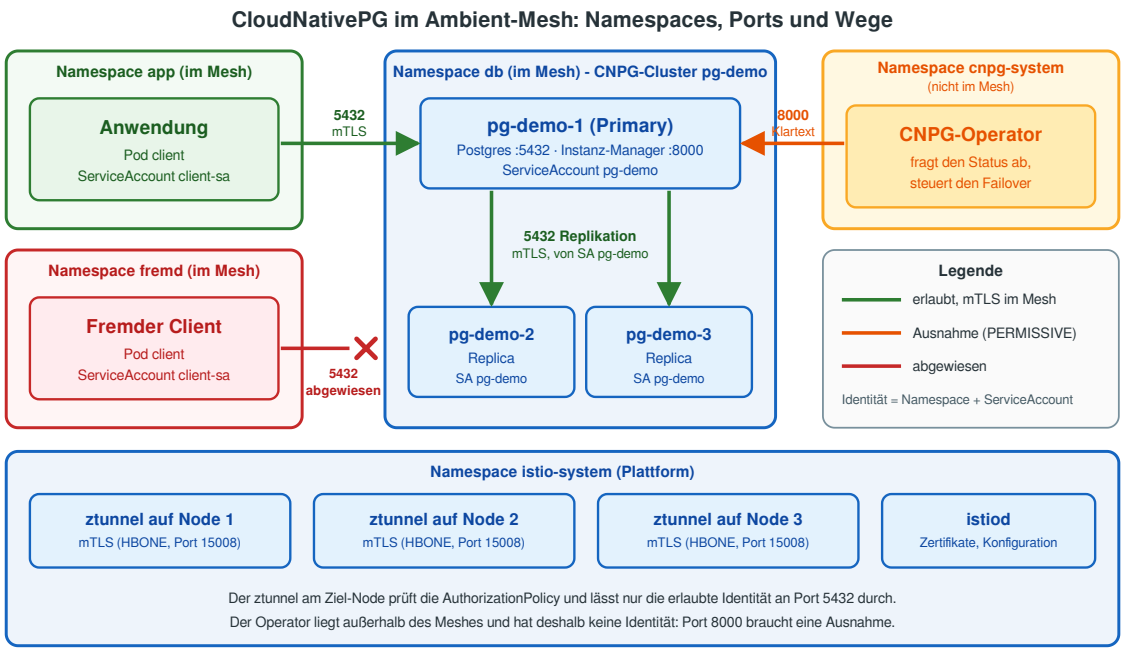
- CNPG-Diskussion #1280: Operator und Cluster unter Istio (Sidecar-Modus: Operator greift per Pod-IP auf Port 8000 zu, Bootstrap-Pods erben keine Annotationen)
- Istio: Ambient und NetworkPolicy (HBONE-Port 15008)

Übung (Ambient): CloudNativePG im Ambient-Mesh absichern (Operator Port 8000, Replikation, NetworkPolicy Port 15008)

Ziel

Du betreibst eine Postgres-Datenbank mit **CloudNativePG (CNPG)** und legst sie ins Ambient-Mesh. Danach darf **nur die Anwendung** an die Datenbank, und du lernst die drei Stellen kennen, an denen das gerne schiefgeht:

1. Der **Operator** erreicht die Datenbank-Pods auf Port 8000. STRICT-mTLS sperrt ihn aus.
2. Die **Replikation** zwischen den Datenbank-Pods laeuft ueber Port 5432. Eine zu strenge `AuthorizationPolicy` stoppt sie, CNPG meldet trotzdem "healthy".
3. Eine **NetworkPolicy** muss im Ambient-Mesh den HBONE-Port 15008 erlauben.



Hintergrund und Entscheidungshilfe (lohnt sich das Mesh fuer die Datenbank?) steht in CloudNativePG im Ambient-Mesh: Ueberblick.

Dauer: ca. 40 Minuten (CNPG braucht beim Start und bei Neustarts jeweils ein bis zwei Minuten).

Voraussetzung

- Dein Cluster hat **Istio Ambient** (Profil `ambient`, siehe Installation) und **Calico** als CNI (kennt NetworkPolicies).

- Der Cluster hat Zugriff ins Internet (Images, Manifeste von GitHub).
- Du arbeitest in **einem** Terminal, weil wir kleine Hilfsfunktionen definieren.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir cnpg-ambient
cd cnpg-ambient
```

Schritt 2: Speicher und Operator installieren

CNPG legt fuer jede Datenbank-Instanz ein PersistentVolume an. Auf dem Trainings-Cluster gibt es noch keine StorageClass, deshalb installieren wir den `local-path-provisioner` (er wird **nicht** zum Default).

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-
provisioner/v0.0.37/deploy/local-path-storage.yaml
kubectl -n local-path-storage rollout status deploy/local-path-provisioner --
timeout=120s
kubectl get storageclass
```

Der CNPG-Operator (Version 1.30.1). `--server-side` ist noetig, weil die CRDs sehr gross sind:

```
kubectl apply --server-side -f https://github.com/cloudnative-pg/cloudnative-
pg/releases/download/v1.30.1/cnpg-1.30.1.yaml
kubectl -n cnpg-system rollout status deploy/cnpg-controller-manager --timeout=180s
kubectl -n cnpg-system get pods
```

Schritt 3: Datenbank und Clients ohne Mesh

Drei Namespaces:

Namespace	Inhalt
`db`	die Datenbank `pg-demo` (CNPG-Cluster)
`app`	die Anwendung (erlaubter Client)
`fremd`	ein fremder Client, der **nicht** an die Datenbank darf

```
kubectl create ns db
kubectl create ns app
kubectl create ns fremd
```

Ein Test-Passwort fuer den Datenbank-Benutzer `app` (nur fuer diese Uebung):

```
kubectl -n db create secret generic pg-demo-appuser --type=kubernetes.io/basic-auth \
  --from-literal=username=app --from-literal=password=lab-only-cnpg-test
```

Die Datenbank mit zwei Instanzen (ein Primary, eine Replica):

```
## vi 01-cluster.yml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-demo
spec:
  instances: 2
  storage:
    size: 1Gi
    storageClass: local-path
  bootstrap:
    initdb:
      database: app
      owner: app
      secret:
        name: pg-demo-appuser
```

```
kubectl apply -f 01-cluster.yml -n db
```

Zwei Clients mit dem **gleichen** ServiceAccount-Namen `client-sa`, aber in verschiedenen Namespaces:

```
## vi 02-client.yml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: client-sa
---
apiVersion: v1
kind: Pod
metadata:
  name: client
spec:
  serviceAccountName: client-sa
  containers:
  - name: psql
    image: ghcr.io/cloudnative-pg/postgresql:17
    command: ["sleep", "infinity"]
    env:
    - name: PGPASSWORD
      value: lab-only-cnpg-test
    - name: PGCONNECT_TIMEOUT
      value: "5"
```

```
kubectl apply -f 02-client.yml -n app
kubectl apply -f 02-client.yml -n fremd
```

Warten, bis alles laeuft:

```
kubectl -n db wait --for=condition=Ready cluster/pg-demo --timeout=400s
kubectl -n app wait --for=condition=Ready pod/client --timeout=200s
```

```
kubectl -n fremd wait --for=condition=Ready pod/client --timeout=200s
kubectl -n db get cluster,pods
```

Erwartet: `Cluster in healthy state`, `pg-demo-1` und `pg-demo-2` sind `1/1 Running`.

Zwei Hilfsfunktionen. `sql` verbindet sich von einem Client-Pod aus mit der Datenbank, `repl` zaehlt auf dem Primary die Replikas, die wirklich streamen:

```
sql() { kubectl -n $1 exec $2 -- psql -h pg-demo-rw.db -U app -d app -tAc "select 'ok, Primary='||not pg_is_in_recovery()" 2>&1 | tail -1; }
repl() { kubectl -n db exec $(kubectl -n db get pod -l cnpg.io/instanceRole=primary -o name) -c postgres -- psql -U postgres -tAc "select count(*)||' Replikas streamen' from pg_stat_replication where state='streaming'" 2>&1 | tail -1; }
```

Ausgangszustand: Beide Clients kommen an die Datenbank. Genau das wollen wir aendern.

```
sql app client
sql fremd client
repl
```

Erwartet: zweimal `ok, Primary=true`, dann `1 Replikas streamen`.

Schritt 4: Die Datenbank ins Ambient-Mesh legen

Ein Label am Namespace genuegt. Die Pods muessen **nicht** neu gestartet werden.

```
kubectl label ns db istio.io/dataplane-mode=ambient
sleep 10
```

Der ztunnel kennt die Datenbank-Pods jetzt und spricht HBONE (mTLS) mit ihnen:

```
istioctl ztunnel-config workloads $(kubectl -n istio-system get pod -l app=ztunnel -o jsonpath='{.items[0].metadata.name}').istio-system | grep -E 'NAMESPACE|pg-demo'
```

Erwartet: `pg-demo-1` und `pg-demo-2` mit Protokoll `HBONE`.

Die Clients sind noch **nicht** im Mesh. Der Namespace steht auf PERMISSIVE (Standard), also geht Klartext weiter:

```
sql app client
```

Erwartet: `ok, Primary=true`.

Schritt 5: STRICT einschalten

Jetzt nimmt die Datenbank nur noch mTLS an:

```
## vi 03-peerauthentication-strict.yml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT
```

```
kubectl apply -f 03-peerauthentication-strict.yml -n db
sleep 15
sql app client
sql fremd client
```

Erwartet: zweimal `command terminated with exit code 2`. Die Clients sind noch nicht im Mesh und sprechen kein mTLS.

Schritt 6: Stolperfalle 1 - der Operator wird ausgesperrt

Der CNPG-Operator liegt in `cnpg-system`, **ausserhalb** des Meshes. Er fragt die Datenbank-Pods direkt ueber die Pod-IP auf **Port 8000** nach ihrem Status. Wir lassen ihn arbeiten, indem wir die Datenbank auf drei Instanzen vergroessern:

```
kubectl -n db patch cluster pg-demo --type merge -p '{"spec":{"instances":3}}'
sleep 90
kubectl -n db get cluster
kubectl -n db get pods
```

Erwartet: Status `Instance Status Extraction Error: HTTP communication issue`, es gibt nur `pg-demo-1` und `pg-demo-2`, **kein** `pg-demo-3`.

Der Operator-Log bestaetigt es:

```
kubectl -n cnpg-system logs deploy/cnpg-controller-manager --since=100s | grep -c
"Cannot extract Pod status"
```

Erwartet: eine Zahl groesser als 0.

Loesung: Port 8000 darf Klartext annehmen (PERMISSIVE), alle anderen Ports bleiben STRICT. Das Port-Level-Mapping braucht einen `selector`:

```
## vi 04-peerauthentication-port8000.yml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: pg-8000-permissive
spec:
  selector:
    matchLabels:
      cnpg.io/cluster: pg-demo
  mtls:
    mode: STRICT
  portLevelMtls:
    "8000":
      mode: PERMISSIVE
```

```
kubectl apply -f 04-peerauthentication-port8000.yml -n db
kubectl -n db wait --for=condition=Ready cluster/pg-demo --timeout=300s
kubectl -n db get cluster
kubectl -n db get pods
```

Erwartet: `Cluster in healthy state`, drei Instanzen. Der neue Pod `pg-demo-3` ist ohne Sonderbehandlung ins Mesh gekommen (mit einem Sidecar gaebe es hier Probleme mit den Init-Pods).

Schritt 7: Die Clients ins Mesh holen

Damit die Clients mTLS sprechen und eine **Identitaet** haben, kommen auch ihre Namespaces ins Ambient-Mesh:

```
kubectl label ns app istio.io/dataplane-mode=ambient
kubectl label ns fremd istio.io/dataplane-mode=ambient
sleep 10
sql app client
sql fremd client
```

Erwartet: zweimal `ok, Primary=true`. Beide sind jetzt im Mesh, und es gibt noch keine Regel, die einen davon sperrt.

Schritt 8: Nur die Anwendung darf an die Datenbank

Die Identitaet eines Clients ist der **Namespace plus ServiceAccount**. Beide Clients heissen `client-sa`, aber nur der in `app` ist erlaubt:

```
## vi 05-authorizationpolicy-app.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: pg-allow-app
spec:
  selector:
    matchLabels:
      cnpg.io/cluster: pg-demo
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/app/sa/client-sa"]
    to:
    - operation:
        ports: ["5432"]
  - to:
    - operation:
        ports: ["8000"]
```

Die zweite Regel laesst den Operator weiter auf Port 8000. Er hat keine Mesh-Identitaet und wird deshalb nur ueber den Port erkannt.

```
kubectl apply -f 05-authorizationpolicy-app.yml -n db
sleep 15
sql app client
sql fremd client
```

Erwartet: `ok, Primary=true` fuer `app`, `command terminated with exit code 2` fuer `fremd`. Das Ziel ist erreicht, aber schau jetzt, was mit der Replikation passiert.

Schritt 9: Stolperfalle 2 - die Replikation steht, CNPG meldet "healthy"

```
repl
```

Erwartet: `2 Replikas streamen`. Die bestehenden Verbindungen laufen weiter. Wir starten die Replikas neu, damit sie sich **neu** mit dem Primary verbinden muessen:

```
kubectl -n db delete pod -l cnpg.io/instanceRole=replica --wait=false
sleep 70
kubectl -n db get cluster
kubectl -n db get pods
repl
```

Erwartet: Cluster `Cluster in healthy state`, alle Pods `1/1 Running` - und trotzdem `0 Replikas streamen`. Die Replikas stehen bereit, bekommen aber keine Daten mehr. Faellt jetzt der Primary aus, fehlen Daten.

Die Ursache: Die Datenbank-Pods sprechen untereinander ueber Port 5432, ihre Identitaet ist der ServiceAccount `pg-demo`. Unsere Regel erlaubt nur `app`. **Der Cluster-Status zeigt das nicht, `pg_stat_replication` schon.**

Loesung: Die Datenbank-Pods untereinander erlauben:

```
## vi 06-authorizationpolicy-replikation.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: pg-allow-replication
spec:
  selector:
    matchLabels:
      cnpg.io/cluster: pg-demo
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/db/sa/pg-demo"]
    to:
    - operation:
        ports: ["5432"]
```

```
kubectl apply -f 06-authorizationpolicy-replikation.yml -n db
kubectl -n db wait --for=condition=Ready cluster/pg-demo --timeout=240s
sleep 45
repl
sql app client
sql fremd client
```

Erwartet: `2 Replikas streamen`, `ok, Primary=true` fuer `app`, weiterhin Fehler fuer `fremd`.

Schritt 10: Failover testen

Wir schalten den Primary hart ab. CNPG muss eine Replica befoerdern, der Operator muss dafuer Port 8000 erreichen, und die Anwendung muss danach wieder an die Datenbank kommen:

```
kubectl -n db get cluster pg-demo -o jsonpath='{.status.currentPrimary}{"\n"}'
kubectl -n db delete pod -l cnpg.io/instanceRole=primary --wait=false
sleep 60
kubectl -n db wait --for=condition=Ready cluster/pg-demo --timeout=200s
kubectl -n db get cluster pg-demo -o jsonpath='{.status.currentPrimary}{"\n"}'
kubectl -n db get pods
sql app client
repl
```

Erwartet: Der neue Primary hat einen **anderen** Namen als der alte, `ok, Primary=true` fuer `app`, `2 Replikas streamen`.

Schritt 11: Stolperfalle 3 - NetworkPolicy und Port 15008

Zusaetzlich zum Mesh sperren wir die Datenbank auf Netzwerk-Ebene: alles verboten, nur 5432 und 8000 erlaubt. Das ist die Sicht, die viele Sicherheitsvorgaben verlangen ("Defense in Depth").

```
## vi 07-networkpolicy-deny.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
  - Ingress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-pg
spec:
  podSelector:
    matchLabels:
      cnpg.io/cluster: pg-demo
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: app
    - podSelector: {}
    ports:
    - port: 5432
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: cnpg-system
    - podSelector: {}
    ports:
    - port: 8000
```

```
kubectl apply -f 07-networkpolicy-deny.yml -n db
sleep 60
```

Bestehende Verbindungen laufen weiter, und der ztunnel nutzt Tunnel wieder. Deshalb testen wir mit einem **neuen** Client-Pod (neue IP, neue Verbindung):

```
kubectl -n app run client2 --image=ghcr.io/cloudnative-pg/postgresql:17 --
overrides='{"spec":{"serviceAccountName":"client-sa"}}' --env=PGPASSWORD=lab-only-
cnpg-test --env=PGCONNECT_TIMEOUT=5 --command -- sleep 3600
kubectl -n app wait --for=condition=Ready pod/client2 --timeout=120s
sql app client2
```

Erwartet: `command terminated with exit code 2`, obwohl 5432 erlaubt ist.

Der Grund: Im Ambient-Mesh kommt der Verkehr nicht auf Port 5432 an. Der ztunnel packt ihn in einen Tunnel (HBONE) auf **Port 15008** am Ziel-Pod. Die NetworkPolicy sieht nur diesen Port.

Loesung: Port 15008 fuer den Namespace `app` und fuer die Datenbank-Pods untereinander erlauben:

```
## vi 08-networkpolicy-hbone.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-hbone
spec:
  podSelector:
    matchLabels:
      cnpg.io/cluster: pg-demo
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: app
    - podSelector: {}
    ports:
    - port: 15008
```

```
kubectl apply -f 08-networkpolicy-hbone.yml -n db
sleep 20
sql app client2
sql fremd client
repl
```

Erwartet: `ok, Primary=true` fuer `app`, Fehler fuer `fremd`, `2 Replikas streamen`.

Aufraeumen

```
kubectl delete ns app fremd db --timeout=180s
kubectl delete -f https://github.com/cloudnative-pg/cloudnative-
pg/releases/download/v1.30.1/cnpg-1.30.1.yaml --timeout=120s
```

```
kubectl delete -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path-storage.yaml --timeout=120s
```

Zusammenfassung

Stolperfalle	Symptom	Loesung
Operator -> Port 8000 unter STRICT	`Instance Status Extraction Error`, Scale-up haengt	`portLevelMtls "8000": PERMISSIVE` (mit `selector`)
`AuthorizationPolicy` ohne die Datenbank-Pods	Cluster `healthy`, aber `0 Replikas streamen`	Regel fuer ServiceAccount `pg-demo` auf Port 5432
`NetworkPolicy` ohne Port 15008	neue Verbindungen scheitern, obwohl 5432 erlaubt ist	HBONE-Port 15008 erlauben (Clients und Datenbank-Pods)

Merksatz: **Nach jeder Policy die Replikation mit `pg_stat_replication` pruefen, nicht den Cluster-Status.**

Sicherheit, Fehlertoleranz & Observability

Übung (Ambient): Egress-Gateway mit Waypoint - externe Ziele steuern, TLS-Origination, Weg per NetworkPolicy erzwingen

Ziel

Pods im Mesh sollen externe Dienste (hier `httpbin.org`) nur noch **ueber einen zentralen Punkt** erreichen. An diesem Punkt pruefen wir Methode und Pfad, bauen TLS zum Ziel auf und koennen mitlesen. Im Sidecar-Mode ist das ein eigener Egress-Gateway-Pod. **In Ambient uebernimmt diese Rolle ein Waypoint.**

 Egress in Ambient: Waypoint als Egress-Gateway

Voraussetzung

- Istio Ambient ist installiert, siehe istio-installation mit istioctl - ambient. Bookinfo wird fuer diese Uebung **nicht** gebraucht.
- Die Gateway-API-CRDs sind da (sie werden fuer jeden Waypoint gebraucht):

```
kubectl get crd gateways.gateway.networking.k8s.io
```

- Das Cluster darf ins Internet (`httpbin.org`, `example.com`).

Hintergrund

Frage	Antwort in Ambient
Wo laeuft der Egress-Gateway?	Als Waypoint in einem eigenen Namespace, hier `istio-egress`
Wie kommt der Verkehr dorthin?	Eine `ServiceEntry` im Namespace des Waypoints beschreibt das Ziel. Der Namespace traegt das Label `istio.io/use-waypoint`, damit gehen alle Ziele darin ueber den Waypoint.
Wer schaut auf Methode und	Der Waypoint (Layer 7), per `AuthorizationPolicy` auf die `ServiceEntry`

Pfad?	
Wer baut TLS zum Ziel auf?	Der Waypoint, per `DestinationRule`. Die Anwendung spricht weiter HTTP.
Wer erzwingt den Weg?	**Nicht der Waypoint.** Das macht eine `NetworkPolicy` (Schritt 8).

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir egress-ambient
cd egress-ambient
```

Schritt 2: Client im Mesh anlegen und Ausgangslage pruefen

Ein Namespace im Ambient-Mesh mit einem kleinen curl-Pod:

```
kubectl create namespace egress-demo
kubectl label namespace egress-demo istio.io/dataplane-mode=ambient
kubectl -n egress-demo apply -f ~/istio/samples/curl/curl.yaml
kubectl -n egress-demo rollout status deploy/curl
```

Zugriff ins Internet, so wie es heute ist:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -o /dev/null -w "httpbin.org
-> %{http_code}\n" http://httpbin.org/get
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -o /dev/null -w "example.com
-> %{http_code}\n" http://example.com/
```

Erwartet:

```
httpbin.org -> 200
example.com -> 200
```

Ausgangslage: Der Pod darf **alles** erreichen. Es gibt keinen zentralen Punkt, an dem wir pruefen oder mitlesen koennten.

Schritt 3: Waypoint als Egress-Gateway anlegen

Eigener Namespace fuer den Egress-Gateway, darin ein Waypoint. `--enroll-namespace` setzt das Label `istio.io/use-waypoint` auf den Namespace:

```
kubectl create namespace istio-egress
istioctl waypoint apply --for service --enroll-namespace -n istio-egress --wait
```

Erwartet (die Warnung zu `dataplane-mode` kannst du ignorieren, der Waypoint selbst gehoert nicht ins Mesh):

```
✓ waypoint istio-egress/waypoint is ready!
✓ namespace istio-egress labeled with "istio.io/use-waypoint: waypoint"
```

```
istioctl waypoint list -n istio-egress
kubectl get namespace istio-egress --show-labels
```

`PROGRAMMED` muss `True` sein.

Schritt 4: Externen Dienst als ServiceEntry beschreiben

```
## vi 01-serviceentry-httpbin.yml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: httpbin
spec:
  hosts:
  - httpbin.org
  ports:
  - number: 80
    name: http
    protocol: HTTP
  resolution: DNS
```

```
kubectl apply -f 01-serviceentry-httpbin.yml -n istio-egress
```

Aufruf wie vorher:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -o /dev/null -w "httpbin.org
-> %{http_code}\n" http://httpbin.org/get
```

Erwartet: `httpbin.org -> 200` . Fuer die Anwendung hat sich nichts geaendert. Aber der Weg ist jetzt ein anderer. Das sehen wir im ztunnel-Log auf dem Node des curl-Pods:

```
NODE=$(kubectl -n egress-demo get pod -l app=curl -o jsonpath="
{.items[0].spec.nodeName}")
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
kubectl -n istio-system logs $ZT --tail=200 | grep egress-demo | grep httpbin | tail
-2
```

Erwartet (gekuerzt):

```
src.workload="curl-..." src.namespace="egress-demo" ... dst.addr=192.168.x.x:15008
dst.service="httpbin.org"
dst.workload="waypoint-..." dst.namespace="istio-egress" ... direction="outbound"
```

Das Ziel ist der **Waypoint auf Port 15008** (HBONE), nicht `httpbin.org` . Der Waypoint macht von dort aus den Aufruf ins Internet.

Noch ein Hinweis, wer nach draussen auftritt:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 http://httpbin.org/get
```

Im Feld `origin` steht die oeffentliche IP des **Nodes, auf dem der Waypoint laeuft**. Beim Zielsystem kommen alle Aufrufe von dort an, egal von welchem Pod sie stammen.

Schritt 5: Nur GET auf /get erlauben

Layer 7 ist jetzt moeglich, weil der Waypoint im Weg ist. Die Policy haengt an der `ServiceEntry`:

```
## vi 02-authorizationpolicy-httpbin.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: httpbin-nur-get
spec:
  targetRefs:
  - kind: ServiceEntry
    group: networking.istio.io
    name: httpbin
  action: ALLOW
  rules:
  - to:
    - operation:
        methods: ["GET"]
        paths: ["/get"]
```

```
kubectl apply -f 02-authorizationpolicy-httpbin.yml -n istio-egress
```

Ein paar Sekunden warten, dann drei Aufrufe:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -X GET  -o /dev/null -w "GET
/get      -> %{http_code}\n" http://httpbin.org/get
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -X GET  -o /dev/null -w "GET
/headers  -> %{http_code}\n" http://httpbin.org/headers
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -X POST -o /dev/null -w "POST
/post    -> %{http_code}\n" http://httpbin.org/post
```

Erwartet:

```
GET /get      -> 200
GET /headers  -> 403
POST /post    -> 403
```

Erwarteter Fehler bei einem abgelehnten Aufruf:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -X POST
http://httpbin.org/post
```

```
RBAC: access denied
```

Die Ablehnung kommt vom Waypoint. Den Zaehler dazu siehst du so:

```
kubectl -n istio-egress exec deploy/waypoint -- pilot-agent request GET stats | grep -
E "rbac\.(allowed|denied)"
```

Erwartet: `rbac.allowed: 1` und `rbac.denied: 2` oder mehr (die Zahlen haengen davon ab, wie oft du aufgerufen hast).

Schritt 6: TLS zum Ziel am Waypoint aufbauen (TLS-Origination)

Die Anwendung bleibt bei HTTP auf Port 80. Der Waypoint macht daraus HTTPS auf Port 443. `targetPort` in der `ServiceEntry` und eine `DestinationRule` reichen dafuer:

```
## vi 03-serviceentry-httpbin-tls.yml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: httpbin
spec:
  hosts:
  - httpbin.org
  ports:
  - number: 80
    name: http
    protocol: HTTP
    targetPort: 443
  resolution: DNS
```

```
## vi 04-destinationrule-httpbin-tls.yml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin-tls
spec:
  host: httpbin.org
  trafficPolicy:
    tls:
      mode: SIMPLE
```

```
kubectl apply -f 03-serviceentry-httpbin-tls.yml -n istio-egress
kubectl apply -f 04-destinationrule-httpbin-tls.yml -n istio-egress
```

Ein paar Sekunden warten, dann:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 http://httpbin.org/get
```

Erwartet (gekuerzt):

```
  "url": "https://httpbin.org/get"
```

Wir haben `http://` aufgerufen, beim Ziel kam aber `https://` an. Das TLS hat der Waypoint aufgebaut. Die Anwendung muss dafuer nichts wissen, und Zertifikatsthemen fuer externe Ziele liegen an **einer** Stelle.

Schritt 7: Die Luecke zeigen - ein Ziel ohne ServiceEntry

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 15 -o /dev/null -w "example.com
-> %{http_code}\n" http://example.com/
```

Erwartet: `example.com -> 200`.

`example.com` hat keine `ServiceEntry`. Der Aufruf geht **am Waypoint vorbei direkt ins Internet**. Auch eine Anwendung, die absichtlich eine andere Route nimmt, kommt so raus. Die Einbindung des Namespaces in den Waypoint **lenkt** den Verkehr, sie **erzwingt** ihn nicht.

Schritt 8: Den Weg erzwingen mit einer NetworkPolicy

Der Pod darf nur noch zu zwei Zielen: DNS und Waypoint (Port 15008, HBONE). Alles andere wird vom CNI (hier Calico) verworfen.

```
## vi 05-networkpolicy-egress.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: egress-nur-ueber-waypoint
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: istio-egress
      podSelector:
        matchLabels:
          gateway.networking.k8s.io/gateway-name: waypoint
    ports:
    - protocol: TCP
      port: 15008
```

```
kubectl apply -f 05-networkpolicy-egress.yml -n egress-demo
```

Ein paar Sekunden warten, dann beide Ziele:

```
kubectl -n egress-demo exec deploy/curl -- curl -s -m 10 -o /dev/null -w "httpbin.org
-> %{http_code}\n" http://httpbin.org/get
kubectl -n egress-demo exec deploy/curl -- curl -s -m 10 -o /dev/null -w "example.com
-> %{http_code}\n" http://example.com/
```

Erwartet:

```
httpbin.org -> 200
example.com -> 000
command terminated with exit code 28
```

`httpbin.org` geht weiter, weil der Weg ueber den Waypoint erlaubt ist. `example.com` laeuft nach 10 Sekunden in den Timeout (curl-Exit-Code 28), weil der direkte Weg gesperrt ist.

Soll `example.com` auch erlaubt werden, ist der Weg immer derselbe: eine `ServiceEntry` im Namespace `istio-egress` anlegen, und, wenn noetig, eine `AuthorizationPolicy` dazu. Eine Anwendung kann sich den Weg nicht selbst aufmachen.

Aufraeumen

Beide Namespaces enthalten alles, was wir angelegt haben (Waypoint, ServiceEntry, Policies, curl-Pod):

```
kubectl delete namespace egress-demo istio-egress
```

```
kubectl get namespace | grep egress
```

Erwartet: keine Ausgabe.

```
cd ~/manifests
rm -rf egress-ambient
```

Zusammenfassung

Schritt	Was wir gemacht haben	Ergebnis
2	Pod im Mesh, kein Egress-Gateway	`httpbin.org` und `example.com` erreichbar
4	ServiceEntry im Namespace mit Waypoint	200, ztunnel-Log zeigt Waypoint `:15008` als Ziel
5	AuthorizationPolicy auf die ServiceEntry	`GET /get` 200, `GET /headers` und `POST /post` 403
6	`targetPort` 443 und DestinationRule	HTTP in der App, HTTPS zum Ziel
7	Ziel ohne ServiceEntry	200, der Waypoint wird umgangen
8	NetworkPolicy: nur DNS und Waypoint	`httpbin.org` 200, `example.com` Timeout

Merksatz: **Der Waypoint steuert den Weg und die Regeln (Layer 7, TLS). Erzwingen muss ihn eine NetworkPolicy.**

Referenz

- Istio: Egress gateways (Ambient)
- Istio: Simplifying Egress Routing to Wildcard Destinations (Wildcard-Ziele wie `*.amazonaws.com`, ServiceEntry mit `istio.io/use-waypoint` und `DYNAMIC_DNS`)

Debugging

Debugging mit debug/run pod

Why like this ?

- We need a way that a service is mounted into the pod (service-account is used)
- Same serviceAccount that would be used by productpage - pod itself

Variante 1: Debug-Container zum Debuggen

- Debug Container in productpage - pod starten, um Verbindung zu pod -> Review zu debuggen

```
kubectl -n bookinfo get pods | grep productpage
## diesen entsprechend hier verwenden
kubectl -n bookinfo debug productpage-v1-54bb874995-rr7cv -it --image=busybox
```

```
## in der bash
wget -O - http://reviews:9080/reviews/1
```

```
exit
```

V2 - Eigener Pod - Podtester

```
kubectl -n bookinfo run --rm -it podtester --image=busybox --overrides='{ "spec": {
"serviceAccount": "bookinfo-productpage" }  }'
```

Übung: Fehlersuche im Ambient-Mesh - ztunnel-Log und Waypoint-Log

Ziel

Du baust in Bookinfo drei Fehler ein. Von aussen sehen alle gleich aus: Auf der Produktseite fehlen die Ratings. Die Ursache steht aber jedes Mal in einem anderen Log. Danach weisst du, **wo du bei welchem Fehler zuerst hinschaust**.

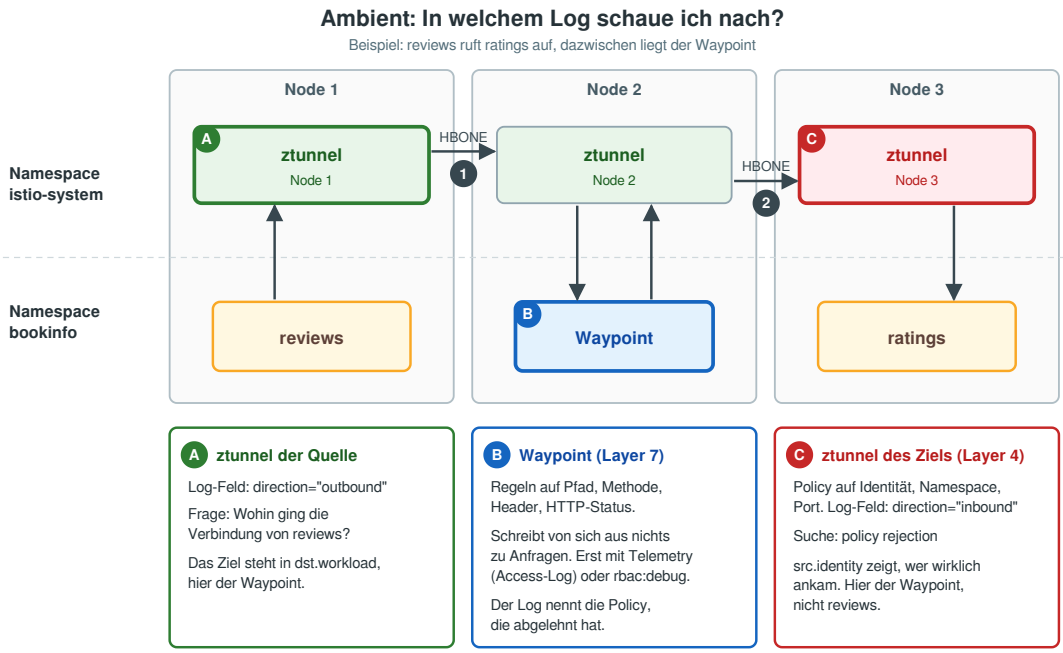
Voraussetzung

- Uebung Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo` mit Waypoint, alle Pods `1/1 Running`).
- Im Namespace gibt es keine AuthorizationPolicies:

```
kubectl -n bookinfo get authorizationpolicy
```

Erwartet: `No resources found in bookinfo namespace.` Sonst vorher loeschen: `kubectl -n bookinfo delete authorizationpolicy --all`

Wegweiser: Wer entscheidet wo?



Frage oder Symptom	Wo schaue ich nach?	Woran erkenne ich es?
Verbindung wird abgelehnt (Policy auf Identitaet, Namespace, Port)	ztunnel auf dem Node des **Ziel-Pods**	`policy rejection`, `direction="inbound"`
Wer kommt beim Ziel wirklich an?	dieselbe Zeile, Feld `src.identity`	bei Waypoint im Pfad steht dort `sa/waypoint`, nicht der Aufrufer
Regel auf Pfad, Methode oder Header greift (HTTP 403)	**Waypoint**-Log (Access-Log per `Telemetry`)	`403 ... rbac_access_denied_matched_policy[...]` nennt die Policy
Ziel hat keine Pods (HTTP 503)	**Waypoint**-Log	`503 UH no_healthy_upstream`, im ztunnel steht nichts
Wohin ging die Verbindung der Quelle?	ztunnel auf dem Node der **Quelle**	`direction="outbound"`, Ziel in `dst.workload`
Wird der Pod vom ztunnel bedient?	`istioctl ztunnel-config workload`	Spalte `PROTOCOL`: `HBONE` (im Mesh) oder `TCP` (nicht im Mesh)

Merksatz: **Layer 4 (Identitaet, Namespace, Port) entscheidet der ztunnel des Ziels. Layer 7 (Pfad, Methode, Header) entscheidet der Waypoint.**

Der Waypoint schreibt von sich aus nichts zu einzelnen Anfragen. Das schalten wir in Fehler 2 ein.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
```

```
mkdir ztunnel-log
cd ztunnel-log
```

Ein kleiner Client im Mesh, von dem aus wir die Produktseite aufrufen:

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo rollout status deploy/sleep
```

Eine Pruefung, die wir immer wieder verwenden. Sie ruft die Produktseite 8-mal auf und zaehlt, wie oft die Ratings fehlen:

```
probe() {
  for i in $(seq 1 8); do
    kubectl -n bookinfo exec deploy/sleep -- curl -s -m 20
productpage:9080/productpage \
      | grep -q "Ratings service is currently unavailable" && echo FEHLER || echo ok
  done | sort | uniq -c
}
```

```
probe
```

Erwartet: `8 ok`

Hinweis: `reviews-v1` ruft `ratings` gar nicht auf. Bei einem Fehler siehst du deshalb nicht 8-mal `FEHLER`, sondern nur bei den Aufrufen, die an `reviews-v2` oder `reviews-v3` gehen.

Wo laeuft welcher ztunnel? Pro Node gibt es einen:

```
kubectl -n istio-system get pods -l app=ztunnel -o wide
kubectl -n bookinfo get pods -o wide
```

Fehler 1: Policy mit falscher Identitaet

Fehler einbauen

Ein Kollege will, dass nur `reviews` auf `ratings` zugreifen darf:

```
## vi 01-ratings-nur-reviews.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ratings-nur-reviews
spec:
  selector:
    matchLabels:
      app: ratings
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/bookinfo-reviews"]
```

```
kubectl -n bookinfo apply -f .
sleep 10
probe
```

Erwartet: mindestens ein `FEHLER`. Die Seite laedt, aber bei den Reviews steht "Ratings service is currently unavailable".

Diagnose: erst der Waypoint

Der erste Impuls: Der Waypoint steht im Pfad, also dort nachsehen.

```
kubectl -n bookinfo logs deploy/waypoint --since=5m | grep ratings
```

Erwartet: keine Ausgabe. Der Waypoint schreibt standardmaessig nichts zu Anfragen, und die Ablehnung passiert hinter ihm.

Diagnose: ztunnel des Ziels

Die Ablehnung entscheidet der ztunnel auf dem Node des `ratings`-Pods. Den finden wir so:

```
NODE=$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
echo $NODE $ZT
```

```
probe
kubectl -n istio-system logs $ZT --since=2m | grep "policy rejection" | tail -2
```

Erwartet (gekuerzt):

```
error  access  connection complete  src.workload="waypoint-..."
src.identity="spiffe://cluster.local/ns/bookinfo/sa/waypoint" ...
dst.workload="ratings-v1-..." direction="inbound" ... error="connection closed due to
policy rejection: allow policies exist, but none allowed"
```

So liest du die Zeile:

Feld	Bedeutung hier
`direction="inbound"`	Die Entscheidung fiel beim Ziel (richtiger ztunnel)
`dst.workload`	Wer abgelehnt wurde: `ratings`
`src.identity`	**Wer wirklich ankam: `sa/waypoint`**, nicht `bookinfo-reviews`
`error=...none allowed`	Es gibt eine ALLOW-Policy, aber die Identitaet steht nicht darin

Ursache: `reviews` ruft `ratings` nicht direkt auf, sondern ueber den Waypoint. Am Ziel kommt deshalb die Identitaet des Waypoints an.

Gegenprobe: Auf den anderen Nodes steht dazu nichts. Das ist der falsche ztunnel.

```
for p in $(kubectl -n istio-system get pod -l app=ztunnel -o name); do echo "-- $p";
kubectl -n istio-system logs $p --since=2m | grep -c "policy rejection"; done
```

Ist `ratings` ueberhaupt im Mesh, und welche Policy kennt der ztunnel?

```
istioctl ztunnel-config workload | grep -E "NAMESPACE|ratings"
istioctl ztunnel-config policies | grep -E "NAMESPACE|ratings"
```

Erwartet: `ratings` mit `PROTOCOL HBONE` und die Policy `ratings-nur-reviews`.

Abhilfe

Die Policy muss die Identitaet erlauben, die tatsaechlich ankommt:

```
## vi 01-ratings-nur-reviews.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ratings-nur-reviews
spec:
  selector:
    matchLabels:
      app: ratings
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/bookinfo/sa/waypoint"]
```

```
kubectl -n bookinfo apply -f .
sleep 10
```

Verifizieren

```
probe
kubectl -n istio-system logs $ZT --since=15s | grep -c "policy rejection"
```

Erwartet: `8 ok` und danach `0`. Keine Ablehnung mehr im ztunnel des Ziels.

Fehler 2: Tippfehler im Pfad einer Layer-7-Regel

Layer-7-Regeln gehoeren an den Waypoint. Hier schreibt der Waypoint zunaechst nichts. Wir schalten sein Access-Log ein:

```
## vi 02-telemetry-waypoint.yml
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: waypoint-logging
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint
  accessLogging:
```

```
  - providers:
    - name: envoy
```

Den Fehler aus Fehler 1 raeumen wir weg, dann bauen wir den neuen ein:

```
kubectl -n bookinfo delete authorizationpolicy ratings-nur-reviews
rm 01-ratings-nur-reviews.yml
```

Fehler einbauen

Ein Kollege will auf `ratings` nur `GET` auf `/ratings/...` erlauben und haengt die Regel an den Waypoint. Er vertippt sich im Pfad (`rating` statt `ratings`):

```
## vi 03-ratings-l7.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ratings-nur-get
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: ratings
  action: ALLOW
  rules:
  - to:
    - operation:
        methods: ["GET"]
        paths: ["/rating/*"]
```

```
kubectl -n bookinfo apply -f .
sleep 10
probe
```

Erwartet: mindestens ein `FEHLER` , wieder fehlen die Ratings.

Diagnose: ztunnel schweigt, der Waypoint redet

Wir rufen die Seite auf und sehen sofort in beide Logs. Das kurze Zeitfenster (15 Sekunden) sorgt dafuer, dass keine alten Zeilen aus Fehler 1 mitzaehlen.

```
probe
kubectl -n istio-system logs $ZT --since=15s | grep -c "policy rejection"
kubectl -n bookinfo logs deploy/waypoint --since=15s | grep '"GET /ratings' | tail -2
```

Erwartet:

- Anzahl `policy rejection` : `0` . Der ztunnel hat nichts abgelehnt.
- Zwei Zeilen aus dem Waypoint (gekuerzt):

```
"GET /ratings/0 HTTP/1.1" 403 - rbac_access_denied_matched_policy[none] ...
"ratings:9080" ... inbound-vip|9080|http|ratings.bookinfo.svc.cluster.local
```

`403` und `rbac_access_denied_matched_policy[none]` heissen: Es gibt eine Policy, aber **keine ihrer Regeln hat auf die Anfrage gepasst**. Der Pfad `/ratings/0` passt nicht auf `/rating/*` . Steht statt `[none]` ein Name in den Klammern, hat genau diese Policy die Anfrage abgelehnt.

Alternative ohne Telemetry, mit mehr Details (laut, nur zum Debuggen):

```
istioctl proxy-config log deploy/waypoint -n bookinfo --level rbac:debug | grep rbac
probe
kubectl -n bookinfo logs deploy/waypoint --since=15s | grep "enforced" | tail -2
istioctl proxy-config log deploy/waypoint -n bookinfo --reset | grep rbac
```

Erwartet: `rbac: debug` , dann `enforced denied, matched policy none` , zuletzt `rbac: warning` . Die Warnung zu `istio-sidecar-injector` beim Zuruecksetzen ist harmlos.

Abhilfe

```
## vi 03-ratings-l7.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ratings-nur-get
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: ratings
  action: ALLOW
  rules:
  - to:
    - operation:
        methods: ["GET"]
        paths: ["/ratings/*"]
```

```
kubectl -n bookinfo apply -f .
sleep 10
```

Verifizieren

```
probe
kubectl -n bookinfo logs deploy/waypoint --since=15s | grep '"GET /ratings' | tail -1
```

Erwartet: `8 ok` und eine Zeile `"GET /ratings/0 HTTP/1.1" 200 ... via_upstream`

Fehler 3: Ziel hat keine Pods

Die Policy aus Fehler 2 raeumen wir weg. Dann nehmen wir `ratings` die Pods weg:

```
kubectl -n bookinfo delete authorizationpolicy ratings-nur-get
rm 03-ratings-l7.yml
kubectl -n bookinfo scale deploy ratings-v1 --replicas=0
sleep 10
```

Dasselbe Bild wie vorher, aber die Ursache steht woanders. Wieder beide Logs:

```
probe
kubectl -n istio-system logs $ZT --since=15s | grep -c "policy rejection"
kubectl -n bookinfo logs deploy/waypoint --since=15s | grep '"GET /ratings' | tail -1
```

Erwartet: mindestens ein `FEHLER`, dann `0` und eine Zeile mit `503 UH no_healthy_upstream`. Der Waypoint findet keinen Pod, an den er weiterleiten kann. Der ztunnel hat nichts abgelehnt.

Abhilfe und Verifizieren

```
kubectl -n bookinfo scale deploy ratings-v1 --replicas=1
kubectl -n bookinfo rollout status deploy/ratings-v1
sleep 5
probe
```

Erwartet: `8 ok`

Zusammenfassung

Fehler	Symptom	Welches Log	Zeile
1: falsche Identitaet in der Policy (Layer 4)	Ratings fehlen	ztunnel auf dem Node des Ziels	`policy rejection ... none allowed`, `src.identity=sa/waypoint`
2: Tippfehler im Pfad (Layer 7)	Ratings fehlen	Waypoint (Access-Log)	`403 rbac_access_denied_matched_policy[none]`
3: Ziel hat keine Pods	Ratings fehlen	Waypoint (Access-Log)	`503 UH no_healthy_upstream`

Vorgehen bei "Ratings fehlen" oder aehnlichen Fehlern:

1. Waypoint-Access-Log ansehen (Status 403 oder 503, nennt die Policy oder fehlende Upstreams).
2. Steht dort nichts, ist die Verbindung schon vorher abgelehnt worden: ztunnel auf dem Node des Ziels (`policy rejection`).
3. Im Zweifel: `istioctl ztunnel-config workload` (ist der Pod im Mesh?) und `istioctl ztunnel-config policies` .

Troubleshooting

Problem	Pruefen
`grep "policy rejection"` ist leer	Richtiger Node? `echo $NODE $ZT`, Pod mit `-o wide` nochmal ansehen. `--since` laenger waehlen
`probe` zeigt trotz Fehler `8 ok`	10 Sekunden warten, Policy braucht einen Moment. `reviews-v1` ruft `ratings` nicht auf, `probe` nochmal ausfuehren
Waypoint-Log zeigt nichts zu `GET /ratings`	`Telemetry` angelegt? `kubectl -n bookinfo get telemetry`
`istioctl x authz check` zeigt einen Fehler	Normal bei Ambient: `ratings` hat keinen Envoy. Stattdessen `istioctl ztunnel-config policies`

Aufraeumen

```
kubectl -n bookinfo delete authorizationpolicy --all
kubectl -n bookinfo delete telemetry waypoint-logging
kubectl -n bookinfo delete -f ~/istio/samples/sleep/sleep.yaml
```

```
kubectl -n bookinfo get pods
```

Erwartet: alle Bookinfo-Pods und der Waypoint `1/1 Running`, kein `sleep`.

Ref

- https://istio.io/latest/docs/ambient/usage/troubleshoot-ztunnel/
- https://istio.io/latest/docs/ambient/usage/l7-features/

Übung (Ambient): Gateway umgeht den Waypoint - in den Logs erkennen und beheben (ingress-use-waypoint)

Ziel

Du legst am Waypoint eine Layer-7-Regel an. Aus dem Mesh greift sie, ueber das Gateway von aussen aber nicht. Es gibt dabei **keine Fehlermeldung**. Du lernst, die Umgehung in drei Logs zu erkennen, behebst sie mit einem Label und pruefst in denselben Logs nach.

Voraussetzung

- Uebung Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo` mit Waypoint und `bookinfo-gateway`, alle Pods `1/1 Running`).
- Im Namespace gibt es keine AuthorizationPolicies und das Label aus dieser Uebung ist noch nicht gesetzt:

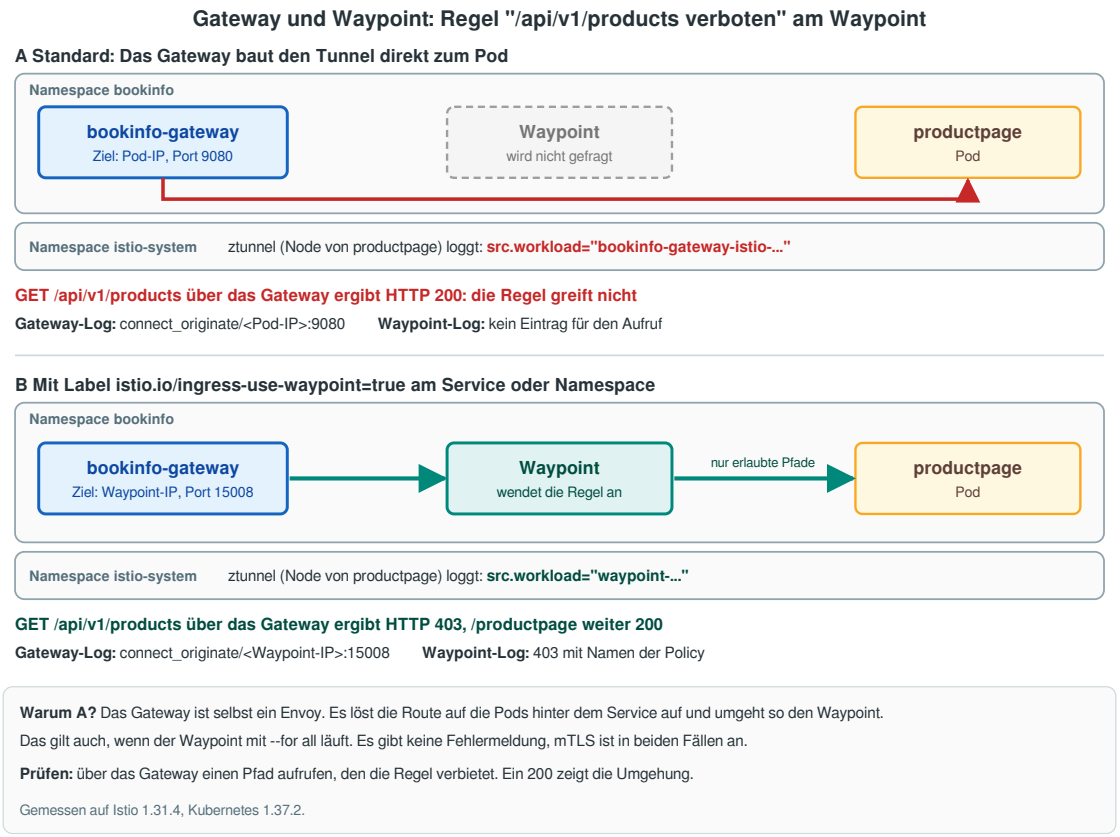
```
kubectl -n bookinfo get authorizationpolicy
kubectl get namespace bookinfo --show-labels
kubectl -n bookinfo get service productpage --show-labels
```

Erwartet: `No resources found in bookinfo namespace.` und nirgends `istio.io/ingress-use-waypoint`

```
## zur Sicherheit führen wir das Kommando aus, um das Label zu entfernen, wenn es da ist
kubectl label namespace bookinfo istio.io/ingress-use-waypoint-
```

.

Hintergrund



Frage	Antwort
Warum umgeht das Gateway den Waypoint?	Das Gateway ist selbst ein Envoy. Es loest die Route auf die Pods hinter dem Service auf und baut den Tunnel (HBONE) direkt zum Pod.
Hilft ein Waypoint mit `--for all`?	Nein. Unser Waypoint laeuft mit `--for all`, das Gateway geht trotzdem daran vorbei.
Warum faellt das nicht auf?	Der Aufruf klappt (HTTP 200), mTLS ist an. Nur die Regel am Waypoint wird nicht gefragt.
Was hilft?	Das Label `istio.io/ingress-use-waypoint=true` am Service oder am Namespace.

Die drei Spuren, die wir uns ansehen:

Log	Umgehung (ohne Label)	Ueber den Waypoint (mit Label)
Gateway (Access-Log)	Ziel ist `<IP des Pods>:9080`	Ziel ist `<IP des Waypoints>:15008`
Waypoint (Access-Log)	kein Eintrag fuer den Aufruf	Eintrag, bei verbotenem Pfad `403` mit Namen der Policy
ztunnel auf dem Node des Ziel-Pods	`src.workload="bookinfo-gateway-istio-..."`	`src.workload="waypoint-..."`

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir gateway-waypoint
cd gateway-waypoint
```

Ein kleiner Client im Mesh fuer den Vergleich:

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo rollout status deploy/sleep
```

Die Adresse des Gateways und der ztunnel auf dem Node von `productpage`:

```
GW=$(kubectl -n bookinfo get gateway bookinfo-gateway -o
jsonpath='{.status.addresses[0].value}')
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
echo $GW $NODE $ZT
```

Die IP-Adressen, die gleich in den Logs auftauchen. Notiere dir die von `productpage` und die von `waypoint`:

```
kubectl -n bookinfo get pods -o wide | grep -E "NAME|productpage|waypoint"
```

Schritt 2: Access-Logs einschalten

Gateway und Waypoint schreiben von sich aus nichts zu einzelnen Anfragen. Wir schalten das fuer beide ein:

```
## vi 01-telemetry.yml
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: waypoint-logging
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint
  accessLogging:
  - providers:
    - name: envoy
---
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: gateway-logging
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: bookinfo-gateway
```

```
  accessLogging:
  - providers:
    - name: envoy
```

```
kubectl -n bookinfo apply -f .
```

Schritt 3: Regel am Waypoint

Der Pfad `/api/v1/products` von `productpage` soll gesperrt sein. Die Regel haengt per `targetRefs` am Service, also wertet sie der Waypoint aus:

```
## vi 02-api-verboten.yml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: productpage-api-verboten
spec:
  targetRefs:
  - kind: Service
    group: ""
    name: productpage
  action: DENY
  rules:
  - to:
    - operation:
        ports: ["9080"]
        paths: ["/api/v1/products*"]
```

```
kubectl -n bookinfo apply -f .
sleep 10
```

Warum diese Regel?

Die Regel ist hier nur ein Messfuehler. Sie macht sichtbar, ob ein Aufruf durch den Waypoint laeuft. Ohne sie liefern beide Wege `200`, und mTLS ist in beiden Faellen an. Mit ihr gilt:

- `403` auf `/api/v1/products`: Der Aufruf lief durch den Waypoint, die Regel wurde gefragt.
- `200` auf `/api/v1/products`: Der Aufruf ging am Waypoint vorbei.

Teil	Grund
`targetRefs` auf den Service `productpage`	Damit wertet der **Waypoint** die Regel aus. Mit `selector` (am Pod) wuerde der ztunnel sie auswerten, und der kann keine Pfade pruefen.
`action: DENY`	Sperrt nur diesen einen Pfad. Bei `ALLOW` waere alles andere automatisch verboten, und Bookinfo liefe nicht mehr.
`paths: ["/api/v1/products*"]`	Ein Pfad ist Layer 7, das kann nur der Waypoint. Der Pfad ist ueber die `HTTPRoute` von aussen erreichbar, und die Produktseite selbst braucht ihn nicht. Deshalb bleibt `/productpage` bei `200`.
`ports: ["9080"]`	Ohne die Angabe warnt Istio beim Anlegen: "will deny all traffic to TCP ports under its scope". Mit dem Port ist die Regel auf den HTTP-Port begrenzt.

In der Praxis steht hier eine echte Regel, zum Beispiel " `/admin` nur fuer bestimmte Aufrufer" oder eine JWT-Pruefung. Das Problem ist dasselbe: Sie gilt fuer Aufrufe aus dem Mesh, aber nicht fuer Aufrufe ueber das Gateway, solange das Label aus Schritt 6 fehlt.

Schritt 4: Aus dem Mesh greift die Regel, ueber das Gateway nicht

Aufruf aus dem Mesh:

```
kubectl -n bookinfo exec deploy/sleep -- curl -s -o /dev/null -w "%{http_code}\n"
productpage:9080/api/v1/products
```

Erwartet: `403`

Derselbe Pfad ueber das Gateway. Wir merken uns vorher die Uhrzeit, damit wir in den Logs nur die neuen Zeilen sehen (das `sleep 2` trennt den Aufruf von eben sicher davon ab):

```
sleep 2
T0=$(date -u +%Y-%m-%dT%H:%M:%SZ)
for i in 1 2 3; do curl -s -o /dev/null -w "%{http_code}\n"
http://$GW/api/v1/products; done
sleep 5
```

Erwartet: 3-mal `200` . **Die Regel greift nicht, und nichts weist darauf hin.**

Schritt 5: Die Umgehung in den Logs erkennen

Spur 1: Gateway-Log - wohin schickt das Gateway?

```
kubectl -n bookinfo logs deploy/bookinfo-gateway-istio --since-time=$T0 | grep
"/api/v1/products" | tail -1
```

Erwartet (gekuerzt):

```
"GET /api/v1/products HTTP/1.1" 200 - via_upstream ...
"envoy://connect_originate/192.168.93.138:9080"
outbound|9080||productpage.bookinfo.svc.cluster.local ...
```

Nur das Ziel, einmal pro Adresse gezaehlt:

```
kubectl -n bookinfo logs deploy/bookinfo-gateway-istio --since-time=$T0 | grep
"/api/v1/products" | grep -o 'connect_originate/[0-9.:]*' | sort | uniq -c
```

Erwartet: `3 connect_originate/<IP von productpage>:9080` . Das Gateway spricht **direkt mit dem Pod** (Port der Anwendung).

Woran erkenne ich die direkte Verbindung?

Feld in der Zeile	Bedeutung
`200`	Der Pfad ist verboten, die Antwort kommt trotzdem. Die Regel wurde nicht gefragt.
`"envoy://connect_originate/<IP>:9080"`	**Das Ziel des Gateways.** Port `9080` ist der Port der Anwendung, die IP ist die von `productpage`.

	Ueber den Waypoint stuende hier `<IP des Waypoints>:15008`.
`outbound|9080||productpage.bookinfo.svc.cluster.local`	Name des Ziels im Envoy (Cluster). Er ist in beiden Faellen gleich und sagt nichts ueber den Weg.
`<IP>:80` danach	Adresse des Gateways selbst, auf der die Anfrage ankam.

Was heisst connect_originate?

`connect_originate` heisst sinngemaess "hier beginnt ein CONNECT-Tunnel". Es ist der Name eines internen Listeners im Envoy des Gateways. Das Gateway schickt die Anfrage nicht direkt ins Netz. Es uebergibt sie intern an diesen Listener, und der baut den Tunnel auf: HTTP/2 `CONNECT` ueber mTLS, also HBONE. Die Adresse hinter dem Schraegstrich ist das Ziel des Tunnels.

- `connect` steht fuer die HTTP-Methode `CONNECT`, mit der ein Tunnel geoeffnet wird.
- `originate` heisst, dass der Tunnel hier beginnt.
- `envoy://...` ist eine Adresse innerhalb des Envoy, keine Netzwerkadresse.

Das Feld zeigt also zweierlei: Der Aufruf geht verschluesselt per HBONE raus, und er geht an den Pod statt an den Waypoint.

Was ist ein Listener?

Ein Listener ist in Envoy die Stelle, an der Verbindungen angenommen werden: eine Adresse mit Port, auf der Envoy lauscht, dazu die Regeln, was mit dem Verkehr passiert. Envoy arbeitet in einer festen Kette:

Baustein	Aufgabe	Beispiel im Gateway
Listener	nimmt Verbindungen an	`0.0.0.0:80`
Route	entscheidet nach Host und Pfad, wohin es geht	`/api/v1/products` geht an `productpage`
Cluster	das Ziel als Gruppe, entspricht einem Service	`outbound|9080||productpage.bookinfo.svc.cluster.local`
Endpoint	die konkrete Adresse hinter dem Cluster	`<IP von productpage>:9080`

Du schreibst die Listener nicht selbst. istiod erzeugt sie aus deinen Objekten: aus dem Listener im `Gateway` (Port 80) wird der Listener `0.0.0.0:80`, aus der `HTTPRoute` die Route, aus dem `Service` der Cluster, aus seinen Pods die Endpoints.

```
istioctl proxy-config listeners deploy/bookinfo-gateway-istio -n bookinfo
```

Erwartet:

```
ADDRESSES PORT  MATCH DESTINATION
          0     ALL   Cluster: connect_originate
0.0.0.0   80    ALL   Route: http.80
0.0.0.0   15021 ALL   Inline Route: /healthz/ready*
0.0.0.0   15090 ALL   Inline Route: /stats/prometheus*
```

Die erste Zeile ohne Adresse und mit Port `0` ist der **interne Listener** `connect_originate`. Er lauscht nicht im Netz und nimmt nur Verkehr an, den derselbe Envoy ihm uebergibt. Der Weg einer Anfrage im Gateway-Pod: Listener Port 80, Route auf `productpage`, interner Listener `connect_originate`, HBONE-Tunnel nach draussen. Das ist ein einziger Envoy-Prozess. Der ztunnel auf dem Node des Gateways ist daran nicht beteiligt.

Die anderen Bausteine siehst du mit `routes`, `clusters` und `endpoints` statt `listeners`.

Spur 2: Waypoint-Log - kam der Aufruf vorbei?

```
kubectl -n bookinfo logs deploy/waypoint --since-time=$T0 | grep -c "/api/v1/products"
```

Erwartet: `0`. Der Waypoint hat die drei Aufrufe nie gesehen.

Zum Vergleich der Aufruf aus dem Mesh von vorhin (der lag vor `T0`, deshalb mit `--since=5m`):

```
kubectl -n bookinfo logs deploy/waypoint --since=5m | grep "/api/v1/products" | tail
-1
```

Erwartet (gekuerzt): `"GET /api/v1/products HTTP/1.1" 403 -
rbac_access_denied_matched_policy[ns[bookinfo]-policy[productpage-api-verboten]-rule[0]]
...`

Spur 3: ztunnel - wer kam beim Pod an?

```
kubectl -n istio-system logs $ZT --since-time=$T0 | grep 'dst.workload="productpage' |
grep 'direction="inbound"' | tail -1
```

Erwartet (gekuerzt):

```
info  access  connection complete  src.workload="bookinfo-gateway-istio-..."
src.identity="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio" ...
dst.hbone_addr=192.168.93.138:9080 dst.workload="productpage-v1-..."
direction="inbound" ...
```

Nur die Quelle, gezaehlt:

```
kubectl -n istio-system logs $ZT --since-time=$T0 | grep 'dst.workload="productpage' |
grep -o 'src.workload="[^"]*"' | sort | uniq -c
```

Erwartet: nur `src.workload="bookinfo-gateway-istio-..."`. Die Zahl ist kleiner als 3, weil das Gateway eine Verbindung fuer mehrere Anfragen nutzt.

So liest du die Zeile:

Feld	Bedeutung hier
`direction="inbound"`	Sicht des Ziels (ztunnel auf dem Node von `productpage`)
`src.workload`, `src.identity`	**Das Gateway selbst kam an**, nicht der Waypoint
`dst.hbone_addr`	IP des Pods mit Port `9080`

Hinweise:

- Der ztunnel schreibt die Zeile erst, wenn die Verbindung endet. Beim Gateway dauert das etwa 2 Sekunden, deshalb das `sleep 5`.
- Einen `outbound`-Eintrag vom Gateway gibt es nicht. Das Gateway baut den Tunnel selbst und laeuft nicht ueber den ztunnel (Label `istio.io/dataplane-mode=none`).

Schritt 6: Beheben mit dem Label ingress-use-waypoint

```
## kubectl -n bookinfo label service productpage istio.io/ingress-use-waypoint=true
## besser
kubectl label namespace bookinfo istio.io/ingress-use-waypoint=true
sleep 10
```

```
T0=$(date -u +%Y-%m-%dT%H:%M:%SZ)
for i in 1 2 3; do curl -s -o /dev/null -w "%{http_code}\n"
http://$GW/api/v1/products; done
curl -s -o /dev/null -w "%{http_code}\n" http://$GW/productpage
sleep 5
```

Erwartet: 3-mal `403`, danach `200` fuer `/productpage`. Die Regel greift jetzt auch von aussen, der Rest der Seite laeuft weiter.

Schritt 7: Gegenprobe in denselben Logs

Gateway: Das Ziel ist jetzt der Waypoint, Port `15008` (HBONE):

```
kubectl -n bookinfo logs deploy/bookinfo-gateway-istio --since-time=$T0 | grep -o
'connect_originate/[0-9.:]*' | sort | uniq -c
```

Erwartet: `4 connect_originate/<IP des Waypoints>:15008`

Waypoint: Die Aufrufe sind da, mit Namen der Policy:

```
kubectl -n bookinfo logs deploy/waypoint --since-time=$T0 | grep "/api/v1/products" |
tail -1
kubectl -n bookinfo logs deploy/waypoint --since-time=$T0 | grep -c "/api/v1/products"
```

Erwartet: `403 - rbac_access_denied_matched_policy[ns[bookinfo]-policy[productpage-api-verboten]-rule[0]]` und `3`

ztunnel: Beim Pod kommt jetzt der Waypoint an. Abgelehnte Aufrufe erreichen den Pod gar nicht, die Zeile stammt vom Aufruf auf `/productpage`:

```
kubectl -n istio-system logs $ZT --since-time=$T0 | grep 'dst.workload="productpage' |
grep -o 'src.workload="[^"]*"' | sort | uniq -c
```

Erwartet: nur `src.workload="waypoint-..."`

Aufraeumen

```
kubectl label namespace bookinfo istio.io/ingress-use-waypoint-
kubectl -n bookinfo delete authorizationpolicy productpage-api-verboten
kubectl -n bookinfo delete telemetry waypoint-logging gateway-logging
kubectl -n bookinfo delete -f ~/istio/samples/sleep/sleep.yaml
```

```
sleep 10
curl -s -o /dev/null -w "%{http_code}\n" http://$GW/api/v1/products
kubectl -n bookinfo get pods
```

Erwartet: `200`, alle Bookinfo-Pods, das Gateway und der Waypoint `1/1 Running`, kein `sleep`.

Zusammenfassung

Aufruf auf `/api/v1/products`	**Ohne Label**	**Mit `istio.io/ingress-use-waypoint=true`**
Aus dem Mesh (`sleep`)	`403`	`403`
Ueber das Gateway	`200` (Regel umgangen)	`403`
Gateway-Log, Ziel	Pod-IP, Port `9080`	Waypoint-IP, Port `15008`
Waypoint-Log	kein Eintrag	`403` mit Namen der Policy
ztunnel am Ziel, `src.workload`	`bookinfo-gateway-istio-...`	`waypoint-...`

Merksatz: **Eine Regel am Waypoint immer auch von aussen ueber das Gateway testen, und zwar mit einem Pfad, den sie verbietet.** Ein `200` auf dem verbotenen Pfad zeigt die Umgehung.

Gemessen mit Istio 1.31.4 und Kubernetes 1.37.2.

Ref

- https://istio.io/latest/docs/ambient/usage/waypoint/
- https://istio.io/latest/docs/ambient/usage/troubleshoot-ztunnel/
- Uebung: Fehlersuche im Ambient-Mesh - ztunnel-Log und Waypoint-Log
- Uebung: Kong als API-Gateway vor Istio Ambient (dort umgeht Kong den Waypoint, mit anderer Loesung)

Monitoring, Logging & Observability

Installation Jaeger

Ziel

Jaeger laeuft im Cluster und ist danach unter `https://jaeger.<dein-user>.do.t3isp.de/jaeger/` erreichbar, geschuetzt mit Benutzer und Passwort.

Hintergrund

Das Gateway von Istio kann kein Basic Auth. Deshalb laeuft im Cluster zusaetzlich **Traefik** als zweiter Gateway-API-Controller. Beide arbeiten mit denselben Objekten (`Gateway`, `HTTPRoute`), nur die `GatewayClass` ist eine andere.

Was	Istio	Traefik
`GatewayClass`	`istio`	`traefik`
Gateway	`bookinfo-gateway` (Namespace `bookinfo`)	`traefik-gateway` (Namespace `traefik`)
Aufgabe hier	Verkehr ins Mesh (Bookinfo)	Addons wie Jaeger veroeffentlichen
Basic Auth	nein	ja, ueber eine `Middleware`

Die `HTTPRoute` ist Standard-Gateway-API. Nur der Verweis auf die `Middleware` ist eine Erweiterung von Traefik (Filter `ExtensionRef`) und funktioniert mit keinem anderen Controller.

Voraussetzung

- Istio ist installiert (Namespace `istio-system`).
- Traefik, DNS und Zertifikat sind schon da (siehe Merkkasten).

> ***Merkkasten: Traefik, DNS und Zertifikat sind schon eingerichtet***
>
> *Der Trainer hat fuer jeden Cluster vorab eingerichtet:*
>
> - *Traefik im Namespace `traefik` mit dem Gateway `traefik-gateway` (Listener `web` fuer HTTP, `websecure` fuer HTTPS)*
> - *den DNS-Eintrag `*.<dein-user>.do.t3isp.de`, z. B. `jaeger.tln5.do.t3isp.de`*
> - *ein Zertifikat von Let's Encrypt fuer `*.<dein-user>.do.t3isp.de`*
> - *die Weiterleitung von `http://` auf `https://`*
>
> *Du musst **nichts** im DNS eintragen und kein Zertifikat holen.*

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir jaeger
cd jaeger
```

```
export HOST=jaeger.$(whoami).do.t3isp.de
echo $HOST $(dig +short $HOST)
```

Erwartet: Hostname und eine IP. Ist die IP leer: Trainer fragen.

```
kubectl get gatewayclass traefik
kubectl -n traefik get gateway traefik-gateway
```

Erwartet: `ACCEPTED True` und `PROGRAMMED True`, `ADDRESS` ist die IP von oben.

Schritt 2: Jaeger installieren

```
kubectl apply -f ~/istio/samples/addons/jaeger.yaml
kubectl -n istio-system rollout status deployment jaeger
```

```
kubectl -n istio-system get pods -l app=jaeger
kubectl -n istio-system get svc tracing jaeger-collector
```

Erwartet: 1 Pod `Running`. Der Service `tracing` (Port 80) ist die Oberflaeche, `jaeger-collector` (Port 4317) nimmt spaeter die Traces von Istio an.

Schritt 3: Benutzer und Passwort (Basic Auth)

Benutzer `training`, Passwort `myS3cr3t!`. Im Secret steht nicht das Passwort, sondern eine htpasswd-Zeile (bcrypt).

```
## vi 01-basic-auth-secret.yml
apiVersion: v1
kind: Secret
metadata:
  name: addons-basic-auth
type: Opaque
stringData:
  users: "training:$2b$12$CfOZaJ.Tr0zu6PfpbuCjzeKiQ2PzZARfP.CbC6tRU/7OvEHCIOREm"
```

```
## vi 02-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: basic-auth
spec:
  basicAuth:
    secret: addons-basic-auth
    realm: Training
```

```
kubectl -n istio-system apply -f 01-basic-auth-secret.yml -f 02-middleware.yml
```

Schritt 4: HTTPRoute anlegen

Die Route haengt am Listener `websecure` (HTTPS) des Traefik-Gateways. Jaeger laeuft unter dem Pfad `/jaeger`, deshalb leitet die erste Regel `/` dorthin weiter. Die zweite Regel fragt das Passwort ab und reicht an Jaeger weiter.

```
## vi 03-httproute.yml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: jaeger
spec:
  parentRefs:
  - name: traefik-gateway
    namespace: traefik
    sectionName: websecure
  hostnames:
  - ${HOST}
  rules:
  - matches:
    - path:
        type: Exact
        value: /
    filters:
    - type: RequestRedirect
      requestRedirect:
        path:
          type: ReplaceFullPath
          replaceFullPath: /jaeger/
        statusCode: 302
  - filters:
```

```
    - type: ExtensionRef
      extensionRef:
        group: traefik.io
        kind: Middleware
        name: basic-auth
    backendRefs:
    - name: tracing
      port: 80
```

```
envsubst < 03-httproute.yml | kubectl -n istio-system apply -f -
```

```
kubectl -n istio-system get httproute jaeger
kubectl -n istio-system get httproute jaeger -o
jsonpath='{.status.parents[0].conditions[*].type}{"\n"}
{.status.parents[0].conditions[*].status}{"\n"}'
```

Erwartet: `Accepted ResolvedRefs` und darunter `True True` .

Schritt 5: Testen

```
curl -s -o /dev/null -w '%{http_code}\n' https://$HOST/jaeger/
```

Erwartet: `401` (ohne Benutzer und Passwort kommt niemand durch).

```
curl -s -o /dev/null -w '%{http_code}\n' -u 'training:myS3cr3t!' https://$HOST/jaeger/
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' http://$HOST/jaeger/
```

Erwartet: `200` , danach `301 https://...` (HTTP wird auf HTTPS umgeleitet).

```
echo https://$HOST/jaeger/
```

Die URL im Browser oeffnen und mit `training` / `myS3cr3t!` anmelden: Die Oberflaeche von Jaeger erscheint, Schloss in der Adressleiste, Aussteller `Let's Encrypt` .

Traces siehst du noch keine. Die kommen in der Uebung Distributed Tracing mit Jaeger.

Fehlersuche

Beobachtung	Pruefen
`404` statt `401`	Stimmt der Hostname der Route? `kubectl -n istio-system get httproute jaeger`
`401` trotz richtigem Passwort	Secret hat den Schluessel `users` (nicht `auth` wie bei ingress-nginx)
`500` oder `404` mit Passwort	Gibt es die `Middleware` im selben Namespace wie die Route? `kubectl -n istio-system get middleware`
Hostname wird nicht gefunden	`dig +short $HOST`, sonst Trainer fragen

Ref

- https://istio.io/latest/docs/ops/integrations/jaeger/

- https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/basicauth/
- https://gateway-api.sigs.k8s.io/api-types/httproute/

Distributed Tracing mit Jaeger

Ziel

Istio schickt Traces an Jaeger. Du siehst in Jaeger den Weg einer Anfrage an Bookinfo: vom Gateway zu `productpage` und von dort ueber den Waypoint zu `details`, `reviews` und `ratings`.

Voraussetzung

- Installation Jaeger ist durchgefuehrt (Jaeger laeuft, `https://jaeger.<dein-user>.do.t3isp.de/jaeger/` fragt nach Benutzer und Passwort).
- Bookinfo demo in ambient ausrollen ist durchgefuehrt (Namespace `bookinfo` mit Waypoint, Gateway `bookinfo-gateway`).
- Istio ist mit `istioctl install --set profile=ambient` installiert.

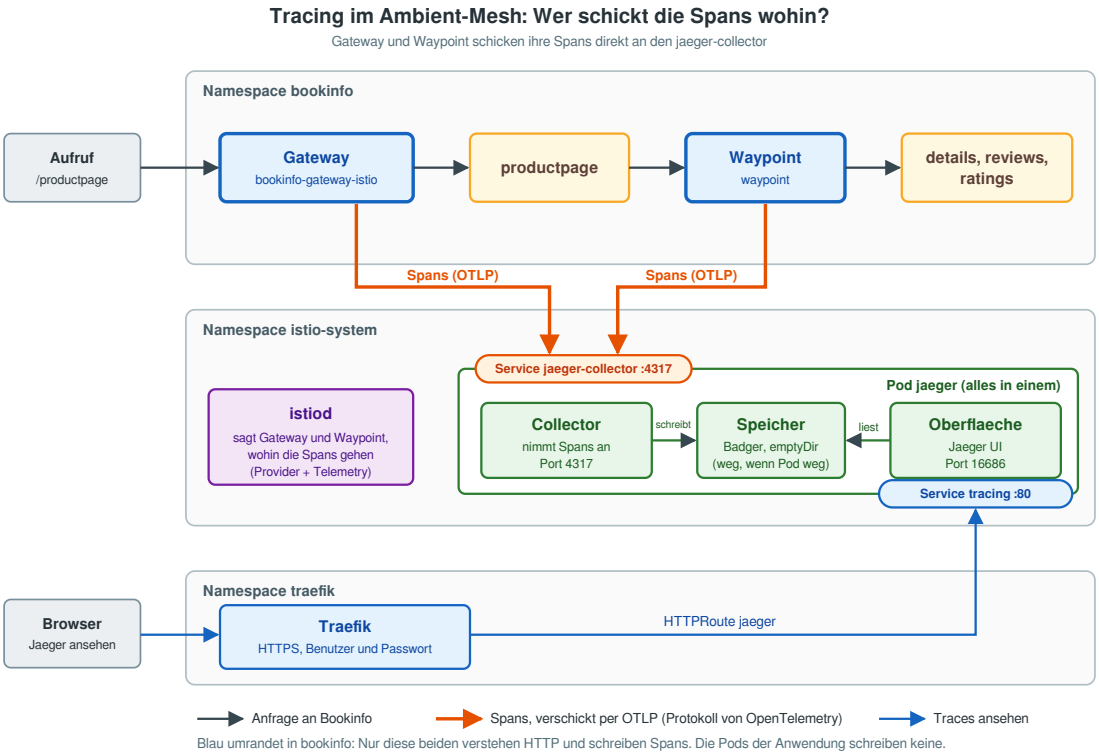
Hintergrund

Zwei Dinge fehlen nach der Installation noch:

Was	Womit	Wirkung
Istio kennt Jaeger nicht	`meshConfig.extensionProviders` (Datei fuer `istioctl install -f`)	Provider `jaeger` zeigt auf `jaeger-collector`, Port 4317 (OpenTelemetry)
Niemand nutzt den Provider	`Telemetry` in `istio-system`	Tracing fuer das ganze Mesh einschalten

Im Ambient-Mode schreiben nur **Gateway** und **Waypoint** Spans, weil nur sie HTTP verstehen (Layer 7). Der `ztunnel` arbeitet auf Layer 4 und erzeugt keine Spans. Ohne Waypoint gibt es deshalb nur den Span vom Gateway.

Von wo nach wo gehen die Spans?



1. Eine Anfrage laeuft durch Gateway und Waypoint. Jeder der beiden schreibt dazu einen Span.
2. Beide schicken ihre Spans direkt an den Service `jaeger-collector`, Port 4317. Dahinter nimmt der Collector im Pod `jaeger` sie an und speichert sie.
3. Die Oberflaeche von Jaeger liest aus demselben Speicher. Du erreichst sie ueber Traefik und den Service `tracing`.

Der Pod `jaeger` aus den Istio-Samples ist "alles in einem": Collector, Speicher und Oberflaeche. Der Speicher liegt in einem `emptyDir`. Wird der Pod geloescht, sind die Traces weg. Fuer Produktion ist das nicht gedacht.

Welche Rolle spielt OpenTelemetry (OTel)?

Begriff	Was es ist	In dieser Uebung
OpenTelemetry	Offener Standard fuer Traces, Metriken und Logs	Gibt das Format und das Protokoll der Spans vor
OTLP	Das Protokoll von OpenTelemetry	Gateway und Waypoint schicken ihre Spans per OTLP (gRPC) an Port 4317
Provider `opentelemetry`	Eintrag in `meshConfig.extensionProviders`	Sagt Istio: "sprich OTLP mit diesem Service". Der Name `jaeger` ist frei gewaehlt
OpenTelemetry Collector	Eigene Komponente, die Spans annimmt und weiterverteilt	**Laeuft hier nicht.** Jaeger nimmt OTLP selbst an

OTel ist hier also nur die **Sprache** zwischen Istio und Jaeger, keine eigene Komponente im Cluster. In Produktion steht oft ein OpenTelemetry Collector dazwischen, z. B. um Spans an mehrere Ziele zu schicken oder zu filtern. Dann zeigt der Provider auf den Collector statt auf Jaeger, sonst aendert sich an Istio nichts.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir tracing-jaeger
cd tracing-jaeger
```

```
export HOST=jaeger.$(whoami).do.t3isp.de
export GATEWAY_IP=$(kubectl -n bookinfo get gateway bookinfo-gateway -o
jsonpath='{.status.addresses[0].value}')
echo $HOST $GATEWAY_IP
```

Beide Werte muessen gefuellt sein.

Schritt 2: Istio den Provider Jaeger mitgeben

Vorher: Es gibt noch keinen Provider.

```
kubectl -n istio-system get cm istio -o jsonpath='{.data.mesh}' | grep -A4
extensionProviders
```

Erwartet: keine Ausgabe.

Die Einstellung kommt in eine Datei. `kind: IstioOperator` ist dabei nur das Dateiformat, das `istioctl install` liest. Im Cluster laeuft **kein** Operator (den gibt es in aktuellen Istio-Versionen nicht mehr), und die Datei wird auch nicht mit `kubectl apply` angewendet.

```
## vi 01-istio-tracing.yml
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: ambient
  meshConfig:
    extensionProviders:
    - name: jaeger
      opentelemetry:
        service: jaeger-collector.istio-system.svc.cluster.local
        port: 4317
  values:
    pilot:
      traceSampling: 100
```

```
istioctl install -f 01-istio-tracing.yml -y
```

`traceSampling: 100` zeichnet jede Anfrage auf (Standard: 1 von 100). Gut zum Ueben, in Produktion zu viel.

```
kubectl -n istio-system rollout status deployment istiod
kubectl -n istio-system get cm istio -o jsonpath='{.data.mesh}' | grep -A4
extensionProviders
```

Erwartet:

```
extensionProviders:
- name: jaeger
  opentelemetry:
    port: 4317
    service: jaeger-collector.istio-system.svc.cluster.local
```

Schritt 3: Tracing einschalten

```
## vi 02-telemetry-jaeger.yml
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
spec:
  tracing:
  - providers:
    - name: jaeger
```

```
kubectl -n istio-system apply -f 02-telemetry-jaeger.yml
kubectl -n istio-system get telemetry
```

Ein `Telemetry` im Namespace `istio-system` gilt fuer das ganze Mesh.

Nachsehen, was beim Waypoint angekommen ist:

```
istioctl proxy-config listener deploy/waypoint -n bookinfo -o json | grep -m1 -A9
envoy.tracers.opentelemetry
```

Erwartet: `envoy.tracers.opentelemetry` mit dem Ziel `outbound|4317||jaeger-collector.istio-system.svc.cluster.local` und `"serviceName": "waypoint.bookinfo"`. Das ist der Name, unter dem der Waypoint spaeter in Jaeger steht.

Schritt 4: Anfragen erzeugen

```
for i in $(seq 1 100); do curl -s -o /dev/null "http://$GATEWAY_IP/productpage"; done
curl -s -o /dev/null -w '%{http_code}\n' "http://$GATEWAY_IP/productpage"
```

Erwartet: `200`.

Hast du die Uebung mit HTTPS und Let's Encrypt gemacht, antwortet das Gateway nur noch unter seinem Namen. Dann stattdessen:

```
## nur nach der Uebung HTTPS mit Let's Encrypt
## for i in $(seq 1 100); do curl -s -o /dev/null
"https://bookinfo.$(whoami).do.t3isp.de/productpage"; done
```

Schritt 5: Traces in Jaeger ansehen

Zuerst auf der Kommandozeile: Welche Services kennt Jaeger?

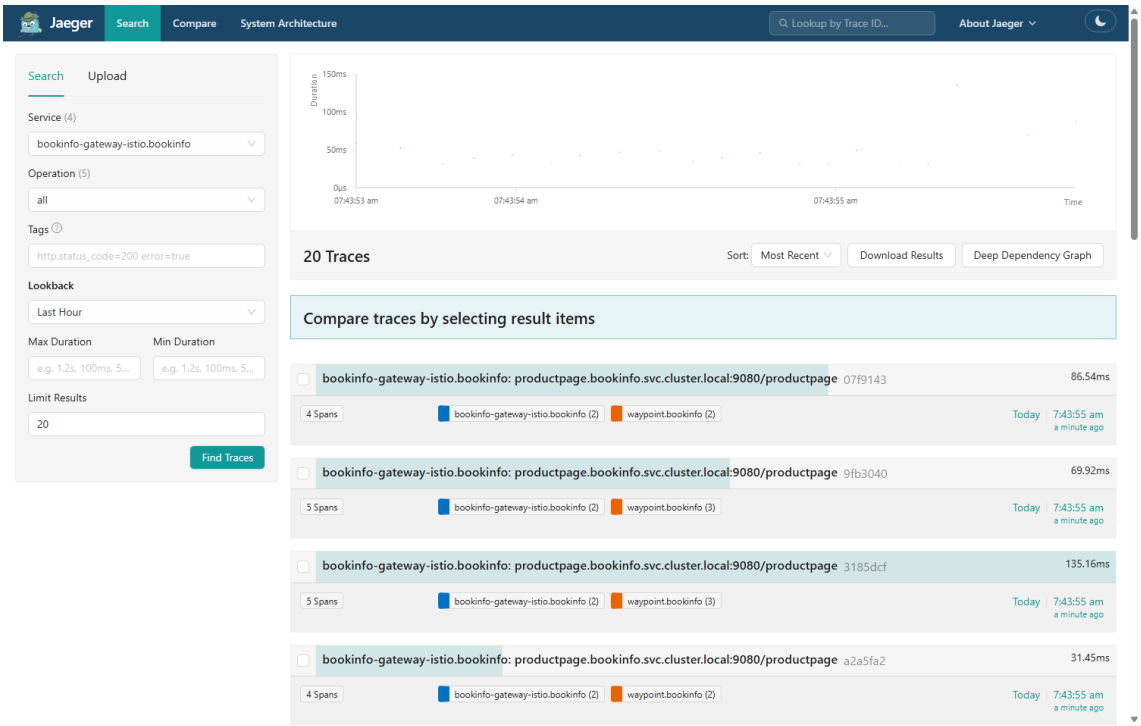
```
sleep 10
curl -s -u 'training:myS3cr3t!' https://$HOST/jaeger/api/services
```

Erwartet: `bookinfo-gateway-istio.bookinfo` und `waypoint.bookinfo` (dazu `jaeger` selbst).

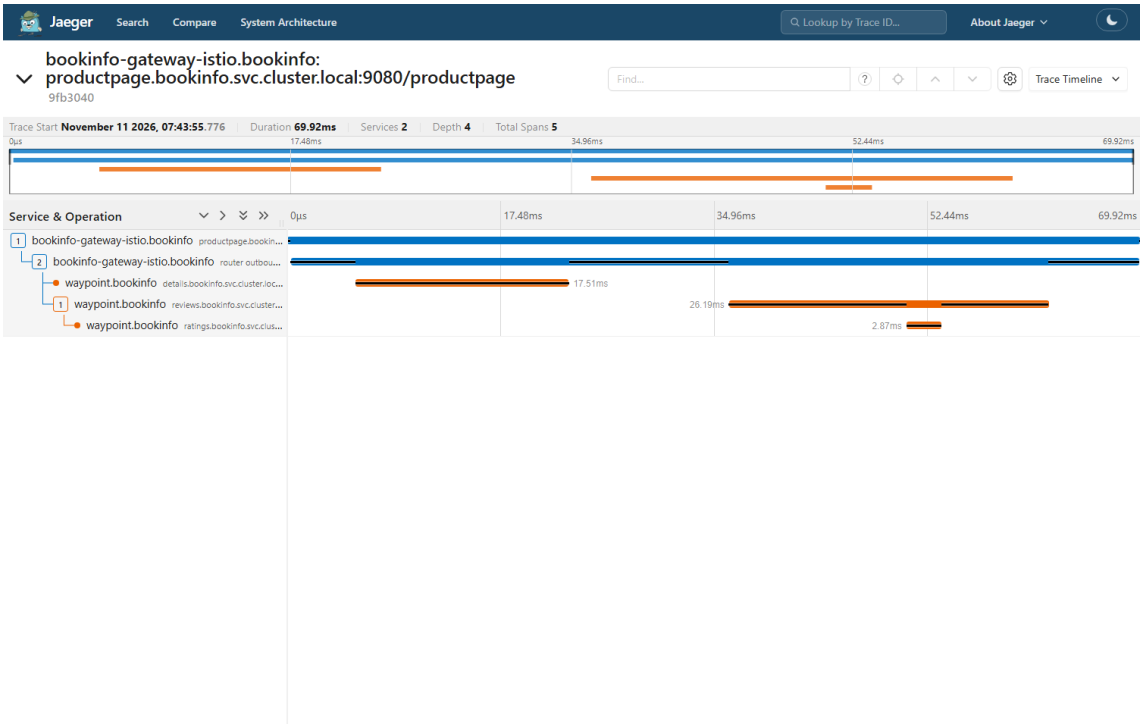
```
echo https://$HOST/jaeger/
```

Die URL im Browser oeffnen (Benutzer `training`, Passwort `myS3cr3t!`):

```
## Links unter "Service": bookinfo-gateway-istio.bookinfo auswaehlen
## Ganz unten: "Find Traces" anklicken
## Oben einen Punkt bzw. einen Trace anklicken
```



Ein Klick auf einen Trace zeigt die einzelnen Spans:



Ein Trace von `/productpage` hat 4 oder 5 Spans:

Wer schreibt den Span	Ziel
`bookinfo-gateway-istio.bookinfo`	`productpage` (2 Spans: Anfrage und Weiterleitung)
`waypoint.bookinfo`	`details`
`waypoint.bookinfo`	`reviews`
`waypoint.bookinfo`	`ratings` (fehlt, wenn `reviews-v1` antwortet: v1 ruft `ratings` nicht auf)

Fragen

1. Warum taucht `productpage` nicht als eigener Service in Jaeger auf?
2. Was siehst du in Jaeger, wenn der Namespace keinen Waypoint hat?
3. Warum haengen die Spans von Gateway und Waypoint in **einem** Trace zusammen?

Loesungen

1. In Jaeger steht unter "Service" der Proxy, der den Span schreibt. Im Sidecar-Mode hat jeder Pod einen eigenen Proxy, der sich mit dem Namen der Anwendung meldet. Im Ambient-Mode gibt es diese Proxys nicht, HTTP sehen nur Gateway und Waypoint. `productpage` taucht deshalb nur als Ziel im Namen eines Spans auf, nicht als eigener Service.
2. Nur noch die Spans vom Gateway. Der `ztunnel` (Layer 4) schreibt keine.
3. Bookinfo reicht die Trace-Header der eingehenden Anfrage (z. B. `traceparent`, `x-b3-traceid`) an seine eigenen Aufrufe weiter. Ohne das waeren es einzelne, unverbundene Traces. Das muss die Anwendung tun, Istio kann es nicht fuer sie.

Aufraeumen

```
kubectl -n istio-system delete -f 02-telemetry-jaeger.yml
```

Istio wieder ohne Provider und mit dem Standard-Sampling:

```
istioctl install --set profile=ambient -y
kubectl -n istio-system rollout status deployment istiod
kubectl -n istio-system get cm istio -o jsonpath='{.data.mesh}' | grep -A4
extensionProviders
```

Erwartet: keine Ausgabe mehr.

Ref

- https://istio.io/latest/docs/tasks/observability/distributed-tracing/jaeger/
- https://istio.io/latest/docs/tasks/observability/telemetry/

Installation Prometheus

Ziel

Prometheus laeuft im Cluster und ist danach unter `https://prometheus.<dein-user>.do.t3isp.de/` erreichbar, geschuetzt mit Benutzer und Passwort.

Hintergrund

Das Gateway von Istio kann kein Basic Auth. Deshalb laeuft im Cluster zusaetzlich **Traefik** als zweiter Gateway-API-Controller. Beide arbeiten mit denselben Objekten (`Gateway` , `HTTPRoute`), nur die `GatewayClass` ist eine andere.

Was	Istio	Traefik
`GatewayClass`	`istio`	`traefik`
Gateway	`bookinfo-gateway` (Namespace `bookinfo`)	`traefik-gateway` (Namespace `traefik`)
Aufgabe hier	Verkehr ins Mesh (Bookinfo)	Addons wie Prometheus veroeffentlichen
Basic Auth	nein	ja, ueber eine `Middleware`

Die `HTTPRoute` ist Standard-Gateway-API. Nur der Verweis auf die `Middleware` ist eine Erweiterung von Traefik (Filter `ExtensionRef`) und funktioniert mit keinem anderen Controller.

Voraussetzung

- Istio ist installiert (Namespace `istio-system`).
- Traefik, DNS und Zertifikat sind schon da (siehe Merkkasten).

> ***Merkkasten: Traefik, DNS und Zertifikat sind schon eingerichtet***
>
> *Der Trainer hat fuer jeden Cluster vorab eingerichtet:*
>
> - *Traefik im Namespace `traefik` mit dem Gateway `traefik-gateway` (Listener `web` fuer HTTP, `websecure` fuer HTTPS)*
> - *den DNS-Eintrag `*.<dein-user>.do.t3isp.de` , z. B. `prometheus.tln5.do.t3isp.de`*
> - *ein Zertifikat von Let's Encrypt fuer `*.<dein-user>.do.t3isp.de`*
> - *die Weiterleitung von `http://` auf `https://`*

> *Du musst **nichts** im DNS eintragen und kein Zertifikat holen.*

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir -p prometheus
cd prometheus
```

```
export HOST=prometheus.$(whoami).do.t3isp.de
echo $HOST $(dig +short $HOST)
```

Erwartet: Hostname und eine IP. Ist die IP leer: Trainer fragen.

```
kubectl get gatewayclass traefik
kubectl -n traefik get gateway traefik-gateway
```

Erwartet: `ACCEPTED True` und `PROGRAMMED True`, `ADDRESS` ist die IP von oben.

Schritt 2: Prometheus installieren

```
kubectl apply -f ~/istio/samples/addons/prometheus.yaml
kubectl -n istio-system rollout status deployment prometheus
```

```
kubectl -n istio-system get pods -l app.kubernetes.io/name=prometheus
kubectl -n istio-system get svc prometheus
```

Erwartet: 1 Pod `Running` (`2/2`). Der Service `prometheus` (Port 9090) ist Oberflaeche und API zugleich.

Prometheus hat von sich aus keinen Login. Den Schutz uebernimmt Traefik.

Schritt 3: Benutzer und Passwort (Basic Auth)

Benutzer `training`, Passwort `myS3cr3t!`. Im Secret steht nicht das Passwort, sondern eine htpasswd-Zeile (bcrypt).

```
## vi 01-basic-auth-secret.yml
apiVersion: v1
kind: Secret
metadata:
  name: addons-basic-auth
type: Opaque
stringData:
  users: "training:$2b$12$CfOZaJ.Tr0zu6PfpbuCjzeKiQ2PzZARfP.CbC6tRU/7OvEHCIOREm"
```

```
## vi 02-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: basic-auth
spec:
```

```
  basicAuth:
    secret: addons-basic-auth
    realm: Training
```

```
kubectl -n istio-system apply -f 01-basic-auth-secret.yml -f 02-middleware.yml
```

Secret und `Middleware` sind fuer alle Addons dieselben. Hast du sie schon bei Jaeger, Grafana oder Kiali angelegt, meldet `kubectl` hier `configured` oder `unchanged`. Beides ist in Ordnung.

Schritt 4: HTTPRoute anlegen

Die Route haengt am Listener `websecure` (HTTPS) des Traefik-Gateways. Die Regel fragt das Passwort ab und reicht an Prometheus weiter.

```
## vi 03-httproute.yml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: prometheus
spec:
  parentRefs:
  - name: traefik-gateway
    namespace: traefik
    sectionName: websecure
  hostnames:
  - ${HOST}
  rules:
  - filters:
    - type: ExtensionRef
      extensionRef:
        group: traefik.io
        kind: Middleware
        name: basic-auth
    backendRefs:
    - name: prometheus
      port: 9090
```

```
envsubst < 03-httproute.yml | kubectl -n istio-system apply -f -
```

```
kubectl -n istio-system get httproute prometheus
kubectl -n istio-system get httproute prometheus -o
jsonpath='{.status.parents[0].conditions[*].type}{"\n"}
{.status.parents[0].conditions[*].status}{"\n"}'
```

Erwartet: `Accepted ResolvedRefs` und darunter `True True`.

Schritt 5: Testen

```
curl -s -o /dev/null -w '%{http_code}\n' https://$HOST/-/ready
```

Erwartet: `401` (ohne Benutzer und Passwort kommt niemand durch).

```
curl -s -o /dev/null -w '%{http_code}\n' -u 'training:myS3cr3t!' https://$HOST/-/ready
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' http://$HOST/-/ready
```

Erwartet: `200`, danach `301 https://...` (HTTP wird auf HTTPS umgeleitet).

```
curl -s -u 'training:myS3cr3t!' "https://$HOST/api/v1/query?query=count(up)"
```

Erwartet: `"status":"success"` und eine Zahl groesser 0 (so viele Ziele fragt Prometheus ab). Ist `result` noch leer (`[]`): 30 Sekunden warten und wiederholen, Prometheus hat dann noch nichts abgefragt.

```
echo https://$HOST/
```

Die URL im Browser oeffnen und mit `training` / `myS3cr3t!` anmelden: Die Oberflaeche von Prometheus erscheint, Schloss in der Adressleiste, Aussteller `Let's Encrypt`.

Abfragen zu Istio machst du in der Uebung Metriken mit Prometheus auswerten.

Fehlersuche

Beobachtung	Pruefen
`404` statt `401`	Stimmt der Hostname der Route? `kubectl -n istio-system get httproute prometheus`
`401` trotz richtigem Passwort	Secret hat den Schluessel `users` (nicht `auth` wie bei ingress-nginx)
`500` oder `404` mit Passwort	Gibt es die `Middleware` im selben Namespace wie die Route? `kubectl -n istio-system get middleware`
Hostname wird nicht gefunden	`dig +short $HOST`, sonst Trainer fragen

Ref

- https://istio.io/latest/docs/ops/integrations/prometheus/
- https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/basicauth/
- https://gateway-api.sigs.k8s.io/api-types/httproute/

Metriken mit Prometheus auswerten

Walkthrough

Commandline: Send Traffic to the mesh

```
## Change to your ip accordingly
GATEWAY_URL=164.90.237.35
curl "http://$GATEWAY_URL/productpage"
```

Search for metrics

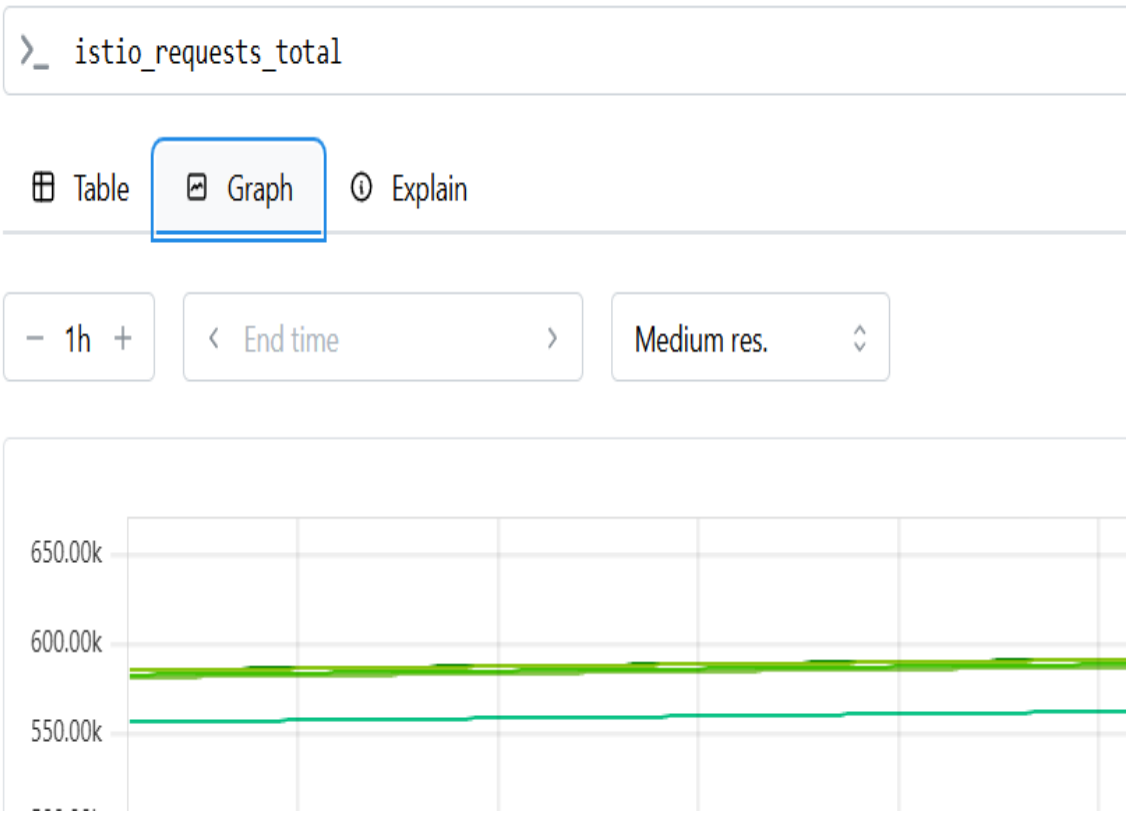
```
## Login to https://prometheus.tlnxx.do.t3isp.de (training / myS3cr3t!)
## Click on Query
```




```
## in enter expression -> enter
istio_requests_total
## THEN -> Click Execute
```

** This will show all the requests to istio **

```
## You can also click on Graph to see the graphical representation
```



Try other queries

```
## 1. Only request to the productpage in namespace bookinfo
istio_requests_total{destination_service="productpage.bookinfo.svc.cluster.local"}
```

- Hinweis, wenn du mehrere Einträge siehst wie **hier:**

istio_requests_total{connection_security_policy="unknown", destination_app="productpage", destination_canonical_revision="v1", destination_canonical_service="productpage", destination_cluster="Kubernetes", destination_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-productpage", destination_service="productpage.bookinfo.svc.cluster.local", destination_service_name="productpage", destination_service_namespace="bookinfo", destination_version="v1", destination_workload="productpage-v1", destination_workload_namespace="bookinfo", gateway_istio_io_managed="istio.io-gateway-controller", gateway_networking_k8s_io_gateway_class_name="istio", gateway_networking_k8s_io_gateway_name="bookinfo-gateway", instance="192.168.197.11:15020", job="kubernetes-pods", namespace="bookinfo", node="k8s-w3", pod="bookinfo-gateway-istio-78f879f6c7-nxkpn", pod_template_hash="78f879f6c7", reporter="source", request_protocol="http", response_code="200", response_flags="-", service_istio_io_canonical_name="bookinfo-gateway-istio", service_istio_io_canonical_revision="latest", sidecar_istio_io_inject="false", source_canonical_revision="latest", source_canonical_service="bookinfo-gateway-istio", source_cluster="Kubernetes", source_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio", source_workload="bookinfo-gateway-istio", source_workload_namespace="bookinfo"}

istio_requests_total{connection_security_policy="unknown", destination_app="productpage", destination_canonical_revision="v1", destination_canonical_service="productpage", destination_cluster="Kubernetes", destination_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-productpage", destination_service="productpage.bookinfo.svc.cluster.local", destination_service_name="productpage", destination_service_namespace="bookinfo", destination_version="v1", destination_workload="productpage-v1", destination_workload_namespace="bookinfo", gateway_istio_io_managed="istio.io-gateway-controller", gateway_networking_k8s_io_gateway_class_name="istio", gateway_networking_k8s_io_gateway_name="bookinfo-gateway", instance="192.168.197.11:15020", job="kubernetes-pods", namespace="bookinfo", node="k8s-w3", pod="bookinfo-gateway-istio-78f879f6c7-nxkpn", pod_template_hash="78f879f6c7", reporter="source", request_protocol="http", response_code="302", response_flags="-", service_istio_io_canonical_name="bookinfo-gateway-istio", service_istio_io_canonical_revision="latest", sidecar_istio_io_inject="false", source_canonical_revision="latest", source_canonical_service="bookinfo-gateway-istio", source_cluster="Kubernetes", source_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio", source_workload="bookinfo-gateway-istio", source_workload_namespace="bookinfo"}

istio_requests_total{connection_security_policy="unknown", destination_app="productpage", destination_canonical_revision="v1", destination_canonical_service="productpage", destination_cluster="Kubernetes", destination_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-productpage", destination_service="productpage.bookinfo.svc.cluster.local", destination_service_name="productpage", destination_service_namespace="bookinfo", destination_version="v1", destination_workload="productpage-v1", destination_workload_namespace="bookinfo", gateway_istio_io_managed="istio.io-gateway-controller", gateway_networking_k8s_io_gateway_class_name="istio", gateway_networking_k8s_io_gateway_name="bookinfo-gateway", instance="192.168.197.11:15020", job="kubernetes-pods", namespace="bookinfo", node="k8s-w3", pod="bookinfo-gateway-istio-78f879f6c7-nxkpn", pod_template_hash="78f879f6c7", reporter="source", request_protocol="http", response_code="304", response_flags="-", service_istio_io_canonical_name="bookinfo-gateway-istio", service_istio_io_canonical_revision="latest", sidecar_istio_io_inject="false", source_canonical_revision="latest", source_canonical_service="bookinfo-gateway-istio", source_cluster="Kubernetes", source_principal="spiffe://cluster.local/ns/bookinfo/sa/bookinfo-gateway-istio", source_workload="bookinfo-gateway-istio", source_workload_namespace="bookinfo"}

- .. unterscheiden sich diese bspw. an dieser Stelle durch den Response - Code (200, 403 etc.)

```
## 2. Total count of all requests to v3 of the reviews service:
istio_requests_total{destination_service="reviews.bookinfo.svc.cluster.local",
destination_version="v3"}
```

```
## 3.  This query returns the current total count of all requests to the v3 of the
reviews service.
## Rate of requests over the past 5 minutes to all instances of the productpage
service:
## =~ <- **ist ein regulärer Ausdruck**
rate(istio_requests_total{destination_service=~"productpage.*", response_code="200"}
[5m])
```

Reference:

- https://istio.io/latest/docs/tasks/observability/metrics/querying-metrics/

Installation Grafana

Ziel

Grafana laeuft im Cluster und ist danach unter `https://grafana.<dein-user>.do.t3isp.de/` erreichbar, geschuetzt mit Benutzer und Passwort.

Hintergrund

Das Gateway von Istio kann kein Basic Auth. Deshalb laeuft im Cluster zusaetzlich **Traefik** als zweiter Gateway-API-Controller. Beide arbeiten mit denselben Objekten (`Gateway` , `HTTPRoute`), nur die `GatewayClass` ist eine andere.

Was	Istio	Traefik
`GatewayClass`	`istio`	`traefik`
Gateway	`bookinfo-gateway` (Namespace `bookinfo`)	`traefik-gateway` (Namespace `traefik`)
Aufgabe hier	Verkehr ins Mesh (Bookinfo)	Addons wie Grafana veroeffentlichen
Basic Auth	nein	ja, ueber eine `Middleware`

Die `HTTPRoute` ist Standard-Gateway-API. Nur der Verweis auf die `Middleware` ist eine Erweiterung von Traefik (Filter `ExtensionRef`) und funktioniert mit keinem anderen Controller.

Voraussetzung

- Istio ist installiert (Namespace `istio-system`).
- Prometheus ist installiert, siehe Installation Prometheus. Grafana holt seine Daten von dort.
- Traefik, DNS und Zertifikat sind schon da (siehe Merkkasten).

> ***Merkkasten: Traefik, DNS und Zertifikat sind schon eingerichtet***
>
> *Der Trainer hat fuer jeden Cluster vorab eingerichtet:*
>
> - *Traefik im Namespace* `traefik` *mit dem Gateway* `traefik-gateway` *(Listener* `web` *fuer HTTP,* `websecure` *fuer HTTPS)*
> - *den DNS-Eintrag* `*.<dein-user>.do.t3isp.de` *, z. B.* `grafana.tln5.do.t3isp.de`
> - *ein Zertifikat von Let's Encrypt fuer* `*.<dein-user>.do.t3isp.de`
> - *die Weiterleitung von* `http://` *auf* `https://`
>
> *Du musst* ***nichts*** *im DNS eintragen und kein Zertifikat holen.*

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir -p grafana
cd grafana
```

```
export HOST=grafana.$(whoami).do.t3isp.de
echo $HOST $(dig +short $HOST)
```

Erwartet: Hostname und eine IP. Ist die IP leer: Trainer fragen.

```
kubectl get gatewayclass traefik
kubectl -n traefik get gateway traefik-gateway
```

Erwartet: `ACCEPTED True` und `PROGRAMMED True` , `ADDRESS` ist die IP von oben.

Schritt 2: Grafana installieren

```
kubectl -n istio-system get deployment prometheus
```

Erwartet: `READY 1/1`. Fehlt Prometheus: zuerst Installation Prometheus.

```
kubectl apply -f ~/istio/samples/addons/grafana.yaml
kubectl -n istio-system rollout status deployment grafana
```

```
kubectl -n istio-system get pods -l app.kubernetes.io/name=grafana
kubectl -n istio-system get svc grafana
```

Erwartet: 1 Pod `Running`. Der Service `grafana` (Port 3000) ist die Oberflaeche.

Grafana selbst fragt nicht nach einem Login (anonymer Zugang im Addon). Den Schutz uebernimmt Traefik.

Schritt 3: Benutzer und Passwort (Basic Auth)

Benutzer `training`, Passwort `myS3cr3t!`. Im Secret steht nicht das Passwort, sondern eine htpasswd-Zeile (bcrypt).

```
## vi 01-basic-auth-secret.yml
apiVersion: v1
kind: Secret
metadata:
  name: addons-basic-auth
type: Opaque
stringData:
  users: "training:$2b$12$CfOZaJ.Tr0zu6PfpbuCjzeKiQ2PzZARfP.CbC6tRU/7OvEHCIOREm"
```

```
## vi 02-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: basic-auth
spec:
  basicAuth:
    secret: addons-basic-auth
    realm: Training
```

```
kubectl -n istio-system apply -f 01-basic-auth-secret.yml -f 02-middleware.yml
```

Secret und `Middleware` sind fuer alle Addons dieselben. Hast du sie schon bei Jaeger, Prometheus oder Kiali angelegt, meldet `kubectl` hier `configured` oder `unchanged`. Beides ist in Ordnung.

Schritt 4: HTTPRoute anlegen

Die Route haengt am Listener `websecure` (HTTPS) des Traefik-Gateways. Die Regel fragt das Passwort ab und reicht an Grafana weiter.

```
## vi 03-httproute.yml
apiVersion: gateway.networking.k8s.io/v1
```

```
kind: HTTPRoute
metadata:
  name: grafana
spec:
  parentRefs:
  - name: traefik-gateway
    namespace: traefik
    sectionName: websecure
  hostnames:
  - ${HOST}
  rules:
  - filters:
    - type: ExtensionRef
      extensionRef:
        group: traefik.io
        kind: Middleware
        name: basic-auth
    backendRefs:
    - name: grafana
      port: 3000
```

```
envsubst < 03-httproute.yml | kubectl -n istio-system apply -f -
```

```
kubectl -n istio-system get httproute grafana
kubectl -n istio-system get httproute grafana -o
jsonpath='{.status.parents[0].conditions[*].type}{"\n"}
{.status.parents[0].conditions[*].status}{"\n"}'
```

Erwartet: `Accepted ResolvedRefs` und darunter `True True`.

Schritt 5: Testen

```
curl -s -o /dev/null -w '%{http_code}\n' https://$HOST/api/health
```

Erwartet: `401` (ohne Benutzer und Passwort kommt niemand durch).

```
curl -s -o /dev/null -w '%{http_code}\n' -u 'training:myS3cr3t!'
https://$HOST/api/health
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' http://$HOST/api/health
```

Erwartet: `200`, danach `301 https://...` (HTTP wird auf HTTPS umgeleitet).

```
curl -s -u 'training:myS3cr3t!' "https://$HOST/api/search?query=Istio" | grep -o
'"title":"[^"]*"'
```

Erwartet: die Dashboards von Istio, z. B. `Istio Mesh Dashboard` und `Istio Service Dashboard`.

```
echo https://$HOST/
```

Die URL im Browser oeffnen und mit `training` / `myS3cr3t!` anmelden: Die Oberflaeche von Grafana erscheint, Schloss in der Adressleiste, Aussteller `Let's Encrypt`.

Die Dashboards siehst du dir in der Uebung Grafana Dashboards fuer istio an.

Fehlersuche

Beobachtung	Pruefen
`404` statt `401`	Stimmt der Hostname der Route? `kubectl -n istio-system get httproute grafana`
`401` trotz richtigem Passwort	Secret hat den Schluessel `users` (nicht `auth` wie bei ingress-nginx)
`500` oder `404` mit Passwort	Gibt es die `Middleware` im selben Namespace wie die Route? `kubectl -n istio-system get middleware`
Hostname wird nicht gefunden	`dig +short $HOST`, sonst Trainer fragen
Dashboards sind leer	Laeuft Prometheus? `kubectl -n istio-system get deployment prometheus`

Ref

- https://istio.io/latest/docs/ops/integrations/grafana/
- https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/basicauth/
- https://gateway-api.sigs.k8s.io/api-types/httproute/

Grafana Dashboards für istio

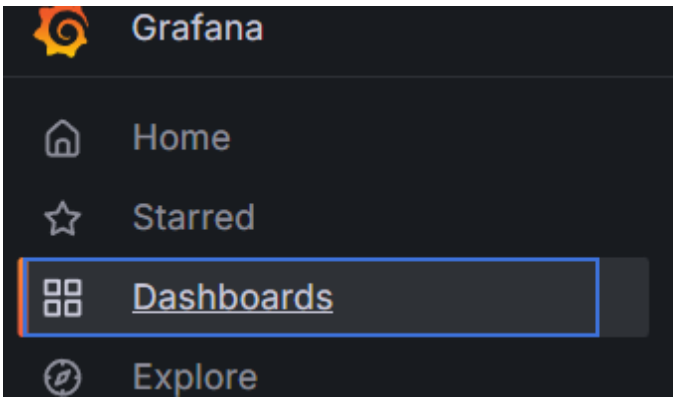
Prerequisites

- bookinfo app is installed
- Grafana dashboard is setup here: https://grafana.tlnxx.do.t3isp.de (Login: training / myS3cr3t!)
- demo profile is used

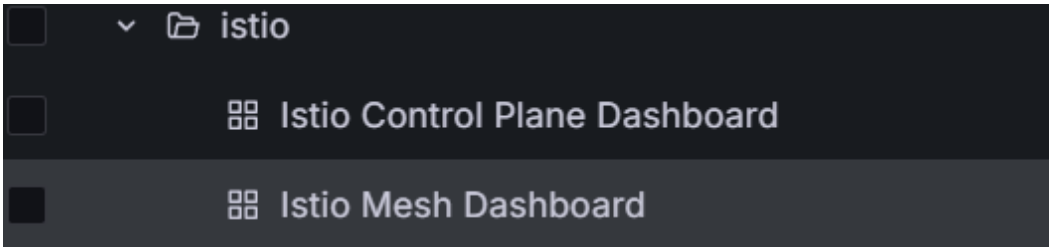
Walkthrough

```
## Open dashboard
https://grafana.tlnxx.do.t3isp.de
```

```
Go to dashboards
```




```
Select Dashboard from folder istio ->
```

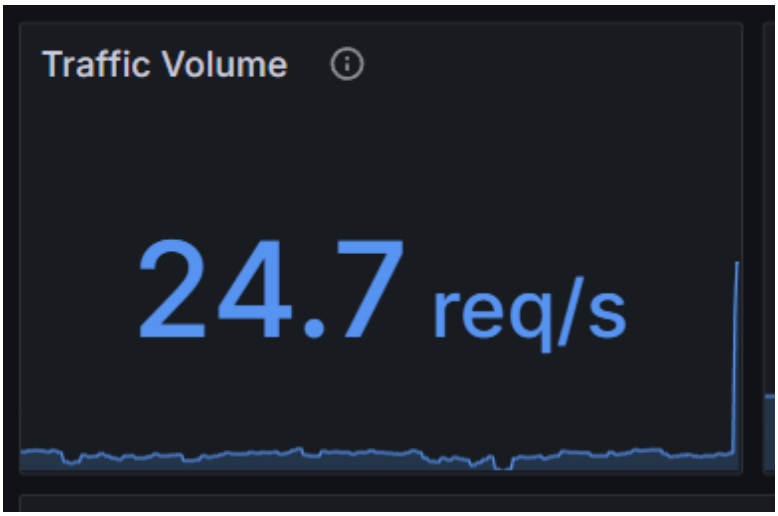



```
## Send traffic to the /productpage
## commandline
## Adjust to your IP
GATEWAY_URL=164.90.237.35
for i in $(seq 1 100); do curl -s -o /dev/null "http://$GATEWAY_URL/productpage"; done
```

```
## In the dashboard click reload (right corner) - a couple of times
```

```
<img width="334" height="288" alt="image" src="https://github.com/user-
attachments/assets/6d117df1-679a-46c5-b52a-82afe68ce138" />
```

```
## The traffic volumen should increase
```



Next stop: Service Dashboard

- Click on Dashboards -> Istio -> Service Dashboard

Next stop: Workload Dashboad

- Click on Dashboard -> Istio -> Workload Dashboard

Sinnvolle istio Metriken und Dashboards - Teil 1

Was ist Latenz?

Die Zeit, die ein Request von Absenden bis Antwort braucht — gemessen in Millisekunden. In Istio misst der **Envoy Sidecar** diese Zeit automatisch für jeden Request.

Inbound vs. Outbound Latenz

Stell dir Service A → Service B → Service C vor.

Aus Sicht von Service B:

- **Inbound-Latenz** = wie lange B insgesamt braucht, um auf einen eingehenden Request zu antworten (gemessen vom Envoy-Proxy *vor* B). Das ist die Gesamtzeit aus Sicht des Aufrufers.
- **Outbound-Latenz** = wie lange die Requests dauern, die B *selbst nach außen* schickt (z.B. an Service C). Gemessen vom Envoy-Proxy *von B* Richtung C.

Warum das beim Troubleshooting hilft

```
Inbound-Latenz von B:  800ms
Outbound-Latenz von B → C:  750ms
```

→ B selbst ist nicht das Problem — er wartet fast die ganze Zeit auf C.

```
Inbound-Latenz von B:  800ms
Outbound-Latenz von B → C:  50ms
```

→ B selbst braucht ~750ms für seine eigene Verarbeitung — **B ist der Flaschenhals**.

Die Differenz zwischen Inbound und Outbound zeigt dir also, ob der Service selbst langsam ist oder ob er auf einen anderen wartet.

Goldene Signale in Prometheus

Latenz pro Service (P99)

```
histogram_quantile(0.99,
  sum(rate(istio_request_duration_milliseconds_bucket{reporter="destination"}[5m]))
  by (destination_service, le)
)
```

Error Rate

```
sum(rate(istio_requests_total{response_code=~"5.."}[5m]))
by (destination_service)
/
sum(rate(istio_requests_total[5m]))
by (destination_service)
```

Throughput / Request Rate

```
sum(rate(istio_requests_total[5m])) by (destination_service, source_workload)
```

Grafana Dashboards

Istio bringt fertige Dashboards mit — die wichtigsten für Troubleshooting:

- **Istio Mesh Dashboard** → Überblick über das gesamte Mesh (Global Request Rate, Error Rate)
- **Istio Service Dashboard** → Latenz-Heatmap pro Service, P50/P90/P99 auf einen Blick
- **Istio Workload Dashboard** → Inbound/Outbound Latenz pro Pod, hier siehst du, *welcher* Workload bremst

---

Systematisch den Engpass eingrenzen

1. **Mesh Dashboard** → welcher Service hat hohe Latenz oder Error Rate?
2. **Service Dashboard** → auf den verdächtigen Service klicken → Inbound vs. Outbound Latenz vergleichen
3. **Workload Dashboard** → einzelne Pods prüfen, ob ein bestimmter Pod langsam ist

Kernlogik: Wenn die Inbound-Latenz eines Service hoch ist, aber seine Outbound-Latenz zu Downstream-Services niedrig → der Service selbst ist das Problem. Wenn die Outbound-Latenz hoch ist → das Problem liegt weiter downstream.

Zusätzlich: Kiali + Tracing

- **Kiali** zeigt den Service Graph mit farbcodierten Edges (rot = Fehler, gelb = langsam)
- **Jaeger**/**Zipkin** für Distributed Tracing → einzelne Requests aufschlüsseln, um zu sehen, *wo genau* im Call-Chain die Zeit verbraucht wird

---

Nützlicher Alert (PrometheusRule)

```
- alert: HighP99Latency
  expr: |
    histogram_quantile(0.99,
      sum(rate(istio_request_duration_milliseconds_bucket{reporter="destination"}
[5m]))
      by (destination_service, le)
    ) > 500
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "P99 Latenz > 500ms für {{ $labels.destination_service }}"
```

Kurzfassung

Mesh Dashboard → auffälligen Service identifizieren → Service/Workload Dashboard → Inbound vs. Outbound Latenz vergleichen → mit Tracing den konkreten Call pinpointen.

Sinnvolle istio Metriken und Dashboards - Teil 2

- https://github.com/jmetzger/training-istio-kubernetes/blob/main/istio/observeability/istio-observeablity-in-prometheus-2.md

Installation Kiali

Ziel

Kiali laeuft im Cluster und ist danach unter `https://kiali.<dein-user>.do.t3isp.de/kiali/` erreichbar, geschuetzt mit Benutzer und Passwort.

Hintergrund

Das Gateway von Istio kann kein Basic Auth. Deshalb laeuft im Cluster zusaetzlich **Traefik** als zweiter Gateway-API-Controller. Beide arbeiten mit denselben Objekten (`Gateway` , `HTTPRoute`), nur die `GatewayClass` ist eine andere.

Was	Istio	Traefik
`GatewayClass`	`istio`	`traefik`
Gateway	`bookinfo-gateway` (Namespace `bookinfo`)	`traefik-gateway` (Namespace `traefik`)
Aufgabe hier	Verkehr ins Mesh (Bookinfo)	Addons wie Kiali veroeffentlichen
Basic Auth	nein	ja, ueber eine `Middleware`

Die `HTTPRoute` ist Standard-Gateway-API. Nur der Verweis auf die `Middleware` ist eine Erweiterung von Traefik (Filter `ExtensionRef`) und funktioniert mit keinem anderen Controller.

Voraussetzung

- Istio ist installiert (Namespace `istio-system`).
- Prometheus ist installiert, siehe Installation Prometheus. Ohne Prometheus startet Kiali, zeigt aber keinen Graphen.
- Traefik, DNS und Zertifikat sind schon da (siehe Merkkasten).

> ***Merkkasten: Traefik, DNS und Zertifikat sind schon eingerichtet***
>
> *Der Trainer hat fuer jeden Cluster vorab eingerichtet:*
>
> - *Traefik im Namespace `traefik` mit dem Gateway `traefik-gateway` (Listener `web` fuer HTTP, `websecure` fuer HTTPS)*
> - *den DNS-Eintrag `*.<dein-user>.do.t3isp.de` , z. B. `kiali.tln5.do.t3isp.de`*
> - *ein Zertifikat von Let's Encrypt fuer `*.<dein-user>.do.t3isp.de`*
> - *die Weiterleitung von `http://` auf `https://`*
>
> *Du musst **nichts** im DNS eintragen und kein Zertifikat holen.*

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir -p kiali
cd kiali
```

```
export HOST=kiali.$(whoami).do.t3isp.de
echo $HOST $(dig +short $HOST)
```

Erwartet: Hostname und eine IP. Ist die IP leer: Trainer fragen.

```
kubectl get gatewayclass traefik
kubectl -n traefik get gateway traefik-gateway
```

Erwartet: `ACCEPTED True` und `PROGRAMMED True` , `ADDRESS` ist die IP von oben.

Schritt 2: Kiali installieren

```
kubectl -n istio-system get deployment prometheus
```

Erwartet: `READY 1/1`. Fehlt Prometheus: zuerst Installation Prometheus.

```
kubectl apply -f ~/istio/samples/addons/kiali.yaml
kubectl -n istio-system rollout status deployment kiali
```

```
kubectl -n istio-system get pods -l app.kubernetes.io/name=kiali
kubectl -n istio-system get svc kiali
```

Erwartet: 1 Pod `Running`. Der Service `kiali` (Port 20001) ist die Oberflaeche.

Kiali selbst fragt nicht nach einem Login (`auth.strategy: anonymous` im Addon). Den Schutz uebernimmt Traefik.

Schritt 3: Benutzer und Passwort (Basic Auth)

Benutzer `training`, Passwort `myS3cr3t!`. Im Secret steht nicht das Passwort, sondern eine htpasswd-Zeile (bcrypt).

```
## vi 01-basic-auth-secret.yml
apiVersion: v1
kind: Secret
metadata:
  name: addons-basic-auth
type: Opaque
stringData:
  users: "training:$2b$12$CfOZaJ.Tr0zu6PfpbuCjzeKiQ2PzZARfP.CbC6tRU/7OvEHCIOREm"
```

```
## vi 02-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: basic-auth
spec:
  basicAuth:
    secret: addons-basic-auth
    realm: Training
```

```
kubectl -n istio-system apply -f 01-basic-auth-secret.yml -f 02-middleware.yml
```

Secret und `Middleware` sind fuer alle Addons dieselben. Hast du sie schon bei Jaeger, Prometheus oder Grafana angelegt, meldet `kubectl` hier `configured` oder `unchanged`. Beides ist in Ordnung.

Schritt 4: HTTPRoute anlegen

Die Route haengt am Listener `websecure` (HTTPS) des Traefik-Gateways. Kiali laeuft unter dem Pfad `/kiali`, deshalb leitet die erste Regel `/` dorthin weiter. Die zweite Regel fragt das Passwort ab und reicht an Kiali weiter.

```
## vi 03-httproute.yml
apiVersion: gateway.networking.k8s.io/v1
```

```
kind: HTTPRoute
metadata:
  name: kiali
spec:
  parentRefs:
  - name: traefik-gateway
    namespace: traefik
    sectionName: websecure
  hostnames:
  - ${HOST}
  rules:
  - matches:
    - path:
        type: Exact
        value: /
    filters:
    - type: RequestRedirect
      requestRedirect:
        path:
          type: ReplaceFullPath
          replaceFullPath: /kiali/
        statusCode: 302
  - filters:
    - type: ExtensionRef
      extensionRef:
        group: traefik.io
        kind: Middleware
        name: basic-auth
    backendRefs:
    - name: kiali
      port: 20001
```

```
envsubst < 03-httproute.yml | kubectl -n istio-system apply -f -
```

```
kubectl -n istio-system get httproute kiali
kubectl -n istio-system get httproute kiali -o
jsonpath='{.status.parents[0].conditions[*].type}{"\n"}
{.status.parents[0].conditions[*].status}{"\n"}'
```

Erwartet: `Accepted ResolvedRefs` und darunter `True True`.

Schritt 5: Testen

```
curl -s -o /dev/null -w '%{http_code}\n' https://$HOST/kiali/
```

Erwartet: `401` (ohne Benutzer und Passwort kommt niemand durch).

```
curl -s -o /dev/null -w '%{http_code}\n' -u 'training:myS3cr3t!' https://$HOST/kiali/
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' http://$HOST/kiali/
```

Erwartet: `200`, danach `301 https://...` (HTTP wird auf HTTPS umgeleitet).

```
curl -s -u 'training:myS3cr3t!' https://$HOST/kiali/api/istio/status
```

Erwartet: eine Liste, darin `istiod` und `prometheus`, beide mit `"status":"Healthy"`.

```
echo https://$HOST/kiali/
```

Die URL im Browser oeffnen und mit `training` / `myS3cr3t!` anmelden: Die Oberflaeche von Kiali erscheint, Schloss in der Adressleiste, Aussteller `Let's Encrypt`.

Den Graphen mit Verkehr siehst du in der Uebung Visualisierung mit Kiali.

Fehlersuche

Beobachtung	Pruefen
`404` statt `401`	Stimmt der Hostname der Route? `kubectl -n istio-system get httproute kiali`
`401` trotz richtigem Passwort	Secret hat den Schluessel `users` (nicht `auth` wie bei ingress-nginx)
`500` oder `404` mit Passwort	Gibt es die `Middleware` im selben Namespace wie die Route? `kubectl -n istio-system get middleware`
Hostname wird nicht gefunden	`dig +short $HOST`, sonst Trainer fragen
Kiali meldet `Could not fetch ... Prometheus`	Laeuft Prometheus? `kubectl -n istio-system get deployment prometheus`

Ref

- https://istio.io/latest/docs/ops/integrations/kiali/
- https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/basicauth/
- https://gateway-api.sigs.k8s.io/api-types/httproute/

Visualisierung mit Kiali

Start Traffic on commandline

```
## Set your ip here, you will get it from
## Look for the line with external ip
kubectl -n bookinfo get gateway bookinfo-gateway
GATEWAY_URL=164.90.237.35

## This consistantly shows 200, without much a-do
watch -n 1 curl -o /dev/null -s -w %{http_code} $GATEWAY_URL/productpage
```

Open kiali, and see how it goes

```
in browser:
## Replace xx with your Teilnehmer-Nr. (Login: training / myS3cr3t!)
https://kiali.tlnxx.do.t3isp.de/kiali/

## click on Traffic Graph and choose namespace bookinfo
```

Mesh

- Click on mesh to see if everything is functional

Analyse & Debugging von Service-Mesh-Daten

Ziel

Einen wiederholbaren Prozess vermitteln, mit dem Teilnehmer Performance-Probleme und Fehler in einem Istio Service Mesh systematisch eingrenzen und beheben können.

Voraussetzungen

Tools im Cluster

Tool	Zweck	Installation (Demo/Training)
Prometheus + Grafana	Metrics & Dashboards	`kubectl apply -f samples/addons/prometheus.yaml` / `grafana.yaml`
Kiali	Service Graph & Health	`kubectl apply -f samples/addons/kiali.yaml`
Jaeger oder Tempo	Distributed Tracing	`kubectl apply -f samples/addons/jaeger.yaml`

> ***Hinweis:*** *Die Addon-Manifeste aus* `samples/addons/` *sind nur für Demo/Training geeignet. Für Produktion: eigene Deployments mit Persistence, Retention und HA.*

Demo-Applikation

Bookinfo-App mit aktiviertem Istio-Sidecar-Injection:

```
kubectl label namespace default istio-injection=enabled
kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
```

Der Debugging-Workflow

Schritt 1: Symptom erkennen (Grafana / Prometheus)

Starte mit den **RED-Metrics** pro Service:

Metrik	Frage	Istio-Metrik
Rate	Traffic-Volumen normal?	`istio_requests_total`
Errors	5xx-Rate erhöht?	`istio_requests_total{response_code=~"5.."}`
Duration	Latenz gestiegen?	`istio_request_duration_milliseconds`

Beispiel-PromQL für p95-Latenz eines Service:

```
histogram_quantile(0.95,
  sum(rate(istio_request_duration_milliseconds_bucket{
    destination_service=~"reviews.*"
```

```
  }[5m])) by (le)
)
```

Ergebnis: Du weißt jetzt, *welcher Service* betroffen ist.

Schritt 2: Wo im Call-Graph? (Kiali)

Kiali öffnen:

```
istioctl dashboard kiali
```

Im Service Graph prüfen:

- Welche Edges zeigen erhöhte Latenz (orange/rot)?
- Ist der betroffene Service selbst langsam, oder sein Downstream?
- Gibt es unerwartete Traffic-Pfade?

Ergebnis: Du weißt jetzt, *zwischen welchen Services* das Problem liegt.

Schritt 3: Einzelnen Request verfolgen (Jaeger / Tempo)

Jaeger öffnen:

```
istioctl dashboard jaeger
```

Einen langsamen Trace auswählen und die Spans analysieren:

```
[Client] ──── 500ms ────> [productpage envoy] ── 5ms ──> [productpage app]
                                                                  │
                                                                  ├── 480ms ──> [reviews
envoy] ── 3ms ──> [reviews app]
                                                                  │
│
                                                                  │
└── 450ms ──> [ratings]
                                                                  └── 10ms ──> [details]
```

Hier siehst du pro Span:

- **Envoy-Proxy-Zeit:** Mesh-Overhead (mTLS, Routing, Retries)
- **App-Zeit:** Verarbeitungszeit der Anwendung selbst
- **Downstream-Zeit:** Wartezeit auf nachgelagerte Services

Ergebnis: Du weißt jetzt, ob das Problem im **Netzwerk**, in der **App** oder **downstream** liegt.

Schritt 4: Ursache eingrenzen

Befund	Mögliche Ursache	Nächster Schritt
Envoy-Proxy langsam	mTLS-Overhead, Retry-Storms, Rate Limiting	`istioctl proxy-config`, Envoy Stats prüfen
App selbst langsam	CPU-Limit, Memory-Pressure, langsame DB-Query	`kubectl top pod`, App-Profiling

Downstream langsam	Problem liegt beim Downstream-Service	Workflow dort wiederholen
Viele Retries sichtbar	Flapping Backend, falsche Timeout-Config	DestinationRule / VirtualService prüfen
Connection-Fehler	mTLS-Mismatch, PeerAuthentication falsch	`istioctl authn tls-check`

Schritt 5: Low-Level Debugging (Envoy / istioctl)

Wenn du tiefer gehen musst:

```
## Sync-Status aller Sidecars mit istiod prüfen
istioctl proxy-status

## Cluster-Config eines Pods inspizieren
istioctl proxy-config clusters <pod> -n <ns>

## Routen eines Pods inspizieren
istioctl proxy-config routes <pod> -n <ns>

## Listener eines Pods inspizieren
istioctl proxy-config listeners <pod> -n <ns>

## Envoy-Stats direkt abfragen
kubectl exec <pod> -c istio-proxy -- pilot-agent request GET /stats

## Envoy-Config-Dump (vollständig)
kubectl exec <pod> -c istio-proxy -- pilot-agent request GET /config_dump

## Istio-Analyse auf Fehlkonfigurationen
istioctl analyze -n <ns>
```

Trainings-Lab: "Finde den Fehler"

Setup: Latenz per Fault Injection erzeugen

Fault Injection auf den `reviews`-Service anwenden – 50% der Requests bekommen 3 Sekunden Delay:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-delay
  namespace: default
spec:
  hosts:
  - reviews
  http:
  - fault:
      delay:
        percentage:
          value: 50
```

```
        fixedDelay: 3s
    route:
    - destination:
        host: reviews
```

```
kubectl apply -f reviews-delay.yaml
```

Aufgabe für Teilnehmer

> *"Die Bookinfo-App ist langsam. Findet heraus, welcher Service betroffen ist, wo die Latenz entsteht und was die Ursache ist."*

Erwarteter Lösungsweg

1. **Grafana:** p95-Latenz von `reviews` ist auf ~3s gestiegen
2. **Kiali:** Edge `productpage → reviews` zeigt erhöhte Latenz
3. **Jaeger:** Trace zeigt 3s Delay im `reviews`-Span, aber App-Zeit selbst ist minimal
4. **Erkenntnis:** Delay kommt nicht von der App – muss eine Mesh-Konfiguration sein
5. **Lösung finden:**

   ```
   kubectl get virtualservice -A
   kubectl describe vs reviews-delay
   ```

   → Fault Injection als Ursache identifiziert

Varianten für weitere Labs

Szenario	Injection	Lernziel
HTTP-Fehler	`fault.abort: 503, 30%`	Error-Rate Debugging
Timeout-Kette	Delay 5s + Timeout 2s in DestinationRule	Timeout-Konfiguration verstehen
Circuit Breaker	`outlierDetection` + Last erzeugen	Circuit Breaking analysieren
mTLS-Mismatch	`PeerAuthentication: STRICT` + Client ohne Sidecar	mTLS-Fehler debuggen

Zusammenfassung: Der Workflow auf einen Blick

```
Symptom erkennen          Grafana / Prometheus (RED-Metrics)
        |
        ▼
Ort eingrenzen            Kiali (Service Graph)
        |
        ▼
Request verfolgen         Jaeger / Tempo (Distributed Tracing)
        |
        ▼
Ursache klassifizieren    Netzwerk | App | Downstream | Config
        |
        ▼
Low-Level Debugging       istioctl / Envoy Admin API
```

Merksatz: Von oben nach unten, von breit nach schmal – erst den Überblick, dann den Deep Dive.

Vergleich Sidecar und Ambient

Istio Proxy-Konzepte

1. Was ist Envoy?

Envoy ist ein High-Performance L4/L7 Proxy, entwickelt von Lyft und heute ein CNCF-Graduated-Projekt. Er ist in C++ geschrieben und wurde von Anfang an für dynamische, Cloud-native Umgebungen konzipiert.

Istio nutzt Envoy als **Data Plane** — jeder Proxy im Mesh ist eine Envoy-Instanz. Istio erweitert Envoy über eigene Filter (z.B. für mTLS, Telemetrie) und steuert ihn zentral über istiod (Control Plane).

2. Wo sitzt der Proxy?

Der Envoy Proxy läuft als **Native Sidecar** im gleichen Pod wie die Applikation.

Ab Kubernetes 1.28+ wird `istio-proxy` als Init-Container mit `restartPolicy: Always` definiert (Native Sidecar Feature). Das bedeutet:

- Er erscheint unter `spec.initContainers`, läuft aber für die gesamte Pod-Laufzeit
- Er startet **vor** der App und stoppt **nach** der App
- `kubectl get pods` zeigt ihn trotzdem als laufenden Container (z.B. `2/2 Ready`)

Da er sich das **Network Namespace** mit dem App-Container teilt, sieht er sämtlichen ein- und ausgehenden Traffic des Pods.

```
Pod
├── initContainers
│   ├── istio-init        ← setzt iptables-Regeln (entfällt mit CNI Plugin)
│   └── istio-proxy       ← Native Sidecar (Envoy), läuft dauerhaft
└── containers
    └── app               ← Applikations-Container
```

3. Wie kommt der Traffic zum Proxy?

Der Traffic wird **transparent** zum Envoy umgeleitet — die Applikation bemerkt nichts davon. Dafür werden iptables-Regeln gesetzt, die allen ein- und ausgehenden Traffic über den Envoy-Proxy routen.

Variante A: `istio-init` Init-Container (Standard)

- Läuft einmalig vor dem App-Start
- Setzt iptables-Regeln im Network Namespace des Pods
- Benötigt `NET_ADMIN` und `NET_RAW` Capabilities

Variante B: Istio CNI Plugin (empfohlen)

- Arbeitet als Chained CNI Plugin auf Node-Ebene
- Setzt die gleichen iptables-Regeln, aber **bevor** irgendein Container startet
- `istio-init` Init-Container entfällt komplett

Vorteile CNI Plugin:

- Keine `NET_ADMIN` / `NET_RAW` Capabilities im Pod → kompatibel mit `restricted` Pod Security Standard
- Keine Race Conditions → kein Traffic geht am Proxy vorbei, auch wenn andere Init-Container Netzwerk-Requests machen

- Sicherheitsaspekt: Ohne CNI Plugin können Init-Container, die vor `istio-init` laufen, unverschlüsselt und ohne Policy kommunizieren (kein mTLS, keine AuthorizationPolicy)
- Im Ambient Mode zwingend erforderlich

Hinweis: Das CNI Plugin setzt die identischen iptables-Regeln wie `istio-init` — es gibt keinen Runtime-Performance-Unterschied.

4. Was macht der Proxy?

Envoy übernimmt als Sidecar eine Vielzahl von Aufgaben, ohne dass die Applikation angepasst werden muss:

Bereich	Funktion
Security	mTLS (automatische Verschlüsselung zwischen Services), Zertifikatsrotation
Traffic Management	Routing, Retries, Timeouts, Fault Injection, Traffic Shifting (Canary, Blue/Green)
Resilienz	Circuit Breaking, Outlier Detection, Rate Limiting
Load Balancing	Round Robin, Least Connections, Random, Consistent Hashing
Observability	Metrics (RED: Rate, Errors, Duration), Distributed Traces, Access Logs

All das wird **nicht in der App konfiguriert**, sondern über Istio-Ressourcen (siehe nächster Abschnitt).

5. Wie wird der Proxy konfiguriert?

Envoy wird **nie direkt konfiguriert**. Stattdessen:

1. Der Anwender erstellt Istio CRDs (z.B. VirtualService, DestinationRule, AuthorizationPolicy)
2. **istiod** (Control Plane) übersetzt diese in Envoy-native Konfiguration
3. istiod pusht die Konfiguration über die **xDS API** (gRPC-Stream) an die Envoy-Proxies

xDS API — Hot Reload ohne Neustart

Envoy öffnet eine langlebige gRPC-Verbindung zu istiod (Watch/Subscribe-Pattern). Bei Änderungen pusht istiod die neue Konfiguration über den bestehenden Stream.

Wichtig: Envoy wendet Änderungen **in-memory** an — kein Schreiben auf die Platte, kein Prozess-Reload. Bestehende Connections werden nicht unterbrochen. Das unterscheidet Envoy fundamental von klassischen Proxies wie Nginx oder HAProxy, die ein Config-File und einen Reload benötigen.

Die einzige Datei-basierte Config ist die **Bootstrap-Konfiguration** beim ersten Start. Sie enthält im Wesentlichen nur die Verbindungsdaten zu istiod (Port 15012). Alles Weitere kommt dynamisch über xDS.

6. Sidecar Injection

Der Envoy Sidecar wird über einen **Mutating Admission Webhook** in den Pod injiziert. Es gibt zwei Varianten:

Automatic Injection (empfohlen)

Namespace mit Label versehen:

```
kubectl label namespace <NAMESPACE> istio-injection=enabled
```

Alle neuen Pods in diesem Namespace erhalten automatisch den Sidecar.

Manual Injection

```
istioctl kube-inject -f deployment.yaml | kubectl apply -f -
```

Nützlich für einzelne Workloads oder zum Debugging (um die generierte Pod-Spec zu inspizieren).

7. Memory-Overhead

Envoy bekommt per Default die Konfiguration für **alle erreichbaren Services** im Mesh — nicht nur die, die der Pod tatsächlich anspricht. In großen Meshes führt das zu erheblichem Speicherverbrauch:

Mesh-Größe	Typischer RAM pro Sidecar
Klein (< 20 Services)	50–80 MB
Mittel (50–100 Services)	100–200 MB
Groß (100+ Services)	200–300+ MB

Gegenmaßnahme: Sidecar CRD

Mit der Istio Sidecar CRD kann die Sichtbarkeit eines Envoy-Proxies eingeschränkt werden:

```yaml
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: frontend-sidecar
  namespace: frontend
spec:
  egress:
  - hosts:
    - "./backend.backend.svc.cluster.local"
    - "istio-system/*"
```

Der Envoy kennt dann nur noch die explizit genannten Services → deutlich weniger Memory.

Vorsicht: Falsch konfiguriert bricht die Kommunikation, weil der Envoy den Ziel-Service nicht kennt. Service-Abhängigkeiten müssen explizit gepflegt werden.

Alternative: Ambient Mode

Im Ambient Mode entfällt der Envoy-Sidecar pro Pod komplett. Stattdessen:

- **ztunnel** (pro Node) für L4 (mTLS, TCP-Routing)
- **Waypoint Proxies** (optional, pro Namespace/Service) für L7 (HTTP-Routing, Retries etc.)

Der Memory-Overhead reduziert sich erheblich, da nicht mehr jeder Pod seinen eigenen Envoy betreibt.

8. Jobs und CronJobs im Mesh

Problem (vor Native Sidecars)

Ein Job führt seinen Task aus → App-Container beendet sich → `istio-proxy` Sidecar läuft weiter → Pod bleibt `Running` statt `Completed` → Job wird nie als fertig erkannt.

Folgen:

- CronJobs häufen sich, weil der vorherige nie abschließt
- `activeDeadlineSeconds` / `backoffLimit` greifen → Job wird als **Failed** markiert
- Ressourcen bleiben durch Zombie-Envoys belegt

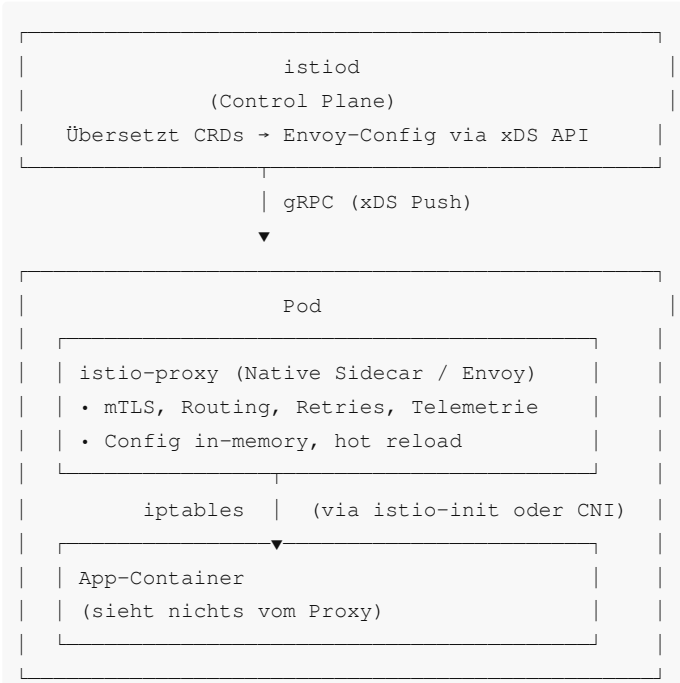
Workarounds (vor K8s 1.28)

```
## Envoy manuell herunterfahren am Ende des Job-Scripts
curl -X POST http://localhost:15000/quitquitquit
```

Lösung: Native Sidecars (K8s 1.28+)

Mit Native Sidecars weiß Kubernetes, dass `istio-proxy` ein Sidecar ist und beendet ihn automatisch, sobald alle regulären Container fertig sind. Keine Workarounds mehr nötig.

9. Zusammenfassung



Istio Sidecar-Injection

1. Verzeichnis anlegen

```
cd
mkdir -p ~/manifests/nginx
```

2. Nginx-Deployment erstellen

```
cat <<'EOF' > ~/manifests/nginx/nginx.yaml
apiVersion: v1
```

```
kind: Namespace
metadata:
  name: nginx-istio
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: nginx-istio
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: nginx-istio
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
EOF
```

3. Sidecar injizieren und anwenden

```
kubectl apply -f <(istioctl kube-inject -f ~/manifests/nginx/nginx.yaml)
```

4. Injection prüfen

```
kubectl get pods -n nginx-istio
```

Erwartetes Ergebnis: `READY 2/2`

Vergleich ohne istio, istio sidecar, istio ambient

- https://livewyer.io/blog/2024/06/06/comparison-of-service-meshes-part-two/
- https://github.com/livewyer-ops/poc-servicemesh2024/blob/main/docs/test-report-istio.md

Was Ambient (noch) nicht kann, was Sidecar kann (Stand 10.11.2026)

Stand: 10.11.2026. Grundlage ist die Istio-Doku zu Version 1.32. Im Training läuft **1.31.4**; die Aussagen gelten vermutlich auch dafür, abgeglichen ist das nicht.

Kurzfassung

Für **einen einzelnen Kubernetes-Cluster** mit Gateway API, ztunnel und Waypoint (unser Aufbau) fehlt fast nichts. Die Lücken liegen bei VMs, EnvoyFilter, SPIRE und Multi-Cluster.

Was es nur mit Sidecar gibt

Thema	Sidecar	Ambient
VMs im Mesh	geht	nicht möglich
EnvoyFilter	geht	nicht unterstützt, auch nicht am Waypoint
SPIRE-Integration	geht	nicht unterstützt
PeerAuthentication `DISABLE`	geht	wird ignoriert
Multi-Cluster	stabil	nur Multi-Primary (Primary-Remote geht nicht)
Multi-Network	stabil	laut Doku Beta
Sidecar-Client → Waypoint	geht	Waypoint wird umgangen (siehe unten)

Was Ambient nur eingeschränkt kann

- **Layer 7 braucht einen Waypoint.** Nur mit ztunnel gibt es mTLS, L4-Autorisierung und TCP-Metriken, aber kein HTTP-Routing, kein HTTP-Tracing.
- **`VirtualService`** ist in Ambient noch Alpha. Besser `HTTPRoute` (Gateway API) nehmen, und beides nicht für denselben Workload mischen.
- **Ingress-Gateways (istio-ingressgateway)** nutzen den Waypoint standardmäßig nicht. Mit dem Label `istio.io/ingress-use-waypoint=true` am Service oder Namespace (Namespace ab Istio 1.25) geht es doch. Laut Istio-Doku gilt das nur, wenn istiod die Variable `ENABLE_INGRESS_WAYPOINT_ROUTING` aktiv hat (Standard `false`). **Auf unserem Cluster mit 1.31.4 ist die Variable nicht gesetzt, das Label wirkte trotzdem** (getestet in Übung Zero Trust Ambient). Warum, ist nicht geklärt. Wirkt das Label bei dir nicht, zuerst die Variable an istiod prüfen.
- **Sicherheitsmodell:** Ambient "stark", Sidecar "am stärksten". Der ztunnel hält pro Node die Schlüssel aller dortigen Workloads, der Sidecar nur die seines Pods.

Ersatz für EnvoyFilter: `TrafficExtension` (Alpha)

Einen 1:1-Ersatz gibt es nicht. Erweiterungen laufen nur **am Waypoint**, mit Wasm oder Lua (beides Alpha).

```
apiVersion: extensions.istio.io/v1alpha1
kind: TrafficExtension
metadata:
  name: basic-auth-at-waypoint
```

```yaml
spec:
  targetRefs:
    - kind: Gateway
      group: gateway.networking.k8s.io
      name: waypoint
  phase: AUTHN
  wasm:
    url: oci://ghcr.io/istio-ecosystem/wasm-extensions/basic_auth:1.12.0
```

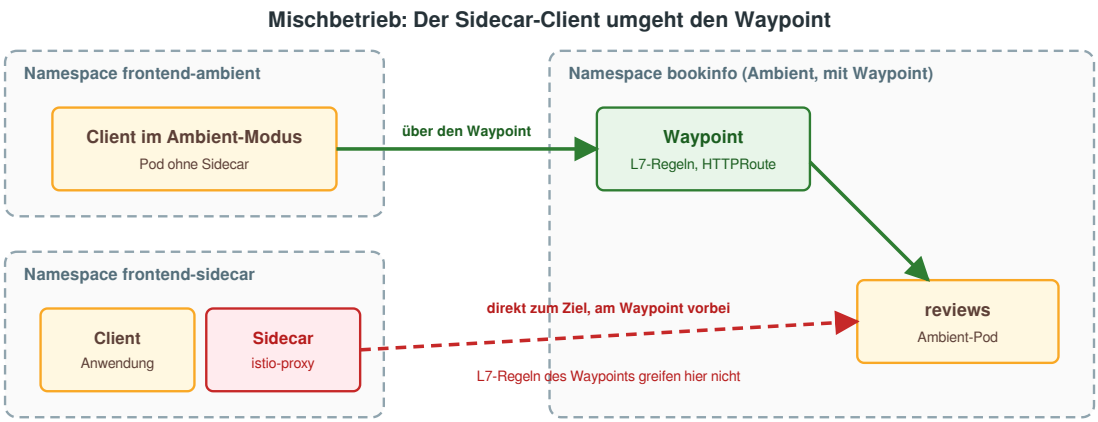
- `targetRefs` zeigt auf den Waypoint (oder einen Service) und ersetzt den `selector` aus dem Sidecar-Modus.
- Für Lua steht das Skript unter `lua.inlineCode` statt `wasm`.
- Es sind nur HTTP-Filter. Cluster, Listener oder Filterketten patchen wie mit `EnvoyFilter` geht nicht.
- Im ztunnel gibt es keine Erweiterungen.

Ob in 1.31 noch `WasmPlugin` der richtige Weg ist, ist nicht geprüft.

Mischbetrieb: Sidecar-Client → Ambient-Ziel

Sidecar und Ambient können im selben Mesh laufen. Die Falle (aus der Doku):

> *Traffic from sidecar mode workloads bypasses waypoint proxies.*



Ambient und Sidecar im selben Mesh. mTLS gilt auf beiden Wegen, L7 nur auf dem grünen.

Es gibt keinen Fehler, die Anfrage kommt an. Aber **L7-Policies, HTTPRoute-Routing und L7-Telemetrie des Waypoints greifen nicht.**

Pfad	L7
Sidecar → Sidecar	wie gewohnt
Ambient → Ambient mit Waypoint	greift am Waypoint
Ambient → Sidecar	vermutlich am Ziel-Sidecar (nicht getestet)
Sidecar → Ambient mit Waypoint	**Waypoint wird umgangen**

Folgen für die Migration

- **Nur mTLS und L4-Policies:** kein Problem, das macht der ztunnel allein.

- **Mit L7-Policies:** Ziel und alle seine Clients **zusammen** umstellen, nicht einzelne Services quer durch den Aufrufgraphen.
- **Zero-Downtime geht mit L7-Policies nicht.** Die alten selektorbasierten Policies müssen weg, bevor die waypoint-basierten greifen. Dazwischen fehlt die L7-Durchsetzung, also Wartungsfenster einplanen.
- **Vorher prüfen, wer den Service aufruft** (Kiali, Metriken, auch Jobs, Cronjobs, andere Namespaces).
- **Waypoint erzwingen:** Eine `AuthorizationPolicy`, die nur die Waypoint-Identität zulässt, lässt umgangenen Verkehr sichtbar scheitern. In der Mischphase bricht das alle Sidecar-Clients ab, also bewusst einsetzen.
- **Dauerzustand ist das fragil:** Ein neuer Client mit Sidecar, und die L7-Regel wirkt nicht mehr. Mischbetrieb nur als Übergang.

Reihenfolge laut Doku: Ambient installieren → Policies migrieren (bei reinem L4 entfällt das) → Namespace für Ambient labeln → Waypoints aktivieren → Sidecar-Injection entfernen → Pods neu starten. Jeder Schritt ist einzeln rückgängig zu machen.

Chaos-Tests / Fault Injection (Delay, Abort)

Nicht verifiziert. In der offiziellen Doku gibt es dazu keine bestätigte Aussage für Ambient.

- Die Aufgabe "Fault Injection" (istio.io) nutzt `VirtualService` (`fault.delay`, `fault.abort`) und sagt nichts zu Ambient oder Waypoint.
- `HTTPRoute` hat kein Feld für Fault Injection. `VirtualService` ist in Ambient Alpha, und Mischen mit Gateway API "will lead to undefined behavior".
- Auf L4 (nur ztunnel) gibt es keine Fault Injection.
- Blogs (keine offizielle Quelle) nennen: `VirtualService` am Waypoint.

Fürs Training: Der Chaos-Link in der Agenda führt in die Sidecar-Welt. Vor der Demo auf dem Ambient-Cluster ausprobieren, ob ein `VirtualService` mit `fault` am Waypoint wirkt.

Gateway API wird intern zu `VirtualService`

istiod übersetzt `HTTPRoute`, `GRPCRoute`, `TCPRoute` und `TLSRoute` intern in `VirtualService`-Objekte (`pilot/pkg/config/kube/gateway/route_collections.go`, Branch `master`). Die gibt es nur im Speicher von istiod, `kubectl get virtualservice` zeigt sie nicht. Für Waypoints ist das nicht gesondert geprüft.

Wann bleibt man bei Sidecar?

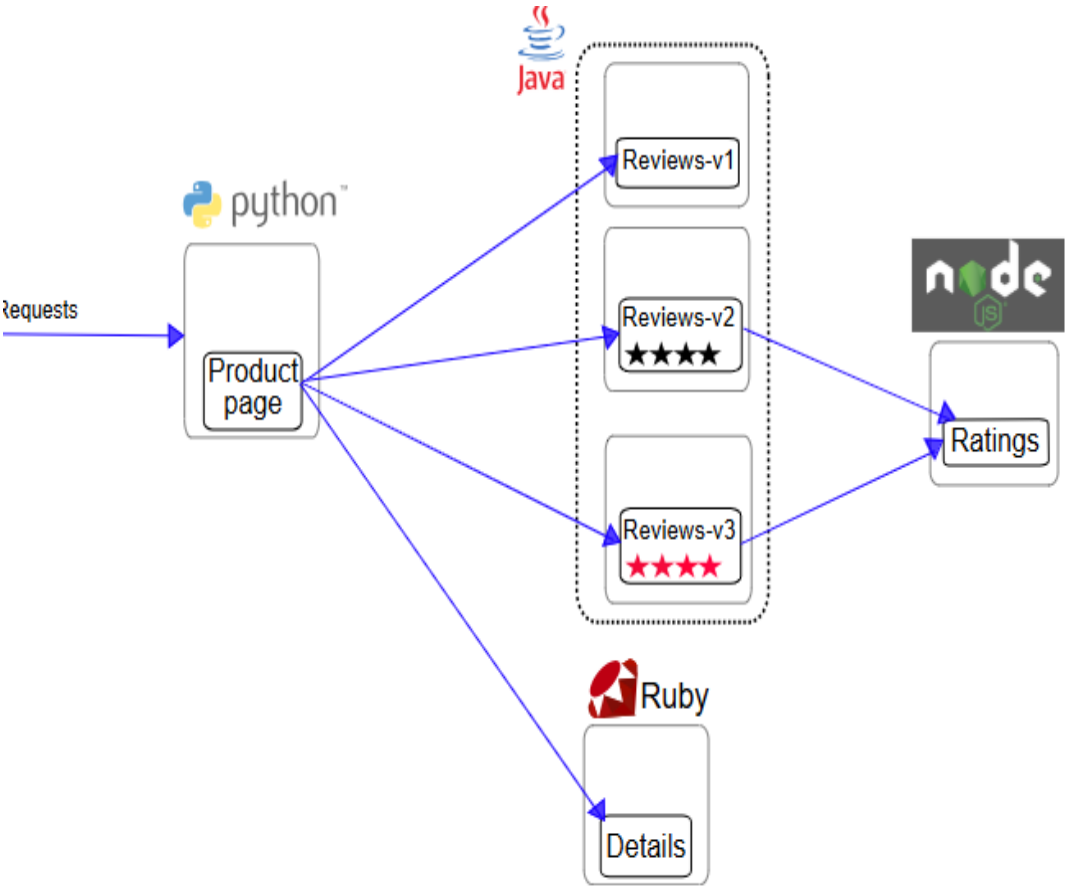
Wenn eines davon zutrifft: **VMs im Mesh, `EnvoyFilter`, SPIRE, Multi-Cluster mit Primary-Remote.** Das sind laut Doku die echten Gründe gegen Ambient.

Quellen

- Istio: Sidecar or ambient?
- Istio: Migrate from Sidecar to Ambient
- Istio: Waypoint-Nutzung
- Istio: Extend waypoints with Wasm
- Istio: Extend waypoints with Lua
- Istio 1.32 Ankündigung

Bookinfo von Sidecar auf Ambient umstellen

Überblick



Prerequisites

- bookinfo ist installiert im modus sidecar

Vorbereitung

```
kubectl label namespace bookinfo istio.io/dataplane-mode=ambient
kubectl label namespace bookinfo istio-injection-
kubectl describe ns bookinfo
```

Waypoint Proxy ausrollen

```
cd
mkdir -p manifests/waypoint
cd manifests/waypoint
```

```
## YAML generieren (dry-run)
istioctl waypoint generate --namespace bookinfo --for all  > waypoint.yaml
```

```
## Anschauen was passiert
cat waypoint.yaml
```

```
## Ausrollen
kubectl apply -f waypoint.yaml
kubectl label namespace bookinfo istio.io/use-waypoint=waypoint
```

```
## Überprüfen
kubectl -n bookinfo get gateways
istioctl waypoint list --namespace bookinfo
```

Bookinfo App ausrollen

```
kubectl -n bookinfo rollout restart deployment
kubectl -n bookinfo get pods
```

Testen ob die App funktioniert

```
kubectl -n bookinfo exec "$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

```
## Einfachere Variante
kubectl -n bookinfo exec deployments/ratings-v1 -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

Von aussen Anfragen über gateway (gateway api)

```
kubectl -n bookinfo get gateways
kubectl -n bookinfo get httproutes -o yaml
```

```
## Das Gateway erstellt automatisch einen Service (die EXTERNAL-IP vergibt MetalLB)
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

```
GATEWAY_IP=$(kubectl -n bookinfo get svc bookinfo-gateway-istio -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
curl -s http://$GATEWAY_IP/productpage | grep -o "<title>.*</title>"
## oder im Browser öffnen: http://<GATEWAY_IP>/productpage
```

Testen aus einem namespace der nicht teil des istio mesh ist

```
kubectl create ns no-mesh
kubectl -n no-mesh apply -f https://raw.githubusercontent.com/istio/istio/release-
1.25/samples/sleep/sleep.yaml
kubectl -n no-mesh wait --for=condition=ready pod --all --timeout=60s

## Zugriff auf productpage versuchen
kubectl -n no-mesh exec deploy/sleep -- \
  curl -s -o /dev/null -w "%{http_code}\n" productpage.bookinfo:9080/productpage
```

Ref

- https://istio.io/latest/docs/ambient/getting-started/deploy-sample-app/

Canary Update von istiod mit tags und revisions

Überblick

Revision Tags sind stabile Aliase für Istio-Revisionen. Namespaces referenzieren nur den Tag-Namen – beim Upgrade wird der Tag auf die neue Revision umgeschwenkt, ohne Namespace-Labels ändern zu müssen.

```
Namespace (istio.io/rev: prod) ──► Tag "prod" ──► istiod Revision 1-22-0
                                                        ↓  (nach Upgrade)
                                                   istiod Revision 1-23-0
```

Voraussetzungen

```
## Helm Repo hinzufügen
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm repo update

## istioctl installieren (wird für Tag-Management benötigt)
curl -sL https://istio.io/downloadIstio | ISTIO_VERSION=1.22.0 sh -
export PATH=$PWD/istio-1.22.0/bin:$PATH
```

Schritt 1: Initiale Installation (Revision 1-22-0)

1.1 Istio Base (CRDs)

```
helm install istio-base istio/base \
  -n istio-system \
  --create-namespace \
  --version 1.22.0
```

1.2 Istiod mit Revision

> ***Wichtig:*** *Bei Revision-basierter Installation wird* `--set revision=` *gesetzt. Der Revisionsname darf nur alphanumerische Zeichen und* `-` *enthalten (keine Punkte).*

```
helm install istiod-1-22-0 istio/istiod \
  -n istio-system \
  --version 1.22.0 \
  --set revision=1-22-0 \
  --wait
```

Prüfen:

```
kubectl get pods -n istio-system -l app=istiod
## NAME                            READY   STATUS
## istiod-1-22-0-xxxxxxxxx-xxxxx   1/1     Running
```

1.3 Gateway (optional)

```
helm install istio-ingressgateway istio/gateway \
  -n istio-ingress \
  --create-namespace \
  --version 1.22.0 \
  --set revision=1-22-0
```

Schritt 2: Revision Tag erstellen

```
## Tag "prod" zeigt auf Revision 1-22-0
istioctl tag set prod --revision 1-22-0

## Prüfen
istioctl tag list
## TAG    REVISION
## prod   1-22-0
```

Technisch erzeugt das eine `MutatingWebhookConfiguration` namens `istio-revision-tag-prod`.

Schritt 3: Namespaces labeln

```
## Namespace mit dem Tag (nicht der Revision!) labeln
kubectl label namespace default istio.io/rev=prod

## Falls noch das alte (nicht-revisionierte) Label existiert, entfernen!
## "istio-injection=enabled" hat Vorrang vor "istio.io/rev" und würde
## den Revision-Tag ignorieren. Nur nötig wenn vorher ohne Revisionen installiert.
kubectl label namespace default istio-injection-
```

Demo-App deployen:

```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-
1.22/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl rollout status deployment -n default
```

Sidecar-Injection prüfen:

```
kubectl get pods -n default -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}
{.spec.containers[*].name}{"\n"}{end}'
## Jeder Pod sollte einen "istio-proxy" Container haben
```

Schritt 4: Canary Upgrade durchführen

4.1 Neue Istiod-Revision installieren

```
helm install istiod-1-23-0 istio/istiod \
  -n istio-system \
  --version 1.23.0 \
  --set revision=1-23-0 \
  --wait
```

Jetzt laufen **zwei** Istiod-Instanzen parallel:

```
kubectl get pods -n istio-system -l app=istiod
## NAME                              READY   STATUS
## istiod-1-22-0-xxxxxxxxx-xxxxx     1/1     Running
## istiod-1-23-0-yyyyyyyyy-yyyyy     1/1     Running
```

4.2 (Optional) Canary-Tag für Testphase

```
## Canary-Tag für einzelne Test-Namespaces
istioctl tag set canary --revision 1-23-0

## Test-Namespace umschalten
kubectl label namespace test-ns istio.io/rev=canary --overwrite
kubectl rollout restart deployment -n test-ns
```

Hier testen, ob alles mit der neuen Version funktioniert.

4.3 Prod-Tag auf neue Revision schwenken

```
## Tag "prod" zeigt jetzt auf 1-23-0
istioctl tag set prod --revision 1-23-0 --overwrite

## Prüfen
istioctl tag list
## TAG      REVISION
## prod     1-23-0
## canary   1-23-0
```

4.4 Workloads neu starten

Die Namespace-Labels bleiben unverändert (`istio.io/rev=prod`). Neue Pods bekommen automatisch den Sidecar der neuen Revision. Bestehende Pods müssen neu gestartet werden:

```
## Alle Deployments im Namespace neu starten
kubectl rollout restart deployment -n default

## Warten bis alle Pods ready sind
kubectl rollout status deployment -n default
```

4.5 Sidecar-Version prüfen

```
## Proxy-Version aller Pods anzeigen
istioctl proxy-status
## NAME                    CLUSTER    CDS    LDS    EDS    RDS    ECDS    ISTIOD
VERSION
## productpage-v1-...      ...        SYNCED ...                     istiod-1-23-0-
yyyyy.istio-system 1.23.0
```

Schritt 5: Alte Revision entfernen

Erst wenn **alle** Workloads auf der neuen Revision laufen:

```
## Prüfen ob noch Proxies an alter Revision hängen
istioctl proxy-status | grep 1-22-0
## Sollte leer sein!

## Canary-Tag entfernen (falls erstellt)
istioctl tag remove canary

## Alte Istiod-Revision deinstallieren
helm uninstall istiod-1-22-0 -n istio-system

## Altes Gateway upgraden (falls installiert)
helm upgrade istio-ingressgateway istio/gateway \
  -n istio-ingress \
  --version 1.23.0 \
  --set revision=1-23-0

## Istio Base CRDs updaten
helm upgrade istio-base istio/base \
  -n istio-system \
  --version 1.23.0
```

Rollback

Falls Probleme auftreten, Tag einfach zurückschwenken:

```
## Tag zurück auf alte Revision
istioctl tag set prod --revision 1-22-0 --overwrite

## Pods neu starten
kubectl rollout restart deployment -n default

## Neue Revision wieder entfernen
helm uninstall istiod-1-23-0 -n istio-system
```

Zusammenfassung der Kommandos

Schritt	Kommando

Neue Revision installieren	`helm install istiod-1-23-0 istio/istiod --set revision=1-23-0`
Tag umschalten	`istioctl tag set prod --revision 1-23-0 --overwrite`
Pods neu starten	`kubectl rollout restart deployment -n <namespace>`
Status prüfen	`istioctl proxy-status`
Alte Revision entfernen	`helm uninstall istiod-1-22-0`
Rollback	`istioctl tag set prod --revision 1-22-0 --overwrite`

Multi-Cluster (nur Theorie)

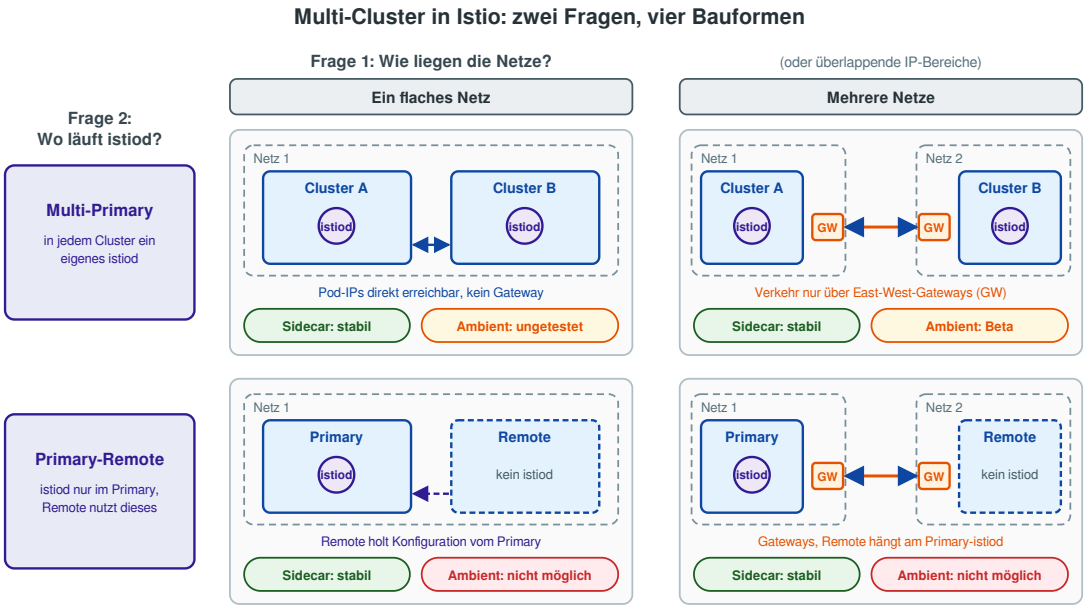
Multi-Cluster- & Hybrid-Umgebungen mit Istio (Überblick mit Grafiken)

Stand: 2026-11-10. Reine Theorie, nichts davon wird im Training auf den Teilnehmer-Clustern aufgebaut. Die Ambient-Aussagen stammen aus der Istio-Doku (siehe Quellen), die Zahlen zum VM-Ausfall aus einem Vorbereitungs-Lab (zwei kubeadm-Cluster, eine VM). Auf unseren 16 Trainings-Clustern wurde nichts davon getestet.

Kurzfassung

- **Zwei Fragen bestimmen die Bauform:** Liegen die Cluster in einem flachen Netz oder in mehreren Netzen? Und hat jeder Cluster sein eigenes istiod (Multi-Primary) oder hängen Cluster an einem fremden istiod (Primary-Remote)?
- **Ein flaches Netz** braucht keine Gateways: Pod-IPs sind direkt erreichbar. **Mehrere Netze** brauchen pro Cluster ein East-West-Gateway.
- **Immer gleich** auf allen Clustern: `meshID`, Root-CA, Trust Domain. **Immer verschieden:** `clusterName`.
- **Ambient** kann Multi-Cluster nur als Multi-Primary über mehrere Netze (Beta). **VMs im Mesh** gehen nur mit Sidecar.

1. Die vier Bauformen

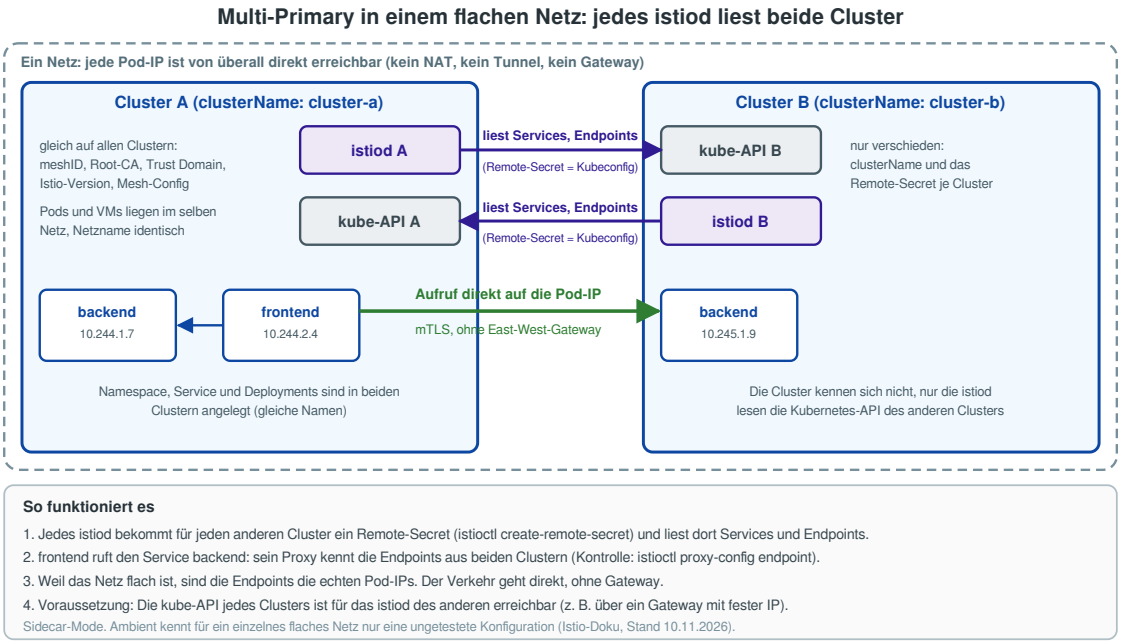


Immer gleich auf allen Clustern: ein Mesh (meshID), gemeinsame Root-CA, gleiche Trust Domain. Je Cluster verschieden: clusterName.
Stand der Ambient-Angaben: Istio-Doku „Ambient multicluster“ (Beta, nur Multi-Network, Primary-Remote nicht unterstützt), Stand 10.11.2026.

Bauform	Sidecar	Ambient
Multi-Primary, ein Netz	stabil	ungetestet, laut Doku möglicherweise fehlerhaft
Multi-Primary, mehrere Netze	stabil	Beta, laut Doku produktionsreif mit Einschränkungen
Primary-Remote, ein Netz	stabil	nicht unterstützt
Primary-Remote, mehrere Netze	stabil	nicht unterstützt

Wann Multi-Cluster überhaupt? Für Ausfallsicherheit (ein Cluster fällt aus, der andere übernimmt), für Trennung nach Teams oder Regionen und für standortnahes Routing. Der Preis ist mehr Betriebsaufwand. Mehrere Control Planes (Multi-Primary) sind laut Doku die ausfallsicherste Variante, weil ein istiod-Ausfall nur die Workloads seines eigenen Clusters trifft.

2. Multi-Primary in einem flachen Netz



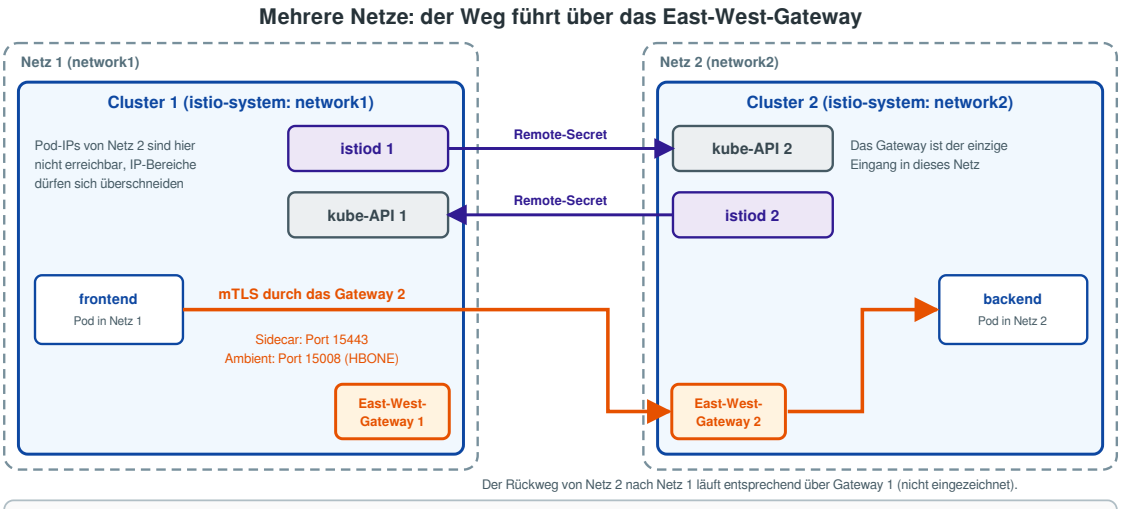
So funktioniert es

1. Jedes istiod bekommt für jeden anderen Cluster ein Remote-Secret (istioctl create-remote-secret) und liest dort Services und Endpoints.
2. frontend ruft den Service backend: sein Proxy kennt die Endpoints aus beiden Clustern (Kontrolle: istioctl proxy-config endpoint).
3. Weil das Netz flach ist, sind die Endpoints die echten Pod-IPs. Der Verkehr geht direkt, ohne Gateway.
4. Voraussetzung: Die kube-API jedes Clusters ist für das istiod des anderen erreichbar (z. B. über ein Gateway mit fester IP).

Sidecar-Mode. Ambient kennt für ein einzelnes flaches Netz nur eine ungetestete Konfiguration (Istio-Doku, Stand 10.11.2026).

- Jedes istiod bekommt ein **Remote-Secret** (Kubeconfig mit ServiceAccount-Token) für jeden anderen Cluster (`istioctl create-remote-secret`). Damit liest es dort Services und Endpoints.
- Der Proxy eines Clients kennt danach die Endpoints **aller** Cluster (Kontrolle: `istioctl proxy-config endpoint <pod>`).
- Weil alle Pod-IPs direkt erreichbar sind, geht der Verkehr **ohne Gateway** auf die Pod-IP.
- Service-Namen müssen in jedem Cluster existieren, der sie aufruft, auch wenn dort keine Pods laufen (DNS).
- Voraussetzung: Die Pod- und Service-Bereiche der Cluster dürfen sich nicht überschneiden.

3. Mehrere Netze: East-West-Gateway



Der Rückweg von Netz 2 nach Netz 1 läuft entsprechend über Gateway 1 (nicht eingezeichnet).

Was zusätzlich nötig ist (gegenüber einem flachen Netz)

1. Netzname je Netz: Label topology.istio.io/network am Namespace istio-system (und am Gateway).
2. Pro Cluster ein East-West-Gateway, das eingehenden mTLS-Verkehr aus dem anderen Netz annimmt und an die Pods weiterreicht.
3. Remote-Secrets wie im flachen Netz, die kube-API des anderen Clusters muss für istiod erreichbar bleiben.
4. Ambient (Beta): Gateway-Klasse istio-east-west, Listener HBONE auf 15008. Gleichnamige Waypoints in allen Clustern, Konfiguration selbst synchron halten (z. B. mit ArgoCD). Der zusätzliche Gateway-Hop kostet Latenz.

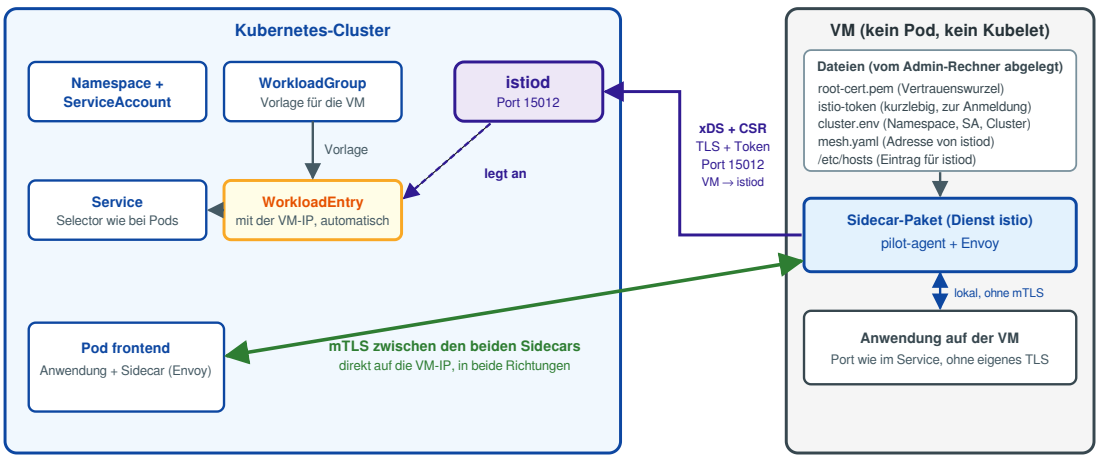
- Jedes Netz bekommt einen Namen: Label `topology.istio.io/network` am Namespace `istio-system`.
- Der Verkehr in das andere Netz geht **nur über dessen East-West-Gateway**. Pod-IPs des fremden Netzes sind nicht erreichbar, dafür dürfen sich IP-Bereiche überschneiden.
- **Sidecar:** mTLS-Verkehr auf Port 15443 (SNI-Passthrough). **Ambient:** Gateway-Klasse `istio-east-west`, Listener `HBONE` auf Port 15008.
- Remote-Secrets wie im flachen Netz: Die Kubernetes-API des anderen Clusters muss für das istiod erreichbar bleiben.
- Der zusätzliche Gateway-Hop kostet Latenz. Mit Locality Load Balancing bleibt der Verkehr nach Möglichkeit im eigenen Cluster oder in der eigenen Zone.

Grenzen bei Ambient (laut Doku)

- Nur Multi-Network wird unterstützt, Primary-Remote nicht.
- Waypoints müssen in allen Clustern **gleich heißen**, ihre Konfiguration muss man selbst synchron halten (z. B. mit ArgoCD).
- Es zählt nur die Konfiguration des **lokalen** Clusters. Globale Waypoints können ungewollten Verkehr zwischen Clustern erzeugen.
- Beim Failover in ein anderes Netz kann ein einzelner Endpoint überproportional viele Anfragen bekommen.
- Die Doku rät davon ab, Ambient-Multicluster in einer bestehenden produktiven Sidecar-Umgebung einzuschalten.

4. Hybrid: eine VM im Mesh

Hybrid: eine VM wird Mitglied im Mesh (Sidecar-Mode)



Ablauf

1. Kubernetes: Namespace, ServiceAccount, WorkloadGroup und Service anlegen. Die Dateien für die VM erzeugt istioctl x workload entry configure.
2. VM: Sidecar-Paket installieren, Dateien ablegen, Dienst istio starten.
3. Der Agent verbindet sich mit istiod auf Port 15012, meldet sich mit dem Token an und holt ein Zertifikat. istiod legt den WorkloadEntry an.
4. Ab da ist die VM ein Endpoint wie ein Pod: Der Service findet sie. mTLS endet im Sidecar der VM, die Anwendung spricht nur lokal mit ihm.

Nur mit Sidecar. Ambient kann keine VMs ins Mesh aufnehmen (Istio-Doku, Stand 10.11.2026). Die VM kennt nur eine istiod-Adresse: siehe nächste Grafik.

Eine VM ist ein **Workload ohne Kubelet**. Istio ist es egal, ob der Sidecar in einem Pod oder auf einer VM läuft, er braucht nur drei Dinge: istiod erreichen, ein Token zur Anmeldung, das Root-Zertifikat.

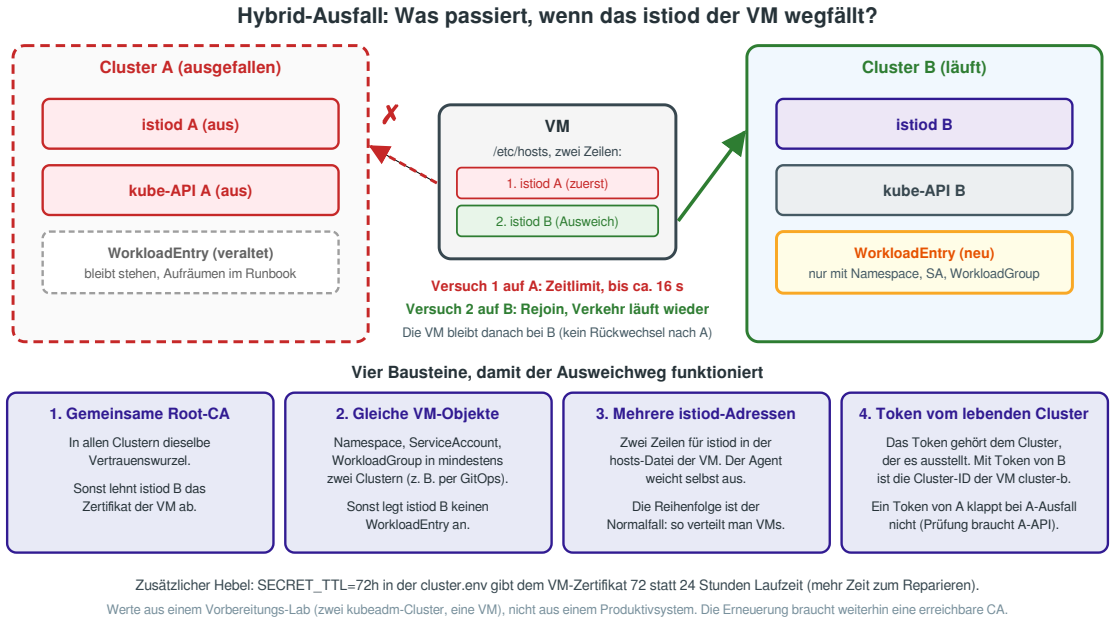
Seite	Was nötig ist
Kubernetes	Namespace, ServiceAccount, `WorkloadGroup` (Vorlage), `Service` mit Selector
VM	Sidecar-Paket (Dienst `istio`: `pilot-agent` + Envoy), `root-cert.pem`, `istio-token` (kurzlebig),

	`cluster.env`, `mesh.yaml`, `/etc/hosts`-Eintrag für istiod
Erzeugung der Dateien	`istioctl x workload entry configure -f <WorkloadGroup> -o <Ordner> --clusterID <Cluster> ...`

Beim Start verbindet sich der Agent mit istiod (Port **15012**, TLS plus Token), holt ein Zertifikat, und istiod legt den `WorkloadEntry` mit der VM-IP an (Auto-Registration). Ab da ist die VM ein Endpoint wie ein Pod. Das mTLS läuft zwischen den Sidecars (Pod-Sidecar und Sidecar der VM), nie bis in die Anwendung: Sie spricht nur lokal mit ihrem Sidecar. Der Verkehr geht in beide Richtungen direkt auf die VM-IP, wenn das Netz das erlaubt (sonst über ein East-West-Gateway).

Nur mit Sidecar. Ambient kann keine VMs ins Mesh aufnehmen (siehe Was Ambient noch nicht kann).

5. Hybrid: was passiert, wenn istiod ausfällt?



Die VM kennt standardmäßig **genau eine** istiod-Adresse. Fällt dieser Cluster aus, bekommt der Sidecar keine Konfigurationsänderungen mehr und kann sich nicht neu anmelden. Das VM-Zertifikat gilt standardmäßig 24 Stunden und wird nach der Hälfte erneuert. Ohne erreichbare CA fällt die VM nach Ablauf aus dem Mesh.

Ergebnisse aus dem Lab (zwei Cluster A und B, Multi-Primary, gemeinsame Root-CA, VM mit zwei istiod-Adressen):

Situation	Ergebnis
istiod A aus, API von A erreichbar, Zertifikat gültig	VM wechselt nach etwa 5 s zu istiod B
Cluster A komplett aus, Agent neu gestartet, Zertifikat gültig	Rejoin über B nach etwa 16 s (erster Versuch läuft in ein Zeitlimit)
A aus, Zertifikat entfernt, **Token von A**	Kein Rejoin (`Unauthenticated`), B kann das Token ohne die API von A nicht prüfen
A aus, Zertifikat entfernt, **Token von B**	Rejoin nach etwa 4 s

(Cluster-ID `cluster-b`)	

Die vier Bausteine für den Ausweichweg stehen in der Grafik. Offen bleiben: Die VM wechselt nicht von selbst zurück nach A, und der veraltete `WorkloadEntry` in A bleibt stehen, ein Aufräumen gehört ins Runbook. Nicht getestet: Ausfall während eines laufenden Token-Wechsels, mehr als zwei Cluster, IPv6, `istio-csr` statt Istio-eigener CA.

Wo passt das zu unserem Training?

- Wir bauen **einen** Cluster pro Teilnehmer mit Ambient, also weder Multi-Cluster noch VMs.
- Für den Kunden-Alltag gilt: Multi-Cluster mit Ambient nur als Multi-Primary über mehrere Netze (Beta). Wer **VMs im Mesh** oder **Primary-Remote** braucht, bleibt beim Sidecar-Mode.
- Hub-and-Spoke auf Azure mit Rancher: der Mesh-Verkehr bleibt im Cluster, siehe Rancher im Hub-and-Spoke-Netz.

Quellen

- Istio: Multicluster installieren
- Istio: Ambient multicluster und Multi-Primary über zwei Netze
- Istio: Deployment-Modelle
- Istio: Virtuelle Maschinen installieren

Nicht gegen die Doku geprüft (aus Istio-Wissen): Port 15443 beim Sidecar-East-West-Gateway, Standardlaufzeit 24 h des VM-Zertifikats, Auto-Registration als Standard.

Istio Federation & Cross-Cluster Traffic

Stand: 10.11.2026. Grundlage sind die Istio-Doku zu den Deployment-Modellen und die Doku zu Ambient Multicluster. Im Training läuft Istio **1.31.4**.

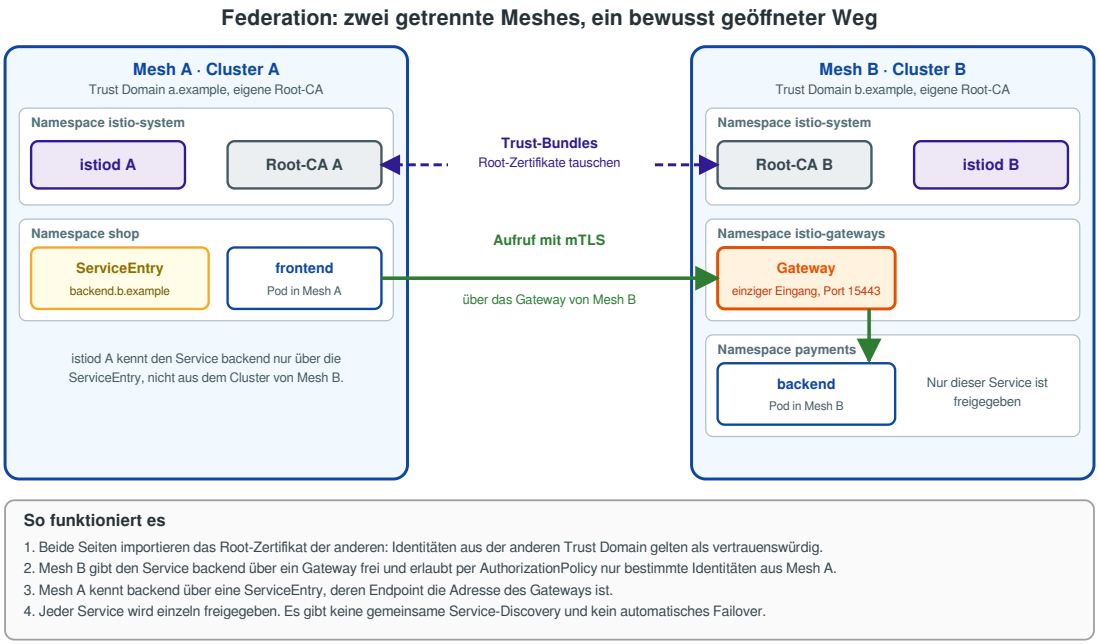
Cross-Cluster-Traffic: was ist das?

Cross-Cluster-Traffic ist jeder Aufruf, bei dem ein Pod in Cluster A einen Service in Cluster B erreicht. Es gibt zwei Wege dorthin. Der Unterschied ist, ob beide Cluster **ein Mesh** bilden oder **zwei getrennte Meshes** bleiben.

	Multi-Cluster (ein Mesh)	Federation (mehrere Meshes)
Aufbau	ein Mesh über mehrere Cluster	jeder Cluster ist ein eigenes Mesh
Trust Domain und Root-CA	gleich	verschieden
Service-Discovery	automatisch, istiod liest die Services des anderen Clusters	keine, jeder Service wird einzeln eingetragen
Welche Services sind sichtbar?	alle (mit gleichem Namen und Namespace)	nur die, die du freigibst
Failover zwischen Clustern	ja	nein
Typischer Fall	ein Team, Hochverfügbarkeit	getrennte Teams, Firmen oder Sicherheitszonen

Was ist Federation?

Federation heißt: **Zwei Meshes bleiben getrennt und öffnen gezielt einen Weg füreinander.** Jedes Mesh behält seinen eigenen istiod, seine eigene Root-CA und seine eigene Trust Domain. Sie tauschen nur zwei Dinge aus: **Vertrauen** (die Root-Zertifikate) und **die Adresse eines freigegebenen Services**.



Was kann Federation?

- **mTLS über die Meshgrenze.** Weil beide Seiten das Root-Zertifikat der anderen kennen, prüft jede Seite die Identität der Gegenseite.
- **Freigabe einzelner Services.** Mesh B macht nur `backend` sichtbar, alles andere bleibt unsichtbar.
- **Zugriffsregeln pro Identität.** Mesh B erlaubt per `AuthorizationPolicy` nur bestimmten Identitäten aus Mesh A den Zugriff (`principals: ["a.example/ns/shop/sa/frontend"]`).
- **Gleiche Namen in beiden Meshes.** Namespaces und Services dürfen in beiden Meshes gleich heißen. Eine eindeutige Mesh-ID sorgt für verschiedene FQDNs.

Wie läuft ein Aufruf?

Das ist das übliche Muster. **Istio liefert dafür kein fertiges Werkzeug.** Die Doku beschreibt nur das Prinzip (Identitäten und Trust-Bundles föderieren), die Bausteine setzt du selbst zusammen.

1. **Trust austauschen.** Jedes Mesh importiert die Root-Zertifikate des anderen, zum Beispiel über `meshConfig.caCertificates` und `trustDomainAliases` . Der Austausch läuft manuell oder per SPIFFE Trust Domain Federation.
2. **Service freigeben.** Mesh B stellt ein Gateway bereit (TLS-Passthrough, Port 15443), das den Verkehr an `backend` weiterreicht.
3. **Service importieren.** In Mesh A legst du eine `ServiceEntry` an. Der Host ist zum Beispiel `backend.b.example` , der Endpoint ist die Adresse des Gateways von Mesh B.
4. **Zugriff erlauben.** Mesh B schränkt per `AuthorizationPolicy` ein, wer durch das Gateway darf.

Eine **einfachere Variante** gibt es auch: Mesh A behandelt Mesh B wie eine **externe API** (`ServiceEntry` plus TLS am Ingress-Gateway von B). Dann gibt es keine Identitäts-Föderation, aber auch kein Zertifikats-Management zwischen den Meshes.

Wann setze ich Federation ein?

Ja, wenn:

- die Meshes **verschiedenen Teams oder Firmen** gehören und keiner dem anderen die Kontrolle über istiod oder die CA geben will,
- **Test und Produktion** strikt getrennt bleiben sollen, aber ein einzelner Service geteilt wird,
- nach einer **Fusion oder bei Partnern** zwei bestehende Meshes mit eigener CA zusammenarbeiten müssen,
- nur **wenige, genau definierte Services** den Cluster verlassen sollen.

Nein, wenn:

- es um **Hochverfügbarkeit** oder **Failover** eines Teams geht. Dann ist Multi-Cluster mit einem Mesh der richtige Weg.
- alle Services überall sichtbar sein sollen. Mit Federation müsstest du jeden einzeln freigeben.
- du keine Zeit für den laufenden Betrieb der Trust-Bundles hast.

Vorteile

- **Starke Isolation.** Ein Fehler oder eine Kompromittierung in Mesh A erreicht Mesh B nur über den freigegebenen Weg.
- **Klare Verantwortung.** Jedes Team betreibt seinen eigenen istiod, seine CA und seine Policies.
- **Freigabe nach dem Minimalprinzip.** Es ist nur sichtbar, was ausdrücklich eingetragen ist.
- **Keine Abstimmung bei Namen, Versionen und Konfiguration.** Die Meshes können verschiedene Istio-Versionen und Mesh-Configs haben.

Nachteile

- **Viel Handarbeit.** Trust-Bundles, `ServiceEntry`, Gateway und `AuthorizationPolicy` pflegst du selbst. Istio bringt dafür keine Automatik mit.
- **Zertifikate laufen ab.** Wird eine Root-CA erneuert, müssen alle Partner das neue Zertifikat bekommen. Sonst bricht der Verkehr ab.
- **Kein Failover und keine automatische Discovery.** Fällt der Service in Mesh B aus, springt Mesh A nicht von selbst auf eine andere Kopie.
- **Zusätzlicher Hop.** Jeder Aufruf geht über das Gateway. Das kostet Latenz.
- **Zwei Stellen für Beobachtung und Fehlersuche.** Traces und Metriken enden an der Meshgrenze, außer du verbindest sie.

Und Multi-Cluster mit einem Mesh?

Das ist der andere Weg für Cross-Cluster-Traffic: automatische Service-Discovery und Failover, dafür mehr Komplexität. Die Istio-Doku sagt: *"Multicluster deployments give you a greater degree of isolation and availability but increase complexity."* Die Bauformen, die Gateways und der Stand unter Ambient stehen auf der Seite Multi-Cluster und Hybrid im Überblick.

Federation unter Ambient: Dazu habe ich in der Istio-Doku keine Anleitung gefunden. Plane das nur mit einem Test auf deinem Cluster ein.

Merksatz

- **Ein Team, ein Ziel, Ausfallsicherheit:** Multi-Cluster mit einem Mesh.
- **Getrennte Verantwortung, wenige Services teilen:** Federation.

Quellen

- Istio: Deployment Models (Multiple Meshes, Multiple Clusters)
- Istio: Ambient Multicluster installieren

Erweiterbarkeit von Istio (nur Theorie)

Was ist Wasm, wofür brauche ich es in Istio, geht es mit Ambient? (mit Anwendungsfall)

Stand: 11.11.2026. Grundlage ist die Istio-Doku zu Version 1.31 (unser Stand: 1.31.4). Das Beispiel unten stammt aus der Doku und ist **auf dem Trainings-Cluster nicht ausgeführt**. Geprüft ist nur: Die CRD `TrafficExtension` ist auf unserem Cluster vorhanden.

Kurzfassung

- **Wasm** (WebAssembly) ist ein Binärformat. Du schreibst eine Erweiterung in Go, Rust oder C++, übersetzt sie in ein `.wasm`-Modul, und Envoy führt sie in einer Sandbox aus.
- **Wofür:** für Logik im Datenpfad, die Istio nicht mitbringt, ohne Envoy neu zu bauen und ohne die Anwendung zu ändern.
- **Ambient:** ja, aber **nur am Waypoint** (oder am Gateway). Im ztunnel läuft kein Wasm. Das Feature ist **Alpha**.

Was ist Wasm?

- Ursprünglich für den Browser gebaut: Code aus vielen Sprachen läuft dort in einer Sandbox, fast so schnell wie nativ.
- Envoy hat dieselbe Idee übernommen. In Envoy steckt eine Wasm-Laufzeit (V8).
- Die Schnittstelle zwischen Envoy und dem Modul heißt **Proxy-Wasm**. Das Modul bekommt Callbacks wie "Request-Header sind da" und darf Header lesen, ändern, den Request ablehnen oder selbst einen HTTP- oder gRPC-Aufruf machen.
- Das Modul liegt als OCI-Image in einer Registry (oder unter einer HTTP-URL). Der Proxy lädt es zur Laufzeit, ein Neustart ist nicht nötig.

Wofür brauche ich das in Istio?

Erst prüfen, ob Istio es schon kann. Meistens reicht das:

Aufgabe	Mittel in Istio
Wer darf wen aufrufen	`AuthorizationPolicy`
JWT prüfen	`RequestAuthentication`
Routing, Retries, Timeouts, Header setzen	`HTTPRoute`
Eigener Auth-Dienst entscheidet	`AuthorizationPolicy` mit `action: CUSTOM` (ext_authz)

Wasm ist für den Rest: eigene Logik, die für jeden Request im Proxy laufen soll. Typische Fälle:

- Anmeldung, die Istio nicht kennt (Basic Auth, API-Key, firmeneigenes Token-Format)
- Rate Limiting mit eigener Regel (siehe Übung im Backlog)
- Header oder Body umschreiben, zum Beispiel personenbezogene Daten maskieren
- Eigene Metriken oder Logzeilen aus dem Request ableiten

Wasm oder Lua?

Istio kennt zwei Arten von Erweiterungen. Beide werden mit `TrafficExtension` eingehängt.

	Wasm	Lua
Sprache	Go, Rust, C++ und andere	nur Lua
Auslieferung	Modul aus der Registry	Skript steht direkt im Objekt

Speicher	höher, jedes Plugin hat seine Sandbox	etwa 10x niedriger
Absturz	bleibt im Plugin	kann den Worker-Thread mitreißen
Verhalten bei Fehler	einstellbar, Standard ist "Request ablehnen"	immer "Request durchlassen"
Passt für	größere Logik, getestet und versioniert	kleine Änderung, Übergangslösung

Geht das mit Ambient?

Ja, mit drei Einschränkungen.



- **Nur am Waypoint oder Gateway.** Der ztunnel ist kein Envoy und arbeitet nur auf Layer 4. Ein Namespace ohne Waypoint hat keine Erweiterungen.
- **`targetRefs` statt `selector`.** Die `TrafficExtension` zeigt auf den Waypoint (alle Services dahinter), auf einen einzelnen Service oder auf ein Gateway. Ein `selector` wird am Waypoint ignoriert.
- **Alpha.** Für Entwickler und erfahrene Anwender gedacht, die Schnittstelle kann sich ändern.

Dazu die Falle aus dem Mischbetrieb: Verkehr, der den Waypoint umgeht, läuft auch an der Erweiterung vorbei. Das betrifft Sidecar-Clients und Gateways ohne `ingress-use-waypoint` (siehe Was Ambient (noch) nicht kann und Übung: Gateway umgeht den Waypoint).

`WasmPlugin` oder `TrafficExtension`?

- `WasmPlugin` ist das ältere Objekt, nur für Wasm.
- `TrafficExtension` ist das neue Objekt für Wasm und Lua. Die Referenz schreibt: "This API supersedes WasmPlugin".
- Beide CRDs sind in 1.31.4 vorhanden. Die Ambient-Doku nutzt nur noch `TrafficExtension`.
- Die Übung im Backlog nutzt noch `WasmPlugin`.

Konkreter Anwendungsfall: Basic Auth vor einem alten Dienst

Ausgangslage: Der Service `reviews` hat keine eigene Anmeldung. Jeder Pod im Mesh, der ihn erreichen darf, bekommt eine Antwort. Ein Batch-Job eines anderen Teams soll ihn künftig nur noch mit Benutzer und Passwort aufrufen dürfen. Das Team von `reviews` kann den Code gerade nicht ändern.

Warum hier Wasm: Istio prüft von Haus aus mTLS-Identitäten und JWT, aber kein Basic Auth. Für genau diesen Fall gibt es ein fertiges Modul aus dem Istio-Ökosystem.

Lösung: Der Waypoint prüft den Header `Authorization`, bevor der Request `reviews` erreicht.

```yaml
apiVersion: extensions.istio.io/v1alpha1
kind: TrafficExtension
metadata:
  name: basic-auth-for-service
  namespace: bookinfo
spec:
  targetRefs:
    - kind: Service
      group: ""
      name: reviews
  phase: AUTHN
  wasm:
    url: oci://ghcr.io/istio-ecosystem/wasm-extensions/basic_auth:1.12.0
    pluginConfig:
      basic_auth_rules:
        - prefix: "/reviews"
          request_methods:
            - "GET"
            - "POST"
          credentials:
            - "ok:test"
            - "MXQtaW4zOmFkbWluMw=="
```

Was die Felder tun:

- `targetRefs`: gilt nur für den Service `reviews`. Mit `kind: Gateway` und `name: waypoint` gälte es für alle Services hinter dem Waypoint.
- `phase: AUTHN`: der Filter läuft vor den Authentifizierungs-Filtern von Istio. Weitere Werte: `AUTHZ`, `STATS`.
- `wasm.url`: das Modul als OCI-Image. Der Waypoint lädt es selbst aus der Registry.
- `pluginConfig`: Einstellungen, die das Modul liest. Hier: Pfad, Methoden und erlaubte Zugangsdaten (Klartext `user:passwort` oder Base64).

Ergebnis laut Doku:

Aufruf von `http://reviews:9080/reviews/1`	**Antwort**
ohne Zugangsdaten	`401`
mit `Authorization: Basic MXQtaW4zOmFkbWluMw==`	`200`

Was du dafür brauchst:

- einen Waypoint im Namespace, und `reviews` muss ihn nutzen
- Zugriff vom Waypoint auf die Registry (`ghcr.io` oder eine eigene Registry mit `imagePullSecret`)
- in Produktion: Zugangsdaten nicht im Klartext im Objekt ablegen, und das Modul per `sha256` festnageln

Referenz

- Istio: Extensibility (Wasm und Lua)
- Istio: Extend waypoints with WebAssembly plugins
- Istio: Referenz TrafficExtension
- Istio: Referenz WasmPlugin
- Modul basic_auth (Quellcode)
- Proxy-Wasm Spezifikation

Wo läuft WASM (WebAssembly) im Rahmen von istio ?

- Direkt in envoy eingebuat

WASM-Runtime in Envoy:

- Envoy hat eine eingebettete WASM-Runtime (meist **V8** oder **Wasmtime**)
- Läuft im gleichen Prozess wie Envoy selbst
- WASM-Module werden zur Laufzeit geladen und im Sandbox ausgeführt

Architektur:

```
Pod
├── App-Container
└── Istio-Sidecar (Envoy)
    └── WASM-Runtime (V8/Wasmtime)
        └── Dein WASM-Plugin
```

Vorteile dieser Integration:

- Kein extra Container nötig
- Schnelle Kommunikation (keine Netzwerk-Calls)
- Memory-isoliert durch WASM-Sandbox
- Hot-Reload möglich ohne Envoy-Neustart

Runtime-Optionen:

- **V8** (Standard): Von Chrome, gut getestet
- **Wasmtime**: Leichtgewichtiger, Rust-basiert
- **WAVM**: Veraltet, nicht mehr empfohlen

Die Runtime wird beim Envoy-Build festgelegt - als User kannst du das normalerweise nicht ändern.

Automatisierung mit GitOps & ArgoCD

Was ist ArgoCD

Was ist ArgoCD?

ArgoCD ist ein deklarativer GitOps-Controller für Kubernetes. Er überwacht Git-Repositories und synchronisiert den dort definierten Soll-Zustand automatisch mit dem Cluster. Abweichungen (Drift) werden erkannt, gemeldet und optional selbstständig korrigiert.

Kernprinzip: **Git ist die Single Source of Truth** – kein `kubectl apply` oder `helm install` mehr von Hand.

Architektur-Übersicht



Warum ArgoCD für Istio?

Istio bringt eine Vielzahl an Custom Resources mit (VirtualService, DestinationRule, AuthorizationPolicy, Gateway, PeerAuthentication, EnvoyFilter, WasmPlugin …). Diese manuell über mehrere Cluster oder Namespaces konsistent zu halten, ist fehleranfällig. ArgoCD löst genau dieses Problem.

Konkrete Einsatzszenarien

Szenario	Ohne ArgoCD	Mit ArgoCD
Istio-Installation & Upgrades	`istioctl install` oder Helm manuell ausführen, Version im Cluster unklar	Istio Helm Charts im Git versioniert, Upgrade = Commit + PR
Traffic Management	VirtualServices per `kubectl apply` deployen, kein Review-Prozess	Canary Releases, Traffic Shifting als PR reviewbar und auditierbar
Security Policies	AuthorizationPolicy/PeerAuthentication ad hoc anwenden, Drift möglich	Zero-Trust-Policies versioniert, Drift wird erkannt und korrigiert
Multi-Cluster / Multi-Env	Copy-Paste von Manifests zwischen Staging und Prod	Kustomize-Overlays pro Umgebung, ein Git-Repo als Quelle
Observability-Stack	Kiali, Prometheus, Grafana manuell installieren	Gesamter Observability-Stack deklarativ verwaltet

Vorteile im Überblick

- **Auditierbarkeit** – Jede Änderung an Istio-Konfiguration ist ein Git-Commit mit Autor, Timestamp und Diff.
- **Rollback in Sekunden** – Git revert → ArgoCD synchronisiert den vorherigen Zustand.
- **Drift Detection** – Manuelle Änderungen im Cluster (`kubectl edit`) werden erkannt und gemeldet.

- **Self-Healing** – Optional: ArgoCD stellt den Git-Zustand automatisch wieder her.
- **PR-basierte Workflows** – Istio-Konfigurationsänderungen durchlaufen Code Review, bevor sie live gehen.
- **Multi-Cluster-Fähigkeit** – Ein ArgoCD verwaltet Istio-Konfiguration über mehrere Cluster hinweg.
- **Helm & Kustomize nativ** – Istio Helm Charts und Kustomize-Overlays werden direkt unterstützt.

ArgoCD vs. Flux CD

Kriterium	ArgoCD	Flux CD
UI	Vollständiges Web-UI mit Ressourcen-Baum	Kein eigenes UI (Weave GitOps als Add-on)
Architektur	Zentraler Server + Application CRD	Dezentrale Controller (source, kustomize, helm)
Multi-Tenancy	App Projects mit RBAC	Namespace-basierte Isolation
Sync-Modell	Pull + manueller/automatischer Sync	Reiner Pull-basierter Reconciliation-Loop
Istio-Kompatibilität	Volle CRD-Unterstützung, Resource Hooks für Reihenfolge	Volle CRD-Unterstützung, Depends-on für Reihenfolge

Beide Tools sind für Istio gleich gut geeignet. ArgoCD punktet durch das UI (hilfreich für Trainings und Troubleshooting), Flux CD durch geringere Komplexität und bessere Composability.

Typische Repo-Struktur für Istio + ArgoCD

```
gitops-repo/
├── base/
│   ├── istio/
│   │   ├── istio-base/          # Helm values für istio/base
│   │   ├── istiod/              # Helm values für istiod
│   │   └── gateway/             # Helm values für Istio Gateway
│   ├── bookinfo/
│   │   ├── deployment.yaml
│   │   ├── virtualservice.yaml
│   │   └── destinationrule.yaml
│   └── policies/
│       ├── authz-deny-all.yaml
│       └── peer-authn-strict.yaml
├── overlays/
│   ├── staging/
│   │   └── kustomization.yaml
│   └── production/
│       └── kustomization.yaml
└── argocd/
    ├── app-istio.yaml           # ArgoCD Application für Istio
    ├── app-bookinfo.yaml
    └── app-policies.yaml
```

Fazit

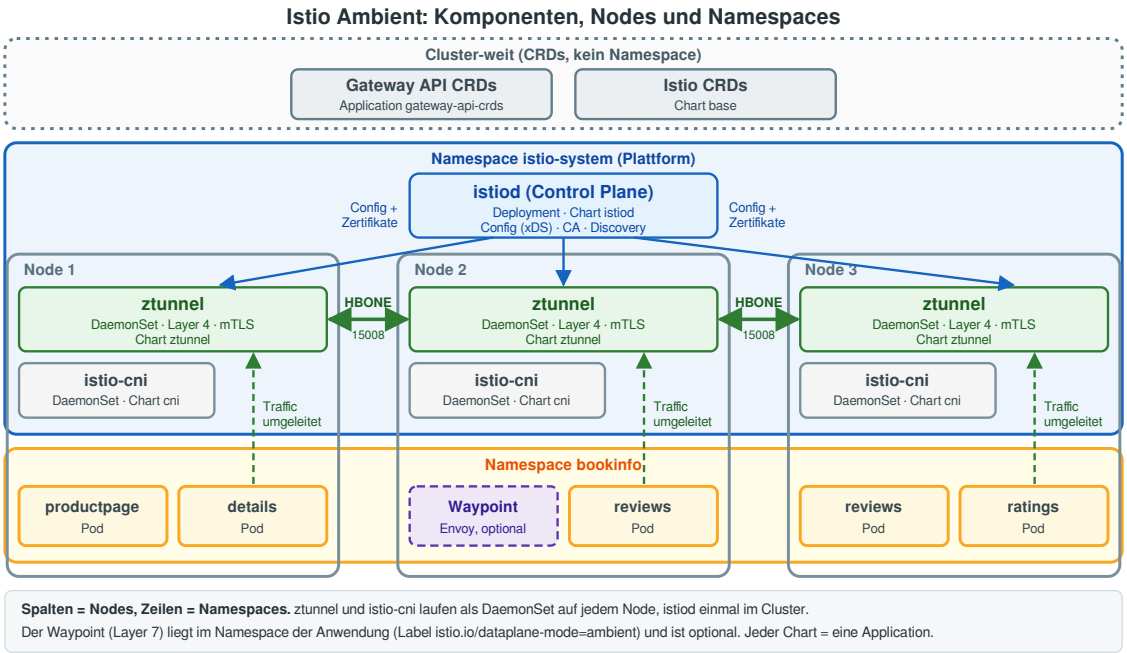
ArgoCD ist kein Muss für Istio, aber ein starker Enabler: Es macht Istio-Konfiguration nachvollziehbar, reviewbar und reproduzierbar. Gerade bei komplexeren Setups (Multi-Cluster, viele Policies, häufige Traffic-Shifts) reduziert es Fehler und erhöht die Geschwindigkeit.

istio ambient und bookinfo mit argocd ausrollen

1. Architektur-Überblick

Istio Ambient Mode – Komponenten

Im Gegensatz zum Sidecar-Modus gibt es bei Ambient Mode **kein Envoy-Sidecar** pro Pod. Stattdessen wird die Mesh-Funktionalität in zwei Schichten aufgeteilt:



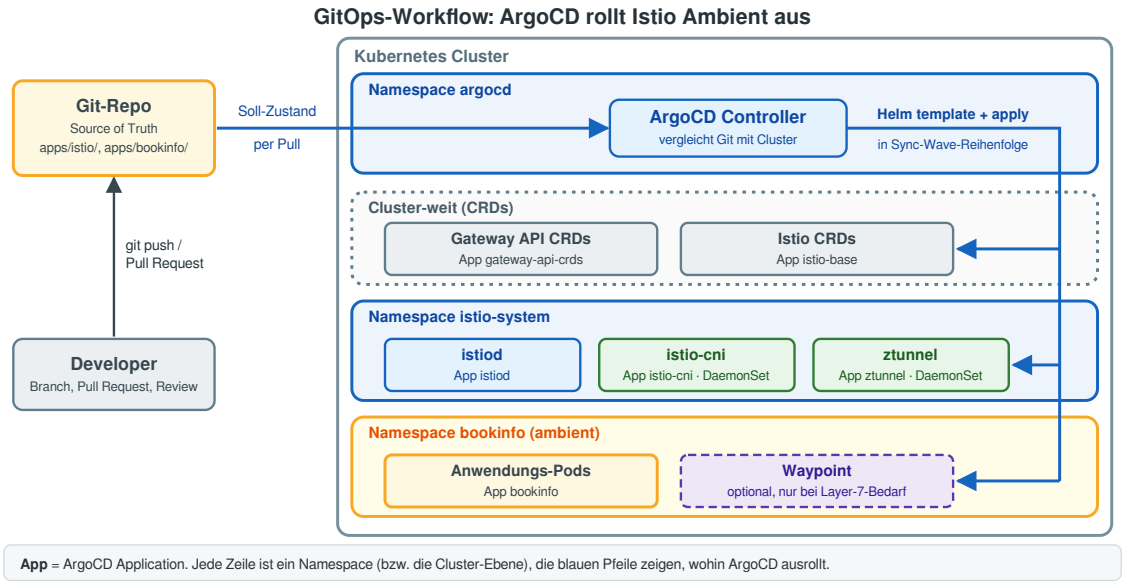
Zusammenfassung der Komponenten:

Helm Chart	Funktion	Typ
`istio/base`	CRDs (VirtualService, DestinationRule...)	Cluster
`istio/istiod`	Control Plane (xDS, Cert-Mgmt)	Cluster
`istio/cni`	CNI Plugin, Traffic-Redirect zu ztunnel	DaemonSet
`istio/ztunnel`	L4-Proxy, mTLS, HBONE-Tunnel	DaemonSet
Gateway API	CRDs für Waypoint/Ingress-Konfiguration	Cluster

---

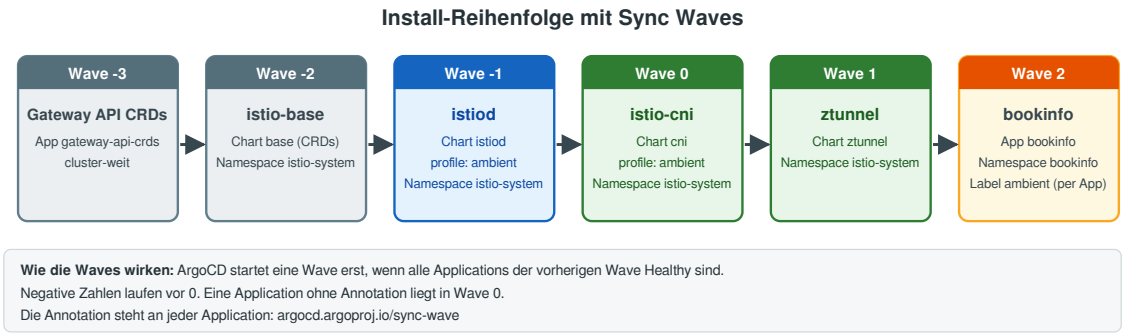
2. Wie ArgoCD mit Istio Ambient zusammenspielt

GitOps-Workflow



Install-Reihenfolge (kritisch!)

ArgoCD deployed Helm Charts. Bei Istio Ambient gibt es **Abhängigkeiten**:



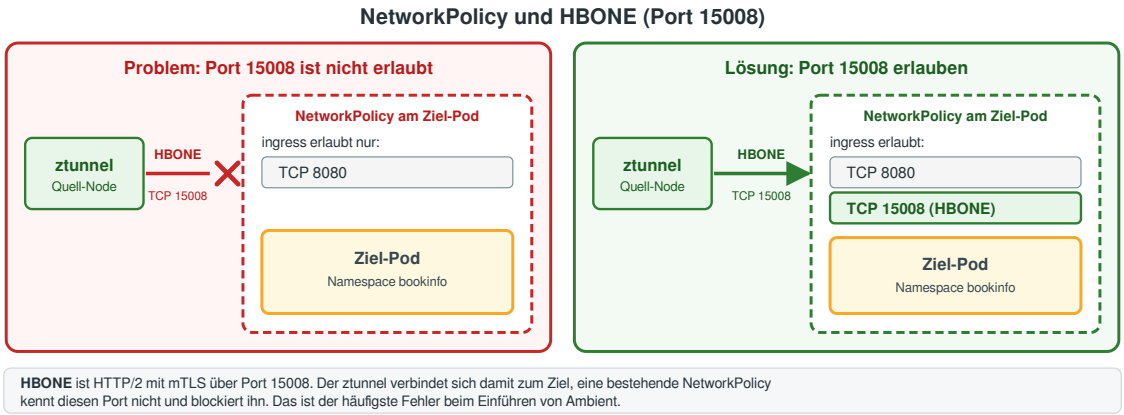
Reihenfolge: Gateway API CRDs → istio-base → istiod → cni → ztunnel → Apps

---

3. Gotchas & Fallstricke

NetworkPolicy + HBONE Port 15008

Das ist der häufigste Fehler! Ambient Mode nutzt HBONE (HTTP/2 + mTLS) über **Port 15008**. Bestehende NetworkPolicies blockieren diesen Port oft.

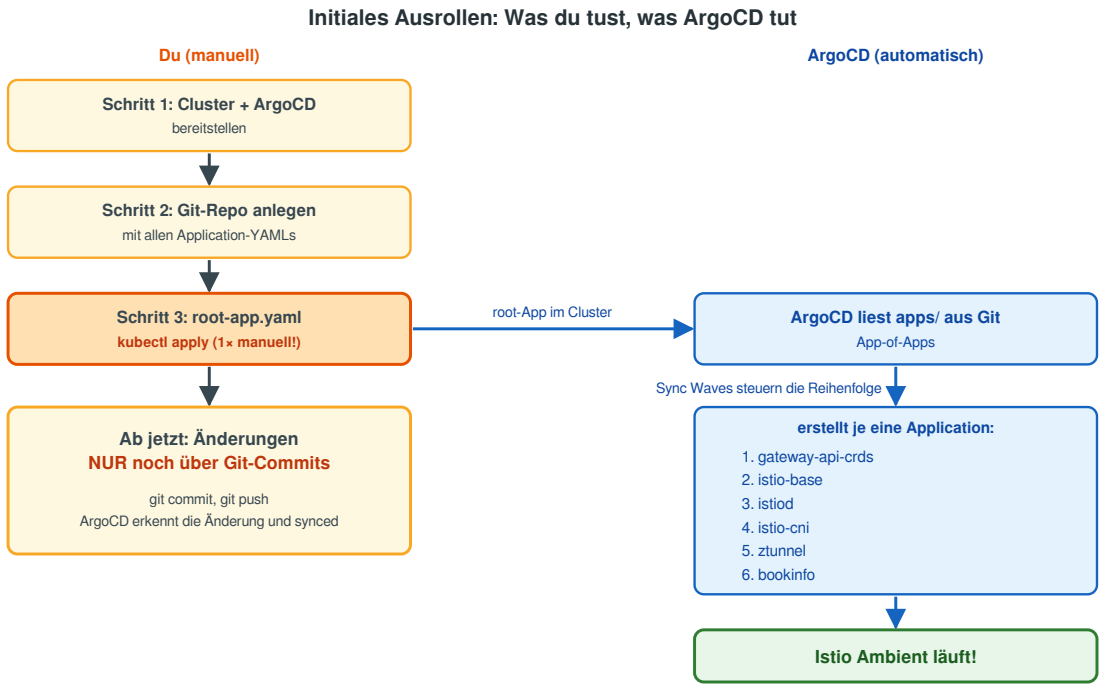


ArgoCD HA + Ambient Mode

ArgoCD HA nutzt Redis-HA mit Sentinel. Wenn der ArgoCD-Namespace ins Ambient Mesh eingebunden wird, muss Port 15008 in **allen** ArgoCD-NetworkPolicies erlaubt werden.

Empfehlung: ArgoCD-Namespace **NICHT** ins Ambient Mesh einbinden – ArgoCD ist die Management-Ebene für Istio und muss unabhängig davon funktionieren. ArgoCD verwaltet Istio – es muss nicht selbst im Mesh sein.

4. Initiales Ausrollen – Schritt für Schritt



Getestete Fassung: fertiges Repo

Das Repo jmetzger/istio-ambient-gitops enthält alles, was ArgoCD braucht: Root-Application, je Komponente eine Application mit Sync Wave, die Helm-Charts von Istio 1.31.4 und Bookinfo (Namespace mit Ambient-Label, Waypoint, Gateway, HTTPRoute). Getestet am 11.11.2026 auf einem leeren kubeadm-Cluster (ArgoCD v3.6.1, Istio 1.31.4, Gateway API v1.5.1, Kubernetes v1.37.2, MetalLB). Istio darf vorher **nicht** installiert sein.

```
## 1. ArgoCD installieren
kubectl create namespace argocd
kubectl apply --server-side --force-conflicts -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/v3.6.1/manifests/install.yaml
kubectl -n argocd wait --for=condition=available deployment --all --timeout=240s

## 2. Health-Prüfung für Applications (sonst warten die Sync Waves zwischen den
Applications nicht)
curl -sLO https://raw.githubusercontent.com/jmetzger/istio-ambient-
gitops/main/bootstrap/argocd-cm-app-health.yaml
kubectl -n argocd patch configmap argocd-cm --type merge --patch-file argocd-cm-app-
health.yaml

## 3. Root-Application: der einzige Handgriff am Cluster
kubectl apply -f https://raw.githubusercontent.com/jmetzger/istio-ambient-
gitops/main/bootstrap/root-app.yaml

## 4. Zuschauen (rund 3 Minuten), am Ende sieben Applications Synced / Healthy
kubectl -n argocd get applications -w
```

```
## Prüfen
kubectl -n istio-system get pods
kubectl -n bookinfo get pods,gateway
IP=$(kubectl -n bookinfo get gateway bookinfo-gateway -o
jsonpath='{.status.addresses[0].value}')
curl -s -o /dev/null -w '%{http_code}\n' http://$IP/productpage
```

Für eigene Änderungen das Repo forken und in `bootstrap/root-app.yaml` sowie in `apps/*.yaml` die `repoURL` anpassen.

Was beim Testen aufgefallen ist (die Lösungen stehen im Repo):

Beobachtung	Ursache	Lösung
Die Sync Waves zwischen den Applications wirken nicht, alles entsteht fast gleichzeitig.	ArgoCD bewertet den Zustand einer `Application` seit Version 1.8 nicht mehr von sich aus.	Health-Prüfung in `argocd-cm` (Schritt 2)
`istiod` steht auf `Degraded`.	Der HPA bekommt ohne metrics-server keinen Messwert.	`autoscaleEnabled: false`
`istio-base` und `istiod` bleiben `OutOfSync`.	istiod ändert `failurePolicy` und das CA-Bundle seiner Webhooks selbst.	`ignoreDifferences`, `RespectIgnoreDifferences=true`
Die CRD-Applications bleiben `OutOfSync`.	Geschrieben wird per Server-Side Apply, verglichen clientseitig.	Annotation `compare-options: ServerSideDiff=true`
Für Istio 1.31.4 gibt es keine Charts im Helm-Repo.	Stand 11.11.2026 endet der Index bei 1.30.8.	Charts aus dem Release-Archiv liegen im Git-Repo

> ***Achtung bei `argocd-cm`:*** *Die Einstellung als Patch einspielen, nicht als Teil-ConfigMap mit `kubectl apply --server-side`. Das entfernt alle anderen Felder samt Labels, ArgoCD meldet dann `configmap "argocd-`*

> *cm" not found*.

Abbauen: `kubectl -n argocd delete application root`. Es bleiben der Namespace `istio-system`, die Istio-CRDs (`helm.sh/resource-policy: keep`) und die Gateway-API-CRDs.

Die folgenden Schritte zeigen denselben Aufbau zum Selbermachen. Die Manifeste dort sind **nicht** mit Istio 1.31.4 getestet (Version 1.24.2 aus dem Helm-Repo, ohne die Korrekturen aus der Tabelle).

Schritt 1: Voraussetzungen prüfen

```
## Kubernetes >= 1.28
kubectl version

## Helm >= 3.6 (nur zum Prüfen, ArgoCD nutzt Helm intern)
helm version
```

Schritt 2: ArgoCD installieren

```
## Falls noch kein ArgoCD im Cluster:
kubectl create namespace argocd
## --server-side ist nötig: das CRD applicationsets.argoproj.io ist für ein normales
## "kubectl apply" zu groß ("metadata.annotations: Too long")
kubectl apply --server-side --force-conflicts -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

## Warten bis alles läuft:
kubectl wait --for=condition=available deployment/argocd-server \
  -n argocd --timeout=120s

## Admin-Passwort holen:
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo

## Optional: Port-Forward für UI
kubectl port-forward svc/argocd-server -n argocd 8080:443
## → https://localhost:8080  (User: admin)
```

Schritt 3: Git-Repo anlegen und befüllen

```
## Neues Repo erstellen (z.B. auf GitHub/GitLab)
mkdir istio-ambient-gitops && cd istio-ambient-gitops
git init

## Verzeichnisstruktur anlegen
mkdir -p apps/istio/values apps/bookinfo
```

Jetzt alle Application-YAMLs und Values-Dateien erstellen (siehe Abschnitt 4.5 für den Inhalt jeder Datei):

```
## Dateien erstellen:
touch apps/istio/gateway-api-crds.yaml
```

```
touch apps/istio/istio-base-app.yaml
touch apps/istio/istiod-app.yaml
touch apps/istio/istio-cni-app.yaml
touch apps/istio/ztunnel-app.yaml
touch apps/istio/values/istiod-values.yaml
touch apps/bookinfo/bookinfo-app.yaml

## Alles committen und pushen:
git add -A
git commit -m "Initial Istio Ambient + ArgoCD setup"
git remote add origin https://github.com/DEIN-USER/istio-ambient-gitops.git
git push -u origin main
```

Schritt 4: root-app.yaml erstellen und anwenden

Diese Datei liegt NICHT im Git-Repo – sie ist der einzige manuelle Schritt:

```
cat <<'EOF' | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/DEIN-USER/istio-ambient-gitops.git
    targetRevision: main
    path: apps/
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
```

Schritt 5: Zuschauen und verifizieren

```
## ArgoCD Applications beobachten:
kubectl get applications -n argocd -w

## Erwartete Ausgabe nach ein paar Minuten:
## NAME                 STATUS   HEALTH
## root                 Synced   Healthy
## gateway-api-crds     Synced   Healthy
## istio-base           Synced   Healthy
## istiod               Synced   Healthy
## istio-cni            Synced   Healthy
## ztunnel              Synced   Healthy
```

```
## bookinfo           Synced   Healthy

## Istio-Pods prüfen:
kubectl get pods -n istio-system

## CRDs prüfen:
kubectl get crds | grep -E 'istio|gateway'
```

Ab jetzt: Alles über Git

```
## Beispiel: Neue App hinzufügen
vim apps/bookinfo/bookinfo-app.yaml
git add -A && git commit -m "Add bookinfo app"
git push
## → ArgoCD erkennt die Änderung und deployed automatisch

## Beispiel: Istio upgraden
## In apps/istio/*-app.yaml: targetRevision von 1.24.2 auf 1.25.0 ändern
git add -A && git commit -m "Upgrade Istio to 1.25.0"
git push
## → ArgoCD rolled die neuen Versionen aus
```

4.4 Git-Repo Struktur

```
istio-ambient-gitops/
├── apps/
│   ├── istio/
│   │   ├── gateway-api-crds.yaml       # ArgoCD Application
│   │   ├── istio-base-app.yaml         # ArgoCD Application
│   │   ├── istiod-app.yaml             # ArgoCD Application
│   │   ├── istio-cni-app.yaml          # ArgoCD Application
│   │   ├── ztunnel-app.yaml            # ArgoCD Application
│   │   └── values/
│   │       ├── istiod-values.yaml
│   │       ├── cni-values.yaml
│   │       └── ztunnel-values.yaml
│   ├── bookinfo/
│   │   └── bookinfo-app.yaml           # ArgoCD Application
│   └── root-app.yaml                   # App-of-Apps
```

4.5 ArgoCD Application Manifeste

Gateway API CRDs (Sync Wave -3)

```
## apps/istio/gateway-api-crds.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: gateway-api-crds
  namespace: argocd
  annotations:
```

```
    argocd.argoproj.io/sync-wave: "-3"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/kubernetes-sigs/gateway-api.git
    targetRevision: v1.4.0
    path: config/crd/standard
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - ServerSideApply=true
      - CreateNamespace=false
```

istio-base (Sync Wave -2)

```
## apps/istio/istio-base-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-base
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://istio-release.storage.googleapis.com/charts
    chart: base
    targetRevision: 1.24.2     # oder 1.29.x je nach Version
  destination:
    server: https://kubernetes.default.svc
    namespace: istio-system
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true    # wichtig für CRDs!
```

istiod (Sync Wave -1)

```
## apps/istio/istiod-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
```

```yaml
metadata:
  name: istiod
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  sources:
    - repoURL: https://istio-release.storage.googleapis.com/charts
      chart: istiod
      targetRevision: 1.24.2
      helm:
        valueFiles:
          - $values/apps/istio/values/istiod-values.yaml
    - repoURL: https://github.com/DEIN-USER/istio-ambient-gitops.git
      targetRevision: main
      ref: values
  destination:
    server: https://kubernetes.default.svc
    namespace: istio-system
  syncPolicy:
    automated:
      selfHeal: true
```

istiod-values.yaml:

```yaml
## apps/istio/values/istiod-values.yaml
profile: ambient
pilot:
  resources:
    requests:
      cpu: 200m
      memory: 256Mi
meshConfig:
  accessLogFile: /dev/stdout
```

> ***Externe CA statt der eigenen CA von istiod?*** *Dann muss das Secret* `cacerts` *vor dem ersten Start von istiod vorhanden sein. Wie das mit ArgoCD und cert-manager geht, steht in der Übung Externe CA für Istio mit ArgoCD ausliefern.*

istio-cni (Sync Wave 0)

```yaml
## apps/istio/istio-cni-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-cni
  namespace: argocd
  annotations:
```

```
    argocd.argoproj.io/sync-wave: "0"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://istio-release.storage.googleapis.com/charts
    chart: cni
    targetRevision: 1.24.2
    helm:
      values: |
        profile: ambient
  destination:
    server: https://kubernetes.default.svc
    namespace: istio-system       # oder kube-system je nach Setup
  syncPolicy:
    automated:
      selfHeal: true
```

ztunnel (Sync Wave 1)

```
## apps/istio/ztunnel-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ztunnel
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "1"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://istio-release.storage.googleapis.com/charts
    chart: ztunnel
    targetRevision: 1.24.2
  destination:
    server: https://kubernetes.default.svc
    namespace: istio-system
  syncPolicy:
    automated:
      selfHeal: true
```

bookinfo (Sync Wave 2)

Die Beispiel-Anwendung kommt **nach** ztunnel (Wave 2), damit die Pods beim Start bereits im Ambient Mesh sind. Die Application legt den Namespace samt Ambient-Label selbst an (`managedNamespaceMetadata` , braucht `CreateNamespace=true`):

```
## apps/bookinfo/bookinfo-app.yaml
apiVersion: argoproj.io/v1alpha1
```

```yaml
kind: Application
metadata:
  name: bookinfo
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "2"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/istio/istio.git
    targetRevision: 1.24.2          # gleiche Version wie die Istio-Charts
    path: samples/bookinfo/platform/kube
    directory:
      include: bookinfo.yaml        # nur die Anwendung, nicht die Varianten im
Ordner
  destination:
    server: https://kubernetes.default.svc
    namespace: bookinfo
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
    managedNamespaceMetadata:
      labels:
        istio.io/dataplane-mode: ambient    # Namespace ins Ambient Mesh aufnehmen
```

Multi-Source Variante (alles in einer Application)

Alternativ kann man alle Charts in einer einzigen Application mit `sources` bündeln. Das funktioniert, aber man verliert die Sync-Wave-Kontrolle:

```yaml
## Kompakte Variante - ACHTUNG: keine Reihenfolge-Garantie!
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-ambient
  namespace: argocd
spec:
  project: default
  destination:
    server: https://kubernetes.default.svc
    namespace: istio-system
  sources:
    - repoURL: https://istio-release.storage.googleapis.com/charts
      chart: base
      targetRevision: 1.24.2
      helm:
        releaseName: istio-base
    - repoURL: https://istio-release.storage.googleapis.com/charts
```

```
      chart: istiod
      targetRevision: 1.24.2
      helm:
        releaseName: istiod
        parameters:
          - name: profile
            value: ambient
    - repoURL: https://istio-release.storage.googleapis.com/charts
      chart: cni
      targetRevision: 1.24.2
      helm:
        releaseName: istio-cni
        parameters:
          - name: profile
            value: ambient
    - repoURL: https://istio-release.storage.googleapis.com/charts
      chart: ztunnel
      targetRevision: 1.24.2
      helm:
        releaseName: ztunnel
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
```

4.6 App-of-Apps (Root Application)

```
## apps/root-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/DEIN-USER/istio-ambient-gitops.git
    targetRevision: main
    path: apps/
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
```

4.7 Namespace ins Ambient Mesh einbinden

```yaml
## bookinfo-namespace.yaml (im Git-Repo!)
apiVersion: v1
kind: Namespace
metadata:
  name: bookinfo
  labels:
    istio.io/dataplane-mode: ambient    # ← Das ist der Schlüssel!
```

4.8 Waypoint Proxy deployen (für L7-Features)

```yaml
## Nur nötig wenn AuthorizationPolicy, Retries, etc. auf L7 gebraucht werden
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: bookinfo-waypoint
  namespace: bookinfo
  labels:
    istio.io/waypoint-for: service      # oder "all" für den ganzen Namespace
spec:
  gatewayClassName: istio-waypoint
  listeners:
    - name: mesh
      port: 15008
      protocol: HBONE
```

4.9 Verifizierung

```bash
## Istio-Komponenten prüfen
kubectl get pods -n istio-system
## Erwartung: istiod-xxx, ztunnel-xxx (pro Node), istio-cni-node-xxx (pro Node)

## CRDs prüfen
kubectl get crds | grep -E 'istio|gateway'

## Ambient-Enrollment prüfen
kubectl get namespace bookinfo --show-labels | grep dataplane-mode

## ztunnel-Logs: mTLS-Verbindungen sichtbar
kubectl logs -n istio-system -l app=ztunnel --tail=20

## Mesh-Status mit istioctl
istioctl ztunnel-config workloads
```

---

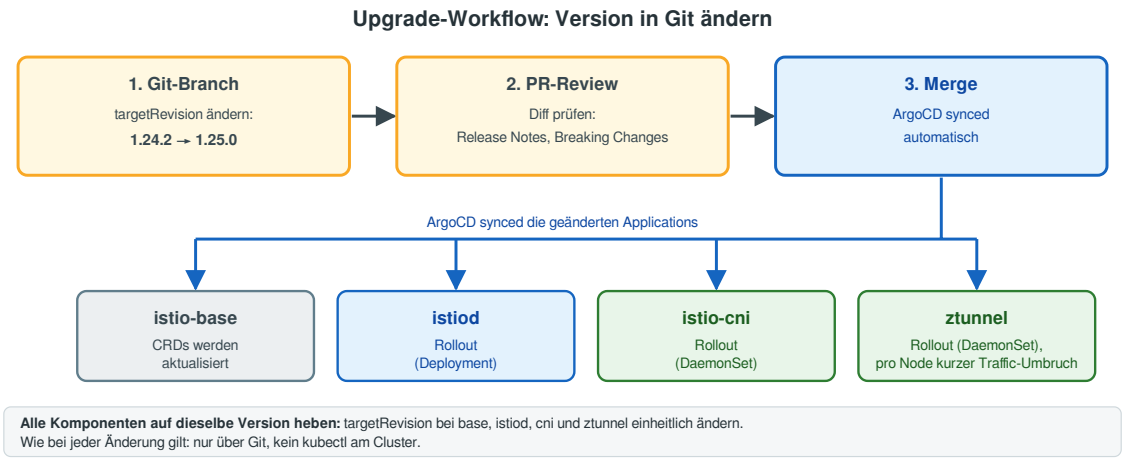
5. Checkliste: Was muss ich beachten?

```
✓  Gateway API CRDs VOR Istio installieren (Sync Wave!)
✓  Helm Charts in korrekter Reihenfolge: base → istiod → cni → ztunnel
✓  ServerSideApply=true für CRD-haltige Charts setzen
```

```
✓ profile: ambient bei istiod und cni setzen
✓ NetworkPolicies: Port 15008 (HBONE) freigeben
✓ ArgoCD-Namespace NICHT ins Ambient Mesh (Management-Ebene bleibt unabhängig)
✓ Namespace-Label: istio.io/dataplane-mode=ambient
✓ ArgoCD extern nur über HTTPS und mit geändertem admin-Passwort (Abschnitt 7)
✓ Waypoint nur deployen, wenn L7-Features nötig sind
✓ Alle Istio-Versionen einheitlich halten (base, istiod, cni, ztunnel)
✓ Bei Upgrades: targetRevision in Git ändern → ArgoCD synced automatisch
```

---

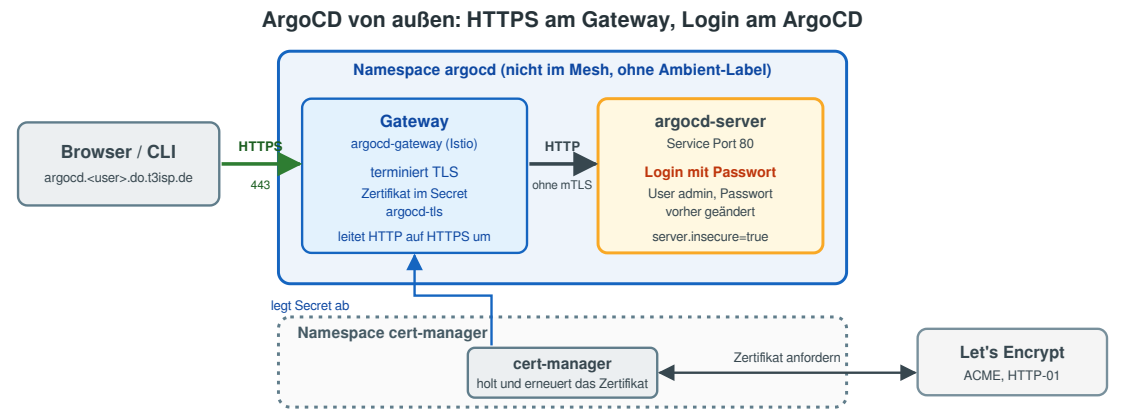
6. Upgrade-Workflow



Wichtig: ztunnel ist ein DaemonSet – beim Upgrade gibt es einen kurzen Traffic-Umbruch pro Node. In Produktion: `maxUnavailable: 1` in den Values setzen.

---

7. ArgoCD von außen erreichbar machen (HTTPS + Passwort)



> ***Merkkasten: Das Gateway gehört nicht zum Mesh***
>
> *Ein Gateway mit `gatewayClassName: istio` ist ein normaler Envoy-Ingress. Er kann an **jeden** Service im Cluster weiterleiten, auch an Anwendungen außerhalb des Meshs. Backends im Ambient Mesh erreicht er per HBONE/mTLS, Backends außerhalb per einfachem HTTP. Mesh-Policies (`PeerAuthentication` ,*

> *`AuthorizationPolicy` für Workloads) gelten nur für Pods im Mesh. Liegt das Backend in einem anderen Namespace als die Route, braucht es ein `ReferenceGrant`.*

Wer schützt hier was?

- **Das Passwort** kommt von ArgoCD selbst (Login-Seite, User `admin`). Istio bringt kein Basic-Auth mit.
- **Das Gateway** sorgt nur für HTTPS (Let's Encrypt). Ohne HTTPS würde das Passwort im Klartext übers Netz gehen.
- Der Namespace `argocd` bleibt **außerhalb des Meshs** (siehe Abschnitt 3). Das Gateway ist ein normaler Envoy-Pod und spricht mit `argocd-server` per einfachem HTTP, ohne mTLS. Das geschieht nur innerhalb des Clusters.

Voraussetzungen

- cert-manager ist installiert (siehe Übung Bookinfo mit HTTPS und Let's Encrypt, Schritt 2).
- Ein DNS-Eintrag `argocd.<dein-user>.do.t3isp.de` zeigt auf die öffentliche IP des Gateways (Trainer fragen).
- Das Gateway braucht eine **eigene IP**, auf der noch kein anderes Gateway auf Port 80/443 lauscht (zwei Gateways mit derselben IP gehen nicht). Im Training hat jeder Cluster zwei Adressen im MetalLB-Pool: Worker 1 nutzt das `bookinfo-gateway`, für ArgoCD nimmst du die von **Worker 2**. Der DNS-Eintrag `argocd.<dein-user>.do.t3isp.de` muss auf diese zweite IP zeigen.
- **Reihenfolge beim DNS:** Den Namen erst abfragen (`dig`, Browser), **nachdem** der Eintrag existiert. Eine Abfrage vorher wird als "nicht vorhanden" zwischengespeichert (Router, Windows, Browser), bei dieser Zone bis zu 30 Minuten.

```
export HOST=argocd.$(whoami).do.t3isp.de
export IP=$(dig +short $HOST)
echo $HOST $IP
kubectl -n metallb-system get ipaddresspool first-pool -o
jsonpath='{.spec.addresses}'; echo   # IP muss im Pool stehen
```

Schritt 1: ArgoCD hinter dem Gateway ohne eigenes TLS betreiben

Das Gateway terminiert TLS. `argocd-server` soll deshalb selbst kein TLS sprechen (sonst gibt es Redirect-Schleifen):

```
kubectl -n argocd patch configmap argocd-cmd-params-cm \
  --type merge -p '{"data":{"server.insecure":"true"}}'
kubectl -n argocd rollout restart deployment argocd-server
kubectl -n argocd rollout status deployment argocd-server
```

Schritt 2: Issuer, Gateway und Routen

```
## 01-argocd-extern.yaml
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: letsencrypt
  namespace: argocd
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
```

```
    privateKeySecretRef:
      name: letsencrypt-account-key
    solvers:
    - http01:
        gatewayHTTPRoute:
          parentRefs:
          - kind: Gateway
            name: argocd-gateway
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: argocd-gateway
  namespace: argocd
  annotations:
    cert-manager.io/issuer: letsencrypt
spec:
  gatewayClassName: istio
  addresses:
  - type: IPAddress
    value: ${IP}
  listeners:
  - name: http
    hostname: ${HOST}
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same
  - name: https
    hostname: ${HOST}
    port: 443
    protocol: HTTPS
    tls:
      mode: Terminate
      certificateRefs:
      - name: argocd-tls
    allowedRoutes:
      namespaces:
        from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: http-to-https
  namespace: argocd
spec:
  parentRefs:
  - name: argocd-gateway
    sectionName: http
  rules:
  - filters:
```

```
    - type: RequestRedirect
      requestRedirect:
        scheme: https
        statusCode: 301
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: argocd
  namespace: argocd
spec:
  parentRefs:
  - name: argocd-gateway
    sectionName: https
  rules:
  - backendRefs:
    - name: argocd-server
      port: 80
```

```
envsubst < 01-argocd-extern.yaml | kubectl apply -f -
kubectl -n argocd get gateway argocd-gateway       # PROGRAMMED True
kubectl -n argocd get certificate                  # READY True (ca. 1 Minute)
```

Schritt 3: Passwort setzen

Das Initial-Passwort ist nur für den ersten Login gedacht. Ändere es, bevor ArgoCD im Internet steht (am besten schon vor Schritt 2). Das Initial-Passwort steht im Secret `argocd-initial-admin-secret` (Abschnitt 4, Schritt 2) und lässt sich auch mit `argocd admin initial-password -n argocd` anzeigen.

```
## Einloggen und Passwort ändern
## (--grpc-web: mit einer einfachen HTTPRoute auf Port 80 läuft die CLI über gRPC-Web
statt HTTP/2)
argocd login $HOST --grpc-web --username admin
argocd account update-password --grpc-web
```

Alternativ in der Oberfläche: **User Info → Update Password**. Danach das Initial-Secret löschen:

```
kubectl -n argocd delete secret argocd-initial-admin-secret
```

Schritt 4: Testen

```
## Weiterleitung auf HTTPS
curl -sI http://$HOST | head -n 2
## Erwartet: 301, location: https://...

## Ohne Login keine Daten (Erwartet: 401 oder 403)
curl -s -o /dev/null -w '%{http_code}\n' https://$HOST/api/v1/applications
```

Im Browser `https://<HOST>` öffnen: Schloss in der Adressleiste und die ArgoCD-Login-Seite.

Gut zu wissen

Thema	Hinweis
Brute-Force	ArgoCD sperrt Logins nach mehreren Fehlversuchen zeitweise.
Weitere Nutzer	Statt alle mit `admin` arbeiten zu lassen: lokale Accounts in `argocd-cm` (`accounts.<name>: login`) mit RBAC in `argocd-rbac-cm`, oder SSO (OIDC).
Zusätzlicher Vorab-Schutz	Soll schon vor der Login-Seite ein zweites Passwort oder SSO greifen: `oauth2-proxy` plus `AuthorizationPolicy` mit `CUSTOM`-Provider (ext_authz) am Gateway. Das ist ein eigenes Thema.
Zugriff einschränken	Die ArgoCD-API ist mächtig (sie deployed in den Cluster). Wenn möglich zusätzlich nach Quell-IP einschränken (Firewall oder `AuthorizationPolicy` mit `ipBlocks`).

Quellen

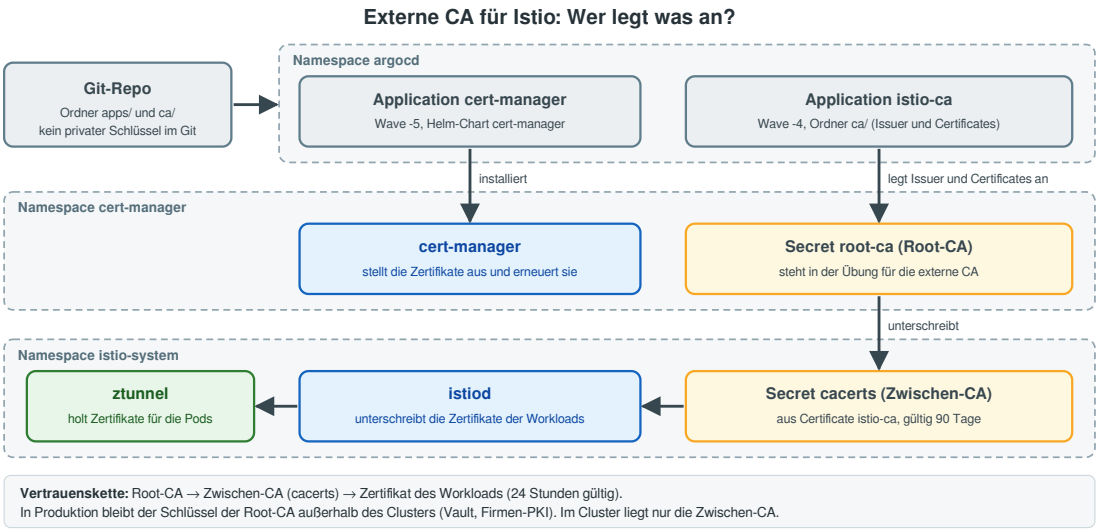
- ArgoCD, Ingress-Konfiguration (`server.insecure` , Gateway-API-Beispiel mit Gateway, HTTPRoute, GRPCRoute, BackendTLSPolicy): https://argo-cd.readthedocs.io/en/stable/operator-manual/ingress/
- ArgoCD, Initial-Passwort ändern und Secret löschen: https://argo-cd.readthedocs.io/en/stable/getting_started/
- ArgoCD, `managedNamespaceMetadata` : https://argo-cd.readthedocs.io/en/stable/user-guide/sync-options/
- Istio, Gateway API: https://istio.io/latest/docs/tasks/traffic-management/ingress/gateway-api/
- cert-manager, Gateway API: https://cert-manager.io/docs/usage/gateway/

Übung: Externe CA für Istio mit ArgoCD ausliefern (cert-manager, Secret cacerts)

Worum geht es?

Ohne weiteres Zutun erzeugt istiod beim ersten Start eine eigene, selbstsignierte CA (Secret `istio-ca-secret`). In Firmen soll das Mesh meist an der vorhandenen PKI hängen. Dafür liest istiod eine **Zwischen-CA** aus dem Secret `cacerts` im Namespace `istio-system` .

Der private Schlüssel gehört nicht ins Git. Über ArgoCD wird deshalb nicht das Secret ausgeliefert, sondern ein `Certificate` für cert-manager. cert-manager erzeugt daraus das Secret im Cluster und erneuert es.

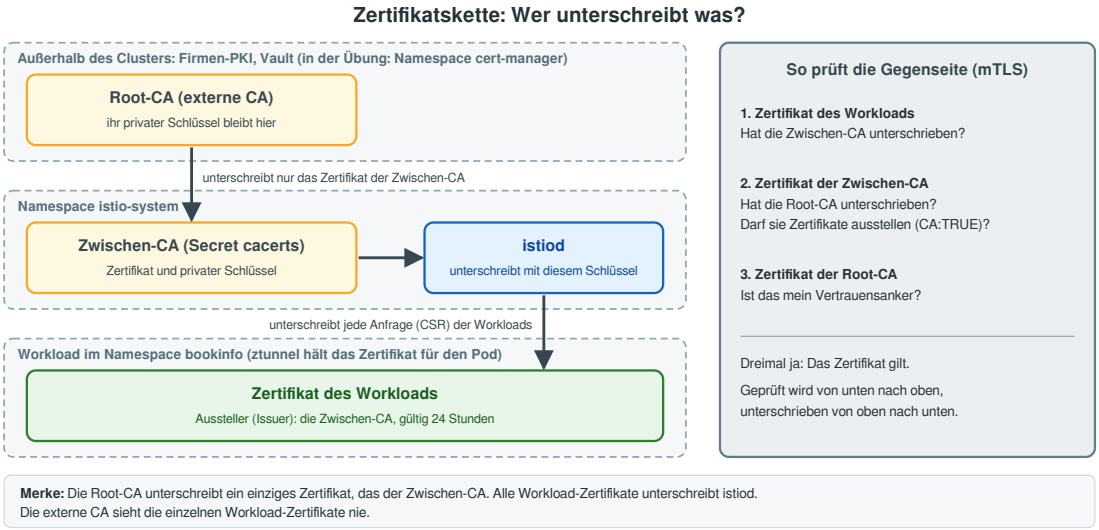


> ***Merkkasten: Die Root-CA ist in der Übung nur ein Platzhalter***

> *In der Übung erzeugt cert-manager eine selbstsignierte Root-CA im Cluster. Sie steht für die externe CA. In Produktion ersetzt du genau diese Stelle durch den Issuer eurer PKI (z. B. Vault). Alles danach bleibt gleich.*

Wer unterschreibt was? Die Zertifikatskette

Die externe CA unterschreibt **nicht** die Zertifikate der Workloads. Sie unterschreibt nur ein einziges Zertifikat: das der Zwischen-CA. Alle Workload-Zertifikate unterschreibt weiterhin istiod, jetzt mit dem Schlüssel der Zwischen-CA.



Mit ihrer Unterschrift sagt die Root-CA: "Diese Zwischen-CA darf in meinem Namen Zertifikate ausstellen." Im Zertifikat der Zwischen-CA steht das als `CA:TRUE`.

Was bedeutet `CA:TRUE`?

`CA:TRUE` ist X.509-Standard (Erweiterung **Basic Constraints**, RFC 5280) und keine Besonderheit von cert-manager. `isCA: true` im `Certificate` ist nur der Schalter dafür: cert-manager schreibt den Wunsch in die Zertifikatsanfrage, die Root-CA übernimmt ihn ins Zertifikat.

- Mit `CA:TRUE` darf der **private Schlüssel** zu diesem Zertifikat andere Zertifikate unterschreiben. Das Zertifikat ist der öffentliche Nachweis dafür.
- Mit `CA:FALSE` (oder ohne die Erweiterung) ist es ein Endzertifikat, z. B. das Zertifikat eines Workloads.
- Durchgesetzt wird die Regel **beim Prüfen**, nicht beim Ausstellen: Die Gegenseite verlangt bei jedem ausstellenden Zertifikat in der Kette `CA:TRUE`, sonst lehnt sie ab.

`CA:TRUE` allein reicht nicht. Damit ein ausgestelltes Zertifikat gilt, muss außerdem stimmen:

Bedingung	Bedeutung
Kette	Das CA-Zertifikat ist selbst von einer vertrauten CA unterschrieben, bis hoch zur Root-CA
Key Usage	Ist die Erweiterung gesetzt, muss `Certificate Sign` enthalten sein (cert-manager setzt das bei `isCA: true` mit)
Gültigkeit	Das CA-Zertifikat ist nicht abgelaufen
Pfadlänge	Ein `pathlen` weiter oben in der Kette wird nicht überschritten (`pathlen:0` = nur Endzertifikate, keine weiteren CAs darunter)

Ein selbst gebautes Zertifikat mit `CA:TRUE` nützt also nichts, solange keine vertraute CA es unterschrieben hat. Einzige Ausnahme ist die Root-CA: Sie unterschreibt sich selbst, und man vertraut ihr, weil sie als Vertrauensanker hinterlegt ist.

Ob die Zwischen-CA `CA:TRUE` bekommt, entscheidet die Root-CA. Eine Firmen-PKI kann das ablehnen oder mit `pathlen:0` begrenzen. Für Istio reicht `pathlen:0`.

Was liegt im Secret `cacerts`?

Weil istiod selbst unterschreibt, liegt im Secret `cacerts` mehr als nur ein Zertifikat der externen CA:

Feld	Inhalt	Wozu
`tls.crt`	Zertifikat der Zwischen-CA	wird als Teil der Kette mit ausgeliefert
`tls.key`	**privater Schlüssel** der Zwischen-CA	damit unterschreibt istiod die Workload-Zertifikate
`ca.crt`	Zertifikat der Root-CA	Vertrauensanker, gegen den alle prüfen

Wie wird die Kette geprüft?

Wer ein Workload-Zertifikat prüft, geht die Kette von unten nach oben:

1. Das Workload-Zertifikat ist von der Zwischen-CA unterschrieben (also von istiod).
2. Das Zertifikat der Zwischen-CA ist von der Root-CA unterschrieben.
3. Der Root-CA vertraue ich, also vertraue ich auch dem Workload-Zertifikat.

Es wird dabei nichts vorgetäuscht: Als Aussteller (Issuer) steht im Workload-Zertifikat die Zwischen-CA, nicht die Root-CA. Die Kette siehst du in Schritt 6 mit `istioctl ztunnel-config certificates`.

> ***Merkkasten: Woher kommt der Schlüssel der Zwischen-CA?***
>
> *cert-manager erzeugt ihn im Cluster und legt ihn im Secret `cacerts` ab. Zur externen CA geht nur die Zertifikatsanfrage (CSR) mit dem öffentlichen Schlüssel, zurück kommt das unterschriebene Zertifikat. Der private Schlüssel der Root-CA kommt nie in den Cluster.*

Getestet mit Istio 1.31.4 (Ambient), cert-manager v1.21.3, Kubernetes 1.37.

Voraussetzungen

- Istio Ambient und Bookinfo laufen (Namespace `bookinfo`).
- ArgoCD ist installiert, siehe istio ambient und bookinfo mit argocd ausrollen, Abschnitt 4, Schritt 2.
- Die Manifeste liegen fertig in diesem Repo unter `istio/argocd/externe-ca/`. Du brauchst kein eigenes Git-Repo.

Schritt 1: Ausgangslage ansehen

```
kubectl -n istio-system get secret
```

Es gibt nur `istio-ca-secret`, also die eigene CA von istiod.

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
```

```
subject=O = cluster.local
issuer=O = cluster.local
```

Schritt 2: cert-manager per ArgoCD ausrollen

> *Hast du cert-manager schon per Helm installiert (Übung Let's Encrypt)? Dann **überspringe** diesen Schritt.*

```yaml
## externe-ca/apps/cert-manager-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cert-manager
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-5"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://charts.jetstack.io
    chart: cert-manager
    targetRevision: v1.21.3
    helm:
      releaseName: cert-manager
      values: |
        crds:
          enabled: true
        config:
          apiVersion: controller.config.cert-manager.io/v1alpha1
          kind: ControllerConfiguration
          enableGatewayAPI: true
  destination:
    server: https://kubernetes.default.svc
    namespace: cert-manager
  syncPolicy:
    automated:
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
```

```
REPO=https://raw.githubusercontent.com/jmetzger/training-istio-
kubernetes/main/istio/argocd/externe-ca
kubectl apply -f $REPO/apps/cert-manager-app.yaml
```

Schritt 3: Die CA per ArgoCD ausrollen

Die Application zeigt auf den Ordner `externe-ca/ca/` in diesem Repo. Dort liegen vier Manifeste:

Datei	Was	Wozu
`01-selfsigned-issuer.yaml`	`ClusterIssuer selfsigned`	Hilfs-Issuer, stellt nur die Root-CA aus
`02-root-ca.yaml`	`Certificate root-ca` (Namespace `cert-manager`)	Root-CA, steht für die externe CA
`03-root-ca-issuer.yaml`	`ClusterIssuer root-ca`	unterschreibt mit der Root-CA
`04-istio-ca.yaml`	`Certificate istio-ca` (Namespace `istio-system`)	Zwischen-CA, ergibt das Secret `cacerts`

Das entscheidende Manifest:

```yaml
## externe-ca/ca/04-istio-ca.yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: istio-ca
  namespace: istio-system
spec:
  isCA: true
  commonName: Istio Intermediate CA
  subject:
    organizations:
      - Training
  secretName: cacerts      # diesen Namen erwartet istiod
  duration: 2160h          # 90 Tage
  renewBefore: 360h        # 15 Tage vorher erneuern
  privateKey:
    algorithm: RSA
    size: 4096
  issuerRef:
    kind: ClusterIssuer
    name: root-ca
```

Die Application dazu:

```yaml
## externe-ca/apps/istio-ca-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-ca
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-4"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
```

```
  source:
    repoURL: https://github.com/jmetzger/training-istio-kubernetes.git
    targetRevision: main
    path: istio/argocd/externe-ca/ca
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
    retry:
      limit: 5
      backoff:
        duration: 10s
        factor: 2
        maxDuration: 2m
```

```
kubectl apply -f $REPO/apps/istio-ca-app.yaml
```

> *Mit eigenem Git-Repo: Ordner `externe-ca/` kopieren und in `istio-ca-app.yaml` die `repoURL` und den `path` anpassen.*

Schritt 4: Prüfen, was ArgoCD und cert-manager angelegt haben

```
kubectl -n argocd get applications
```

```
NAME           SYNC STATUS   HEALTH STATUS
cert-manager   Synced        Healthy
istio-ca       Synced        Healthy
```

Das dauert ein bis zwei Minuten. `istio-ca` steht zuerst auf `OutOfSync / Missing`, bis cert-manager läuft.

```
kubectl get clusterissuer
kubectl get certificate -A
```

```
NAMESPACE      NAME       READY   SECRET    AGE
cert-manager   root-ca    True    root-ca   10s
istio-system   istio-ca   True    cacerts   10s
```

```
kubectl -n istio-system describe secret cacerts
```

```
Type:  kubernetes.io/tls

Data
====
ca.crt:   1854 bytes
tls.crt:  1907 bytes
tls.key:  3243 bytes
```

`ca.crt` ist die Root-CA, `tls.crt` und `tls.key` sind die Zwischen-CA. istiod versteht dieses Format direkt.

Darf die Zwischen-CA Zertifikate ausstellen? Das steht in ihrem Zertifikat:

```
kubectl -n istio-system get secret cacerts -o jsonpath='{.data.tls\.crt}' \
  | base64 -d | openssl x509 -noout -text | grep -A1 -E "Basic Constraints|Key Usage"
```

Erwartet: unter `Basic Constraints` steht `CA:TRUE`, unter `Key Usage` steht `Certificate Sign`.

Das Mesh nutzt die neue CA **noch nicht**:

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
```

```
subject=O = cluster.local
issuer=O = cluster.local
```

Schritt 5: istiod auf die neue CA umstellen

istiod entscheidet **beim Start**, welche CA es nimmt. Deshalb neu starten:

```
kubectl -n istio-system rollout restart deployment istiod
kubectl -n istio-system rollout status deployment istiod
```

```
kubectl -n istio-system logs deploy/istiod | grep -E "cacerts|x509 cert -"
```

```
Added cacerts files watcher at ./etc/cacerts
Use root cert from etc/cacerts/ca.crt
x509 cert - Issuer: "CN=Training Root CA,O=Training", Subject: "CN=Istio Intermediate
CA,O=Training", ...
x509 cert - Issuer: "CN=Training Root CA,O=Training", Subject: "CN=Training Root
CA,O=Training", ...
```

Jetzt verteilt istiod die neue Root-CA in alle Namespaces:

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
```

```
subject=O = Training, CN = Training Root CA
issuer=O = Training, CN = Training Root CA
```

Schritt 6: Datenebene neu starten und prüfen

ztunnel, Waypoint und Gateway halten noch Zertifikate der alten CA. Neu starten, damit sie neue holen:

```
kubectl -n istio-system rollout restart daemonset ztunnel
kubectl -n istio-system rollout status daemonset ztunnel
kubectl -n bookinfo rollout restart deployment waypoint bookinfo-gateway-istio
kubectl -n bookinfo rollout status deployment waypoint
```

(Hast du keinen Waypoint oder ein anderes Gateway, passe die Namen an: `kubectl -n bookinfo get deploy`.)

```
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].spec.nodeName}')
ZTUNNEL=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
istioctl ztunnel-config certificates -n istio-system $ZTUNNEL
```

```
CERTIFICATE NAME                                        TYPE           STATUS
VALID CERT   SERIAL NUMBER   NOT AFTER              NOT BEFORE
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Leaf           Available
true         ae97e7...       2026-11-11T20:02:36Z   2026-11-10T20:00:36Z
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Intermediate   Available
true         369dec...       2027-02-08T20:01:20Z   2026-11-10T20:01:20Z
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Intermediate   Available
true         2ae1fe...       2036-11-07T20:01:20Z   2026-11-10T20:01:20Z
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Root           Available
true         2ae1fe...       2036-11-07T20:01:20Z   2026-11-10T20:01:20Z
```

Vorher gab es pro Workload nur `Leaf` und `Root`. Jetzt siehst du die ganze Kette: Zertifikat des Workloads (1 Tag), Zwischen-CA (90 Tage), Root-CA (10 Jahre).

Funktioniert die Anwendung noch?

```
kubectl -n bookinfo exec deploy/productpage-v1 -- \
  python3 -c "import urllib.request as u;
print(u.urlopen('http://reviews:9080/reviews/1').status)"
```

Erwartet: `200`. Im ztunnel-Log steht die Verbindung mit beiden Identitäten (`src.identity`, `dst.identity`):

```
kubectl -n istio-system logs -l app=ztunnel --tail=20 | grep reviews
```

Neuer Cluster: Reihenfolge mit Sync Waves

Rollst du Istio selbst mit ArgoCD aus (siehe ArgoCD-Übung), muss `cacerts` da sein, **bevor** istiod zum ersten Mal startet. Dann entfallen die Neustarts aus Schritt 5 und 6. Die beiden Applications haben dafür schon die passenden Waves:

Wave	Application	Inhalt
-5	`cert-manager`	Helm-Chart cert-manager
-4	`istio-ca`	Issuer, Root-CA, Zwischen-CA (`cacerts`)
-3	`gateway-api-crds`	Gateway API CRDs
-2	`istio-base`	CRDs von Istio
-1	`istiod`	liest `cacerts` beim Start
0 bis 2	`istio-cni`, `ztunnel`, `bookinfo`	wie gehabt

Zwei Dinge sind dabei zu beachten (dieser Abschnitt ist **nicht** auf dem Trainings-Cluster getestet):

- Der Namespace `istio-system` muss schon in Wave -4 existieren, weil das `Certificate` dort liegt. Lege dafür ein Namespace-Manifest mit in den Ordner `ca/` .
- Kopiere beide Dateien aus `externe-ca/apps/` in den Ordner `apps/istio/` deines GitOps-Repos, dann rollt die Root-App sie mit aus.

Gut zu wissen

Thema	Hinweis
Erneuerung der Zwischen-CA	cert-manager erneuert `cacerts` 15 Tage vor Ablauf. istiod beobachtet die Dateien (`Added cacerts files watcher`) und lädt sie nach. Die Root-CA bleibt gleich, also bleibt das Vertrauen erhalten.
Wechsel der Root-CA	Das ist eine geplante Migration. Alte und neue Root müssen eine Zeit lang gemeinsam vertraut werden, sonst bricht mTLS zwischen alten und neuen Zertifikaten.
Umstellung im laufenden Betrieb	Wie in Schritt 5 und 6 wechselt die Root-CA. Plane dafür ein Wartungsfenster ein und starte alles zügig nacheinander neu.
Produktion	Ersetze `01` bis `03` im Ordner `ca/` durch den Issuer eurer PKI (z. B. Vault). `04-istio-ca.yaml` zeigt dann per `issuerRef` darauf.
Schlüssel der Zwischen-CA	Er liegt als Secret in `istio-system`. Wer dort Secrets lesen darf, kann Zertifikate für jeden Workload ausstellen. Zugriff per RBAC eng halten.
Gar kein CA-Schlüssel im Cluster	Mit `istio-csr` unterschreibt cert-manager jede Anfrage der Workloads selbst, istiod hat dann keine CA mehr. Siehe Übung: cert-manager als CA für das Mesh mit istio-csr.

Aufräumen

Zurück zur eigenen CA von istiod:

```
kubectl -n argocd delete application istio-ca
kubectl -n istio-system delete secret cacerts
```

(Das Löschen der Application entfernt Issuer und Certificates. Die Secrets bleiben liegen, deshalb `cacerts` von Hand löschen.)

```
kubectl -n istio-system rollout restart deployment istiod
kubectl -n istio-system rollout status deployment istiod
kubectl -n istio-system rollout restart daemonset ztunnel
kubectl -n istio-system rollout status daemonset ztunnel
kubectl -n bookinfo rollout restart deployment waypoint bookinfo-gateway-istio
```

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
## subject=O = cluster.local
```

cert-manager entfernen (nur, wenn du es in Schritt 2 per ArgoCD ausgerollt hast):

```
kubectl -n argocd delete application cert-manager
kubectl delete namespace cert-manager
kubectl get crd -o name | grep cert-manager.io | xargs kubectl delete
```

Quellen

- Istio, eigene CA-Zertifikate einbinden (`cacerts`): https://istio.io/latest/docs/tasks/security/cert-management/plugin-ca-cert/
- cert-manager, CA-Issuer: https://cert-manager.io/docs/configuration/ca/
- cert-manager, SelfSigned-Issuer (Root-CA erzeugen): https://cert-manager.io/docs/configuration/selfsigned/
- ArgoCD, Sync Waves: https://argo-cd.readthedocs.io/en/stable/user-guide/sync-waves/

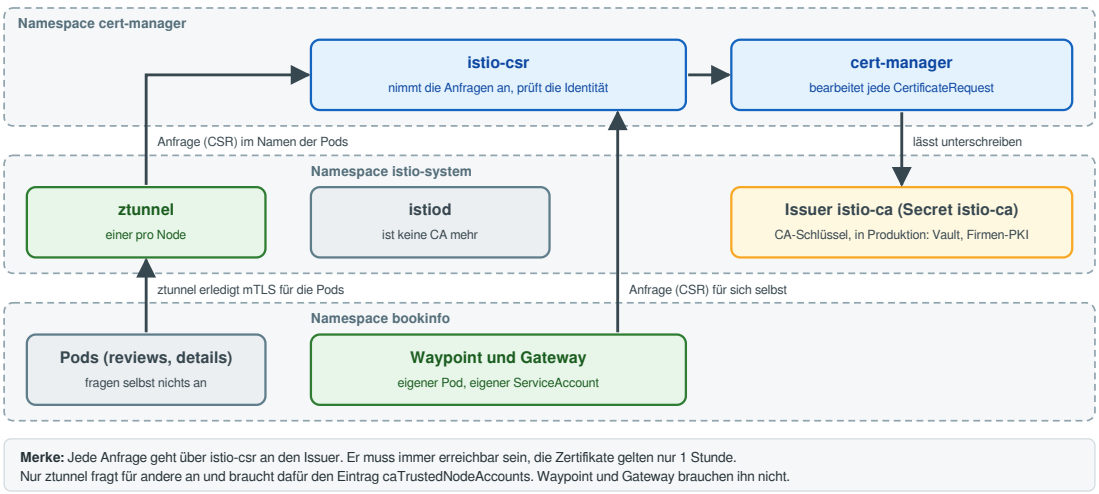
Übung (Ambient): cert-manager als CA für das Mesh mit istio-csr (istiod ist keine CA mehr; von Hand und mit ArgoCD)

Worum geht es?

In der Übung Externe CA für Istio mit ArgoCD bleibt istiod die CA: Es bekommt eine Zwischen-CA (Secret `cacerts`) und unterschreibt damit die Zertifikate der Workloads selbst.

Mit **istio-csr** ist istiod **keine CA mehr**. Jede Zertifikatsanfrage (CSR) geht an istio-csr, cert-manager lässt sie vom Issuer unterschreiben. Im Cluster liegt dann kein CA-Schlüssel mehr bei istiod.

istio-csr im Ambient-Mesh: Wer fragt wen nach einem Zertifikat?



Die Übung zeigt zuerst den Weg von Hand (`helm` , `istioctl`), damit jeder Schritt sichtbar ist. Wie du dasselbe mit ArgoCD ausrollst, steht im Abschnitt Variante: Dasselbe mit ArgoCD.

	`cacerts` (Zwischen-CA)	istio-csr
Wer unterschreibt die Workload-Zertifikate?	istiod	der Issuer von cert-manager
Wie oft wird der Issuer gefragt?	einmal je Erneuerung der Zwischen-CA (z. B. alle 90 Tage)	bei **jedem** Workload-Zertifikat
Gültigkeit der Workload-	24 Stunden	1 Stunde

Zertifikate		
Issuer nicht erreichbar	fällt lange nicht auf	nach spätestens 1 Stunde gibt es keine neuen Zertifikate
Zusätzliche Komponente	keine (cert-manager nur zum Anlegen des Secrets)	cert-manager **und** istio-csr

> ***Merkkasten: Was ist istio-csr?***
>
> *Ein eigenes Helm-Chart des cert-manager-Projekts (`cert-manager-istio-csr`). Es gehört weder zu Istio noch zum Chart von cert-manager und wird nur für diese Variante installiert.*

> ***Merkkasten: Let's Encrypt geht hier nicht***
>
> *Die Zertifikate der Workloads tragen eine SPIFFE-ID (`spiffe://cluster.local/ns/bookinfo/sa/...`) statt eines öffentlichen DNS-Namens. Solche Zertifikate stellt Let's Encrypt (ACME) nicht aus. Du brauchst einen Issuer vom Typ `ca` (wie hier), Vault oder eine Firmen-PKI.*

Getestet mit Istio 1.31.4 (Ambient), cert-manager v1.21.3, istio-csr v0.19.1, Kubernetes 1.37.

ztunnel und Waypoint: Wer fragt für wen?

Stell dir das Zertifikat als Ausweis vor und die CA als Ausweisbehörde.

Wer	Beantragt den Ausweis für	Braucht eine Sondererlaubnis?
Waypoint, Gateway	sich selbst (eigener ServiceAccount)	nein
ztunnel	die Pods auf seinem Node (z. B. `reviews`), je ServiceAccount einmal	**ja**

ztunnel verlangt einen Ausweis auf einen fremden Namen. Das darf nicht jeder, sonst könnte sich jeder Pod als `reviews` ausgeben. Die CA braucht deshalb eine Liste, wer für andere anfragen darf. Bei istio-csr ist das die Einstellung `app.server.caTrustedNodeAccounts=istio-system/ztunnel` (bei istiod heißt sie `CA_TRUSTED_NODE_ACCOUNTS`).

Beide haben auch je eine eigene CA-Adresse: ztunnel die Variable `CA_ADDRESS` , Waypoint und Gateway `CA_ADDR` .

Voraussetzungen

- Istio Ambient und Bookinfo laufen (Namespace `bookinfo` , mit Waypoint und Gateway).
- `helm` ist auf dem Client installiert.
- cert-manager läuft (`kubectl -n cert-manager get pods`). Wenn nicht: Schritt 2 der Übung Bookinfo mit HTTPS und Let's Encrypt ausführen.

> ***Achtung: Umstellung im laufenden Mesh***
>
> *cert-manager nennt den Einbau in ein laufendes Istio **nicht unterstützt**. Die Root-CA wechselt, deshalb kann mTLS während der Umstellung kurz brechen. Im Trainings-Cluster hat es funktioniert, in Produktion gehört istio-csr vor die Installation von Istio.*

Schritt 1: Ausgangslage ansehen

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
```

```
subject=O = cluster.local
issuer=O = cluster.local
```

Wo holen ztunnel und Waypoint ihre Zertifikate?

```
kubectl -n istio-system get ds ztunnel \
  -o jsonpath='{range .spec.template.spec.containers[0].env[*]}{.name}={.value}{"\n"}
{end}' | grep CA_ADDRESS
kubectl -n bookinfo get deploy waypoint \
  -o jsonpath='{range .spec.template.spec.containers[0].env[*]}{.name}={.value}{"\n"}
{end}' | grep CA_ADDR
```

```
CA_ADDRESS=istiod.istio-system.svc:15012
CA_ADDR=istiod.istio-system.svc:15012
```

Schritt 2: Issuer und Root-CA anlegen

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned
  namespace: istio-system
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: istio-ca
  namespace: istio-system
spec:
  isCA: true
  commonName: Training Root CA
  subject:
    organizations:
      - Training
  secretName: istio-ca
  duration: 87600h        # 10 Jahre
  privateKey:
    algorithm: ECDSA
    size: 256
  issuerRef:
    kind: Issuer
    name: selfsigned
---
```

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: istio-ca          # diesen Namen erwartet istio-csr
  namespace: istio-system
spec:
  ca:
    secretName: istio-ca
EOF
```

```
kubectl -n istio-system get issuer,certificate
```

```
NAME                                  READY
issuer.cert-manager.io/istio-ca       True
issuer.cert-manager.io/selfsigned     True

NAME                                      READY   SECRET
certificate.cert-manager.io/istio-ca      True    istio-ca
```

Der Issuer `istio-ca` ist der Platzhalter für die externe CA. In Produktion zeigt er auf Vault oder die Firmen-PKI.

Schritt 3: Zertifikat der Root-CA für istio-csr bereitstellen

istio-csr soll die Root-CA fest vorgegeben bekommen. Sonst übernimmt es, was der Issuer als Root-CA meldet.

```
kubectl -n istio-system get certificaterequest istio-ca-1 -o
jsonpath='{.status.certificate}' \
  | base64 -d > ca.pem
openssl x509 -in ca.pem -noout -subject -issuer
```

```
subject=O = Training, CN = Training Root CA
issuer=O = Training, CN = Training Root CA
```

```
kubectl -n cert-manager create secret generic istio-root-ca --from-file=ca.pem=ca.pem
```

(In der Datei steht nur das öffentliche Zertifikat, kein Schlüssel.)

Schritt 4: istio-csr installieren

```
helm upgrade --install cert-manager-istio-csr oci://quay.io/jetstack/charts/cert-
manager-istio-csr \
  --version v0.19.1 --namespace cert-manager --wait \
  --set "app.tls.rootCAFile=/var/run/secrets/istio-csr/ca.pem" \
  --set "app.server.caTrustedNodeAccounts=istio-system/ztunnel" \
  --set "volumeMounts[0].name=root-ca" \
  --set "volumeMounts[0].mountPath=/var/run/secrets/istio-csr" \
  --set "volumes[0].name=root-ca" \
  --set "volumes[0].secret.secretName=istio-root-ca"
```

```
kubectl -n cert-manager get pods
kubectl -n istio-system get certificate
```

```
NAME                                     READY   SECRET
certificate.cert-manager.io/istio-ca     True    istio-ca
certificate.cert-manager.io/istiod       True    istiod-tls
```

istio-csr hat gleich ein Zertifikat für istiod angelegt (`istiod-tls`). istiod braucht es, weil es sich selbst keins mehr ausstellt.

> *Mach jetzt zügig mit Schritt 5 weiter. Bis dahin überschreiben sich istiod und istio-csr gegenseitig die ConfigMap `istio-ca-root-cert` in allen Namespaces.*

Schritt 5: Istio auf istio-csr umstellen

```
cat > istio-csr-iop.yaml <<'EOF'
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: ambient
  values:
    global:
      # hier holen ztunnel, Waypoint und Gateway ihre Zertifikate
      caAddress: cert-manager-istio-csr.cert-manager.svc:443
  components:
    pilot:
      k8s:
        env:
          # istiod ist keine CA mehr
          - name: ENABLE_CA_SERVER
            value: "false"
EOF
istioctl install -f istio-csr-iop.yaml -y
```

```
kubectl -n istio-system rollout status deployment istiod
kubectl -n istio-system rollout status daemonset ztunnel
```

Du musst nichts von Hand neu starten: istiod und ztunnel werden durch die Änderung neu ausgerollt, Waypoint und Gateway ebenfalls, weil sich ihre CA-Adresse ändert.

Schritt 6: Prüfen

```
kubectl -n istio-system logs deploy/istiod | grep "x509 cert -" | head -n 1
```

```
x509 cert - Issuer: "CN=Training Root CA,O=Training", Subject: "CN=istiod.istio-
system.svc", ...
```

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
```

```
subject=O = Training, CN = Training Root CA
issuer=O = Training, CN = Training Root CA
```

```
kubectl -n istio-system get ds ztunnel \
  -o jsonpath='{range .spec.template.spec.containers[0].env[*]}{.name}={.value}{"\n"}
{end}' | grep CA_ADDRESS
kubectl -n bookinfo get deploy waypoint \
  -o jsonpath='{range .spec.template.spec.containers[0].env[*]}{.name}={.value}{"\n"}
{end}' | grep CA_ADDR
```

```
CA_ADDRESS=cert-manager-istio-csr.cert-manager.svc:443
CA_ADDR=cert-manager-istio-csr.cert-manager.svc:443
```

Zertifikate, die ztunnel für die Pods hält (ein ztunnel hält nur die Zertifikate der Pods auf seinem Node, deshalb den ztunnel auf dem Node von `productpage` nehmen):

```
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].spec.nodeName}')
ZTUNNEL=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
istioctl ztunnel-config certificates -n istio-system $ZTUNNEL
```

```
CERTIFICATE NAME                                          TYPE    STATUS      VALID CERT
SERIAL NUMBER    NOT AFTER              NOT BEFORE
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Leaf    Available   true
d28ec5...        2026-11-11T05:45:22Z   2026-11-11T04:45:22Z
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details  Root    Available   true
581bc3...        2036-11-08T04:43:56Z   2026-11-11T04:43:56Z
```

Die Zertifikate gelten jetzt nur **1 Stunde**. Das Zertifikat, das der Waypoint für sich selbst hält:

```
istioctl proxy-config secret deploy/waypoint -n bookinfo
```

Funktioniert die Anwendung noch?

```
kubectl -n bookinfo exec deploy/productpage-v1 -- \
  python3 -c "import urllib.request as u;
print(u.urlopen('http://reviews:9080/reviews/1').status)"
```

Erwartet: `200` .

> *In der ersten Minute nach der Umstellung kann ztunnel im Log `invalid peer certificate: UnknownIssuer` gegenüber istiod melden. Das hört von selbst auf (`Stream established`).*

Schritt 7: Die Anfragen sichtbar machen

Normalerweise löscht istio-csr jede `CertificateRequest` sofort wieder. Zum Ansehen lassen wir sie stehen:

```
helm upgrade cert-manager-istio-csr oci://quay.io/jetstack/charts/cert-manager-istio-csr \
  --version v0.19.1 --namespace cert-manager --reuse-values --wait \
  --set "app.certmanager.preserveCertificateRequests=true"
```

Neue Anfragen auslösen:

```
kubectl -n istio-system rollout restart daemonset ztunnel
kubectl -n istio-system rollout status daemonset ztunnel
kubectl -n bookinfo rollout restart deployment waypoint
kubectl -n bookinfo rollout status deployment waypoint
```

```
kubectl -n istio-system get certificaterequest -o custom-columns='NAME:.metadata.name,ISSUER:.spec.issuerRef.name,READY:.status.conditions[?(@.type=="Ready")].status,IDENTITAET:.metadata.annotations.istio\.cert-manager\.io/identities'
```

```
NAME              ISSUER       READY   IDENTITAET
istio-ca-1        selfsigned   True    <none>
istio-csr-6gb6q   istio-ca     True    spiffe://cluster.local/ns/bookinfo/sa/bookinfo-ratings
istio-csr-d6gnv   istio-ca     True    spiffe://cluster.local/ns/bookinfo/sa/waypoint
istio-csr-kqdhb   istio-ca     True    spiffe://cluster.local/ns/bookinfo/sa/bookinfo-productpage
istio-csr-pn2q8   istio-ca     True    spiffe://cluster.local/ns/bookinfo/sa/bookinfo-reviews
istio-csr-wf7nm   istio-ca     True    istio-csr-serving
istio-csr-wnz6f   istio-ca     True    spiffe://cluster.local/ns/bookinfo/sa/bookinfo-details
istiod-1          istio-ca     True    <none>
```

- `bookinfo-ratings`, `bookinfo-reviews` usw.: von **ztunnel** im Namen der Pods angefragt, je ServiceAccount eine Anfrage.
- `waypoint`: vom **Waypoint** für sich selbst angefragt.
- `istio-csr-serving` und `istiod-1`: die eigenen Zertifikate von istio-csr und istiod.

> *`preserveCertificateRequests` ist nur zum Ansehen gedacht. Auf Dauer füllt es etcd mit Anfragen.*

Schritt 8 (optional): Gegenprobe ohne Erlaubnis für ztunnel

Was passiert, wenn ztunnel nicht für andere anfragen darf?

```
helm upgrade cert-manager-istio-csr oci://quay.io/jetstack/charts/cert-manager-istio-csr \
  --version v0.19.1 --namespace cert-manager --reuse-values --wait \
  --set "app.server.caTrustedNodeAccounts="
kubectl -n istio-system rollout restart daemonset ztunnel
kubectl -n istio-system rollout status daemonset ztunnel
```

```
kubectl -n cert-manager logs deploy/cert-manager-istio-csr --since=2m | grep
impersonation | tail -n 2
```

```
warn    impersonation not allowed, as node authorizer (CA_TRUSTED_NODE_ACCOUNTS) is
not configured
```

```
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o
jsonpath='{.items[0].spec.nodeName}')
ZTUNNEL=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
istioctl ztunnel-config certificates -n istio-system $ZTUNNEL
```

Die Liste ist leer, ztunnel hat keine Zertifikate. Der Aufruf von `reviews` aus Schritt 6 schlägt fehl, das Gateway antwortet mit `503`.

Wieder erlauben:

```
helm upgrade cert-manager-istio-csr oci://quay.io/jetstack/charts/cert-manager-istio-
csr \
  --version v0.19.1 --namespace cert-manager --reuse-values --wait \
  --set "app.server.caTrustedNodeAccounts=istio-system/ztunnel"
kubectl -n istio-system rollout restart daemonset ztunnel
kubectl -n istio-system rollout status daemonset ztunnel
```

Danach liefert der Aufruf aus Schritt 6 wieder `200`.

Variante: Dasselbe mit ArgoCD

Bisher hast du Issuer, Root-CA und istio-csr von Hand angelegt (Schritt 2 bis 4). Mit ArgoCD liegt alles im Git, auch die beiden Applications selbst. Von Hand legst du nur noch **eine** Wurzel-Application an (App of Apps), die auf den Ordner `apps/` zeigt. Die Manifeste liegen fertig in diesem Repo unter `istio/argocd/istio-csr/`. Du brauchst kein eigenes Git-Repo.

```
Wurzel-Application istio-csr-root   (einziger Handgriff am Cluster)
  └── Ordner apps/ im Git
        ├── Application istio-csr-ca  →  Ordner ca/ im Git (Issuer, Root-CA)
        └── Application istio-csr     →  Helm-Chart cert-manager-istio-csr
```

Datei	Was	Ersetzt
`ca/01-selfsigned-issuer.yaml`, `ca/02-root-ca.yaml`, `ca/03-istio-ca-issuer.yaml`	Issuer und Root-CA	Schritt 2
`apps/istio-csr-ca-app.yaml`	Application `istio-csr-ca`, zeigt auf den Ordner `ca/` (Wave -4)	Schritt 2
`apps/istio-csr-app.yaml`	Application `istio-csr`, Helm-Chart `cert-manager-istio-csr` (Wave -3)	Schritt 4

Voraussetzungen: ArgoCD ist installiert (siehe ArgoCD-Übung, Abschnitt 4, Schritt 2), cert-manager läuft, und der Stand von Hand ist aufgeräumt (Abschnitt "Aufräumen").

ArgoCD Schritt 1: Die Wurzel-Application anlegen

Die beiden Applications liegen im Git. So sieht die für istio-csr aus (nur zum Ansehen, nicht anwenden):

```
## istio-csr/apps/istio-csr-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-csr
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-3"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://charts.jetstack.io
    chart: cert-manager-istio-csr
    targetRevision: v0.19.1
    helm:
      releaseName: cert-manager-istio-csr
      values: |
        app:
          server:
            # ztunnel darf Zertifikate im Namen der Pods anfragen
            caTrustedNodeAccounts: istio-system/ztunnel
  destination:
    server: https://kubernetes.default.svc
    namespace: cert-manager
  syncPolicy:
    automated:
      selfHeal: true
    retry:
      limit: 5
      backoff:
        duration: 10s
        factor: 2
        maxDuration: 2m
```

Die Wurzel-Application liegt nicht im Git, sie ist der einzige Handgriff am Cluster. Sie zeigt auf den Ordner `apps/`, ArgoCD legt daraus die beiden Applications an:

```
cat <<'EOF' | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: istio-csr-root
  namespace: argocd
```

```
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/jmetzger/training-istio-kubernetes.git
    targetRevision: main
    path: istio/argocd/istio-csr/apps
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
EOF
```

> *Die Warnung `prefer a domain-qualified finalizer name` kannst du ignorieren.*

```
kubectl -n argocd get applications
```

```
NAME             SYNC STATUS   HEALTH STATUS
istio-csr        Synced        Healthy
istio-csr-ca     Synced        Healthy
istio-csr-root   Synced        Healthy
```

> *Mit eigenem Git-Repo: Ordner `istio-csr/` kopieren, in der Wurzel-Application und in `istio-csr-ca-app.yaml` die `repoURL` und den `path` anpassen. Änderungen an den Applications machst du dann per Commit, nicht per `kubectl`.*

Das dauert etwa eine halbe Minute.

```
kubectl -n istio-system get issuer,certificate
kubectl -n cert-manager get pods
```

> ***Unterschied zum Weg von Hand: keine fest vorgegebene Root-CA***
>
> *Schritt 3 (Secret `istio-root-ca`) ist ein Handgriff am Cluster und passt nicht zu GitOps. Die Application lässt ihn weg, istio-csr übernimmt die Root-CA dann vom Issuer. In Produktion ist das Zertifikat der Root-CA bekannt und öffentlich: Lege es als ConfigMap ins Git und binde es über `volumes`, `volumeMounts` und `app.tls.rootCAFile` ein (in dieser Übung nicht getestet).*

ArgoCD Schritt 2: Istio umstellen

Istio ist mit `istioctl` installiert (Normalfall im Training): Führe Schritt 5 von oben aus, danach Schritt 6 zum Prüfen.

> *Schritt 7 und 8 ändern istio-csr mit `helm upgrade`. Unter ArgoCD geht das nicht, ArgoCD stellt den Stand aus dem Git wieder her (`selfHeal`). Solche Änderungen gehören dann in die `values` der Application.*

Istio wird selbst von ArgoCD ausgerollt (wie in der ArgoCD-Übung): Dann gehören die beiden Einstellungen in die Werte der Helm-Charts im Git-Repo.

```
## apps/istio/values/istiod-values.yaml
profile: ambient
global:
  # hier holen Waypoint und Gateway ihre Zertifikate
  caAddress: cert-manager-istio-csr.cert-manager.svc:443
pilot:
  env:
    # istiod ist keine CA mehr
    ENABLE_CA_SERVER: "false"
```

```
## apps/istio/ztunnel-app.yaml (Ausschnitt)
  source:
    chart: ztunnel
    helm:
      values: |
        # hier holt ztunnel die Zertifikate für die Pods
        caAddress: cert-manager-istio-csr.cert-manager.svc:443
```

Die Reihenfolge regeln die Sync Waves:

Wave	Application	Inhalt
-5	`cert-manager`	Helm-Chart cert-manager (`externe-ca/apps/cert-manager-app.yaml`)
-4	`istio-csr-ca`	Issuer und Root-CA
-3	`istio-csr`	Helm-Chart istio-csr, legt auch `istiod-tls` an
-2	`istio-base`	CRDs von Istio
-1	`istiod`	startet gleich ohne eigene CA
0 bis 2	`istio-cni`, `ztunnel`, `bookinfo`	wie gehabt

Das ist der Weg, den cert-manager vorsieht: istio-csr ist da, bevor Istio zum ersten Mal startet.

> *Dieser zweite Fall ist **nicht** auf dem Trainings-Cluster ausgerollt. Geprüft ist nur, dass die Charts von Istio 1.31.4 mit diesen Werten `ENABLE_CA_SERVER=false` und die neue CA-Adresse erzeugen (`helm template`). Der Namespace `istio-system` muss in Wave -4 schon existieren, weil Issuer und Root-CA dort liegen.*

ArgoCD: Aufräumen

```
kubectl -n argocd delete application istio-csr-root
```

Das Löschen der Wurzel entfernt die beiden Applications und damit istio-csr, Issuer und Certificates. Die Secrets und das Lease bleiben liegen, deshalb von Hand:

```
kubectl -n istio-system delete secret istio-ca istiod-tls
kubectl -n istio-system delete lease istio-csr
rm -f istio-csr-iop.yaml
```

Danach Istio zurückstellen wie im Abschnitt "Aufräumen" (`istioctl install --set profile=ambient -y` und die Prüfungen).

Gut zu wissen

Thema	Hinweis
Verfügbarkeit	Ist istio-csr, cert-manager oder der Issuer länger als die Restlaufzeit der Zertifikate weg, bekommt kein Workload ein neues Zertifikat. In Produktion istio-csr mit mehreren Replicas betreiben.
Last	Jeder ServiceAccount je Node, jeder Waypoint und jedes Gateway fragt mindestens einmal pro Stunde an. Der Issuer muss das aushalten.
Reihenfolge	Vorgesehen ist: cert-manager, Issuer, istio-csr, **dann** Istio. Der nachträgliche Einbau wie hier ist laut cert-manager nicht unterstützt.
Root-CA	Sie wechselt bei der Umstellung. Das ist eine geplante Migration, kein Schalter für den laufenden Betrieb.
CA-Schlüssel	In der Übung liegt er als Secret `istio-ca` in `istio-system`. Mit Vault oder Firmen-PKI als Issuer liegt gar kein CA-Schlüssel mehr im Cluster.
Einfacher	Reicht es, dass das Mesh an der Firmen-PKI hängt, ist die Variante mit `cacerts` einfacher und hängt im Betrieb nicht am Issuer.

Aufräumen

(Hast du die ArgoCD-Variante benutzt, beginne mit "ArgoCD: Aufräumen" weiter oben.)

Zurück zur eigenen CA von istiod. Die Reihenfolge ist wichtig: `istiod-tls` muss weg sein, **bevor** istiod neu startet.

```
helm -n cert-manager uninstall cert-manager-istio-csr
kubectl -n istio-system delete certificate istio-ca
kubectl -n istio-system delete issuer istio-ca selfsigned
kubectl -n istio-system delete certificaterequest --all
kubectl -n istio-system delete secret istio-ca istiod-tls
kubectl -n istio-system delete lease istio-csr
kubectl -n cert-manager delete secret istio-root-ca
rm -f ca.pem istio-csr-iop.yaml
```

```
istioctl install --set profile=ambient -y
kubectl -n istio-system rollout status deployment istiod
kubectl -n istio-system rollout status daemonset ztunnel
```

Auch hier werden Waypoint und Gateway von selbst neu ausgerollt.

```
kubectl -n bookinfo get cm istio-ca-root-cert -o jsonpath='{.data.root-cert\.pem}' \
  | openssl x509 -noout -subject -issuer
## subject=O = cluster.local
```

```
kubectl -n bookinfo exec deploy/productpage-v1 -- \
  python3 -c "import urllib.request as u;
print(u.urlopen('http://reviews:9080/reviews/1').status)"
```

Erwartet: `200`. cert-manager bleibt installiert.

Quellen

- cert-manager, istio-csr: https://cert-manager.io/docs/usage/istio-csr/
- cert-manager, istio-csr installieren (mit Abschnitt zu Ambient): https://cert-manager.io/docs/usage/istio-csr/installation/
- Helm-Chart: https://github.com/cert-manager/istio-csr

Installation von istio (ambient mode) mit helm

istio-installation with helm - ambient

Voraussetzung

- Helm-Charts `base`, `cni` und `istiod` bereits installiert (Version 1.29.1)
- Helm-Profile ambient: https://github.com/istio/istio/blob/master/manifests/helm-profiles/ambient.yaml

Step 1: Gateway API CRDs (v1.4.0 - Standard)

```
kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml
```

Step 2: istiod und cni auf Ambient-Profil upgraden

```
## Zur Sicherheit, manchmal gibt es Probleme mit dem Mutating Webhook
kubectl delete validatingwebhookconfigurations istio-validator-istio-system

helm upgrade istiod istio/istiod -n istio-system --set profile=ambient --version 1.29.1
helm upgrade istio-cni istio/cni -n istio-system --set profile=ambient --version 1.29.1
```

Step 3: ztunnel installieren

```
helm install ztunnel istio/ztunnel \
  --namespace istio-system \
  --version 1.29.1 \
  --wait
```

Setup Cluster (Self-Service)

Self-Service Cluster ausrollen

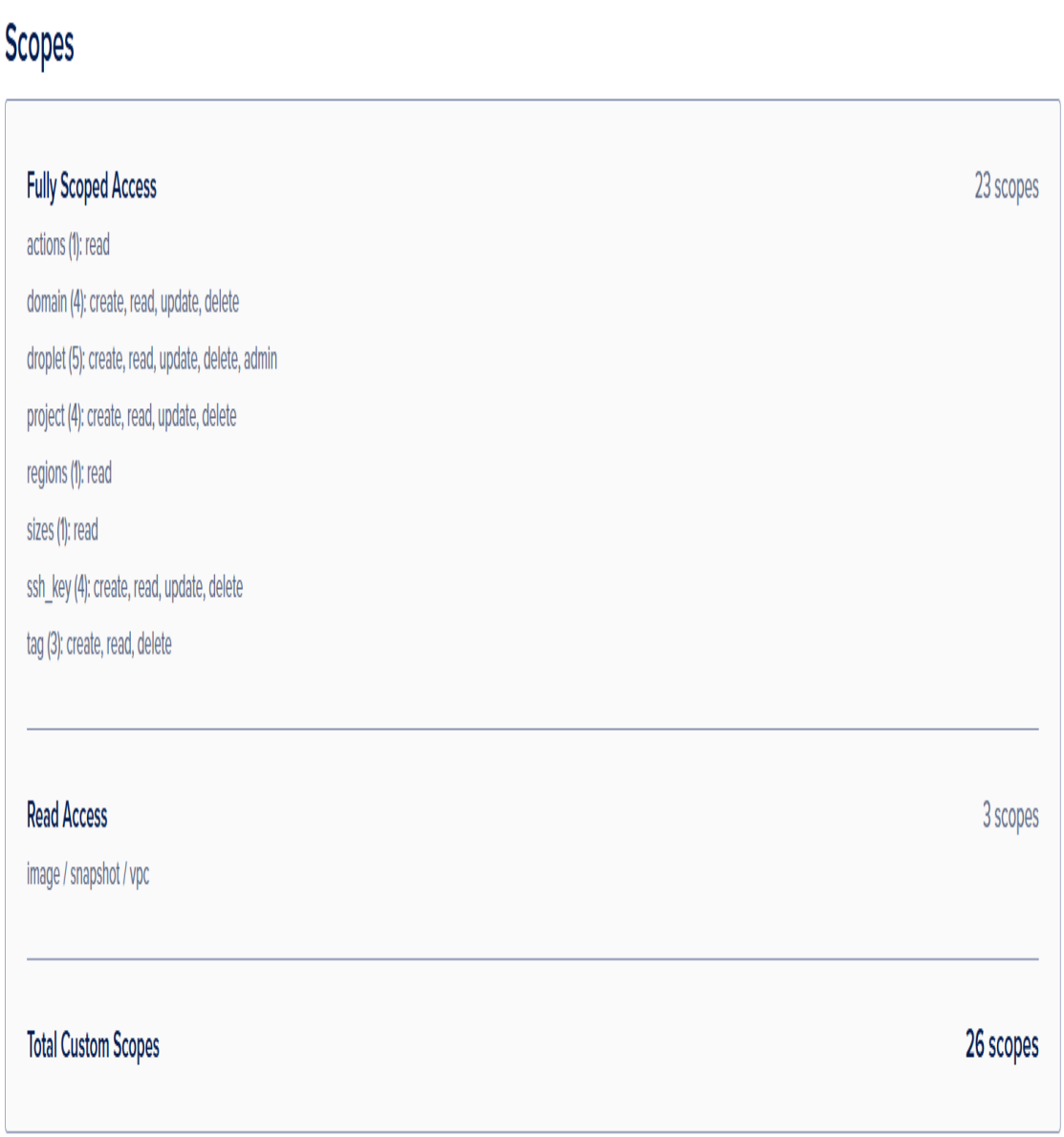
- ausgerollt mit terraform (binary ist installiert) - snap install --classic terraform
- beinhaltet

1. 1 controlplane
2. 3 worker nodes
3. metallb mit ip's (IP-Adressen) der Nodes (hacky but works)
4. ingress mit wildcard-domain: *.tlnx.do.t3isp.de

Vorbereitung seitens des Trainers

```
## /tmp/.env - Datei wurde vom Trainer vorbereitet
## Inhalt / export -> damit Umgebungsvariable
export TF_VAR_do_token="DAS_TOKEN_FUER_DIGITALOCEAN"
```

```
Folgende Berechtigungen wurden für das Token gesetzt
```



Walktrough

- Setup takes about 6-7 minutes
- Hinweis: /tmp/.env beinhaltet Digitalocean Access Token der für das einrichten benötigt wird.

```
cd
git clone https://github.com/jmetzger/training-istio-kubernetes-stack-do-terraform.git
install
cd install
cat /tmp/.env
source /tmp/.env
tofu init
tofu apply -auto-approve
```

Hinweis

```
## Sollte es nicht sauber durchlaufen
## Cluster nochmal löschen
./scripts/safe-destroy.sh
tofu apply -auto-approve
```

Self-Service Cluster destroy

```
cd
cd install
source /tmp/.env
terraform destroy -auto-approve
```

istio sidecar (Grundlagen)

Systemanforderungen & Kubernetes-Cluster-Vorbereitung

Hardware-Anforderungen

Master Node (Control Plane)

- **Minimum:** 2 CPU, 2 GB RAM, 20 GB Disk
- **Empfohlen:** 4 CPU, 8 GB RAM, 50 GB Disk
- **Schulung:** 2 CPU, 4 GB RAM, 30 GB Disk

Worker Node

- **Minimum:** 2 CPU, 2 GB RAM, 20 GB Disk
- **Empfohlen:** 4 CPU, 16 GB RAM, 100 GB Disk
- **Schulung:** 2 CPU, 4 GB RAM, 30 GB Disk

Software-Voraussetzungen

Betriebssystem

- Ubuntu 20.04/22.04 LTS
- Debian 11/12
- RHEL/Rocky/Alma Linux 8/9
- 64-bit Architektur erforderlich

Container Runtime

- containerd (empfohlen)
- CRI-O
- Docker Engine (über cri-dockerd)

System-Tools

```
curl, wget, apt-transport-https, ca-certificates
```

Netzwerk-Anforderungen

Ports Control Plane

- 6443: Kubernetes API Server
- 2379-2380: etcd
- 10250: Kubelet API
- 10259: kube-scheduler
- 10257: kube-controller-manager

Ports Worker Nodes

- 10250: Kubelet API
- 30000-32767: NodePort Services

Cluster-Vorbereitung

System-Updates

```
apt update && apt upgrade -y
## oder
dnf update -y
```

Swap deaktivieren

```
swapoff -a
sed -i '/ swap / s/^/#/' /etc/fstab
```

Kernel-Module laden

```
cat <<EOL | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOL

modprobe overlay
modprobe br_netfilter
```

Sysctl-Parameter

```
cat <<EOL | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.ipv4.ip_forward                 = 1
EOL

sysctl --system
```

Firewall

- UFW/firewalld deaktivieren (Schulung)
- Oder: Benötigte Ports freischalten

lsmod | grep br_netfilter

```
## Installation/Konfiguration/Deinstallation von istio (sidecar - Installation mit
istioctl)

### Istio-Installation mit istioctl und der IstioOperator - Resource


  * Most simplistic way
  * Doing the right setup is done with profiles
  * Interestingly it uses an compile-in helm chart  (see also: Show what a profile
does)

### Hint for production

  * Best option (in most cases) is default

### in our case: Including demo (tracing is activated)

  * Not suitable for production !!

### Show what a profile does
```

istioctl manifest generate > istio-manifest.yaml

If not profile is mentioned, it uses the default profile

it does not use an operator

cat istio-manifest.yaml | grep -i -A20 "^Kind" | less

If you want you can apply it like so:

kubectl apply -f istio-manifest.yaml

```
### Installation including Demo

> [!CAUTION]
> This profile (demo) enables high levels of tracing and access logging so it is not
suitable for performance tests.
```

```
#### Schritt 1: istio runterladen und installieren
```

cd

current version of istio is 1.28.0

curl -L https://istio.io/downloadIstio | sh - ln -s ~~/istio-1.28.0 ~/istio echo "export PATH=~~/istio-1.28.0/bin:$PATH" >> ~/.bashrc source ~/.bashrc

```
#### Schritt 2: bash completion integrieren
```

cp ~/istio/tools/istioctl.bash ~/istioctl.bash echo "source ~/istioctl.bash" >> ~/.bashrc source ~/istioctl.bash

```
#### Schritt 2.5. See what it would install
```

dry-run

istioctl install -f ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml -y --dry-run

```
#### Schritt 3: Installation with demo (by using operator)
```

cat ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml

Wird vom ControlPlane ausgewertet

Hier wird das ingressgateway abgeschaltet,

Weil wir das nicht benötigen, wenn wir

die Kubernetes Gateway API verwenden

apiVersion: install.istio.io/v1alpha1 kind: IstioOperator spec: profile: demo components: ingressGateways: - name: istio-ingressgateway enabled: false egressGateways: - name: istio-egressgateway enabled: false

```

```

Der Trend geht Richtung Kubernetees Gateway API

istioctl install -f ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml -y

```
#### Schritt 4: Gateway API's CRD's installieren
```

kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null ||
{ kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?ref=v1.4.0" | kubectl apply -f -; }

Reference: Get started

  * https://istio.io/latest/docs/setup/getting-started/

Installations-Config-Profile

  * istio verwendet verschiedene vorgefertigte Profile, die das Ausrollen (installieren) erleichtern
  * Diese können in istioctl verwendet werden, aber auch mit dem helm-chart

Welcher Profile gibt es ?

  * Es gibt deployment - profile und platform profile

Übersicht der Deployment - Profile

Core components	default	demo	minimal	remote	empty	preview	ambient
istio-egressgateway		✓					
istio-ingressgateway	✓	✓				✓	
istiod	✓	✓	✓			✓	✓
CNI							✓
Ztunnel							✓

Welches Deployment - Profile nehme ich am besten

  * Für Productiion am besten Default (mit Sidecar) oder Ambient (für den Ambient - Modus)
  * Zum Testen / Üben demo (mit Sidecar), aber nicht ! für die Production. Schlecht für Performance (Hier ist ganz viel Debuggen und Tracing aktiviert)

Übersicht Platform - Profile

Plattform	Beschreibung
gke	Sets chart options required or recommended for installing Istio in Google Kubernetes Engine (GKE) environments.
eks	Sets chart options required or recommended for installing Istio in Amazon's Elastic Kubernetes Service (EKS) environments.
openshift	Sets chart options required or recommended for installing Istio in OpenShift environments.
k3d	Sets chart options required or recommended for installing Istio in k3d environments.

```
| k3s | Sets chart options required or recommended for installing Istio in K3s
environments. |
| microk8s | Sets chart options required or recommended for installing Istio in
MicroK8s environments. |
| minikube | Sets chart options required or recommended for installing Istio in
minikube environments. |

### Reference:

  * https://istio.io/latest/docs/setup/additional-setup/config-profiles/

### Wie ändere ich die Config/Installation von istio - Beispiel egressGateway


  * Always !! mention the profile
  * Overlays work for: Properties and Scalars, but not for lists
  * To avoid problems always ! list the complete config

### Exercise: Enable egress gateway

#### Step 1: Prepare istioOperator - config
```

cd mkdir -p manifests/istio cd manfifests/istio nano istioOperator.yaml

```

```

apiVersion: install.istio.io/v1alpha1 kind: IstioOperator spec: profile: demo components: ingressGateways: - name: istio-ingressgateway enabled: false # keep this, so it does NOT come back egressGateways: - name: istio-egressgateway enabled: true # turn this on

```
### Step 2: Dry-run first
```

See what is done

istiocl install -f istioOperator.yaml --dry-run

```
### Step 3: Install (Change now)
```

istioclt install -f istioOperator.yaml

```
### Deinstallation von Istio mit istioctl
```

istioctl uninstall -y --purge

```
## Installation von istio (sidecar) mit helm-charts
```

```
### Systemanforderungen & Kubernetes-Cluster-Vorbereitung


### Hardware-Anforderungen

#### Master Node (Control Plane)
- **Minimum:** 2 CPU, 2 GB RAM, 20 GB Disk
- **Empfohlen:** 4 CPU, 8 GB RAM, 50 GB Disk
- **Schulung:** 2 CPU, 4 GB RAM, 30 GB Disk

#### Worker Node
- **Minimum:** 2 CPU, 2 GB RAM, 20 GB Disk
- **Empfohlen:** 4 CPU, 16 GB RAM, 100 GB Disk
- **Schulung:** 2 CPU, 4 GB RAM, 30 GB Disk

### Software-Voraussetzungen

#### Betriebssystem
- Ubuntu 20.04/22.04 LTS
- Debian 11/12
- RHEL/Rocky/Alma Linux 8/9
- 64-bit Architektur erforderlich

#### Container Runtime
- containerd (empfohlen)
- CRI-O
- Docker Engine (über cri-dockerd)

#### System-Tools
```bash
curl, wget, apt-transport-https, ca-certificates
```

Netzwerk-Anforderungen

Ports Control Plane

- 6443: Kubernetes API Server
- 2379-2380: etcd
- 10250: Kubelet API
- 10259: kube-scheduler
- 10257: kube-controller-manager

Ports Worker Nodes

- 10250: Kubelet API
- 30000-32767: NodePort Services

Cluster-Vorbereitung

System-Updates

```
apt update && apt upgrade -y
## oder
dnf update -y
```
```

Swap deaktivieren

```
swapoff -a
sed -i '/ swap / s/^/#/' /etc/fstab
```

Kernel-Module laden

```
cat <<EOL | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOL

modprobe overlay
modprobe br_netfilter
```

Sysctl-Parameter

```
cat <<EOL | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOL

sysctl --system
```

Firewall

- UFW/firewalld deaktivieren (Schulung)
- Oder: Benötigte Ports freischalten

lsmod | grep br_netfilter

```
### Istio-Installation mit helm charts


### Voraussetzungen

- Kubernetes-Cluster läuft und `kubectl` ist konfiguriert
- Helm ist installiert (`helm version`)

### Profile

  * Die Charts von istio verwenden das "profile"-Konzept
  * Diese werden nachher einfach mit dem defaults - Block gemerged
  * Wenn wir z.B. eine Einstellungen für die Demo haben wollen (z.B. sampleTracing auf
100 statt auf 1), können wir das demo-profile verwenden
  * Die Profile für die Helm-Charts findest Du hier (nicht alles, was es für die
Installation mit istioctl gibt, gibt es auch mit helm !!):
    [Profile für Helm]https://github.com/istio/istio/tree/release-
```

```
1.29.1/manifests/helm-profiles)
  * Wichtig ! Sie werden nicht in allen Charts verwenden, wenn wir es überall angeben,
müssen wir uns aber keine Gedanken machen, in welchem Start


### Schritt 1: Helm-Repository hinzufügen

```bash
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm repo update
```

Schritt 2: Namespace erstellen und Base-Chart installieren

Das Base-Chart installiert die Istio CRDs (Custom Resource Definitions):

```
helm install istio-base istio/base -n istio-system --set defaultRevision=default --
set profile=demo --create-namespace --version 1.29.1
```

Prüfen:

```
helm ls -n istio-system
```

Der Status von `istio-base` sollte `deployed` sein.

Schritt 3: Istiod (Control Plane) installieren

```
helm install istiod istio/istiod -n istio-system --wait --version 1.29.1  --set
profile=demo
```

Prüfen:

```
helm ls -n istio-system
kubectl get deployments -n istio-system
```

`istiod` sollte `READY 1/1` anzeigen.

Schritt 4 : Ingress Gateway installieren

```
helm install istio-ingress istio/gateway -n istio-ingress --wait --version 1.29.1 --
set profile=demo --create-namespace
```

> ***Hinweis:*** *Der Gateway-Namespace darf kein Label* `istio-injection=disabled` *haben.*

Schritt 4.5: (Optional) Metrics-Server installieren

- Der HorizontalPodAutoscaler der vom Gateway - Chart verwendet wird braucht den Metrics-Server um die CPU vom istio-ingressgateway - Pod auszulesen
- kubectl -n istio-ingress get hpa # Hier steht aktuell **Unknown**
```

```
jmetzger@powerhouse(ubuntu24):~/bin$ kubectl -n istio-ingress get hpa
NAME            REFERENCE                  TARGETS              MINPODS   MAXPODS   REPLICAS   AGE
istio-ingress   Deployment/istio-ingress   cpu: <unknown>/80%   1         5         1          8m22s
```

- Um das zu beheben müssen wir den Metrics-Server installieren

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
```

```
cd
mkdir -p helm-charts/metrics-server
cd helm-charts/metrics-server
nano values.yaml
```

```
args:
  - --kubelet-insecure-tls
```

```
helm -n kube-system upgrade --install metrics-server metrics-server/metrics-server --
version 3.13.0 --create-namespace -f values.yaml
```

```
## Überprüfen, es dauert ein bisschen, bis er Ready
kubectl -n kube-system get pods
```

```
## hpa sollte jetzt die richtigen Infos haben
kubectl -n istio-ingress get hpa
```

```
jmetzger@powerhouse(ubuntu24):~/helm-charts/metrics-server$ kubectl -n istio-ingress get hpa
NAME            REFERENCE                  TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
istio-ingress   Deployment/istio-ingress   cpu: 7%/80%   1         5         1          28m
```

Konfiguration anpassen

Default-Werte eines Charts anzeigen:

```
helm show values istio/istiod
helm show values istio/gateway
```

Eigene Werte beim Installieren mitgeben:

```
helm install istiod istio/istiod -n istio-system --values my-values.yaml --wait
```

Deinstallation

In umgekehrter Reihenfolge deinstallieren:

```
## 1. Gateway (falls installiert)
helm delete istio-ingress -n istio-ingress
kubectl delete namespace istio-ingress

## 2. Istiod
helm delete istiod -n istio-system

## 3. Base-Chart
helm delete istio-base -n istio-system

## 4. Namespace aufräumen
kubectl delete namespace istio-system
```

Optional – CRDs entfernen (Achtung: löscht alle Istio-Ressourcen!):

```
kubectl get crd -oname | grep 'istio.io' | xargs kubectl delete
```

Referenz

- Offizielle Doku
- Helm Upgrade Guide
- Advanced Helm Customization

istioctl als debugging - tool installieren

- Good tool for debugging
- Download includes demo and additional tools

Schritt 1: istio runterladen und installieren

```
cd
## current version of istio is 1.29.1
curl -L https://istio.io/downloadIstio | sh -
ln -s ~/istio-1.29.1 ~/istio
echo 'export PATH=~/istio/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

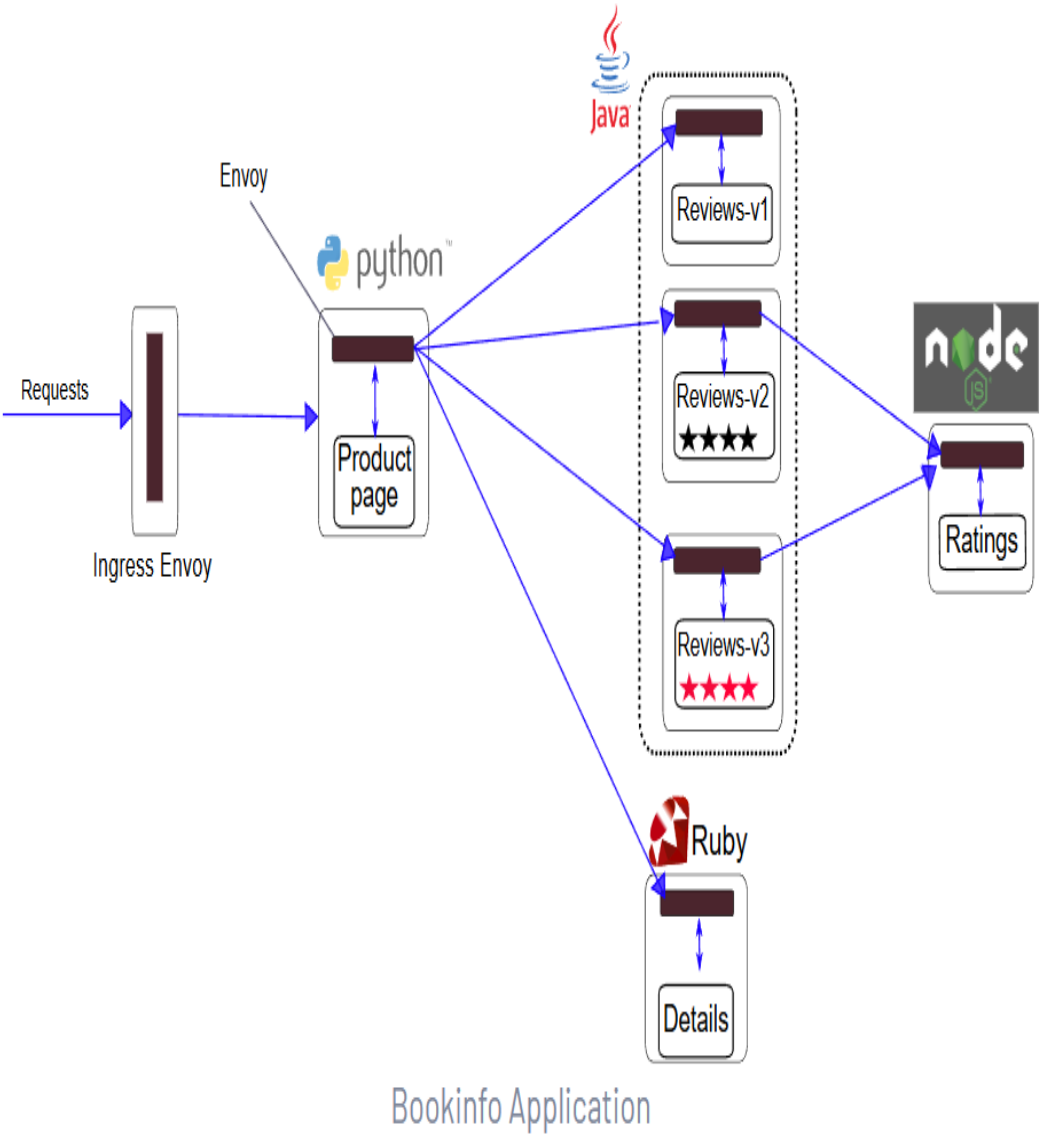
Schritt 2: bash completion integrieren

```
cp ~/istio/tools/istioctl.bash ~/istioctl.bash
echo "source ~/istioctl.bash" >> ~/.bashrc
source ~/istioctl.bash
```

Demo-App (Gateway API, sidecar)

Istio demo-app *bookinfo* installieren (Gateway API - Variante)

Überblick



Vorbereitung

Gateway API - CRD's installieren (Stand 2026-11-08, v1.5.x ab Istio 1.30)

- falls nicht bereits vorher geschehen

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.5.1/standard-install.yaml
```

```
kubectl create ns bookinfo
kubectl label namespace bookinfo istio-injection=enabled
```

bookdemo app ausrollen

```
kubectl -n bookinfo apply -f  ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo get all
```

testen ob die app funktioniert

```
kubectl -n bookinfo exec "$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

App mit gateway api nach aussen öffnen

```
## That's what we do ....
cat  ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
```

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/gateway-api/bookinfo-
gateway.yaml
kubectl -n bookinfo get gateways
kubectl -n bookinfo get httproutes -o yaml
```

```
## note the external-ip from this output
## gateway automatically creates a service
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

```
http://<external-ip>/productpage
## or in your browser
```

CNI-Plugin istio

Vorteile und Installation mit helm

Warum das Istio CNI Plugin?

Das Istio CNI Plugin ersetzt den `istio-init` Init-Container, der normalerweise die iptables-Regeln für die Traffic-Umleitung zum Envoy Sidecar setzt.

Vorteile

- **Keine erhöhten Privileges im Pod** — `NET_ADMIN` und `NET_RAW` Capabilities entfallen, da die iptables-Regeln auf Node-Ebene gesetzt werden
- **Kompatibel mit `restricted` Pod Security Standard** — wichtig für sicherheitskritische Umgebungen
- **Keine Race Conditions** — Traffic-Regeln stehen bereits bevor der erste Container im Pod startet. Ohne CNI Plugin können Init-Container, die vor `istio-init` laufen, Traffic am Envoy vorbeischicken (kein mTLS, keine AuthorizationPolicy)
- **Weniger Ressourcen** — kein zusätzlicher Init-Container-Image-Pull
- **Im Ambient Mode zwingend erforderlich** — wer später auf Ambient Mode migrieren möchte, braucht das CNI Plugin ohnehin

Voraussetzungen

- Bestehende Istio-Installation via Helm (base + istiod) mit `profile=demo`
- Helm >= 3.x

- `kubectl` Zugriff auf den Cluster
- Das CNI Plugin arbeitet als **Chained CNI Plugin** — es hängt sich an das bestehende CNI an (Flannel, Calico etc.). Bei Cilium-basierten Clustern (z.B. DigitalOcean DOKS, GKE Dataplane v2) vorher Kompatibilität prüfen.

Schritt-für-Schritt-Anleitung

1. Aktuelle Installation prüfen

```
## Installierte Istio Helm Releases anzeigen
helm list -n istio-system

## Aktuelle istiod-Werte prüfen
helm get values istiod -n istio-system
```

Verifiziere, dass bestehende Pods einen `istio-init` Init-Container haben:

```
## alle pods im bookinfo - namespace
kubectl -n bookinfo get pods details-v1-6cc9f5cc44-c5zqf -o
jsonpath='{.spec.initContainers[*].name}'
```

Erwartete Ausgabe (vor CNI Plugin): `istio-init`

2. Istio Helm Repo aktualisieren

```
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm repo update
```

3. Istio CNI Plugin installieren

```
helm install istio-cni istio/cni -n istio-system \
  --set profile=demo
```

Warten bis das CNI DaemonSet auf allen Nodes läuft:

```
kubectl rollout status daemonset/istio-cni-node -n istio-system
kubectl -n istio-system get ds istio-cni-node
```

4. istiod upgraden — CNI Plugin aktivieren

```
## evtl gibt es sonst Probleme - Fehlermeldung, wenn man den Hook vorher nicht löscht
kubectl delete validatingwebhookconfiguration istio-validator-istio-system
helm upgrade istiod istio/istiod -n istio-system \
  --set profile=demo \
  --set pilot.cni.enabled=true
```

- Achtung istiod wird nicht neugestartet, weil
- die Einstellung pilot.cni.enabled in der configmap istio-sidecar-injector landet
- nur diese muss verändert werden

- Diese configmap wird geladen, wenn ein neuer Pod erstellt wird. Das macht istio über den Kubernetes Api-Server

```
kubectl -n istio-system get cm istio-sidecar-injector
```

5. Bestehende Workloads neu starten

Bestehende Pods haben noch den alten `istio-init` Init-Container. Erst nach einem Restart werden sie ohne Init-Container und mit CNI Plugin Traffic-Umleitung gestartet.

```
## Alle Deployments in einem Namespace neu starten
kubectl rollout restart deployment -n bookinfo

## Oder gezielt ein einzelnes Deployment
kubectl rollout restart deployment/<DEPLOYMENT_NAME> -n bookinfo
```

6. Verifizierung

CNI DaemonSet läuft auf allen Nodes:

```
kubectl get daemonset istio-cni-node -n istio-system
```

Neue Pods haben keinen `istio-init` Init-Container mehr:

```
kubectl get pod <NEUER_POD> -n bookinfo -o jsonpath='{.spec.initContainers[*].name}'
```

Erwartete Ausgabe: istio-validation istio-proxy

- istio-validation überprüft ob ide iptables - Regeln richtig gesetzt wurden

Traffic funktioniert weiterhin:

```
## Aus einem Pod heraus testen
## istio-proxy hat curl mit drauf
kubectl -n bookinfo exec deploy/productpage-v1 -c istio-proxy -- curl -s
http://reviews:9080/reviews/1r.local
```

Troubleshooting

CNI DaemonSet startet nicht

```
kubectl describe daemonset istio-cni-node -n istio-system
kubectl logs -l k8s-app=istio-cni-node -n istio-system
```

Häufige Ursache: CNI-Bin- oder Conf-Verzeichnis stimmt nicht mit dem Cluster-Setup überein. Kann über Helm Values angepasst werden:

```
helm upgrade istio-cni istio/cni -n istio-system \
  --set profile=demo \
```

```
  --set cni.cniBinDir=/opt/cni/bin \
  --set cni.cniConfDir=/etc/cni/net.d
```

Pods haben nach Restart immer noch `istio-init`

Prüfen, ob `pilot.cni.enabled=true` gesetzt ist:

```
helm get values istiod -n istio-system | grep cni
```

Netzwerk-Konnektivität unterbrochen nach CNI Installation

Rollback möglich:

```
helm upgrade istiod istio/istiod -n istio-system \
  --set profile=demo \
  --set pilot.cni.enabled=false

kubectl rollout restart deployment -n bookinfo

helm uninstall istio-cni -n istio-system
```

Service Resilience & Fehlertoleranz (nur VirtualService)

Bemerkungen zu nur VirtualService und nicht Gateway API - httproute

- Some features are only implemented in VirtualService (like FaultInjection, Retries)

Vorgehensweise

- httpRoute wird für NorthSouth Traffic verwendet (das was ins Cluster reinkommt)
- Im Mesh kann ich entweder httpRoute oder virtualService verwenden
- Wenn eine Feature nur in virtualService und noch nicht für die Gateway Api implementiert ist, muss ich virtualService im Mesh verwenden (Beispiel Fault Injection, Retries)

Warum kann ich im Mesh httpRoute und und VirtualService nicht mischen

- Um nicht alles neu zu entwickeln, hat sich istio httpRoute (Mesh) -> in -> virtualService umzuschreiben
- D.h. für einen Service entweder virtualService oder httpRoute
- Wann immer möglich (weil alle Feature, die ich brauche drin sind), würde ich httpRoute nehmen

Retries VirtualService only !

- Retries in der Gateway API noch nicht implementiert
- Wenn benötigt, dann für den Service (intern-mesh)-> keine httproute verwenden, sondern VirtualService
- So haben wir es hier auch implementiert

Wofür

- Wenn ein Pod nicht erreicht werden kann, und ein anderer HTTP-Code als 200 zurückkommt, dann nochmal probieren

Vorbereitung: Test-Server erstellen, der 500er erstellt

```
cd
mkdir -p manifests/retry-flaky
```

```
cd manifests/retry-flaky
```

```
nano flaky-server.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: flaky-server
  namespace: bookinfo
data:
  server.py: |
    from http.server import BaseHTTPRequestHandler, HTTPServer
    import random

    class Handler(BaseHTTPRequestHandler):
        def do_GET(self):
            # 50% Chance auf 500, sonst 200
            if random.random() < 0.5:
                self.send_response(500)
                self.end_headers()
                self.wfile.write(b"oops, random 500\n")
            else:
                self.send_response(200)
                self.end_headers()
                self.wfile.write(b"all good, 200\n")

        def log_message(self, format, *args):
            # Einfaches Logging auf STDOUT
            print("%s - - [%s] %s" % (self.client_address[0],
self.log_date_time_string(), format % args))

    if __name__ == "__main__":
        port = 8080
        server = HTTPServer(("", port), Handler)
        print(f"Starting flaky server on port {port}")
        server.serve_forever()

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flaky-server
  namespace: bookinfo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flaky-server
  template:
    metadata:
      labels:
        app: flaky-server
```

```
    spec:
      containers:
      - name: flaky-server
        image: python:3.11-alpine
        command: ["python", "/app/server.py"]
        ports:
        - containerPort: 8080
        volumeMounts:
        - name: server-code
          mountPath: /app
      volumes:
      - name: server-code
        configMap:
          name: flaky-server
          items:
          - key: server.py
            path: server.py
---
apiVersion: v1
kind: Service
metadata:
  name: flaky-server
  namespace: bookinfo
spec:
  selector:
    app: flaky-server
  ports:
  - name: http
    port: 8080
    targetPort: 8080
```

```
kubectl apply -f flaky-server.yaml
```

Schritt 2: VirtualHost ohne retries

- Hier könnte man auch httproute (east-west) verwenden

```
nano virtual-no-retries.yaml
```

```
## Istio VirtualService mit Retries für flaky-server
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: flaky-server
  namespace: bookinfo
spec:
  hosts:
  - flaky-server
  http:
  - route:
    - destination:
        host: flaky-server
```

```
        port:
          number: 8080
```

```
kubectl apply -f virtual-no-retries.yaml
```

Schritt 3: Testscript flaky erstellt und ausführen

```
touch flaky-test.sh
chmod u+x flaky-test.sh
nano flaky-test.sh
```

```
##!/usr/bin/env bash

set -e

NAMESPACE="bookinfo"
SERVICE="flaky-server"
PORT=8080

REVIEWS_POD=$(kubectl get pod -n "$NAMESPACE" -l app=reviews -o
jsonpath='{.items[0].metadata.name}')
echo "Using REVIEWS_POD: $REVIEWS_POD"
echo "Calling http://$SERVICE:$PORT/ ..."
echo "Zum Abbrechen: STRG+C"
echo

kubectl exec -n "$NAMESPACE" -c reviews "$REVIEWS_POD" -- sh -c '
  i=0
  while true; do
    i=$((i+1))
    code=$(curl -s -o /dev/null -w "%{http_code}\n in %{time_total}s" http://flaky-
server:8080/ || echo "000 in 0")
    echo "[Request $i] -> $code"
    sleep 1
  done
'
```

```
## 500er beobachte
## Wieviele 500er
./flaky-test.sh
```

```
CTRL + C
```

Schritt 4: virtualservice zu mit retries modifizieren

```
nano virtualservice-mit-retries.yaml
```

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
```

```
  name: flaky-server
  namespace: bookinfo
spec:
  hosts:
  - flaky-server
  http:
  - route:
    - destination:
        host: flaky-server
        port:
          number: 8080
    retries:
      attempts: 3
      perTryTimeout: 2s
      retryOn: "5xx,connect-failure,refused-stream,reset"
```

```
kubectl apply -f virtualservice-mit-retries.yaml
```

```
## nochmal testen
## Wieviele 500er jetzt
./flaky-test.sh
```

```
CTRL + C
```

Verifizieren direkt im Flaky-Pod

- Er gibt 200 oder 500 zurück
- Für alle gleichen Anfragen, gibt es es eine Request_id

 Retries in flaky-server

```
FLAKY_POD=$(kubectl get pod -n bookinfo -l app=flaky-server -o
jsonpath='{.items[0].metadata.name}')
kubectl logs -n bookinfo -c istio-proxy "$FLAKY_POD"
```

Aufräumen

```
kubectl delete -f .
```

Fehlerinjektion (z.B. 500er) VirtualService only !

- The gateway api does not support fault-injection (so the http-route object)
- We are using the gateway api, but can still use the VirtualService for Service-2-Service call inside the mesh

Walkthrough

Step 1: Prepare

```
cd
mkdir -p manifests/fault-injection
cd manifests/fault-injection
```

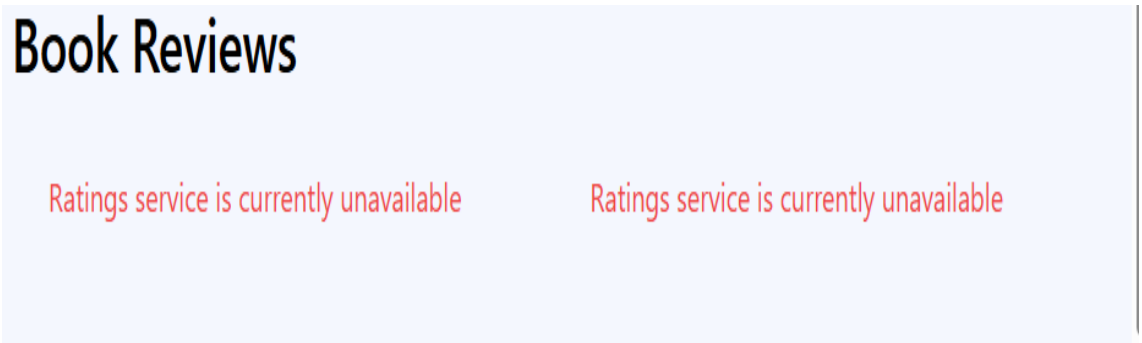
```
cp -a ~/istio/samples/bookinfo/networking/virtual-service-all-v1.yaml .
cp -a ~/istio/samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml .
kubectl -n bookinfo apply -f .
```

Step 2: Fault-Injection - Abort (500er)

```
kubectl apply -n bookinfo -f - <<EOF
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2
  - route:
    - destination:
        host: reviews
        subset: v3
EOF
```

Step 3: Seite aufrufen und als jason einloggen

- /productpage



Step 4: Ausloggen oder als andere Nutzer einloggen

- Reviews gehen wieder

Step 5: Cleanup

```
kubectl delete -f . --ingore-not-found
```

Service Resilience & Fehlertoleranz (nur Sidecar, EnvoyFilter)

Local Rate Limit mit EnvoyFilter - nur Sidecar !

> ***Nur für Sidecar-Mode.*** *Der EnvoyFilter greift im Sidecar (* `context: SIDECAR_INBOUND` *,* `workloadSelector: app=productpage` *). Im Ambient-Mode gibt es diesen Sidecar nicht: ztunnel ist kein Envoy (kein EnvoyFilter), der Waypoint wird von EnvoyFilter nicht offiziell unterstützt. Diese Übung läuft deshalb nur auf einem Cluster mit Sidecar-Injection (* `istio-injection=enabled` *).*

Walkthrough

- Bind EnvoyFilter to workload app=productpage

```
cd
mkdir -p manifests/local-rate-limit
cd manifests/local-rate-limit
## Get gateway ip
kubectl -n bookinfo get gateway bookinfo-gateway
GATEWAY_URL="http://<your-ip-of-gateway>"
```

```
nano rate-limit-240-60s-ok.yaml
```

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: filter-local-ratelimit-svc
  namespace: bookinfo
spec:
  workloadSelector:
    labels:
      app: productpage
  configPatches:
    - applyTo: HTTP_FILTER
      match:
        context: SIDECAR_INBOUND
        listener:
          portNumber: 9080 # Port of Service / best practice -> be specific
          filterChain:
            filter:
              name: "envoy.filters.network.http_connection_manager"
      patch:
        operation: INSERT_BEFORE
        value:
          name: envoy.filters.http.local_ratelimit
          typed_config:
            "@type": type.googleapis.com/udpa.type.v1.TypedStruct
            type_url:
type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.LocalRateLimit
            value:
              stat_prefix: http_local_rate_limiter
              enable_x_ratelimit_headers: DRAFT_VERSION_03 # mehr infos im outpunt
header - wieviele token sind verbraucht
```

```
              token_bucket:
                max_tokens: 240
                tokens_per_fill: 240
                fill_interval: 60s
              filter_enabled:
                runtime_key: local_rate_limit_enabled
                default_value:
                  numerator: 100
                  denominator: HUNDRED
              filter_enforced:
                runtime_key: local_rate_limit_enforced
                default_value:
                  numerator: 100
                  denominator: HUNDRED
              response_headers_to_add:
                - append: false
                  header:
                    key: x-local-rate-limit
                    value: 'true'
```

```
kubectl apply -f rate-limit-240-60s-ok.yaml
```

```
while true; do curl -s "$GATEWAY_URL/productpage" -o /dev/null -w "%{http_code}\n";
done
```

```
## works will out put
```

```
200
200
200
200
200
200
200
200
200
200
200
200
200
200
```

```
## now try with 20 per 60s
```

```
nano rate-limit-20-60s-not.yaml
```

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: filter-local-ratelimit-svc
  namespace: bookinfo
spec:
  workloadSelector:
    labels:
      app: productpage
  configPatches:
    - applyTo: HTTP_FILTER
      match:
        context: SIDECAR_INBOUND
        listener:
          portNumber: 9080 # port of service -> best practice be specific
          filterChain:
            filter:
              name: "envoy.filters.network.http_connection_manager"
      patch:
        operation: INSERT_BEFORE
        value:
```

```
          name: envoy.filters.http.local_ratelimit
          typed_config:
            "@type": type.googleapis.com/udpa.type.v1.TypedStruct
            type_url:
type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.LocalRateLimit
            value:
              stat_prefix: http_local_rate_limiter
              token_bucket:
                max_tokens: 20
                tokens_per_fill: 20
                fill_interval: 60s
              filter_enabled:
                runtime_key: local_rate_limit_enabled
                default_value:
                  numerator: 100
                  denominator: HUNDRED
              filter_enforced:
                runtime_key: local_rate_limit_enforced
                default_value:
                  numerator: 100
                  denominator: HUNDRED
              response_headers_to_add:
                - append: false
                  header:
                    key: x-local-rate-limit
                    value: 'true'
```

```
kubectl apply -f rate-limit-20-60s-not.yaml
```

```
while true; do curl -s "$GATEWAY_URL/productpage" -o /dev/null -w "%{http_code}\n";
done
```

```
## now we have a lot of too many connections
## rate limit helps to throttle
```

```
200
200
200
200
429
429
429
429
429
429
429
200
200
429
200
429
429
429
429
429
429
429
429
200
200
```

Analyse direkt im istio-proxy (sidecar)

```
istioctl proxy-config listener -n bookinfo deploy/productpage-v1 --port 15006 -o json
| grep -A40 local_ratelimit
```

Erklärung - Ablauf

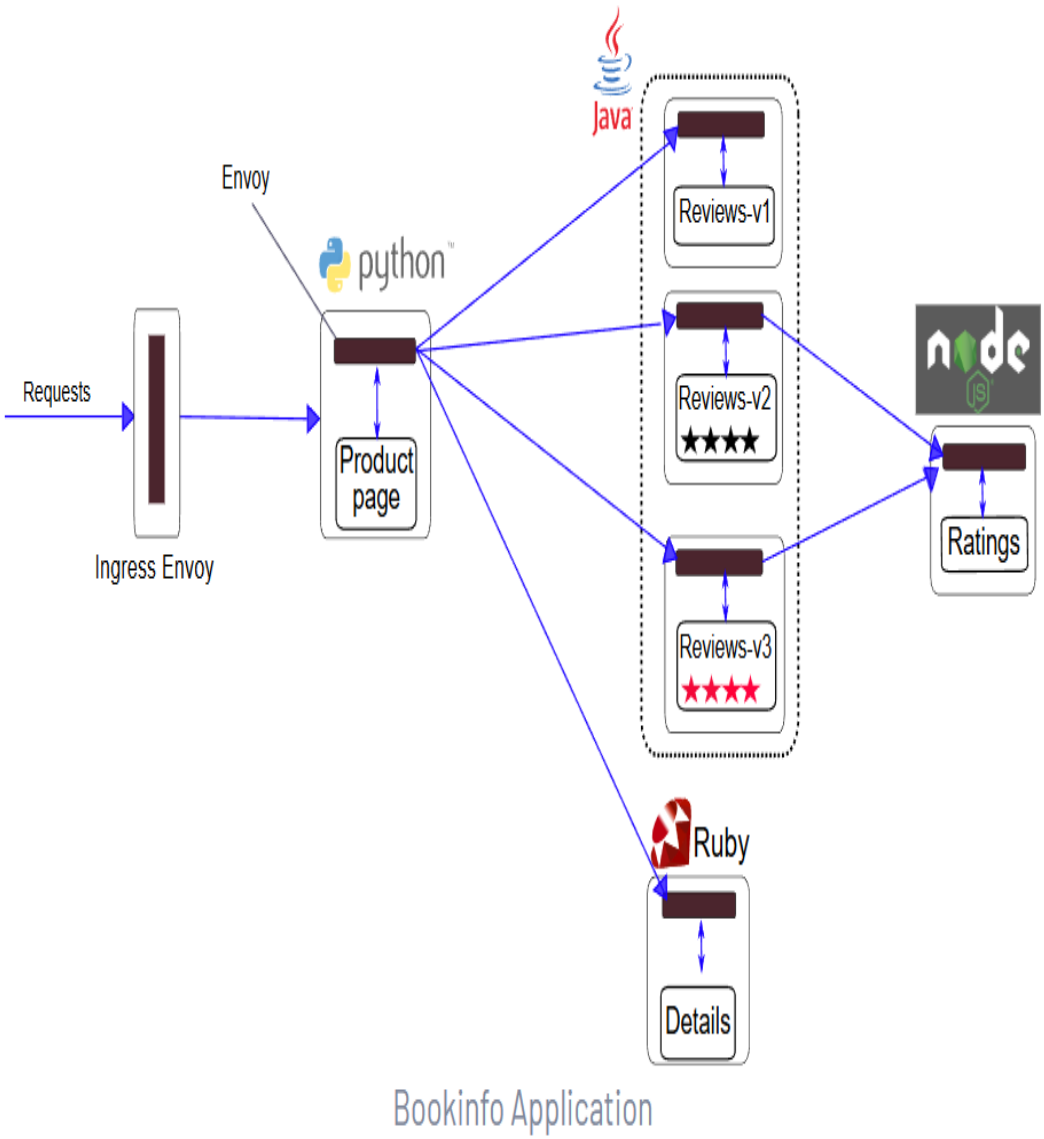
1. Ein Request kommt am Pod an (z.B. auf Port 9080)

2. **iptables-Regeln** (von `istio-init` eingerichtet) fangen den Traffic ab und leiten ihn auf Port **15006** um
3. Der `virtualInbound` Listener auf 15006 nimmt den Request entgegen
4. Envoy schaut sich TLS, ALPN und den **Original-Zielport** an (via `original_dst` Listener Filter)
5. Basierend darauf wird die passende **Filter Chain** ausgewählt (z.B. `0.0.0.0_9080` mTLS)
6. Der Request durchläuft die HTTP Filter (inkl. Rate Limiting) und wird an den lokalen App-Container weitergeleitet

Demo-App (istio api)

Istio demo-app *bookinfo* installieren (istio API - Variante)

Überblick



Vorbereitung

Namespace erstellen und labeln.

```
kubectl create ns bookinfo
kubectl label namespace bookinfo istio-injection=enabled
```

bookdemo app ausrollen

```
kubectl -n bookinfo apply -f  ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo get all
```

testen ob die app funktioniert

```
kubectl -n bookinfo exec "$(kubectl -n bookinfo get pod -l app=ratings -o
jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS
productpage:9080/productpage | grep -o "<title>.*</title>"
```

App mit istio-ingress-gateway nach aussen öffnen

```
cd
mkdir -p manifests/istio-gateway
cd manifests/istio-gateway
nano gateway-virtualservice.yaml
```

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  # The selector matches the ingress gateway pod labels.
  # If you installed Istio using Helm following the standard documentation, this would
be "istio=ingress"
  selector:
    istio: ingress # use istio default controller
  servers:
  - port:
      number: 80
      name: http2
      protocol: HTTP
    hosts:
    - "*"
---
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
  - "*"
  gateways:
  - bookinfo-gateway
```

```
  http:
  - match:
    - uri:
        exact: /productpage
    - uri:
        prefix: /static
    - uri:
        exact: /login
    - uri:
        exact: /logout
    - uri:
        prefix: /api/v1/products
    route:
    - destination:
        host: productpage
        port:
          number: 9080
```

```
kubectl -n bookinfo apply -f .
```

```
kubectl -n bookinfo get gw
kubectl -n bookinfo get virtualservice -o yaml
```

```
## get ip from here
kubectl -n istio-ingress get all
kubectl -n istio-ingress get svc
```

```
http://<external-ip>/productpage
## or in your browser
```

Übungen Steuerung des Netzwerkverhaltens (istio api)

Übung: Header-basiertes Routing - istio api

Request Routing

Vorher (ohne request routing)

- Es werden alle Pods angezeigt, die das Label: app:reviews haben
- D.h. jedesmal wenn ich die Seite öffne, wird eine andere Version angegezeigt (v1, v2 oder v3)
- Service (selector: app:reviews)

```
kubectl -n bookinfo get svc reviews -o yaml
kubectl -n bookinfo get pods -l app=reviews --show-labels
```

Übung (jetzt request - routing)

Voraussetzung:

- Bookinfo-App läuft bereits im Namespace `bookinfo` (ansonsten neu einrichten)
- Service Reviews ist definiert

- Es gibt 3 verschieden Pods an Reviews (v1, v2 und v3)
- Ingress/Gateway + `GATEWAY_URL` (IP: http://164.90.237.35/productpage aus der vorherigen Übung vorhanden

0 Vorbereitung

```
mkdir -p ~/manifests/requests
cd ~/manifests/requests

## Die Destinationen-Versionen anlegen
cp -a ~/istio/samples/bookinfo/networking/destination-rule-all.yaml destination-rule-all.yaml
kubectl -n bookinfo apply -f destination-rule-all.yaml
```

---

1. VirtualService: Alle Requests → `reviews-v1`

```
nano reviews-v1.yaml
```

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
```

```
kubectl -n bookinfo apply -f reviews-v1.yaml
kubectl -n bookinfo get virtualservice reviews -n bookinfo
```

```
## Gateway IP ausfinden machen
kubectl -n istio-ingress get svc istio-ingress
GATEWAY_URL=<ip-aus-letzter-ausgabe>
```

Anzeige im Browser - es ist immer die v1

http://$GATEWAY_URL/productpage

```


---

#### 2. Adjust VirtualHost: User `jason` → `reviews-v2`, Rest → `reviews-v1`
```

cp -a ~/istio/samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml virtual-service-reviews-test-v2.yaml cat virtual-service-reviews-test-v2.yaml

kubectl -n bookinfo apply -f virtual-service-reviews-test-v2.yaml kubectl -n bookinfo get vs reviews -n bookinfo -o yaml

```
---

### 3. Testen im Browser

```bash
echo "$GATEWAY_URL"
## Beispiel: http://<IP>:<PORT>

## 1. Im Browser: $GATEWAY_URL/productpage aufrufen (nicht eingeloggt oder anderer
User)
##    → Reviews ohne Sterne (v1)

## 2. Im Browser: als User "jason" einloggen
##    → Reviews mit Sternen (v2)
```

---

4. Aufräumen

```
kubectl -n bookinfo delete vs reviews
```

Reference:

- https://istio.io/latest/docs/examples/bookinfo/#define-the-service-versions

Skalierung, Erweiterbarkeit & Performance-Optimierung

Skalierung von Istio - Möglichkeiten/Tipps&Tricks

Überblick

Istio besteht aus einer **Control Plane** (istiod) und einer **Data Plane** (Envoy-Proxies). Beide müssen unabhängig voneinander skaliert werden. Die größte Herausforderung bei wachsenden Meshes ist nicht CPU oder RAM, sondern die **xDS-Konfigurationsverteilung** – also die Menge an Routing-Regeln, Endpoints und Policies, die istiod an jeden einzelnen Proxy pushen muss.

---

1. Control Plane Skalierung (istiod)

1.1 Horizontale Skalierung

istiod ist stateless bezüglich der Mesh-Konfiguration (sie kommt aus Kubernetes-API-Objekten). Mehrere Replicas können parallel laufen.

```
apiVersion: apps/v1
kind: Deployment
metadata:
```
```

```
  name: istiod
  namespace: istio-system
spec:
  replicas: 3
```

Empfehlungen:

- **Kleine Meshes** (< 100 Pods): 1 Replica reicht
- **Mittlere Meshes** (100–500 Pods): 2–3 Replicas
- **Große Meshes** (500+ Pods): 3+ Replicas mit HPA

1.2 HorizontalPodAutoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: istiod
  namespace: istio-system
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: istiod
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

1.3 Ressourcen-Limits

```
resources:
  requests:
    cpu: 500m
    memory: 2Gi
  limits:
    cpu: "2"
    memory: 4Gi
```

> ***Faustregel:*** *istiod braucht vor allem Memory. Jeder Service/Endpoint verbraucht Speicher für die interne Konfigurationsdatenbank. Bei 1000+ Services sollte man mit 4–8 Gi rechnen.*

---

2. xDS-Konfigurationsumfang reduzieren

Dies ist der **wichtigste Hebel** für Istio-Skalierung. Standardmäßig bekommt jeder Envoy-Proxy die Konfiguration für **alle** Services im Mesh – auch für Services, mit denen er nie kommuniziert.

2.1 Sidecar-Resource

Die `Sidecar`-Resource beschränkt den Sichtbarkeitsbereich eines Workloads auf nur die Services, die er tatsächlich braucht.

Mesh-weiter Default – nur eigenen Namespace + istio-system sehen:

```
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: default
  namespace: istio-system   # mesh-weit wenn in istio-system
spec:
  egress:
    - hosts:
        - "./*"               # eigener Namespace
        - "istio-system/*"    # Istio-interne Services
```

Namespace-spezifisch – z.B. ein Frontend, das nur das Backend-API braucht:

```
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: default
  namespace: frontend
spec:
  egress:
    - hosts:
        - "./*"
        - "istio-system/*"
        - "backend/*"
```

Auswirkung: Reduziert die Anzahl der Cluster/Endpoints/Routes in der Envoy-Konfiguration drastisch. Bei einem Mesh mit 200 Services sieht ein Frontend-Pod statt 200 nur noch 10–20 Service-Konfigurationen.

2.2 Discovery Selectors (meshConfig)

Discovery Selectors steuern, welche **Namespaces** istiod überhaupt überwacht. Standardmäßig watched istiod alle Namespaces im Cluster – auch solche, die gar nicht zum Mesh gehören (z.B. `monitoring`, `logging`, `ci-cd`).

Mit Discovery Selectors sagt man: "istiod, kümmere dich nur um Namespaces mit bestimmten Labels." Alles andere wird komplett ignoriert – keine Watches auf der Kubernetes-API, keine Services/Endpoints in der internen Datenbank, kein xDS-Push dafür.

IstioOperator-Variante:

```
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  meshConfig:
    discoverySelectors:
      - matchLabels:
          istio-discovery: enabled
```

Installation via Helm Chart:

```
helm install istio-base istio/base -n istio-system --create-namespace

helm install istiod istio/istiod -n istio-system \
  --set meshConfig.discoverySelectors[0].matchLabels.istio-discovery=enabled
```

Oder sauberer über eine `values.yaml`:

```
## values-istiod.yaml
meshConfig:
  discoverySelectors:
    - matchLabels:
        istio-discovery: enabled
    - matchExpressions:
        - key: env
          operator: In
          values: ["production", "staging"]
```

```
helm install istiod istio/istiod -n istio-system -f values-istiod.yaml
```

> ***Hinweis:*** *Das `matchExpressions`-Beispiel zeigt, dass man auch Set-basierte Selektoren nutzen kann – z.B. um mehrere Environments gleichzeitig einzuschließen.*

Namespaces labeln:

```
kubectl label namespace frontend istio-discovery=enabled
kubectl label namespace backend istio-discovery=enabled
kubectl label namespace database istio-discovery=enabled
```

Nur Namespaces mit dem Label `istio-discovery: enabled` werden von istiod beobachtet. Das reduziert:

- Watch-Last auf der Kubernetes-API
- Memory-Verbrauch von istiod
- xDS-Push-Umfang

Unterschied zur Sidecar-Resource:

	Discovery Selectors	Sidecar-Resource
Ebene	istiod – was wird überhaupt beobachtet	Pro Proxy – was bekommt er davon zu sehen
Wirkung	Namespace existiert für istiod nicht	Namespace existiert, wird aber gefiltert
Spart	API-Watches, Memory in istiod	xDS-Config-Größe pro Proxy

Am besten kombiniert man beides: Discovery Selectors schließen irrelevante Namespaces komplett aus, Sidecar-Resources filtern innerhalb der relevanten Namespaces die Sichtbarkeit pro Workload.

2.3 exportTo

Auf Ressourcen-Ebene kann man die Sichtbarkeit einschränken:

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: my-api
  namespace: backend
spec:
  exportTo:
    - "."        # nur im eigenen Namespace sichtbar
    - "frontend" # und im frontend Namespace
  hosts:
    - my-api
  http:
    - route:
        - destination:
            host: my-api
```

`exportTo` funktioniert auf: `VirtualService`, `DestinationRule`, `ServiceEntry`.

---

3. Data Plane Skalierung

3.1 Sidecar-Ressourcen pro Workload

Die Envoy-Proxies verbrauchen per Default relativ wenig, aber bei High-Traffic-Workloads muss man die Ressourcen anpassen:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        sidecar.istio.io/proxyCPU: "200m"
        sidecar.istio.io/proxyMemory: "256Mi"
        sidecar.istio.io/proxyCPULimit: "1"
        sidecar.istio.io/proxyMemoryLimit: "512Mi"
```

3.2 Envoy Concurrency

Standardmäßig nutzt Envoy 2 Worker-Threads. Für High-Throughput-Workloads kann man das erhöhen:

```yaml
apiVersion: networking.istio.io/v1
kind: ProxyConfig
metadata:
  name: high-throughput
  namespace: backend
spec:
  concurrency: 4    # Anzahl Worker-Threads
```

> ***Achtung:*** *Mehr Concurrency = mehr CPU-Verbrauch. Nur für Workloads mit hohem RPS sinnvoll.*

3.3 Ambient Mode als Skalierungsalternative

Im **Ambient Mode** entfällt der Sidecar-Proxy pro Pod komplett. Stattdessen:

- **ztunnel** (Node-Level): Übernimmt mTLS und L4-Routing – ein DaemonSet-Pod pro Node statt ein Sidecar pro Pod
- **Waypoint Proxy** (Namespace/Service-Level): Wird nur bei Bedarf für L7-Policies deployt

Vorteile für Skalierung:

Aspekt	Sidecar-Mode	Ambient Mode
Proxy-Anzahl	1 pro Pod	1 ztunnel pro Node + Waypoints
Memory-Overhead	~50–100 MB pro Pod	~50 MB pro Node
xDS-Konfiguration	Jeder Sidecar braucht Konfig	Nur ztunnel + Waypoints
Startup-Latenz	Sidecar muss mit Pod starten	Kein Sidecar-Init nötig

Bei einem Cluster mit 100 Nodes und 2000 Pods:

- **Sidecar-Mode:** 2000 Envoy-Proxies, ~100–200 GB Memory-Overhead
- **Ambient Mode:** 100 ztunnel + wenige Waypoints, ~5–10 GB Memory-Overhead

Aktivierung:

```
kubectl label namespace my-app istio.io/dataplane-mode=ambient
```

---

4. Monitoring der Skalierung

4.1 Wichtige istiod-Metriken

Metrik	Bedeutung	Alarmschwelle
`pilot_xds_pushes`	Anzahl xDS-Pushes	Spikes = häufige Config-Änderungen
`pilot_xds_push_time`	Dauer eines Push-Zyklus	> 5s problematisch
`pilot_proxy_convergence_time`	Zeit bis alle Proxies aktuell	> 10s kritisch
`pilot_virt_services`	Anzahl VirtualServices	Wachstum beobachten
`pilot_services`	Anzahl Services im Mesh	Wachstum beobachten
`pilot_xds_config_size_bytes`	Größe der xDS-Config	Sidecar-Scope prüfen wenn hoch

4.2 Envoy-Metriken pro Proxy

```
## Config-Dump eines Sidecars ansehen
kubectl exec -it deploy/my-app -c istio-proxy -- \
  pilot-agent request GET config_dump | jq '.configs | length'

## Cluster-Anzahl (= bekannte Upstream-Services)
```

```
kubectl exec -it deploy/my-app -c istio-proxy -- \
  pilot-agent request GET clusters | wc -l
```

Wenn ein einzelner Sidecar hunderte Cluster hat, ist die `Sidecar`-Resource nicht richtig konfiguriert.

4.3 Grafana Dashboards

Istio liefert vorkonfigurierte Dashboards:

- **Istio Control Plane Dashboard**: istiod CPU, Memory, Push-Zeiten, xDS-Connections
- **Istio Mesh Dashboard**: Überblick über alle Services und deren Traffic
- **Istio Performance Dashboard**: Proxy-Latenz und Ressourcenverbrauch

---

5. Best Practices Zusammenfassung

1. **Sidecar-Resource ist Pflicht** in produktiven Meshes – mesh-weiten Default setzen, dann pro Namespace verfeinern:

   ```yaml
   # Mesh-weiter Default: restriktiver Ausgangspunkt
   apiVersion: networking.istio.io/v1
   kind: Sidecar
   metadata:
     name: default
     namespace: istio-system      # istio-system = gilt mesh-weit
   spec:
     egress:
       - hosts:
           - "./*"                 # nur eigener Namespace
           - "istio-system/*"      # + Istio-interne Services
   ```

   Dann pro Namespace verfeinern, wo nötig (Namespace-Sidecar überschreibt den mesh-weiten Default):

   ```yaml
   apiVersion: networking.istio.io/v1
   kind: Sidecar
   metadata:
     name: default
     namespace: frontend          # überschreibt mesh-weiten Default
   spec:
     egress:
       - hosts:
           - "./*"
           - "istio-system/*"
           - "backend/*"          # zusätzliche Abhängigkeit
   ```

2. **Discovery Selectors** nutzen, um irrelevante Namespaces (Monitoring, CI/CD, etc.) auszuschließen
3. **exportTo** auf VirtualServices/DestinationRules setzen, um Cross-Namespace-Leaks zu vermeiden
4. **istiod horizontal skalieren** ab ~200 Pods mit HPA
5. **Memory-Limits** großzügig setzen – istiod fällt bei OOM komplett aus
6. **Ambient Mode evaluieren** – besonders bei großen Clusters mit vielen Pods deutlicher Ressourcenvorteil

7. **Monitoring** der Push-Zeiten und Convergence-Time als Frühwarnsystem
8. **Canary-Updates** für istiod – Revision-basierte Upgrades (`istioctl install --revision canary`) ermöglichen parallele Control Planes

---

6. Hands-on Übung: Sidecar-Scope konfigurieren

Ausgangslage

Cluster mit drei Namespaces: `frontend` , `backend` , `database` . Jeder Sidecar sieht aktuell alle Services.

Aufgabe

1. Prüfe die aktuelle Cluster-Anzahl im Frontend-Sidecar:

```
kubectl exec -it deploy/frontend -n frontend -c istio-proxy -- \
  pilot-agent request GET clusters | grep "^[a-z]" | wc -l
```

2. Erstelle eine mesh-weite Sidecar-Resource, die standardmäßig nur den eigenen Namespace erlaubt
3. Erstelle namespace-spezifische Sidecar-Resources:
   - `frontend` → sieht `backend` und `istio-system`
   - `backend` → sieht `database` und `istio-system`
   - `database` → sieht nur `istio-system`
4. Prüfe erneut die Cluster-Anzahl und vergleiche

Erwartetes Ergebnis

Die Cluster-Anzahl pro Sidecar sollte um 50–80% sinken, je nach Mesh-Größe.

Performance-Optimierung

Überblick

Istio fügt durch die Envoy-Proxies einen gewissen Overhead hinzu – Latenz, CPU, Memory und I/O. Ziel der Performance-Optimierung ist es, diesen Overhead so gering wie möglich zu halten, ohne auf die Vorteile des Service Mesh zu verzichten.

---

1. Proxy-Latenz verstehen und messen

Jeder Envoy-Sidecar fügt Latenz hinzu – **typisch ~0,5ms pro Hop**, bei Ausreißern (z.B. TLS-Handshake, xDS-Update, hohe Last) bis zu 1–3ms. Ein Hop = ein Proxy-Durchgang. Da der Traffic sowohl den Sender- als auch den Empfänger-Sidecar passiert, kommen pro Service-Call ca. 1–2ms dazu (bei Ausreißern bis zu 6ms).

Bei langen Call-Chains summiert sich das:

```
Client → Frontend → Backend → Database
         +1-2ms     +1-2ms    +1-2ms
                                      = 3-6ms zusätzliche Latenz (typ.)
```

Latenz messen mit Istio-Metriken:

```
## p99 Latenz pro Service (server-side)
## p99 = 99. Perzentil: 99% aller Requests sind schneller als dieser Wert
histogram_quantile(0.99,
  sum(rate(istio_request_duration_milliseconds_bucket{reporter="destination"}[5m]))
  by (le, destination_service))

## Differenz zwischen client- und server-reported Latenz = Proxy-Overhead
histogram_quantile(0.99,
rate(istio_request_duration_milliseconds_bucket{reporter="source"}[5m]))
-
histogram_quantile(0.99,
rate(istio_request_duration_milliseconds_bucket{reporter="destination"}[5m]))
```

---

2. Protocol Detection vs. explizites appProtocol

Standardmäßig muss Envoy das Protokoll jeder eingehenden Verbindung erraten (Protocol Sniffing). Das kostet Zeit – Envoy wartet kurz, um die ersten Bytes zu inspizieren.

Lösung: Protokoll explizit angeben, dann entfällt die Sniffing-Phase:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-api
spec:
  ports:
    - name: http
      port: 80
      targetPort: 8080
      appProtocol: http    # explizit – spart Sniffing-Phase
```

Alternativ über die Port-Naming-Konvention: `http-`, `grpc-`, `tcp-` als Prefix.

> ***Empfehlung:*** *Immer `appProtocol` setzen. Das ist die zuverlässigste Methode und spart pro Verbindung einige Millisekunden Initialisierung.*

---

3. Access Logging tunen

Per Default loggt Envoy jeden einzelnen Request. Bei hohem RPS (Requests Per Second) erzeugt das erhebliche CPU- und I/O-Last.

Wie Access Logging funktioniert

Access Logs sind die Logs der **Envoy-Proxies** – nicht die Anwendungslogs. Sie zeigen: wer hat wen aufgerufen, mit welchem Ergebnis. In `kubectl logs` sieht man beide, aber aus verschiedenen Containern:

```
## App-Logs
kubectl logs deploy/my-app -c my-app

## Envoy Access Logs
kubectl logs deploy/my-app -c istio-proxy
```

Konfiguration

Access Logging wird über die **Telemetry-API** konfiguriert. Drei Aspekte:

- **Wohin** → `accessLogFile` in meshConfig (Dateipfad oder stdout) für den eingebauten Provider, oder `extensionProviders` für externe Ziele
- **Format** → `accessLogEncoding` in meshConfig (`TEXT` oder `JSON`)
- **Was** → `filter.expression` in der Telemetry-Resource

Logging komplett deaktivieren:

```
meshConfig:
  accessLogFile: ""    # kein Ziel = nichts wird geloggt
```

Nur Fehler loggen (empfohlen für Produktion):

```
## Schritt 1: Ziel und Format konfigurieren
meshConfig:
  accessLogFile: /dev/stdout    # wohin (stdout = sichtbar via kubectl logs)
  accessLogEncoding: JSON       # format
```

```
## Schritt 2: Filter setzen – nur Fehler loggen
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: access-logging
  namespace: istio-system       # istio-system = gilt mesh-weit
spec:
  accessLogging:
    - providers:
        - name: envoy           # eingebauter Provider, nutzt accessLogFile
      filter:
        expression: "response.code >= 400"   # CEL-Expression: nur 4xx und 5xx
```

Die `filter.expression` ist eine CEL-Expression (Common Expression Language), die für jeden Request ausgewertet wird. Nur wenn sie `true` ergibt, wird geloggt. Weitere Beispiele:

- `"response.code >= 500"` – nur Server-Fehler
- `"response.code >= 400 || response.duration > 1000"` – Fehler oder langsame Requests (>1s)

Mehrere Log-Ziele

Man kann zu verschiedenen Zielen mit unterschiedlichen Filtern loggen:

```
## Externe Provider in meshConfig registrieren
meshConfig:
  accessLogFile: /dev/stdout
  extensionProviders:
    - name: otel-logging
      envoyOtelAls:
        service: otel-collector.monitoring.svc.cluster.local
        port: 4317
```

```
## Verschiedene Filter pro Ziel
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: access-logging
  namespace: istio-system
spec:
  accessLogging:
    - providers:
        - name: envoy              # stdout: Client- und Server-Fehler
      filter:
        expression: "response.code >= 400"
    - providers:
        - name: otel-logging       # Collector: nur Server-Fehler
      filter:
        expression: "response.code >= 500"
```

`envoy` (stdout) ist immer verfügbar und muss nicht extra registriert werden. Alle anderen Provider müssen in `extensionProviders` definiert werden.

---

4. Tracing Sampling

Distributed Tracing erzeugt bei hohem RPS erheblichen Overhead – jeder Trace muss gesammelt, serialisiert und exportiert werden.

Sampling-Rate anpassen:

```
meshConfig:
  defaultConfig:
    tracing:
      sampling: 1.0    # 1% der Requests - Default ist oft 100% in Demo-Setups!
```

Umgebung	Empfohlenes Sampling
Development	100%
Staging	10–50%
Produktion (niedrig RPS)	1–10%
Produktion (hoch RPS)	0.1–1%

> ***Achtung:*** *Viele Istio-Demo-Installationen setzen Sampling auf 100%. Das ist ein häufiger Grund für unerklärlich hohen CPU-Verbrauch in Produktion.*

---

5. Metrics Cardinality begrenzen

Was ist Metrics Cardinality?

Istio-Metriken werden mit **Labels** (Key-Value-Paare) versehen. Jede einzigartige Kombination von Label-Werten erzeugt eine eigene Zeitreihe in Prometheus. Beispiel:

```
istio_requests_total{
  source_workload="frontend",
  destination_workload="backend-api",
  response_code="200",
  request_protocol="http",
  connection_security_policy="mutual_tls"
} 48273
```

Das ist ein Zähler: "Frontend hat Backend-API 48.273 mal mit HTTP aufgerufen und 200 zurückbekommen." Nicht jeder Request erzeugt eine neue Zeitreihe – aber jede **neue Kombination** von Label-Werten schon. Danach wird nur der Zähler hochgezählt:

```
...{source="frontend", destination="backend", response_code="200"} 48273  ← Zähler
steigt
...{source="frontend", destination="backend", response_code="500"} 12     ← neue
Kombination
...{source="checkout", destination="backend", response_code="200"} 9500   ← neue
Kombination
```

Warum ist die Anzahl der Labels entscheidend?

Jedes zusätzliche Label **multipliziert** die Anzahl der möglichen Kombinationen:

```
10 Sources × 10 Destinations                              = 100 Zeitreihen
10 Sources × 10 Destinations × 5 Response-Codes           = 500 Zeitreihen
10 Sources × 10 Destinations × 5 Codes × 2 Protocols      = 1.000 Zeitreihen
10 Sources × 10 Destinations × 5 Codes × 2 Prot. × 2 Sec. = 2.000 Zeitreihen
```

Entfernt man `request_protocol` und `connection_security_policy`, fällt man von 2.000 auf 500 Zeitreihen – ein Viertel. Und das pro Metrik. Bei 5 Istio-Metriken spart man 6.000 Zeitreihen.

Hohe Cardinality = Problem für Prometheus

- Mehr Memory-Verbrauch (jede Zeitreihe lebt im RAM)
- Langsamere Queries
- Mehr Disk I/O

Bei 200 Services im Mesh können schnell Millionen von Zeitreihen entstehen.

Labels entfernen

Labels selektiv abschalten per Telemetry-API:

```yaml
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: reduce-metrics
  namespace: istio-system
spec:
  metrics:
    - providers:
        - name: prometheus
      overrides:
        - match:
```

```
      metric: ALL_METRICS
    tagOverrides:
      request_protocol:
        operation: REMOVE
      connection_security_policy:
        operation: REMOVE
```

> ***Faustregel:*** *Labels entfernen, die man nicht aktiv in Dashboards oder Alerts abfragt. Wenn niemand je nach* `connection_security_policy` *filtert, bringt das Label keinen Nutzen, kostet aber Speicher.*

---

6. Envoy Concurrency

Standardmäßig nutzt Envoy 2 Worker-Threads. Für High-Throughput-Workloads kann man das erhöhen:

```yaml
apiVersion: networking.istio.io/v1
kind: ProxyConfig
metadata:
  name: high-throughput
  namespace: backend
spec:
  concurrency: 4   # Anzahl Worker-Threads
```

> ***Achtung:*** *Mehr Concurrency = mehr CPU-Verbrauch. Nur für Workloads mit hohem RPS sinnvoll.*

---

7. mTLS-Overhead

Der TLS-Handshake und die Ver-/Entschlüsselung kosten CPU, aber in der Praxis ist der Overhead gering – typisch **< 0,5ms zusätzliche Latenz** und ~**5–10% mehr CPU** auf dem Proxy.

Relevant wird es bei:

- Sehr hohem Throughput (>10.000 RPS pro Pod)
- Großen Payloads (Verschlüsselung skaliert mit Datenmenge)
- Ressourcen-knappe Umgebungen (Edge, IoT)

> ***Empfehlung:*** *mTLS immer auf STRICT lassen. Der Sicherheitsgewinn überwiegt den minimalen Performance-Overhead bei weitem. Optimierung an anderen Stellen (Sidecar-Scope, Tracing, Logging) bringt deutlich mehr.*

---

8. Connection Pooling: HTTP/2 vs. HTTP/1.1

Envoy nutzt standardmäßig Connection Pooling, aber das Verhalten unterscheidet sich je nach Protokoll:

- **HTTP/1.1:** Eine Connection pro Request (oder Keep-Alive mit begrenzten parallelen Requests)
- **HTTP/2:** Multiplexing – viele Requests über eine Connection

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: high-throughput-pool
spec:
  host: my-api
  trafficPolicy:
```

```
    connectionPool:
      http:
        h2UpgradePolicy: UPGRADE        # HTTP/1.1 → HTTP/2 upgraden
        http2MaxRequests: 1000           # max parallele Requests
        maxRequestsPerConnection: 0      # unbegrenzt (für HTTP/2)
      tcp:
        maxConnections: 100
        connectTimeout: 5s
```

> ***Tipp:*** *Zwischen Sidecars im Mesh kann man bedenkenlos auf HTTP/2 upgraden, auch wenn die App selbst nur HTTP/1.1 spricht – Envoy übernimmt die Protokoll-Translation.*

---

9. Custom Extensions: EnvoyFilter, Lua und WasmPlugin

Wenn man das Verhalten der Envoy-Proxies anpassen will (z.B. Header hinzufügen, Requests transformieren), gibt es drei Ansätze.

9.1 EnvoyFilter (native Config-Patches)

Man patcht direkt die Envoy-Konfiguration. Sehr performant, aber fragil – bricht leicht bei Istio-Upgrades, weil sich interne Strukturen ändern.

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: add-timeout
  namespace: backend
spec:
  configPatches:
    - applyTo: ROUTE_CONFIGURATION
      patch:
        operation: MERGE
        value:
          request_timeout: 30s
```

9.2 Lua (via EnvoyFilter)

Lua-Scripts werden ebenfalls über ein `EnvoyFilter`-Objekt deployt – aber statt die Envoy-Config direkt zu patchen, injiziert man ein Script als HTTP-Filter. Das Script selbst ist stabil, das EnvoyFilter-Objekt drumherum kann bei Upgrades trotzdem brechen.

Zwei Einstiegspunkte: `envoy_on_request` (bevor der Request zum Backend geht) und `envoy_on_response` (bevor die Response zum Client geht).

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: lua-add-header
  namespace: backend
spec:
  workloadSelector:
    labels:
```

```
      app: my-api
  configPatches:
    - applyTo: HTTP_FILTER
      match:
        context: SIDECAR_INBOUND
        listener:
          filterChain:
            filter:
              name: envoy.filters.network.http_connection_manager
              subFilter:
                name: envoy.filters.http.router
      patch:
        operation: INSERT_BEFORE
        value:
          name: envoy.lua
          typed_config:
            "@type": type.googleapis.com/envoy.extensions.filters.http.lua.v3.Lua
            inline_code: |
              function envoy_on_response(response_handle)
                response_handle:headers():remove("server")
                response_handle:headers():add("x-served-by", "mesh")
              end
```

9.3 WasmPlugin (empfohlener Weg)

Seit Istio die `WasmPlugin`-Resource hat, ist das der strategisch beste Ansatz. Vorteile:

- **Eigene Istio-API** – kein EnvoyFilter nötig, deutlich stabiler bei Upgrades
- **Sandbox** – Wasm läuft in einer VM, kann den Proxy nicht crashen
- **Portabel** – Wasm-Module lassen sich als OCI-Image verteilen
- **Mehrere Sprachen** – Go, Rust, C++ (via proxy-wasm SDK)

```
apiVersion: extensions.istio.io/v1alpha1
kind: WasmPlugin
metadata:
  name: add-header
  namespace: backend
spec:
  selector:
    matchLabels:
      app: my-api
  url: oci://registry.example.com/my-wasm-plugin:v1
  phase: STATS               # wann im Filter-Chain: AUTHN, AUTHZ, STATS
  pluginConfig:
    header_name: "x-custom"
    header_value: "hello"
```

Performance-Vergleich

Ansatz	Latenz-Overhead	Memory	Upgrade-Stabilität
EnvoyFilter (native)	Minimal	Gering	Fragil – bricht häufig

Lua (via EnvoyFilter)	Niedrig (~0,1–0,5ms)	Gering	Script stabil, EnvoyFilter-Wrapper fragil
WasmPlugin	Mittel (~0,5–2ms)	Höher (VM)	Stabil – eigene API

Empfehlung

- **WasmPlugin** als Standard für Custom Extensions – stabil, sicher, portabel
- **Lua** für schnelle, einfache Eingriffe (Header-Manipulation, kleine Transformationen)
- **EnvoyFilter (native Config-Patches)** nur als letztes Mittel, wenn es keine andere Option gibt

---

10. Best Practices Zusammenfassung

1. **appProtocol** immer explizit setzen – spart Protocol-Sniffing-Overhead
2. **Access Logging** auf Fehler beschränken oder deaktivieren – spart CPU/IO
3. **Tracing Sampling** in Produktion auf 0,1–1% setzen – häufigste Ursache für unerklärlich hohen CPU-Verbrauch
4. **Metrics Cardinality** bewusst begrenzen – Labels entfernen die nicht abgefragt werden
5. **Envoy Concurrency** nur bei High-Throughput-Workloads erhöhen
6. **mTLS auf STRICT lassen** – Overhead minimal, Sicherheitsgewinn groß
7. **HTTP/2 zwischen Sidecars** aktivieren bei vielen parallelen Requests
8. **WasmPlugin** als Standard für Custom Extensions – Lua für kleine Eingriffe, EnvoyFilter (native) vermeiden

Erweiterte Routing-Techniken & Traffic-Optimierung (Istio API)

Canary Releases & Progressive Deployments / Traffic - Shifting

0. Vorbereitung

```
mkdir -p ~/manifests/traffic-shifting
cd ~/manifests/traffic-shifting

## Die Destinationen-Versionen anlegen
cp -a ~/istio/samples/bookinfo/networking/destination-rule-all.yaml destination-rule-all.yaml
kubectl -n bookinfo apply -f destination-rule-all.yaml
```

1. 100% Traffic -> reviews.v1

```
cat ~/istio/samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/networking/virtual-service-all-v1.yaml
```

```
kubectl get vs -n bookinfo reviews -o yaml | head -n 30
```

2. Testen

```
## Seite öffnen
http://<deine-ip>/productpage

## Egal wie oft du die Seite lädst, es bleibt immer v1
```

3. 50% (v1) /50% (v3) Traffic

```
cat ~/istio/samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
```

```
kubectl -n bookinfo get vs reviews -o yaml
```

4. Testen

```
## Seite öffnen
http://<deine-ip>/productpage

## Abwechselnd bei mehrmals laden v1 (keine Sterne) und v3 (sterne)
```

5. 100% auf v3

```
cat ~/istio/samples/bookinfo/networking/virtual-service-reviews-v3.yaml
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/networking/virtual-service-reviews-v3.yaml
```

```
$BOOKINFO_URL="deine-ip"
for i in {1..10}; do
  curl -fsS "$BOOKINFO_URL/productpage" \
    | grep -c 'glyphicon-star"' \
    | awk '{print $1" Sterne"}'
done
```

6. Aufräumen

```
kubectl delete -n bookinfo vs reviews
```

Reference:

- https://istio.io/latest/docs/tasks/traffic-management/traffic-shifting/

Erweiterbarkeit von Istio (WebAssembly, Übung)

Eigene Istio-Erweiterungen mit WebAssembly schreiben (RateLimit-Plugin)

Lernziele

- WASM-Plugin in Go für Envoy/Istio entwickeln
- Token-Bucket-Algorithmus implementieren
- WasmPlugin-Resource konfigurieren (Sidecar, Ambient, Gateway)

Voraussetzungen

- Kubernetes-Cluster mit Istio ≥ 1.22
- Container-Registry (Harbor, GHCR, Docker Hub)

---

1 — Tooling installieren

```
## Go (https://go.dev/dl/)
wget https://go.dev/dl/go1.22.5.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.22.5.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin

## TinyGo — WASM-fähiger Go-Compiler (Standard-Go kann kein wasi-Target)
wget https://github.com/tinygo-
org/tinygo/releases/download/v0.33.0/tinygo_0.33.0_amd64.deb
sudo dpkg -i tinygo_0.33.0_amd64.deb

## ORAS — OCI-Registry-Tool zum Pushen von WASM-Modulen
curl -LO https://github.com/oras-
project/oras/releases/download/v1.2.0/oras_1.2.0_linux_amd64.tar.gz
tar xzf oras_1.2.0_linux_amd64.tar.gz && sudo mv oras /usr/local/bin/

## Prüfen
go version && tinygo version && oras version
```

---

2 — Projekt anlegen

```
mkdir -p istio-ratelimit-wasm && cd istio-ratelimit-wasm
go mod init github.com/t3company/istio-ratelimit-wasm
go get github.com/tetratelabs/proxy-wasm-go-sdk
```

---

3 — Token Bucket Algorithmus

```
Tokens: ████████░░  (8/10)

  Request → Token verbrauchen → 7/10 → 200 OK
  Request → Token verbrauchen → 6/10 → 200 OK
  ...
  Request → Bucket leer       → 0/10 → 429!

  Timer Tick (1s) → Refill +10 → 10/10
```

Parameter	Beschreibung
`max_tokens`	Burst-Kapazität
`refill_rate`	Tokens/Sekunde nachgefüllt

Vorteil gegenüber Fixed Window: kein Boundary-Problem, Burst-fähig, glättet langfristigen Durchsatz.

---

4 — Plugin implementieren

Proxy-WASM Architektur (Go)

```
VMContext → PluginContext → HttpContext
                │                │
                ├─ OnPluginStart()          Config + Timer
                ├─ OnTick()                 Token Refill
                └─ NewHttpContext()
                        └─ OnHttpRequestHeaders()  Token prüfen
```

main.go

```go
package main

import (
    "encoding/binary"
    "encoding/json"
    "fmt"

    "github.com/tetratelabs/proxy-wasm-go-sdk/proxywasm"
    "github.com/tetratelabs/proxy-wasm-go-sdk/proxywasm/types"
)

func main() { proxywasm.SetVMContext(&vmContext{}) }

// ── Config ──────────────────────────────────────────────────────────

type rateLimitConfig struct {
    MaxTokens        uint64 `json:"max_tokens"`
    RefillRate       uint64 `json:"refill_rate"`
    RefillIntervalMs uint64 `json:"refill_interval_ms"`
    StatusCode       uint32 `json:"status_code"`
    ResponseBody     string `json:"response_body"`
}

func defaultConfig() rateLimitConfig {
    return rateLimitConfig{100, 10, 1000, 429, "rate limit exceeded"}
}

// ── Shared Data Keys + Helpers ──────────────────────────────────────

const (
    bucketKey       = "ratelimit_tokens"
    statsAllowedKey = "ratelimit_allowed"
    statsDeniedKey  = "ratelimit_denied"
)

func toBytes(v uint64) []byte {
    b := make([]byte, 8)
    binary.LittleEndian.PutUint64(b, v)
    return b
}

func fromBytes(b []byte) uint64 {
```

```
    if len(b) < 8 { return 0 }
    return binary.LittleEndian.Uint64(b)
}

func incrementStat(key string) {
    data, cas, err := proxywasm.GetSharedData(key)
    if err != nil { return }
    proxywasm.SetSharedData(key, toBytes(fromBytes(data)+1), cas)
}

// ── VM Context ──────────────────────────────────────────────────────

type vmContext struct{ types.DefaultVMContext }

func (*vmContext) NewPluginContext(contextID uint32) types.PluginContext {
    return &pluginContext{config: defaultConfig()}
}

// ── Plugin Context (Config + Timer + Bucket) ────────────────────────

type pluginContext struct {
    types.DefaultPluginContext
    config rateLimitConfig
}

func (p *pluginContext) OnPluginStart(configSize int) types.OnPluginStartStatus {
    if configSize > 0 {
        if data, err := proxywasm.GetPluginConfiguration(); err == nil {
            var cfg rateLimitConfig
            if err := json.Unmarshal(data, &cfg); err == nil {
                if cfg.MaxTokens == 0        { cfg.MaxTokens = 100 }
                if cfg.RefillRate == 0       { cfg.RefillRate = 10 }
                if cfg.RefillIntervalMs == 0 { cfg.RefillIntervalMs = 1000 }
                if cfg.StatusCode == 0       { cfg.StatusCode = 429 }
                if cfg.ResponseBody == ""    { cfg.ResponseBody = "rate limit
exceeded" }
                p.config = cfg
            }
        }
    }

    proxywasm.LogInfof("Rate Limiter: max_tokens=%d, refill_rate=%d/s",
        p.config.MaxTokens, p.config.RefillRate)

    proxywasm.SetSharedData(bucketKey, toBytes(p.config.MaxTokens), 0)
    proxywasm.SetSharedData(statsAllowedKey, toBytes(0), 0)
    proxywasm.SetSharedData(statsDeniedKey, toBytes(0), 0)
    proxywasm.SetTickPeriodMilliSeconds(uint32(p.config.RefillIntervalMs))
    return types.OnPluginStartStatusOK
}

func (p *pluginContext) OnTick() {
```

```
    data, cas, err := proxywasm.GetSharedData(bucketKey)
    if err != nil { return }

    current := fromBytes(data)
    add := p.config.RefillRate * p.config.RefillIntervalMs / 1000
    newTokens := current + add
    if newTokens > p.config.MaxTokens { newTokens = p.config.MaxTokens }

    if err := proxywasm.SetSharedData(bucketKey, toBytes(newTokens), cas); err != nil
{
        proxywasm.LogWarn("refill CAS conflict, retry next tick")
    }
}

func (p *pluginContext) NewHttpContext(contextID uint32) types.HttpContext {
    return &httpContext{config: p.config}
}

// ── HTTP Context (pro Request) ──────────────────────────────────────

type httpContext struct {
    types.DefaultHttpContext
    config rateLimitConfig
}

func (h *httpContext) OnHttpRequestHeaders(numHeaders int, endOfStream bool)
types.Action {
    for attempt := 0; attempt < 3; attempt++ {
        data, cas, err := proxywasm.GetSharedData(bucketKey)
        if err != nil { continue }

        current := fromBytes(data)

        if current == 0 {
            incrementStat(statsDeniedKey)
            proxywasm.SendHttpResponse(h.config.StatusCode, [][2]string{
                {"x-ratelimit-limit", fmt.Sprintf("%d", h.config.MaxTokens)},
                {"x-ratelimit-remaining", "0"},
                {"retry-after", "1"},
            }, []byte(h.config.ResponseBody), -1)
            return types.ActionPause
        }

        if err := proxywasm.SetSharedData(bucketKey, toBytes(current-1), cas); err !=
nil {
            continue // CAS-Konflikt → retry
        }

        incrementStat(statsAllowedKey)
        proxywasm.AddHttpRequestHeader("x-ratelimit-remaining", fmt.Sprintf("%d",
current-1))
        return types.ActionContinue
```

```
    }

    proxywasm.LogWarn("CAS exhausted, fail-open")
    return types.ActionContinue
}
```

Wichtige Konzepte

- **CAS (Compare-And-Swap):** `GetSharedData` liefert Wert + CAS-Token. `SetSharedData` schreibt nur wenn Token noch gültig. Bei Konflikt → retry.
- **Fail-Open:** Nach 3 CAS-Retries wird der Request durchgelassen — kaputtes Plugin soll nicht allen Traffic blockieren.
- **`types.DefaultHttpContext`:** Go-Embedding, implementiert alle Interface-Methoden als No-Ops. Wir überschreiben nur was wir brauchen.

---

5 — Build & Push

```
## Kompilieren (-scheduler=none: kein Goroutine-Scheduler, -target=wasi: WASM)
tinygo build -o ratelimit.wasm -scheduler=none -target=wasi main.go

ls -lh ratelimit.wasm   # ~500 KB - 1.5 MB

## Optional: ~30% kleiner
wasm-opt -Os ratelimit.wasm -o ratelimit.wasm

## Pushen
oras login registry.example.com
oras push registry.example.com/wasm/ratelimit:v1 \
  ratelimit.wasm:application/vnd.module.wasm.content.layer.v1+wasm
```

---

6 — Deployen

```
kubectl create namespace demo
kubectl label namespace demo istio-injection=enabled
kubectl apply -f k8s/test-deployment.yaml
kubectl -n demo get pods -w   # Warten auf 2/2 Running
```

`k8s/test-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-service
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-service
```

```
  template:
    metadata:
      labels:
        app: my-service
    spec:
      containers:
        - name: httpbin
          image: kennethreitz/httpbin:latest
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: demo
spec:
  selector:
    app: my-service
  ports:
    - port: 8080
      targetPort: 80
```

`k8s/wasmplugin.yaml`:

```
apiVersion: extensions.istio.io/v1alpha1
kind: WasmPlugin
metadata:
  name: local-ratelimit
  namespace: demo
spec:
  selector:
    matchLabels:
      app: my-service
  url: oci://registry.example.com/wasm/ratelimit:v1
  phase: AUTHN                    # Vor Authentication in der Filterkette
  pluginConfig:
    max_tokens: 100
    refill_rate: 10
    refill_interval_ms: 1000
    status_code: 429
    response_body: '{"error": "rate limit exceeded", "retry_after": 1}'
  match:
    - mode: SERVER                # Nur Inbound
      ports:
        - number: 8080
  imagePullPolicy: IfNotPresent
```

```
kubectl apply -f k8s/wasmplugin.yaml
kubectl -n demo logs deploy/my-service -c istio-proxy | grep -i "rate"
```

---

7 — Testen

```
## Einzelner Request
kubectl -n demo run test --rm -it --image=curlimages/curl -- \
  curl -sv http://my-service:8080/get 2>&1 | grep -E "HTTP|x-ratelimit"

## Load Test
kubectl -n demo run loadtest --rm -it --image=curlimages/curl -- sh -c '
  ALLOWED=0; DENIED=0
  for i in $(seq 1 200); do
    STATUS=$(curl -s -o /dev/null -w "%{http_code}\n" http://my-service:8080/get)
    [ "$STATUS" = "429" ] && DENIED=$((DENIED + 1)) || ALLOWED=$((ALLOWED + 1))
  done
  echo "Erlaubt: $ALLOWED | Gedrosselt: $DENIED | Gesamt: 200"
'
```

Erwartet: ~100 erlaubt (Burst), Rest 429. Pro Sekunde kommen ~10 nach.

---

8 — Config ändern

`max_tokens: 10` , `refill_rate: 2` setzen → `kubectl apply` → Load Test wiederholen. Erwartet: nur ~10 erlaubt.

> ***Frage:*** *Was passiert bei 3 Replicas? → Jeder Pod hat eigenen Bucket → 3× Durchsatz. Das ist Local vs. Global Rate Limiting.*

---

9 — Ambient Mode

```
kubectl label namespace demo istio-injection-
kubectl label namespace demo istio.io/dataplane-mode=ambient
istioctl waypoint apply -n demo --name my-service-waypoint --for service
kubectl -n demo label service my-service istio.io/use-waypoint=my-service-waypoint
```

```
apiVersion: extensions.istio.io/v1alpha1
kind: WasmPlugin
metadata:
  name: local-ratelimit-ambient
  namespace: demo
spec:
  targetRefs:                       # statt selector
    - kind: Service
      group: ""
      name: my-service
  url: oci://registry.example.com/wasm/ratelimit:v1
  phase: AUTHN
  pluginConfig:
    max_tokens: 50
    refill_rate: 5
    refill_interval_ms: 1000
```

Waypoint Proxy = ein zentraler Proxy pro Service → ein Bucket → verhält sich wie Global Rate Limiting.

---

10 — Gateway-Level Rate Limiting

```yaml
apiVersion: extensions.istio.io/v1alpha1
kind: WasmPlugin
metadata:
  name: gateway-ratelimit
  namespace: istio-system
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  url: oci://registry.example.com/wasm/ratelimit:v1
  phase: AUTHN
  pluginConfig:
    max_tokens: 1000
    refill_rate: 100
```

Gateway schützt den Cluster, Service-Level schützt einzelne Microservices. In Produktion beide kombinieren.

---

11 — Bonus-Aufgaben

Per Source-IP: `proxywasm.GetHttpRequestHeader("x-forwarded-for")` → Bucket-Key pro IP.

Health-Checks ausnehmen: `:path` Header prüfen, bei `/health` direkt `ActionContinue`.

Per Pfad: Separate Config pro Pfad-Prefix in `pluginConfig`.

Vergleich

	EnvoyFilter	WasmPlugin
API-Stabilität	"break glass"	Stabile Istio-API
Portierbarkeit	Istio-only	Envoy, Istio, Gateway API
Ambient Mode	Nein	Ja

	Go (TinyGo)	Rust
Lernkurve	Flach	Steil
Binary-Größe	~500 KB - 1.5 MB	~200-400 KB
Einschränkungen	Kein `net/http`, kein `reflect`	Keine
Empfehlung	Prototyping, Go-Teams	Produktion, Performance

Aufräumen

```
kubectl -n demo delete wasmplugin --all && kubectl delete namespace demo
```

Referenzen

- Proxy-WASM Go SDK · TinyGo · Istio WasmPlugin API · ORAS

Misc

Grenze bei der Header-Größe Istio vs. Nginx

Problem

Keycloak erzeugt große HTTP-Header (JWTs, Set-Cookie mit Access/ID/Refresh Token). Sowohl nginx als auch Istio/Envoy haben Default-Limits, die dabei überschritten werden können.

Envoy/Istio Defaults

Parameter	Default	Maximum
`max_request_headers_kb`	60 KiB*	**96 KiB** (hartes Limit)
`max_headers_count`	100	frei konfigurierbar

> ***Achtung:*** *In älteren Istio-Versionen (< 1.8) lag der tatsächliche Default im Code bei ~29 KiB, nicht bei den dokumentierten 60 KiB.*

Fehlermeldung bei Überschreitung: `431 Request Header Fields Too Large`

nginx Defaults

Parameter	Default
`large_client_header_buffers`	4 × 8 KB
`proxy_buffer_size`	4k / 8k
`proxy_buffers`	8 × 4k/8k

Fehlermeldungen bei Überschreitung:

- **400 Bad Request** — Request-Header des Clients zu groß (`large_client_header_buffers`)
- **502 Bad Gateway** — Response-Header vom Upstream (Keycloak) zu groß (`proxy_buffer_size`) — der Klassiker mit Keycloak

---

Fix: Istio/Envoy (EnvoyFilter)

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: max-request-headers
  namespace: istio-system
spec:
  configPatches:
```

```
  - applyTo: NETWORK_FILTER
    match:
      context: ANY
      listener:
        filterChain:
          filter:
            name: "envoy.filters.network.http_connection_manager"
    patch:
      operation: MERGE
      value:
        typed_config:
          "@type":
"type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.HttpCo

          max_request_headers_kb: 96
```

Verifizierung:

```
istioctl proxy-config listener <pod-name> -o json | grep max_request_headers
```

Bekannte Probleme

- Das **Verringern** des Limits funktioniert zuverlässig, das **Erhöhen** über den Default hinaus greift manchmal nicht (gemeldetes Istio-Issue)
- Werte > 96 werden ignoriert — **128 ist ungültig!**
- Der `@type` muss zur Istio-Version passen:
  - Istio ≤ 1.6: `...http_connection_manager.v2.HttpConnectionManager`
  - Istio ≥ 1.8: `...http_connection_manager.v3.HttpConnectionManager`

Fix: nginx

```
## Response-Header vom Upstream (Keycloak) — behebt 502
proxy_buffer_size       128k;
proxy_buffers           4 256k;
proxy_busy_buffers_size 256k;

## Request-Header vom Client — behebt 400
large_client_header_buffers 4 32k;
```

Checkliste: Weitere Header-Limits prüfen

Nicht nur der Proxy, auch die **Anwendung selbst** kann Header ablehnen:

Framework / Server	Default Max Header
Spring Boot	8 KiB
Gunicorn	8 KiB
Tomcat	8 KiB

Node.js (http)	16 KiB

Falls nach dem EnvoyFilter-Fix weiterhin Fehler auftreten, liegt es oft am App-Container, nicht an Envoy.

Istio als API Gateway

Was sind Dinge, die istio nicht unterstützt

Features, die Istio NICHT bietet:

- **Request/Response Body Transformation** — kein Umschreiben von JSON-Bodys, Feld-Mapping, XML↔JSON-Konvertierung (nur Header-Manipulation möglich)
- **OpenAPI-Spec-Validierung** — keine automatische Validierung eingehender Requests gegen ein Schema
- **API Key Management** — kein Ausstellen, Rotieren, Widerrufen von API Keys
- **Developer Portal** — keine Self-Service-Oberfläche für API-Consumer
- **Quota Management pro Consumer** — Rate Limiting ja, aber keine feingranularen Quotas pro API-Key/Consumer-Plan
- **API Versionierungs-Management** — Routing auf v1/v2 geht, aber kein automatisches Deprecation/Sunset-Handling
- **Caching** — kein Response-Caching am Gateway
- **GraphQL-Support** — kein nativer GraphQL-zu-REST-Proxy oder Schema-Stitching
- **Monetarisierung** / **Billing** — keine Usage-Tracking-Integration für Abrechnung

Was sind Features, die istio unterstützt

Was oft verwechselt wird — das kann Istio durchaus:

- Rate Limiting (via WASM oder EnvoyFilter)
- JWT-Validierung / OIDC
- Header-basiertes Routing
- mTLS / AuthorizationPolicy

Kernaussage: Istio fehlt vor allem alles rund um **Body-Manipulation, Schema-Validierung und API-als-Produkt-Features**. Auf der Traffic- und Security-Ebene ist es sehr vollständig.

Mehrere Namepaces bei allowedRoutes bei Gateway - Objekt / Gateway API

** Mit Label-Selector (empfohlen)**

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
  namespace: istio-system
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Selector
        selector:
```

```
          matchLabels:
            gateway-access: "true"
```

Dann die gewünschten Namespaces labeln:

```
kubectl label ns app1 gateway-access=true
kubectl label ns app2 gateway-access=true
```

```
Es gibt **keine** Möglichkeit, einzelne Namespaces als Liste anzugeben — die Gateway
API bietet nur `Same`, `All` oder `Selector`. Der Label-Selector ist der Weg, wenn du
gezielt bestimmte Namespaces freischalten willst.

### Global Ratelimiting, je nach Kunde


Komplettes Setup für per-Kunde Rate Limiting im Istio Service Mesh.
Kunden authentifizieren sich per JWT, der `customer_id`-Claim wird als
Envoy-Descriptor an den externen Rate Limit Service gesendet. Monitoring
über StatsD → statsd-exporter → Prometheus/VictoriaMetrics → Grafana.

---

### Architektur
```

Client --[JWT]--> Gateway (istio-ingressgateway) | ├── 1. JWT validieren (RequestAuthentication) ├── 2. Kein JWT? → 403 (AuthorizationPolicy) ├── 3. customer_id Claim extrahieren ├── 4. Descriptor {customer_id: "xyz"} an RLS senden (EnvoyFilter) | | | v | Rate Limit Service (gRPC :8081) | ├── StatsD ──► statsd-exporter (:9102) ──► Prometheus | | | | v v | Redis (:6379) — Counter pro Kunde Grafana | ├── OK → weiter zum Backend └── OVER_LIMIT → 429 Too Many Requests + x-envoy-ratelimited Header

```
#### Begriffe

- **Descriptor**: Key-Value-Paar (z.B. `customer_id: "customer-a"`), das Envoy per
gRPC an den Rate Limit Service schickt. Wird über `actions` im EnvoyFilter definiert.
Mögliche Quellen: JWT Claims, HTTP Header, Client-IP, statische Werte.
- **Cluster-Patch**: Registriert den Rate Limit Service als Envoy-Cluster im Gateway
mit einem kontrollierten Namen (`rate_limit_cluster`). Auch wenn der RLS im Mesh läuft
und Istio automatisch einen Cluster anlegt, ist der explizite Patch der sauberere Weg
— du kontrollierst den Cluster-Namen, der im HTTP Filter referenziert wird.
- **Domain**: Logischer Namespace im RLS, um Configs verschiedener Anwendungen zu
trennen. Muss in EnvoyFilter und ConfigMap identisch sein.

---

### Komponenten

| # | Ressource | Namespace | Beschreibung |
|---|-----------|-----------|--------------|
| 1 | Namespace | — | `ratelimit` Namespace |
```

```
| 2 | Redis | ratelimit | Counter-Backend |
| 3 | ConfigMap ratelimit-config | ratelimit | Rate Limits pro Kunde |
| 4 | ConfigMap statsd-config | ratelimit | StatsD → Prometheus Mapping |
| 5 | Deployment + Service ratelimit | ratelimit | RLS + statsd-exporter Sidecar |
| 6 | PodMonitor | ratelimit | Prometheus Scrape Config (nur mit Operator) |
| 7 | RequestAuthentication | istio-system | JWT Validierung am Gateway |
| 8 | AuthorizationPolicy | istio-system | JWT erzwingen |
| 9 | EnvoyFilter | istio-system | Cluster-Patch + Rate Limit Filter + Descriptor |
| 10 | PeerAuthentication | ratelimit | Metrics-Port PERMISSIVE für Prometheus Scrape
|
| 11 | Alerting Rules | ratelimit | PrometheusRule (Operator) oder ConfigMap (plain) |

---

### 1. Namespace (mit Sidecar Injection)

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ratelimit
  labels:
    istio-injection: enabled
```

> ***Mesh-Entscheidung**: In diesem Setup läuft der ratelimit-Namespace **im Mesh**. Das bedeutet: automatisches mTLS zwischen Gateway und RLS, bessere Observability (Istio Access Logs, Metriken für den gRPC-Traffic), und konsistentes Security-Modell wenn der Rest des Clusters STRICT mTLS fährt.*
>
> *Falls du den Namespace **nicht** im Mesh haben willst: `istio-injection: enabled` weglassen und Schritt 10 (PeerAuthentication) überspringen.*

---

2. Redis

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: ratelimit
spec:
  ports:
  - port: 6379
    targetPort: 6379
  selector:
    app: redis
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: ratelimit
spec:
```
```

```
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        ports:
        - containerPort: 6379
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            memory: 256Mi
```

> ***Produktion**: Redis Sentinel oder Redis Cluster verwenden.*

---

3. Rate Limit Config (pro Kunde)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ratelimit-config
  namespace: ratelimit
data:
  config.yaml: |
    domain: my-api
    descriptors:
      # Kunde A: Premium - 1000 req/min
      - key: customer_id
        value: "customer-a"
        rate_limit:
          unit: minute
          requests_per_unit: 1000

      # Kunde B: Standard - 100 req/min
      - key: customer_id
        value: "customer-b"
        rate_limit:
          unit: minute
          requests_per_unit: 100

      # Kunde C: Free Tier - 10 req/min
      - key: customer_id
        value: "customer-c"
```

```
      rate_limit:
        unit: minute
        requests_per_unit: 10

    # Default: unbekannte Kunden
    - key: customer_id
      rate_limit:
        unit: minute
        requests_per_unit: 5
```

> ***Hot Reload**: Der ratelimit-Service überwacht die Config-Dateien. ConfigMap-Änderungen werden automatisch übernommen (bis zu 60s Delay wegen kubelet sync).*

> ***Dynamische Limits**: Für viele Kunden / Self-Service kann ein Config-Generator (CronJob/Operator) die YAML aus einer Datenbank erzeugen und die ConfigMap updaten. Alternativ: Custom gRPC Rate Limit Service, der das `ShouldRateLimit` -Interface implementiert und Limits direkt aus einer DB liest.*

---

4. StatsD-Exporter Mapping Config

Übersetzt die StatsD-Metriken des ratelimit-Service in Prometheus-Format mit sinnvollen Labels (`domain` , `key1` , `key2`).

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: statsd-config
  namespace: ratelimit
data:
  statsd.yaml: |
    mappings:
    # 2 Key-Levels: domain + key1 (z.B. customer_id_customer-a)
    - match: "ratelimit.service.rate_limit.*.*.near_limit"
      name: "ratelimit_service_rate_limit_near_limit"
      observer_type: "histogram"
      labels:
        domain: "$1"
        key1: "$2"
    - match: "ratelimit.service.rate_limit.*.*.over_limit"
      name: "ratelimit_service_rate_limit_over_limit"
      observer_type: "histogram"
      labels:
        domain: "$1"
        key1: "$2"
    - match: "ratelimit.service.rate_limit.*.*.total_hits"
      name: "ratelimit_service_rate_limit_total_hits"
      observer_type: "histogram"
      labels:
        domain: "$1"
        key1: "$2"
    - match: "ratelimit.service.rate_limit.*.*.within_limit"
      name: "ratelimit_service_rate_limit_within_limit"
      observer_type: "histogram"
```

```
    labels:
      domain: "$1"
      key1: "$2"

  # 3 Key-Levels: domain + key1 + key2 (für verschachtelte Descriptors)
  - match: "ratelimit.service.rate_limit.*.*.*.near_limit"
    name: "ratelimit_service_rate_limit_near_limit"
    observer_type: "histogram"
    labels:
      domain: "$1"
      key1: "$2"
      key2: "$3"
  - match: "ratelimit.service.rate_limit.*.*.*.over_limit"
    name: "ratelimit_service_rate_limit_over_limit"
    observer_type: "histogram"
    labels:
      domain: "$1"
      key1: "$2"
      key2: "$3"
  - match: "ratelimit.service.rate_limit.*.*.*.total_hits"
    name: "ratelimit_service_rate_limit_total_hits"
    observer_type: "histogram"
    labels:
      domain: "$1"
      key1: "$2"
      key2: "$3"
  - match: "ratelimit.service.rate_limit.*.*.*.within_limit"
    name: "ratelimit_service_rate_limit_within_limit"
    observer_type: "histogram"
    labels:
      domain: "$1"
      key1: "$2"
      key2: "$3"

  # Config load Metriken
  - match: "ratelimit.service.config_load_success"
    name: "ratelimit_service_config_load_success"
    match_metric_type: counter
  - match: "ratelimit.service.config_load_error"
    name: "ratelimit_service_config_load_error"
    match_metric_type: counter

  # Alles andere droppen
  - match: "."
    match_type: "regex"
    action: "drop"
    name: "dropped"
```

> ***Quelle**: https://github.com/envoyproxy/ratelimit/blob/main/examples/prom-statsd-exporter/conf.yaml*

---

5. Rate Limit Service + StatsD-Exporter Sidecar

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ratelimit
  namespace: ratelimit
  labels:
    app: ratelimit
spec:
  ports:
  - port: 8080
    targetPort: 8080
    name: http
  - port: 8081
    targetPort: 8081
    name: grpc
  - port: 6070
    targetPort: 6070
    name: http-debug
  - port: 9102
    targetPort: 9102
    name: metrics
  selector:
    app: ratelimit
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ratelimit
  namespace: ratelimit
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ratelimit
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: ratelimit
      annotations:
        # Prometheus Scrape Annotations
        prometheus.io/scrape: "true"
        prometheus.io/port: "9102"
        prometheus.io/path: "/metrics"
        # Istio soll Metriken nicht mergen
        prometheus.istio.io/merge-metrics: "false"
    spec:
      containers:

      # --- Rate Limit Service ---
```

```
      - name: ratelimit
        # Gepinnter Commit-SHA statt :master (kein semantisches Versioning nach
v1.4.0)
        # Aktuelle Tags: https://hub.docker.com/r/envoyproxy/ratelimit/tags
        image: envoyproxy/ratelimit:3fb70258
        command: ["/bin/ratelimit"]
        ports:
        - containerPort: 8080   # HTTP/REST
        - containerPort: 8081   # gRPC
        - containerPort: 6070   # Debug
        env:
        - name: LOG_LEVEL
          value: debug
        - name: LOG_FORMAT
          value: json
        - name: REDIS_SOCKET_TYPE
          value: tcp
        - name: REDIS_URL
          value: redis.ratelimit.svc.cluster.local:6379
        - name: RUNTIME_ROOT
          value: /data
        - name: RUNTIME_SUBDIRECTORY
          value: ratelimit
        - name: RUNTIME_WATCH_ROOT
          value: "false"
        - name: RUNTIME_IGNOREDOTFILES
          value: "true"
        # --- StatsD Konfiguration ---
        - name: USE_STATSD
          value: "true"
        - name: STATSD_HOST
          value: localhost        # Sidecar im selben Pod
        - name: STATSD_PORT
          value: "9125"
        - name: STATS_FLUSH_INTERVAL
          value: "10s"
        - name: GRPC_PORT
          value: "8081"
        volumeMounts:
        - name: config
          mountPath: /data/ratelimit/config
          readOnly: true
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            memory: 256Mi

      # --- StatsD Exporter Sidecar ---
      # Empfängt StatsD-Metriken vom ratelimit-Container auf localhost:9125
      # und exponiert sie als Prometheus-Metriken auf :9102/metrics
```

```
      - name: statsd-exporter
        image: prom/statsd-exporter:v0.27.2
        args:
        - "--statsd.mapping-config=/etc/statsd/statsd.yaml"
        - "--statsd.listen-udp=:9125"
        - "--web.listen-address=:9102"
        ports:
        - containerPort: 9125
          protocol: UDP
          name: statsd-udp
        - containerPort: 9102
          name: metrics
        volumeMounts:
        - name: statsd-config
          mountPath: /etc/statsd
          readOnly: true
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            memory: 128Mi

      volumes:
      - name: config
        configMap:
          name: ratelimit-config
      - name: statsd-config
        configMap:
          name: statsd-config
```

> ***Image-Tag**: `envoyproxy/ratelimit` hat kein semantisches Versioning nach v1.4.0. Immer einen Commit-SHA pinnen (z.B. `3fb70258`), nicht `:master` verwenden.*

---

6. Monitoring Scrape Config

> ***Hinweis**: Das Prometheus aus dem Istio Demo-Stack (`istio/samples/addons`) ist ein **Plain Prometheus ohne Operator**. CRDs wie `PodMonitor` und `PrometheusRule` existieren dort nicht.*

Variante A: Plain Prometheus / Istio Demo (Standard)

Kein extra Manifest nötig. Das Deployment in Schritt 5 hat bereits die Annotations gesetzt, die Plain Prometheus automatisch scraped:

```
## Bereits im Deployment (Schritt 5) enthalten:
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "9102"
  prometheus.io/path: "/metrics"
```

Variante B: Prometheus Operator / VictoriaMetrics Operator

Nur wenn du den Prometheus Operator (z.B. kube-prometheus-stack) oder VictoriaMetrics Operator nutzt:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: ratelimit
  namespace: ratelimit
  labels:
    release: prometheus   # Muss zu deinem Prometheus-Selector passen
spec:
  selector:
    matchLabels:
      app: ratelimit
  podMetricsEndpoints:
  - port: metrics
    path: /metrics
    interval: 15s
```

> ***Nicht beides gleichzeitig*** *— Annotations + PodMonitor führt zu doppeltem Scraping. Bei Nutzung von PodMonitor die Annotations aus dem Deployment entfernen.*

---

7. JWT Authentication (am Gateway)

```yaml
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: jwt-auth
  namespace: istio-system
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  jwtRules:
  - issuer: "https://auth.example.com"
    jwksUri: "https://auth.example.com/.well-known/jwks.json"
    # Claim "customer_id" muss im JWT-Payload auf Top-Level vorhanden sein
    # z.B.: { "sub": "user123", "customer_id": "customer-a", "iss": "...", ... }
    # Verschachtelte Claims: path mit mehreren Keys angeben
```

> ***Hinweis Gateway API****: Falls du ein Gateway API* `Gateway` *-Objekt statt des klassischen* `istio-ingressgateway` *nutzt, ersetze das Label mit:* `istio.io/gateway-name: <name-deines-gateways>`

---

8. AuthorizationPolicy (JWT erzwingen)

```yaml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: istio-system
```

```yaml
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  action: DENY
  rules:
  - from:
    - source:
        notRequestPrincipals: ["*"]
```

---

9. EnvoyFilter: Rate Limit am Gateway

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: ratelimit-filter
  namespace: istio-system
spec:
  workloadSelector:
    labels:
      istio: ingressgateway
      # Für Gateway API stattdessen:
      # istio.io/gateway-name: <name-deines-gateways>
  configPatches:

  # ============================================================
  # CLUSTER-PATCH: Rate Limit Service als Envoy-Cluster registrieren
  # ============================================================
  # Auch wenn der RLS im Mesh läuft und Istio automatisch einen
  # Cluster anlegt, ist der explizite Patch sauberer: du kontrollierst
  # den Cluster-Namen, der im HTTP Filter referenziert wird.
  # Ohne diesen Patch: "unknown cluster 'rate_limit_cluster'" im Log.
  - applyTo: CLUSTER
    match:
      cluster:
        service: ratelimit.ratelimit.svc.cluster.local
    patch:
      operation: ADD
      value:
        name: rate_limit_cluster
        type: STRICT_DNS
        connect_timeout: 10s
        lb_policy: ROUND_ROBIN
        http2_protocol_options: {}        # gRPC braucht HTTP/2
        load_assignment:
          cluster_name: rate_limit_cluster
          endpoints:
          - lb_endpoints:
            - endpoint:
                address:
                  socket_address:
```

```
                    address: ratelimit.ratelimit.svc.cluster.local
                    port_value: 8081

  # ============================================================
  # HTTP FILTER: Rate Limit Filter in die Filter-Chain einfügen
  # ============================================================
  - applyTo: HTTP_FILTER
    match:
      context: GATEWAY
      listener:
        filterChain:
          filter:
            name: envoy.filters.network.http_connection_manager
            subFilter:
              name: envoy.filters.http.router
    patch:
      operation: INSERT_BEFORE
      value:
        name: envoy.filters.http.ratelimit
        typed_config:
          "@type":
type.googleapis.com/envoy.extensions.filters.http.ratelimit.v3.RateLimit
          domain: my-api                    # Muss mit ConfigMap übereinstimmen
          failure_mode_deny: false          # fail-open: bei RLS-Ausfall Requests
durchlassen
          timeout: 0.25s
          rate_limit_service:
            grpc_service:
              envoy_grpc:
                cluster_name: rate_limit_cluster
            transport_api_version: V3

  # ============================================================
  # DESCRIPTOR: JWT Claim "customer_id" als Descriptor an RLS senden
  # ============================================================
  # Istio macht JWT-Claims automatisch als Envoy-Metadata verfügbar
  # unter dem Key "envoy.filters.http.jwt_authn".
  # Die Action extrahiert daraus den Claim "customer_id" und
  # sendet ihn als Descriptor {customer_id: "<wert>"} an den RLS.
  - applyTo: VIRTUAL_HOST
    match:
      context: GATEWAY
    patch:
      operation: MERGE
      value:
        rate_limits:
        - actions:
          - metadata:
              metadata_key:
                key: envoy.filters.http.jwt_authn
                path:
```

```
      - key: customer_id
    descriptor_key: customer_id
```

> ***Ohne Cluster-Patch*** *bekommst du im Gateway-Log:* `unknown cluster 'rate_limit_cluster'` *und Rate Limiting funktioniert nicht.*

> `failure_mode_deny: false` *= fail-open (Requests durchlassen bei RLS-Ausfall). Auf* `true` *setzen für fail-closed (429 bei RLS-Ausfall).*

> ***Default Response****: Envoy gibt bei OVER_LIMIT* ***429 Too Many Requests*** *zurück plus den Header* `x-envoy-ratelimited: true` *. Anpassbar über* `rate_limited_status` *:*
>
> ```
> rate_limited_status:
>   code: 412    # statt 429 (nicht empfohlen)
> ```

---

10. PeerAuthentication (nötig weil ratelimit im Mesh)

Da der ratelimit-Namespace im Mesh läuft (`istio-injection: enabled`), ist STRICT mTLS aktiv. Prometheus läuft aber typischerweise ohne Sidecar und kann den Metrics-Port nicht über mTLS ansprechen. Deshalb Port 9102 auf PERMISSIVE setzen.

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: ratelimit
  namespace: ratelimit
spec:
  selector:
    matchLabels:
      app: ratelimit
  portLevelMtls:
    8081:
      mode: STRICT       # gRPC vom Gateway: mTLS (beide im Mesh)
    9102:
      mode: PERMISSIVE    # Prometheus hat keinen Sidecar, braucht Plain-Text-Zugriff
```

> ***Ohne Mesh****: Wenn der ratelimit-Namespace nicht im Mesh ist (* `istio-injection` *nicht gesetzt), ist diese Ressource nicht nötig — überspringen.*

---

11. Alerting

> ***Hinweis****: Das Prometheus aus dem Istio Demo-Stack (* `istio/samples/addons` *) hat keinen Operator.* `PrometheusRule` *CRDs werden dort nicht unterstützt.*

Variante A: Plain Prometheus / Istio Demo

Die Rules direkt als ConfigMap einbinden:

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: prometheus-ratelimit-rules
  namespace: istio-system    # oder wo dein Prometheus läuft
data:
  ratelimit_rules.yml: |
    groups:
    - name: ratelimit
      rules:
      - alert: CustomerRateLimitExceeded
        expr: |
          rate(ratelimit_service_rate_limit_over_limit_count{domain="my-api"}[5m])
          /
          rate(ratelimit_service_rate_limit_total_hits_count{domain="my-api"}[5m])
          > 0.5
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Kunde {{ $labels.key1 }} überschreitet Rate Limit"
          description: "Mehr als 50% der Requests von {{ $labels.key1 }} werden
abgelehnt."

      - alert: RateLimitServiceDown
        expr: up{job="ratelimit"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "Rate Limit Service ist nicht erreichbar"

      - alert: RateLimitConfigError
        expr: increase(ratelimit_service_config_load_error[5m]) > 0
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Rate Limit Config konnte nicht geladen werden"
```

Die ConfigMap muss als Volume in den Prometheus-Pod gemountet und in `prometheus.yml` referenziert werden:

```
rule_files:
- /etc/prometheus/ratelimit_rules.yml
```

Variante B: Prometheus Operator

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: ratelimit-alerts
  namespace: ratelimit
spec:
  groups:
```

```
  - name: ratelimit
    rules:
    - alert: CustomerRateLimitExceeded
      expr: |
        rate(ratelimit_service_rate_limit_over_limit_count{domain="my-api"}[5m])
        /
        rate(ratelimit_service_rate_limit_total_hits_count{domain="my-api"}[5m])
        > 0.5
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Kunde {{ $labels.key1 }} überschreitet Rate Limit"
        description: "Mehr als 50% der Requests von {{ $labels.key1 }} werden
abgelehnt."

    - alert: RateLimitServiceDown
      expr: up{job="ratelimit"} == 0
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "Rate Limit Service ist nicht erreichbar"

    - alert: RateLimitConfigError
      expr: increase(ratelimit_service_config_load_error[5m]) > 0
      for: 1m
      labels:
        severity: warning
      annotations:
        summary: "Rate Limit Config konnte nicht geladen werden"
```

Deployment-Reihenfolge

```
## 1. Namespace
kubectl apply -f 01-namespace.yaml

## 2. Redis
kubectl apply -f 02-redis.yaml
kubectl -n ratelimit wait --for=condition=ready pod -l app=redis --timeout=60s

## 3. Configs (ratelimit + statsd mapping)
kubectl apply -f 03-ratelimit-config.yaml
kubectl apply -f 04-statsd-config.yaml

## 4. Rate Limit Service + StatsD Exporter
kubectl apply -f 05-ratelimit-deployment.yaml
kubectl -n ratelimit wait --for=condition=ready pod -l app=ratelimit --timeout=60s

## 5. Monitoring (nur bei Prometheus Operator, sonst überspringen - Annotations
reichen)
```

```
## kubectl apply -f 06-podmonitor.yaml

## 6. PeerAuthentication (nötig weil ratelimit im Mesh)
kubectl apply -f 07-peer-authentication.yaml

## 7. JWT Auth + Policy
kubectl apply -f 08-request-authentication.yaml
kubectl apply -f 09-authorization-policy.yaml

## 8. EnvoyFilter (zuletzt!)
kubectl apply -f 10-envoyfilter-ratelimit.yaml

## 9. Alerting (PrometheusRule bei Operator, sonst ConfigMap - siehe Schritt 11)
## kubectl apply -f 11-prometheus-rules.yaml
```

Testen

```
## Token für customer-a generieren
TOKEN_A="eyJhbGciOiJS..."  # JWT mit claim: customer_id: "customer-a"

## Einzelner Request - sollte 200 zurückgeben
curl -v -H "Authorization: Bearer $TOKEN_A" https://api.example.com/endpoint

## Ohne Token - sollte 403 zurückgeben (AuthorizationPolicy)
curl -v https://api.example.com/endpoint

## Last generieren - Rate Limit testen
for i in $(seq 1 50); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -H "Authorization: Bearer $TOKEN_A" \
    https://api.example.com/endpoint
done
## Erwartete Response bei Rate Limit:
## HTTP/1.1 429 Too Many Requests
## x-envoy-ratelimited: true

## Metriken direkt vom statsd-exporter prüfen
kubectl -n ratelimit port-forward deploy/ratelimit 9102:9102 &
curl -s http://localhost:9102/metrics | grep ratelimit_service_rate_limit
## Erwarteter Output:
## ratelimit_service_rate_limit_total_hits_count{domain="my-
api",key1="customer_id_customer-a"} 50
## ratelimit_service_rate_limit_within_limit_count{domain="my-
api",key1="customer_id_customer-a"} 40
## ratelimit_service_rate_limit_over_limit_count{domain="my-
api",key1="customer_id_customer-a"} 10

## Redis Counter direkt prüfen
kubectl -n ratelimit exec -it deploy/redis -- redis-cli KEYS '*'
```

Grafana / Prometheus Queries

Requests pro Kunde (Rate/Minute)

```
rate(ratelimit_service_rate_limit_total_hits_count{domain="my-api"}[1m]) * 60
```

Over-Limit Requests pro Kunde

```
rate(ratelimit_service_rate_limit_over_limit_count{domain="my-api"}[1m]) * 60
```

Near-Limit Warnungen (>80% verbraucht)

```
rate(ratelimit_service_rate_limit_near_limit_count{domain="my-api"}[1m]) > 0
```

Prozent vom Limit verbraucht

Die Metrik weiß nicht, wie hoch das konfigurierte Limit ist. Statische Zuordnung als Recording Rule oder Grafana-Variable nötig:

```
## customer-a: 1000 req/min
rate(ratelimit_service_rate_limit_total_hits_count{key1="customer_id_customer-a"}
[1m]) * 60 / 1000 * 100

## customer-b: 100 req/min
rate(ratelimit_service_rate_limit_total_hits_count{key1="customer_id_customer-b"}
[1m]) * 60 / 100 * 100
```

Config Load Status (Healthcheck)

```
ratelimit_service_config_load_success   # Sollte > 0 sein
ratelimit_service_config_load_error     # Sollte 0 sein
```

Debugging

```
## Rate Limit Service Logs
kubectl -n ratelimit logs -l app=ratelimit -c ratelimit -f

## StatsD Exporter Logs
kubectl -n ratelimit logs -l app=ratelimit -c statsd-exporter -f

## Gateway: Wurde der Cluster-Patch applied?
istioctl proxy-config cluster -n istio-system deploy/istio-ingressgateway | grep rate

## Gateway: Wurde der HTTP Filter applied?
istioctl proxy-config listener -n istio-system deploy/istio-ingressgateway -o json |
grep ratelimit
```

```
## Gateway: Route Config mit Rate Limit Actions?
istioctl proxy-config route -n istio-system deploy/istio-ingressgateway -o json |
grep rate

## Redis Counter prüfen
kubectl -n ratelimit exec -it deploy/redis -- redis-cli KEYS '*'
```

Häufige Fehler

Symptom	Ursache	Lösung
`unknown cluster 'rate_limit_cluster'`	Cluster-Patch fehlt oder falsch	EnvoyFilter Cluster-Patch prüfen
Rate Limit greift nicht	Domain in EnvoyFilter ≠ ConfigMap	`domain: my-api` überall identisch
500 statt 429	RLS nicht erreichbar + `failure_mode_deny: true`	RLS Logs prüfen, auf `false` setzen
Metriken leer	`USE_STATSD=false` oder statsd-exporter nicht da	Env-Var + Sidecar prüfen
Metriken ohne Kunden-Labels	Mapping-Config fehlt oder falsch	statsd-config ConfigMap prüfen

---

Hinweise für Produktion

- **Mesh**: In diesem Setup läuft ratelimit im Mesh (`istio-injection: enabled`). Das gibt dir mTLS und Observability. Ohne Mesh: Label weglassen, Schritt 10 überspringen
- **`domain: my-api`** ist frei wählbar — muss aber in EnvoyFilter, ratelimit-Config und Grafana Queries identisch sein
- **StatsD → localhost**: ratelimit-Container sendet an `localhost:9125` , statsd-exporter Sidecar lauscht dort
- **`STATS_FLUSH_INTERVAL: 10s`** : Metriken werden alle 10s geflushed. Für Produktion 10-30s
- **Redis**: Für Produktion Redis Sentinel/Cluster verwenden
- **Replicas**: ratelimit-Service kann horizontal skaliert werden (State liegt in Redis)
- **Image-Tag**: Immer Commit-SHA pinnen, nicht `:master`
- **Istio-Upgrades**: EnvoyFilter ist eine unversioned API — bei jedem Istio/Envoy-Upgrade testen!
- **Metrik-Labels**: `key1` enthält `customer_id_<value>` . In Grafana filtern: `key1=~"customer_id_.*"`
- **JWT Claim**: `customer_id` muss auf Top-Level im JWT-Payload sein. Verschachtelte Claims über `path` mit mehreren Keys
- **Gateway API**: Label im EnvoyFilter anpassen: `istio.io/gateway-name: <n>` statt `istio: ingressgateway` , context bleibt `GATEWAY`
- **PeerAuthentication**: Nötig weil ratelimit im Mesh. Port 9102 PERMISSIVE damit Prometheus scrapen kann. Ohne Mesh nicht nötig
- **Monitoring**: Entweder PodMonitor (Prometheus Operator) ODER `prometheus.io/*` Annotations — nicht beides, sonst doppeltes Scraping

zu istio beitragen

Der typische Weg:

1. **Einstieg über Docs/UX** — Am einfachsten ist der Einstieg über Issues mit den Labels "User Experience" und "Documentation". Auf `istio/istio.io` gibt es kuratierte "good first issues", die speziell für Erstbeitragende

ausgewählt sind.

2. **CLA unterschreiben** — Bevor PRs akzeptiert werden, muss ein Contributor License Agreement (CLA) bei der CNCF/Linux Foundation unterzeichnet werden.

3. **Working Groups** — Für Feature-Beiträge muss man zuerst die Idee mit der zuständigen Working Group auf Slack diskutieren, dann ein GitHub Issue erstellen, und bei größeren Features ein Design-Dokument über Google Docs einreichen.

4. **Release Manager Rotation** — Wer tiefer einsteigen will, kann der Release-Manager-Rotation beitreten — zwei Releases als Shadow, dann selbst Primary.

Ref:

- https://github.com/istio/community/blob/master/CONTRIBUTING.md#working-groups